INSTITUT POLYTECHNIQUE DE PARIS

NNT : 2026IPPAP020

Thèse de doctorat

Optimization for Transportation Planning and Periodic Task Scheduling:

Rail Freight Crew Management under Train Delays, Line Planning, and Balanced Assignments

Thèse de doctorat de l'Institut Polytechnique de Paris
préparée à École nationale des ponts et chaussées

École doctorale n°574 École doctorale de l'Institut Polytechnique de Paris, École doctorale de mathématiques Hadamard (EDMH)
Spécialité de doctorat : Mathématiques appliquées

Thèse présentée et soutenue à Champs-sur-Marne, le 3 novembre 2026, par

HÉLOÏSE GACHET

Composition du Jury :

Christoph Dürr Researcher, CNRS (LIP6)	Rapporteur
Dominique Feillet Professeur, Mines Saint-Étienne (LIMOS)	Rapporteur
Walid Ben Ameur Professeur, Télécom SudParis (SAMOVAR)	Examinateur
Nadia Brauner Professeur, Université Grenoble Alpes (G-SCOP)	Présidente du jury
Rolf van Lieshout Professeur Assistant, Eindhoven University of Technology (OPAC)	Examinateur
Frédéric Meunier Professeur, École nationale des ponts et chaussées (CERMICS)	Directeur de thèse
Vincent Chmielarski Dirigeant pôle Recherche Opérationnelle, Hexafret	Invité
Juliette Pouzet Cheffe de projet, SNCF (DTIPG)	Invitée

Remerciements

***

J'ouvre la longue liste des remerciements à tou.te.s celles et ceux qui ont contribué de près ou de loin à ces trois années de thèse en adressant un immense merci à mon directeur de thèse Frédéric Meunier. Ça a été un réel plaisir de travailler (un peu) et discuter (beaucoup) avec toi. J'ai beaucoup appris scientifiquement grâce à tous tes précieux conseils et, quoi que tu puisses en penser, humainement avec nos digressions sur l'IA, le cinéma, l'art et j'en passe ! Merci aussi pour avoir rendu cette expérience de la recherche tellement motivante et agréable que je compte bien l'enrichir de mon côté (qui aurait pu le prédire il y a quelques années ?). Et je peux affirmer sans aucun doute que la réponse à la question « Est-ce que j'ai aimé aller travailler pendant ces trois dernières années ? » est un grand « oui ».

Je remercie aussi tout particulièrement mon encadrante à la SNCF Juliette Pouzet, sans qui l'idée de faire une thèse ne m'aurait jamais traversé l'esprit. Merci pour la confiance que tu m'as accordée en stage puis en thèse, merci pour ton soutien sans faille et ta bonne humeur constante (malgré parfois un genou, une cheville ou un dos en vrac) !

Je remercie Christoph Dürr et Dominique Feillet pour avoir accepté de relire cette thèse, pour vos remarques précieuses et votre temps. Je remercie également Walid Ben-Ameur, Nadia Brauner et Rolf van Lieshout pour avoir accepté de faire partie de mon jury et d'avoir fait le déplacement malgré des emplois du temps bien chargés.

Je tiens à remercier toutes les personnes à la SNCF qui ont été impliquées tout au long de cette thèse : Christelle Lérin et les équipes d'Hexafret dont l'expertise et les fréquents retours ont rendu toute la première partie de cette thèse possible. Merci en particulier à Vincent Chmielarski d'avoir aidé au montage et au suivi de cette thèse, pour ta disponibilité dès que j'avais des questions « métier » et pour ta présence lors de la soutenance.

I would like to thank Anita Schöbel, the mobility team (with the Fraunhofer Institute people) and the whole optimization group at RPTU for their really nice welcome in Kaiserslautern for the summer. I had great discussions and felt really included in the team thanks to your kind efforts, and I hope we will have the chance to collaborate again in the near future.

J'ai été ravie d'avoir passé cette thèse entre les équipes de la SNCF et du Cermics avec tous les départs et les arrivées que ces trois années ont compté ! Je ne nommerai pas tou.te.s celles et ceux que ce très large merci inclut, par peur des potentiel.le.s oublié.e.s, mais merci pour les chouettes discussions, les sorties musées/expos, patinoire, escalade, cinéma, théâtre et skate, les jeux de société, le ping-pong, le café chez Jeanine et surtout la bonne ambiance constante ! Merci à la team Boxe Thaï : j'ai appris à envoyer des high kicks à vos côtés et je ne l'oublierai pas.

Merci bien sûr aux "coupaigns" (vous vous reconnaîtrez sans peine) pour avoir été là depuis toutes ces années et toujours traquenardables pour des bêtises ! Merci à la team "musée et autre?" de prépa ! Merci à la fine équipe du patinage !

Et pour finir, merci les parents de m'avoir donné le goût des maths (et je peux vous dire

qu'il n'a pas le goût du comté, vous aurez la référence), merci Cécile pour nos von kohls récurrents malgré la frontière qui nous sépare (pour parler maths, cuisine ou juste passer trois heures au téléphone parce que pourquoi pas), merci Vivien pour les après-midis passés ensemble à faire du roller, des expos, de l'escalade (avec souvent une glace à la fin) ou juste prévoir des trucs sans avoir le temps de les faire (la classique).

Abstract

***

This thesis investigates optimization problems arising in transportation planning and periodic task scheduling, combining integer programming, combinatorial optimization, and stochastic modeling, with a strong application focus on rail freight operations at SNCF.

The first contribution addresses the *crew management* problem for rail freight drivers at SNCF: given periodic tasks (trains to be driven), we aim to build optimal *cyclic rosters* covering all tasks, while complying with numerous rules and regulations. Current industrial practice, motivated by a full body of literature, decomposes this problem into two sequential stages: first, the construction of *duties* (daily task sequences assigned to a single driver), then their organization into cyclic rosters. This sequential approach introduces structural suboptimalities and makes it difficult to account for train delays. This work proposes first a deterministic integrated model formulated as a set-covering integer linear program and solved by column generation. The pricing subproblem is modeled as a Resource-Constrained Shortest Path (RCSP) problem on an acyclic graph, and solved using a label correcting algorithm with additional speeding-up strategies. Numerical experiments on real SNCF instances demonstrate significant cost reductions compared to the sequential pipeline.

Building on this deterministic framework, we then extend the problem to account for train delays. A *fragility* score is introduced to measure the sensitivity of a roster to train delays: as exogenous delays arise and propagate within each duty, this score measures the expected number of disrupted trains and duties that would be operated by backup drivers in practice. This stochastic crew management problem then builds optimal cyclic rosters covering all trains, with an additional threshold constraint on the expected fragility. The corresponding model is solved by column generation with an enriched RCSP pricing subproblem that handles delay distributions. Results confirm that the integrated stochastic approach outperforms the current two-stage sequential approach at SNCF, both in terms of cost and robustness to disruptions.

Shifting to a different application domain, the second contribution of this work tackles the *line planning* problem in public transit networks. Given a passenger origin-destination demand matrix and a pool of candidate lines, the goal is to select lines and assign integer frequencies so as to minimize passengers' travel time, subject to budget and capacity constraints. A Dantzig–Wolfe reformulation of an arc-based model on a "change-and-go network" is solved with branch-and-price-and-cut. Numerical experiments compare the straightforward resolution of the arc-based model with the proposed algorithm.

Finally, the third contribution takes a more theoretical perspective and studies *balanced assignments* of periodic tasks, i.e., such that each worker performs every task with the same frequency over the long term. A structural connection is established between the existence of such assignments and properties of arc-colored Eulerian directed graphs. Existence and periodicity theorems, bounds on the minimal period, and complexity results are established, for a basic version first, then for an extended version introducing additional constraints.

Résumé

***

Cette thèse explore des problèmes d'optimisation en planification des transports et en conception d'emplois du temps périodiques, en combinant programmation en nombres entiers, optimisation combinatoire et modélisation stochastique, avec des applications au fret ferroviaire à la SNCF.

La première contribution traite du problème de gestion des équipes de conducteurs de fret à la SNCF : étant données des tâches périodiques (trains à conduire), l'objectif est de construire des *grilles cycliques* optimales couvrant l'ensemble des tâches, tout en respectant une réglementation riche. La pratique industrielle actuelle décompose ce problème en deux étapes séquentielles — construction des *journées de service* (séquences de tâches journalières affectées à un conducteur) puis leur organisation en grilles cycliques — ce qui introduit des sous-optimalités structurelles et rend difficile la prise en compte des retards. Ce travail propose d'abord un modèle déterministe intégré, formulé comme un programme linéaire en nombres entiers de type "set cover" et résolu par génération de colonnes, avec un sous-problème de pricing modélisé comme un plus court chemin avec contraintes de ressources (RCSP) sur un graphe acyclique. Les expériences numériques sur des instances réelles de la SNCF démontrent des réductions de coûts significatives par rapport à l'approche séquentielle. Ce cadre est ensuite étendu aux retards de trains via un score de *fragilité*, qui mesure le nombre moyen de trains et de journées de service perturbés qui devraient être pris en charge par des conducteurs de remplacement. Le problème stochastique de gestion des équipes minimise alors les coûts sous une contrainte de seuil sur la fragilité moyenne, et est résolu par génération de colonnes avec un RCSP enrichi gérant les distributions de retards. Les résultats confirment que l'approche intégrée stochastique surpasse la méthode séquentielle actuellement en usage à la SNCF, tant en termes de coûts que de robustesse aux perturbations.

La deuxième contribution aborde le problème de *planification de lignes* pour les transports publics. Étant donnés une matrice de demande origine-destination et un ensemble de lignes candidates, l'objectif est de sélectionner des lignes et d'y affecter des fréquences entières afin de minimiser le temps de trajet des voyageurs, sous contraintes de budget et de capacité. Une reformulation de Dantzig–Wolfe d'un modèle basé sur les arcs, sur un graphe appelé "change-and-go network", est résolue par branch-and-price-and-cut, et comparée à la résolution directe du modèle de base dans des expériences numériques.

Enfin, la troisième contribution adopte une perspective plus théorique et étudie les *affectations équilibrées* de tâches périodiques, c'est-à-dire telles que chaque travailleur effectue chaque tâche avec la même fréquence à long terme. Un lien structurel est établi entre l'existence de telles affectations et les propriétés de graphes orientés eulériens arc-colorés. Des théorèmes d'existence et de périodicité, des bornes sur la période minimale et des résultats de complexité sont établis, d'abord pour une version de base, puis pour une version enrichie par des contraintes supplémentaires.

Contents

***

Remerciements **i**

Abstract **iii**

Résumé **v**

List of Figures **xi**

List of Tables **xiv**

1 Introduction **1**

2 Introduction (français) **13**

I Integrating crew scheduling and crew rostering with train delays for rail freight **27**

Introduction **29**

3 Planning process in the railway industry 29
3.1 For rail passenger transportation 29
3.2 For rail freight 30
4 Sequential versus integrated optimization 30
5 Uncertainties in the planning process 31
6 Scientific challenges and contributions 31
7 Outline 32

3 Crew scheduling and crew rostering at SNCF **33**

3.1 Context and contributions 34
3.2 Literature review 34
3.2.1 Crew scheduling problem 34
Resolution methods for the railway industry 35
CSP for rail freight 35
3.2.2 Crew rostering problem 35
Cyclic versus personalized rostering 35
Resolution methods for the railway industry 36
CRP for rail freight 36
3.3 Crew scheduling and crew rostering at SNCF: formulation, specificities and algorithms 36

3.3.1 Crew scheduling at SNCF . . . 36
Problem formulation . . . 36
Problem specificities and algorithm currently used at SNCF . . . 37
3.3.2 Crew rostering at SNCF . . . 38
Problem formulation . . . 38
Problem specificities and algorithm currently used at SNCF . . . 39
Comments and limitations . . . 42
3.4 Alternative model to determine optimal cyclic rosters at SNCF . . . 44
3.5 Numerical experiments . . . 47
3.5.1 Experimental setting and instances . . . 47
3.5.2 Results . . . 47
3.6 Comments . . . 51

4 Integrating crew scheduling and crew rostering for rail freight 53
4.1 Context and contributions . . . 54
4.2 Literature review: integrating crew scheduling and crew rostering . . . 54
4.3 Problem formulation . . . 56
4.4 Model . . . 56
4.4.1 Mathematical program . . . 56
4.5 Solution method . . . 57
4.5.1 Column generation . . . 57
4.5.2 Pricing subproblem as a resource-constrained shortest path problem . . . 58
Resource-constrained shortest path problems: general setting . . . 58
Specialization to the pricing problem . . . 58
Label correcting algorithm . . . 59
4.5.3 Improvements . . . 61
Early feasibility check . . . 61
Warm-starting the label correcting algorithm . . . 61
Set of bounds to discard paths . . . 61
Variable dominance . . . 63
Improving the quality of integer solution . . . 63
4.6 Numerical experiments . . . 64
4.6.1 Experimental setting and instances . . . 64
4.6.2 Results . . . 64
4.6.3 Comments . . . 65

5 Crew management with train delays 67
5.1 Context . . . 68
5.2 Literature review . . . 68
5.3 Problem formulation . . . 69
5.3.1 Setting . . . 69
5.3.2 Modeling delays and “fragility” . . . 69
Modeling delay propagation . . . 69
Modeling “fragility” of rosters . . . 69
5.3.3 Problem formulation . . . 70
5.4 Model . . . 71
5.5 Solution method . . . 71
5.5.1 Column generation . . . 71
Algorithm for the linear relaxation (Covering SCM) . . . 71
From a fractional solution to an integer solution . . . 72
5.5.2 Pricing subproblem with delay distribution as a RCSP problem . . . 72
Poset of resources . . . 72

Extension functions . . . 73
Cost function . . . 73
5.6 Coping with approximated distributions for primary delays . . . 73
5.7 Numerical experiments . . . 76
5.7.1 Experimental setting and instances . . . 76
5.7.2 Results . . . 76
5.7.3 Comments . . . 77
5.8 Final remarks . . . 77
5.8.1 Convergence speed . . . 77
5.8.2 Other risk measures . . . 78
Generalities . . . 78
Conditional value-at-risk . . . 78
5.8.3 Variants of the stochastic crew management problem . . . 79
Minimizing the fragility . . . 79
Delays impacting the number of working days . . . 81

II Branch-and-price(-and-cut) for line planning with minimal traveling time 83

Introduction 85

6 Branch-and-price 87
6.1 Problem formulation . . . 87
6.2 Literature review . . . 88
6.2.1 Passenger-oriented line planning . . . 89
6.2.2 Solution methods . . . 89
6.2.3 Different formulations . . . 89
6.3 Model . . . 90
6.3.1 Change-and-go network . . . 90
6.3.2 Arc-based formulation . . . 91
6.3.3 Path-based formulation . . . 92
6.4 Branch-and-price(-and-cut) . . . 94
6.4.1 Column generation . . . 94
6.4.2 Pricing subproblem . . . 94
6.4.3 Branching strategy . . . 95
6.4.4 Valid inequalities . . . 95
6.4.5 Accelerating for the first integer solution . . . 97
6.5 Numerical experiments . . . 98
6.5.1 Experimental setting and instances . . . 98
6.5.2 Results . . . 98
6.5.3 Comments . . . 98

III Balanced assignments of periodic tasks 101

Introduction 103

7 Basic version: non-overlapping balanced assignments 105
7.1 Introduction . . . 105
7.1.1 Problem formulation and results . . . 105
7.1.2 Positioning with respect to the literature . . . 107
7.2 Main tools for the proofs . . . 109
7.2.1 Building a new feasible assignment from a sequence of feasible assignments 109
7.2.2 Encoding assignments as a directed graph: definition and properties . 110
7.2.3 From a general instance $\mathcal{I}$ to one $\mathcal{I}'$ satisfying $|U(\mathcal{I}')| = q$ . . . 110
7.3 Proof of Theorem 7.1.1 . . . 112
7.4 Proof of Theorem 7.1.2 . . . 117
7.5 Final remarks . . . 119
7.5.1 Discussing possible generalizations of Theorem 7.1.1 . . . 119
7.5.2 Minimizing the number of workers . . . 121
7.5.3 Link to crew rostering . . . 121

8 Extended version: balanced assignments with valid weekly schedules 123
8.1 Introduction . . . 124
8.1.1 Problem formulation and results . . . 124
8.1.2 Positioning with respect to the literature . . . 126
8.2 Main tools: revisions from the basic versions . . . 126
8.2.1 Building a new feasible assignment from a sequence of feasible assignments 126
8.2.2 Encoding assignments as a directed graph: definition and properties . 127
8.2.3 From a general instance $\mathcal{I}$ to one $\mathcal{I}'$ satisfying $|U(\mathcal{I}')| = q$ . . . 127
8.3 A problem of pebbles on an arc-colored Eulerian directed graph . . . 128
8.4 Proofs of Theorems 8.1.1 and 8.1.2 . . . 130
8.5 Concluding remarks . . . 132
8.5.1 Group action and Cayley graph . . . 132
8.5.2 All feasible assignments are balanced. (Almost.) . . . 133
8.5.3 Open question . . . 134
8.5.4 Link to crew rostering . . . 134

9 Complementary discussions 137
9.1 Special case: Cayley graph . . . 137
9.1.1 A problem of pebbles on a Cayley graph . . . 137
9.1.2 Link to periodic balanced feasible assignments . . . 138
9.2 Equivalence between colors and assignments . . . 139
9.3 An ILP solving the PERIODIC RELAXED BALANCED SEQUENCE OF COLORS problem . . . 142
9.4 Other particular feasible assignments . . . 144
9.4.1 Problem formulation . . . 144
9.4.2 First result . . . 145
9.4.3 A special case . . . 145
9.4.4 Skill-balanced sequence of colors . . . 146
Problem of pebbles . . . 146
From a skill-balanced sequence of colors to a skill-balanced assignment 147

10 Conclusion 149

Bibliography 156

A Column generation **157**
A.1 Setting . . . 157
A.2 Algorithm . . . 157
A.2.1 Pricing subproblem . . . 158
A.2.2 Farkas pricing . . . 158
A.3 Integer linear program . . . 159
A.3.1 Branch-and-price . . . 159
A.3.2 Enumeration-based gap closing . . . 160

List of Figures

***

2.1 Operational planning steps for railway passenger transportation 29
2.2 Operational planning steps for railway freight transportation 30

3.1 Example of trains in an instance 37
3.2 Example of a duty 37
3.3 Example of a block 38
3.4 Example of a roster 39
3.5 Illustration of the augmentation of blocks with long on-calls. 43
3.6 Illustration of the three cases occurring for the constraint ensuring that blocks are not overlapping 46
3.7 First part of the heuristic building a roster from blocks 48
3.8 Second part of the heuristic building a roster from blocks: overlap case 49
3.9 Second part of the heuristic building a roster from blocks: non-overlap case . 49

4.1 Example of a RCSPP graph for the pricing subproblem 60
4.2 Illustration of the multi-bounds efficiency 62

5.1 Illustration of the upper and lower finite-support random variables bounding the optimal value under partially known delays 74
5.2 Different problems in the line planning step of the operational planning process 85

6.1 An illustration of a public transportation network with the associated change-and-go network. 91

7.1 Example of an instance with a feasible assignment but no balanced one . . . 107
7.2 Example of an instance with a feasible assignment and a balanced one 108
7.3 Illustration of a merge operation 113
7.4 Illustration of the map χ 115
7.5 Illustration of the assignment encoded by a closed formula when $q \geqslant 2n$ . . . 118
7.6 Illustration of a series of merge operations resulting in a Hamiltonian cycle . 119
7.7 Illustration of the assignment built from an Hamiltonian cycle 120
7.8 Example with interdicted transitions with a balanced feasible assignment but no q-periodic one 120

8.1 Example of an instance for the extended version 125
8.2 Illustration of the transformation of an instance 128
8.3 Example of a regular colored Eulerian digraph 129
8.4 Example of a graph with tight bound on the period 135

9.1 Example of a graph with a skill-balanced sequence of colors but no periodic one147

A.1 Illustration of a branch-and-price tree 161

List of Tables

***

3.1 Working rules for duties 38
3.2 Working rules for blocks 40
3.3 Working rules for rosters 40
3.4 Results for the alternative formulation 50

4.1 Results for the integrated approach 64
4.2 Label correcting algorithm statistics 65

5.1 Primary delay distributions 76
5.2 Primary delay distributions: instance 1,815 trains bis 77
5.3 Results for the stochastic integrated approach 77
5.4 Results for the stochastic integrated approach: variant 80

6.1 Characteristics of the instances 98
6.2 Results of the line planning problem 99

Chapter 1

Introduction

***

This thesis was conducted as an industrial partnership with SNCF, the main French railway company, and Hexafret, a subsidiary of SNCF specialized in rail freight transportation and logistics.

Transportation systems rely on complex infrastructures in which limited resources, physical capacity constraints, operational uncertainty, and strong regulatory requirements make it difficult to provide reliable and cost-efficient services. Railway systems are a particularly rich example of this complexity. From long-term strategic design to daily operations and real-time disruption management, railway operators must make decisions at different time scales: which trains should run, at what frequency and at what time, on which routes, with which rolling stock, and with which crews. These decisions are usually supported by planners' individual expertise, but optimization-based decision-support tools can make transportation planning faster, more systematic, and more efficient. Railway planning and public transport optimization have therefore been widely studied in operations research and transportation science [26, 45, 75].

In practice, many planning processes are organized sequentially. First, train lines, frequencies, and timetables are designed; then infrastructural and rolling-stock resources are assigned; then crew duties and rosters are built to accommodate the previous decisions; and finally disruptions are handled as they occur. Here, a *duty* denotes a sequence of work activities performed by a worker during a working day, typically including one or several tasks, waiting times, breaks, etc. A *roster* denotes a longer-term schedule assigned to a worker, or to a team of workers, specifying the succession of duties and rest days while satisfying labor regulations and operational constraints. This decomposition into sequential planning steps is natural from an organizational and computational viewpoint. However, these decisions are deeply interdependent. A train timetable that looks attractive from the passenger point of view may require resources that are difficult to schedule. A crew duty that is locally efficient may be hard to insert into a roster. A roster that is cost-efficient in nominal conditions may become fragile when delays occur and propagate. These interactions explain why transportation planning is a challenging field for mathematical optimization. A recurring issue is that sequential optimization may lead to suboptimal global decisions when downstream constraints are ignored at earlier planning stages. This has been observed in several personnel scheduling and crew management contexts, where integrating scheduling and rostering decisions can significantly improve the quality of the final solution [13, 36, 37, 59, 63, 86, 98]. This thesis follows this line of research by studying planning and assignment problems in which local

feasibility, global structure, robustness, and computational tractability must be handled simultaneously.

This thesis is organized around three main topics. The first topic deals with freight railway crew management. This planning problem is usually decomposed into a crew scheduling step and a crew rostering step. Crew scheduling consists in building feasible duties or blocks of work that cover the required train services, while crew rostering consists in assembling these duties into rosters assigned to drivers. Such problems belong to the broader field of personnel scheduling and rostering, which has been extensively studied in operations research [36, 86]. In railway applications, the additional presence of operational constraints, labor regulations, geographical compatibility, and cyclicity requirements makes the problem particularly challenging [48]. Instead of solving crew scheduling and crew rostering independently, we investigate integrated models that capture their interactions, with the aim of reducing suboptimalities induced by the sequential process. We also study robustness to train delays through a stochastic constraint. Since the resulting optimization models are too large to be solved by direct enumeration, we rely on column generation, a standard decomposition technique for large-scale integer programming and crew planning applications [30, 67]. The second topic of this thesis deals with line planning, a planning step that appears at the beginning of the transportation planning timeline. The line planning problem consists in selecting lines and frequencies in a public transport network while satisfying demand, capacity, and budget constraints. In passenger-oriented line planning, the objective is not only to control operational cost but also to account for passenger travel time, waiting time, and transfers. A classical way to model passenger choices relies on the *change-and-go network*, introduced by Anita Schöbel and Susanne Scholl in 2004 [81], which explicitly represents boarding, traveling, waiting, transfers, and alighting [12, 60, 80]. However, existing arc-based formulations for this problem can become memory-intensive on large instances. We therefore explore a path-based formulation obtained through a Dantzig–Wolfe reformulation of the arc-based model. This approach is related to previous work on line planning, passenger routing, and path-based formulations. We solve the resulting formulation using a branch-and-price algorithm, strengthened with additional valid inequalities. The third topic of the thesis is more theoretical. It aims to study the combinatorial properties that ensure the existence of balanced assignments of periodic tasks to workers. In this setting, tasks occur periodically and must be assigned to a given number of workers. The goal is to ensure that each worker performs each task with the same asymptotic frequency. Such questions are related to periodic scheduling, graph coloring, and fair assignments [14, 57]. Many transportation planning problems have periodic inputs: passenger demand in line planning is often described over a representative period, while train services in crew management may repeat according to periodic patterns. Sometimes, the output is also required to be periodic, or even cyclic. This is the case for cyclic rosters, where workers in a same team share a single roster, which is repeated cyclically over the planning horizon. The cyclicity is required in order to ensure regular and fair schedules among the workers. The motivation behind balanced assignments is to build on this idea: a balanced assignment preserves fairness among workers while relaxing the cyclicity (or even periodicity) requirement.

The purpose of this introduction is to present the operational and mathematical context of the thesis, to explain the main scientific challenges, to highlight the contributions, and to describe the organization of the manuscript. Before presenting these three topics in detail, we introduce the main mathematical concepts used throughout the thesis. The aim is not to provide a technical survey, but to give the reader the necessary background to understand why assignment problems, graph models, and decomposition methods naturally arise in transportation planning.

1 Some mathematical background

1.1 Mathematical programming

Many transportation planning problems belong to the broad field of mathematical optimization and operations research. The general idea is straightforward: among a set of feasible decisions, one seeks a decision that is best according to a given criterion. In this thesis, decisions may be driver duties, passenger paths, line frequencies, while feasibility may encode physical constraints, operational rules, labor regulations, passenger capacity, or fairness requirements. The criterion, which we want to either minimize or maximize, also referred to as the cost or the objective, may represent economical costs, travel time, robustness, or any performance indicator.

Many of the modern developments in operations research were motivated by transportation, logistics, and industrial planning problems. Linear programming emerged as a central tool in the middle of the twentieth century, notably with the simplex method introduced by Dantzig in 1947. In a linear program, one optimizes a linear objective function subject to linear equality and inequality constraints. A linear program writes in full generality in the following standard form:

$$\begin{aligned} \text{find a vector} \quad & x \in \mathbb{R}^n_{\geqslant 0} \\ \text{minimizing} \quad & \sum_{i=1}^{n} c_i x_i \\ \text{such that} \quad & \sum_{i=1}^{n} A_{i,j} x_i = b_j \quad \forall j \in [m]\,. \end{aligned}$$

This framework is powerful because many transportation, logistics, scheduling, and resource allocation problems can be expressed in this form, and because efficient algorithms exist to solve large instances. Linear programming is known to be solvable in polynomial time (though this result was proven much later, in 1979 by Khachiyan [53], using the ellipsoid algorithm, then by Karmarkar in 1984 [51] with the interior point method, which is still used nowadays in practice). Duality is another fundamental concept in linear programming: every linear program admits a companion problem, called the dual, which provides bounds and optimality certificates, which are the basis for more advanced algorithms. Linear programming is also used in economics, since dual variables can be interpreted as the marginal values of resources.

However, many planning decisions are discrete. A duty is either selected or not, a line is operated at an integer frequency or not operated at all, etc. Such discrete decisions are naturally modeled using integer variables. This leads to integer linear programming, and more generally mixed-integer programming, where some variables are required to take integer values. A mixed-integer linear program writes in full generality as:

$$\begin{aligned} \text{find a vector} \quad & x \in \mathbb{R}^n_{\geqslant 0} \times \mathbb{Z}^{n'}_{\geqslant 0} \\ \text{minimizing} \quad & \sum_{i=1}^{n+n'} c_i x_i \\ \text{such that} \quad & \sum_{i=1}^{n+n'} A_{i,j} x_i = b_j \quad \forall j \in [m]\,. \end{aligned}$$

Integer programming is much harder than linear programming from a computational viewpoint, but it covers many more optimization problems. It is one of the main modeling languages used in transportation optimization [25, 67].

A basic example is the assignment problem. Given a set of tasks and a set of resources, the aim is to assign resources to tasks while minimizing cost. Many transportation planning problems can be interpreted as assignment problems: vehicles are assigned to trips, tasks to duties, passengers to paths, etc. Simple assignment problems have a mathematical structure which makes their resolution efficient, making them natural examples to highlight the strength of combinatorial optimization. Yet, real transportation problems are rarely simple assignments. Assignments might have to follow geographical constraints, compatibility constraints, capacity constraints; a driver cannot be assigned to two trains that overlap in time; a passenger path must be compatible with the selected lines and their frequencies; in a balanced assignment, a worker should not only follow a schedule that is feasible each week, but also balanced in the long run.

This is why many problems in this thesis can be viewed as enriched assignment problems. They keep the intuitive structure of assigning objects to one another, but they add operational, stochastic, and fairness constraints. This enrichment is what makes them mathematically interesting and computationally difficult.

1.2 Graphs and shortest paths

Graphs provide another fundamental language for the problems studied in this thesis. A graph is made of vertices and arcs, and it can represent a network of possible movements (e.g., public transportation network), connections (e.g., change-and-go network), or incompatibilities (e.g., interval graphs). In transportation, graphs appear naturally: vertices could represent stations, or tasks; arcs could represent tracks or task compatibility. In crew management, a graph can represent possible transitions between tasks. In line planning, a graph can represent the actions available to passengers: boarding, traveling, transferring, and alighting. In balanced assignments, graphs can represent incompatible work intervals and schedules.

One of the most classical graph problems is the shortest path problem: given a network with costs on the arcs, find a path of minimum total cost between two vertices. This problem is central in transportation and has been studied since the early days of algorithmic graph theory, with landmark algorithms such as Dijkstra's algorithm in 1959 and the $A^\star$ algorithm in 1968 [46]. Shortest path problems are computationally efficient in their basic form, but they become significantly harder when additional constraints must be tracked along the path. In the context of this thesis, for the line planning problem, a path represents a passenger's itinerary.

This leads to the resource-constrained shortest path problem. In this problem, a path must minimize cost while satisfying constraints on resources such as time, duration, number of tasks, breaks, or capacity. Algorithms solving such problems often use the key ideas of the $A^\star$ algorithm. This resource-constrained shortest path problem is a key ingredient in many industrial applications, as most operational constraints can be modeled within this framework. In the context of this thesis, for the crew management application, a path in a graph may represent consecutive duties for a driver. The resources ensure that operational and labor constraints are respected. Thus, graph algorithms become a way of generating feasible planning objects.

Graphs also play a central role in the theoretical part of the thesis. There, they are not used to represent physical railway networks, but to represent abstract assignment structures. Directed graphs and Eulerian properties make it possible to obtain periodicity and balancedness results for assignments. This illustrates a recurring feature of combinatorial optimization: the same mathematical object, here a graph, can model very different realities depending on the interpretation of its vertices and arcs.

1.3 Large-scale applications and column generation

A recurring obstacle in large-scale optimization is that a formulation as an (integer) linear program may contain too many variables. For example, in the crew management problem, one may define one variable for each feasible duty or block of consecutive duties. In line planning, one may define one variable for each feasible passenger path. While each variable has a direct operational meaning, the set of possible variables is usually far too large to be enumerated explicitly. Alternative formulations with a tractable number of variables could also be used to model the same problem, yet it is sometimes preferable to favor a natural formulation, where feasibility constraints are implicit, at the expense of a large number of variables. As illustrated below by the Dantzig–Wolfe reformulation for integer linear programs, such a formulation could also yield a tighter linear relaxation, which can translate into faster resolution times.

This is where column generation comes into play. This algorithm aims to solve linear programs to optimality and is typically the basis for exact resolution methods for integer linear programs, such as branch-and-price. Instead of enumerating all variables of the original linear program (called the master problem), one starts with a small subset of variables, iteratively solves a restricted linear program, and searches for new variables that could improve the solution.

More precisely, column generation separates the problem into two interacting components. The restricted master problem computes an optimal solution using only the currently available variables (also called columns). The pricing subproblem searches for new columns with negative "reduced cost", that is, columns that could improve the current solution. The reduced cost is computed from the dual information of the restricted master problem. Although this terminology may appear technical, the intuition is the following: the restricted master problem says what kind of object is currently valuable, and the pricing subproblem tries to build such an object.

In the crew management problem, the master problem selects feasible blocks of consecutive duties to cover trains (possibly with an additional stochastic constraint when we consider train delays). The pricing subproblem then generates new feasible blocks. This subproblem can be modeled as a resource-constrained shortest path problem. In line planning, the master problem selects passenger paths and line frequencies, while the pricing subproblem searches for attractive paths in a network, which can be modeled as a shortest path problem.

When faced with a (mixed) integer linear program, column generation can be combined with branching techniques in order to reach optimal integer solutions. This gives rise to branch-and-price algorithms [8]. A branch-and-price algorithm integrates column generation inside a branch-and-bound framework. At each node of a search tree, a linear relaxation (i.e., when $x \in \mathbb{Z}^n_{\geqslant 0}$ is relaxed to $x \in \mathbb{R}^n_{\geqslant 0}$) is solved by column generation, and branching decisions are used to ultimately enforce integrality. Such algorithms are powerful, but achieving good practical performance can be challenging. Branching decisions may destroy or weaken the structure that makes the pricing problem tractable, for instance by turning a simple shortest path problem into a resource-constrained one. In addition, the master problem is often highly degenerate, which may lead to slow convergence. Designing an effective branch-and-price algorithm therefore requires more than applying a generic framework: it calls for custom branching rules, cutting-plane strategies, and acceleration heuristics.

Dantzig–Wolfe reformulations of compact (integer) linear programs provide a principled way to build formulations with large sets of variables, typically solved via column generation based methods [27]. The idea behind such a reformulation is that a convex polyhedron can either be defined by its outer description (using linear constraints) or by its inner description. That being said, this reformulation is performed by identifying a subset of constraints in the original (integer) linear program, which is then, under mild assumptions, replaced by a combination of extreme points and extreme rays of a convex polyhedron (defined by the constraints in the continuous setting, or their convex hull in the integer setting). While such

extreme points and extreme rays are hard to enumerate explicitly, they are often described by a well-understood structure, which the pricing subproblem exploits to generate them via dedicated algorithms. Such reformulations for integer linear programs can yield a stronger linear relaxation than the original formulation, in particular when the reformulated constraints' convex hull is tighter than their linear relaxation.

1.4 Optimization under uncertainty

Classical optimization models often assume that all data are known with certainty. In transportation, this assumption is only an approximation. Trains may be delayed, travel times may vary, demand may fluctuate, and disruptions may occur. A solution that is optimal for nominal data may perform poorly when unexpected events occur.

Several mathematical frameworks have been developed to address uncertainty. Stochastic programming assumes that some parameters are random variables that follow known probability distributions and seeks decisions that perform well under probabilistic constraints or costs [10]. Stochastic models may incorporate risk measures as a way to assess how a solution copes with uncertainty: the expectation captures average behavior, while other risk measures focus instead on extreme outcomes (e.g., quantile-based risk measures). Robust optimization, on the other hand, seeks the best worst-case scenario solutions where unknown parameters belong to an uncertainty set, without requiring knowledge on the probability distributions [9]. Both frameworks are widely used when decisions must be made before uncertainty is fully revealed.

In railway planning, taking into account uncertainties when constructing cyclic rosters is particularly important. A small delay on one train can affect a connection, delaying forthcoming trains and violating the rules and regulations. A plan with many tight connections may be cost-efficient in nominal conditions but fragile in operations. In this thesis, we introduce a stochastic fragility measure for the crew management problem that evaluates how vulnerable cyclic rosters are to delays. This approach is consistent with the broader idea that planning should anticipate operational variability before disruptions occur.

A wide body of literature addresses different stochastic and robust optimization models such as multi-stage stochastic and robust optimization, distributionally robust optimization, decision-dependent robust optimization, etc.

1.5 Integrated optimization of sequential processes

In many applications, the planning process is decomposed into successive decision stages: the output of one optimization problem becomes a parameter or a constraint for the next one. Although this sequential structure is computationally convenient, and often reflects the way planning is performed in practice, it may lead to decisions that are locally optimal but globally suboptimal. In particular, the objective value of an intermediate stage is often not meaningful on its own; what matters is the quality of the final outcome after all stages have been solved. Integrated optimization therefore aims to consider the whole sequential process as a single optimization problem, in which early decisions are evaluated through their consequences on downstream problems. In the crew management problem, constructing duties independently of their subsequent organization into rosters may lead to globally suboptimal solutions: duties that appear efficient in isolation can induce costly or fragile rosters, especially in the presence of delays.

To illustrate the suboptimalities inherent to sequential approaches, consider a two-stage setting. A sequential approach first chooses

$$x^{\text{seq}} \in \arg\min_{x \in X} f(x)\,,$$

and then, given this first-stage decision,

$$y^{\text{seq}} \in \underset{y \in Y(x^{\text{seq}})}{\arg\min}\ g(x^{\text{seq}}, y)\,.$$

The integrated problem deals instead with finding

$$(x^{\text{int}}, y^{\text{int}}) \in \underset{\substack{x \in X \\ y \in Y(x)}}{\arg\min}\, g(x, y)\,.$$

In general, x^{seq} need not coincide with some x^{int}: a decision that minimizes the intermediate cost f may restrict the second-stage feasible set or lead to a poor value of the final objective g.

This effect is even more pronounced under uncertainty, when early decisions also determine the flexibility of the system with respect to future delays or disruptions.

1.6 Periodicity in transportation planning

Transportation systems are often planned over representative periods. Passenger demands are described over a repeating time horizon; timetables are built to be periodic; duties are repeated weekly; and crew rosters cycle periodically to provide yearly schedules. Periodicity is useful because it makes planning easier to understand, communicate, and implement. It also reflects the repeated nature of many transportation operations.

However, periodicity raises mathematical questions. If the input data are periodic, can the solution also be chosen periodic?

This question has been addressed in the literature for different problems: the periodic event scheduling problem [14], the aircraft routing problem [64], etc.

The theoretical part of this thesis answers this question for a new problem, which could be seen as a relaxation of the cyclic rostering requirement. It studies balanced assignments of periodic tasks and shows that, under suitable feasibility assumptions, balanced solutions can be chosen periodic.

2 Organization of the Thesis

This thesis is organized around three main topics. Chapters 3, 4, and 5 are devoted to rail freight crew scheduling and crew rostering and propose deterministic and stochastic integrated methods for a crew management problem. Chapter 6 focuses on a passenger-oriented line planning problem and develops a path-based reformulation solved by branch-and-price. Chapters 7, 8, and 9 examine balanced assignments of periodic tasks from a theoretical perspective, with existence, periodicity, and algorithmic results.

2.1 Rail Freight Crew Management

Context and Scientific Challenges

The first part of the thesis addresses the rail freight crew management problem, motivated by the industrial planning at SNCF. The problem consists in assigning train drivers to freight train driving tasks while satisfying operational constraints, labor regulations, geographical compatibility constraints, and cyclicity requirements. In crew management, a *duty* is a sequence of work activities performed by a driver during a working day. It may include one or several train driving tasks, waiting times, transfers, and breaks. A *roster* is a longer-term schedule assigned to a driver, or to a team of drivers in a cyclic setting. It specifies the succession of duties and rest days over a planning horizon, while satisfying labor regulations

and operational constraints. In practice, the planning process is often decomposed into two successive steps. First, crew scheduling builds feasible duties covering the required train services. Then, crew rostering assembles these duties into rosters.

This sequential decomposition is natural from an operational and computational viewpoint, but it can induce significant suboptimalities. A first challenge is to optimize jointly crew scheduling and crew rostering decisions. In the standard sequential approach, duties are built before their insertion into rosters is evaluated. Duties that are attractive when considered independently may be difficult to combine into cyclic rosters, or may create connections between trains that are fragile when delays happen. Conversely, a slightly more expensive duty may leave more flexibility for the rostering phase and lead to a better global solution. A second challenge comes from uncertainty. Freight trains are frequently subject to delays, and these delays may propagate through planned driver connections. A roster can therefore be feasible and cost-efficient in nominal conditions, but fragile in operation. Incorporating this aspect into the planning model requires a stochastic measure of fragility, while keeping the model computationally tractable on industrial instances.

Scope and Structure

This part contains three chapters devoted to freight railway crew management.

Chapter 3 introduces the industrial and methodological background of this part. It describes the rail freight crew management process, the operational constraints faced by SNCF, and the standard decomposition between crew scheduling and crew rostering. It also presents the main modeling ingredients used in the following chapters, including duties, rosters, cyclicity requirements, labor rules and regulatory constraints, and the role of column generation in large-scale crew management problems.

Chapter 4 describes a deterministic integrated resolution method for the rail freight crew management problem. It proposes an optimization model in which the construction of duties and their assembly into cyclic rosters are handled jointly. The model is based on *blocks*, which are intermediate planning objects grouping one or several duties and encoding part of the compatibility requirements needed to build cyclic rosters.

Chapter 5 extends this framework to a stochastic setting. It considers the impact of train delays on cyclic rosters and introduces a measure of roster fragility. The objective is not only to build feasible and cost-efficient rosters, but also to control their vulnerability to delay propagation.

The three chapters progressively move from the industrial description of the problem to deterministic integrated optimization, and then to stochastic optimization under uncertainty. Chapters 4 and 5 both rely on column generation to solve the proposed models. As seen in Section 1.3, this structure is well suited to large-scale crew management problems, where the number of feasible roster-compatible blocks is too large to be enumerated explicitly.

Contributions

The contributions of this part are both methodological and computational.

Chapter 3 first formalizes the industrial rail freight crew management problem at SNCF. It also introduces an alternative integer linear programming formulation for the crew rostering problem at SNCF and proves a preliminary lemma that clarifies the structure of feasible cyclic rosters. This provides a rigorous modeling basis for the integrated approaches developed in the following chapters.

The first main contribution is the design of a deterministic integrated formulation and a resolution method for the crew management problem at SNCF. This contribution is challenging because it combines two decisions that are usually treated separately: the construction of duties and their assembly into cyclic rosters. The joint optimization of these crew scheduling

and crew rostering problems leads to formulations that are not tractable as is, even on toy instances. Chapter 4 introduces a set-covering block-based formulation, in which blocks are intermediate planning objects that group duties while encoding compatibility information needed to build optimal cyclic rosters. A key point is that this block-based model identifies optimal blocks, whose insertion into cyclic rosters preserves optimality. Thus, blocks are not introduced as a heuristic simplification, but as an exact reformulation tool. The difficulty is then transferred to the generation of blocks. Since the number of feasible blocks is extremely large, they cannot be enumerated explicitly. Chapter 4 therefore develops a column-generation approach in which blocks are generated iteratively. In this framework, the master problem selects optimal blocks covering all trains. The associated pricing problem is a high-dimensional resource-constrained shortest path (RCSP) problem. It is particularly challenging to solve efficiently on large-scale instances because of the high dimensionality of resources and nonlinearities in the feasibility and cost computations. Dedicated algorithmic acceleration techniques are therefore developed to solve this pricing problem efficiently within the column-generation framework.

The second main contribution, developed in Chapter 5, is the extension of this integrated framework to a stochastic setting. A modeling contribution is the introduction of a fragility measure for cyclic rosters. For a block, the fragility is defined as the expected value of a function measuring the impact of random arrival times and delay propagation on this block. The fragility of a roster is then obtained by summing the fragilities of its blocks. This measure is incorporated into the integrated crew management formulation through a stochastic constraint bounding the total fragility. Incorporating this measure into the optimization model leads to a significantly harder pricing problem, since random delays are tracked as distributions in the RCSP problem, with their propagation being performed while generating feasible blocks. This assumes that exogenous delays are independent random variables with finite-support distributions. Chapter 5 then questions this finite-support assumption and shows that the proposed framework can still be used to derive bounds for general delay distributions, using approximating finite-support distributions. Under additional assumptions, it further proves an optimality result that can be exploited within the current algorithmic scheme.

The computational experiments compare the proposed integrated approaches with the sequential method used as a reference, on realistic freight instances. They show that integration can significantly improve nominal costs, and that the stochastic formulation can substantially reduce fragility to delays, while also improving the nominal cost. This part therefore highlights both sources of improvement studied in the thesis: reducing the suboptimality of sequential planning and anticipating operational uncertainty during the construction of rosters.

2.2 Passenger-oriented Line Planning

Context and Scientific Challenges

The second part of the thesis addresses a passenger-oriented line planning problem. Given a transportation network, a set of candidate lines, and passenger demand between origin–destination pairs, it consists in selecting the lines to operate and determining their frequencies. The objective is to minimize the total travel time of passengers, including in-vehicle travel, boarding, alighting, and transfer times. The selected lines and their frequencies must provide sufficient capacity to transport all passengers while satisfying a global operating budget and upper bounds on the total frequency operated on each infrastructure arc. Line planning thus defines the transport offer from which downstream planning steps, such as timetabling, vehicle scheduling, and crew management, are subsequently built.

Passenger itineraries are represented using a *change-and-go network*. This auxiliary network explicitly describes the different stages of a trip: entering a line, traveling along

it, leaving it, and transferring between lines. It therefore makes it possible to account for the lines used by passengers and to link passenger flows with the capacities provided by the corresponding line frequencies.

The problem is difficult from a computational point of view: frequencies are discrete decisions, exact travel time computation requires the introduction of the much larger change-and-go network, and passenger routing decisions have to be explicit in the formulation.

A central challenge is the size of the formulation. In the classical arc-based model, passenger demand is represented as a multicommodity flow in the change-and-go network. Flow variables are introduced for every origin–destination pair and for a large number of network arcs. The size of this formulation therefore grows rapidly with the number of candidate lines and passenger commodities. On large-scale instances, explicitly constructing and storing the complete model may lead to memory overflows before the optimization process can even begin.

The main computational challenge is thus to devise a formulation that avoids the memory bottleneck of the arc-based representation. At the same time, the alternative formulation must preserve the passenger-oriented nature of the model and provide valid lower bounds and integer solutions.

Scope and Structure of the Part

This part consists of a single chapter, Chapter 6, devoted to passenger-oriented line planning. The chapter first defines the problem and presents an arc-based mixed-integer linear programming formulation on a change-and-go network, in which line frequencies and passenger flows are jointly determined.

To overcome the memory limitations of this formulation, the chapter then derives a path-based formulation using Dantzig–Wolfe reformulation. Passenger paths are generated dynamically by solving shortest-path problems. This column-generation procedure is embedded into a branch-price-and-cut algorithm, complemented by branching rules, valid inequalities, and acceleration strategies.

Finally, computational experiments compare the arc-based and path-based approaches on instances from the LinTim library, with particular attention to solution quality, computational time, and memory consumption.

Contributions

The main contribution of this part is the development of an exact path-based optimization framework for passenger-oriented line planning. Its purpose is to overcome the memory limitations of the classical arc-based formulation without modifying the underlying line planning problem.

The first contribution is a Dantzig–Wolfe reformulation of the passenger-flow constraints. This reformulation relies on the structure of the flow polyhedron: feasible passenger flows can be decomposed into elementary paths and simple cycles. Since cycle costs are nonnegative in the problem considered, there always exists an optimal solution containing no such cycles. Passenger routing can therefore be represented using elementary paths only.

This reformulation is not intended to strengthen the linear relaxation of the original formulation. Its main advantage is computational: it replaces a large compact formulation with a path-based formulation suitable for column generation. Passenger paths are generated dynamically rather than enumerated in advance. The corresponding pricing subproblems have a shortest-path structure and can be solved efficiently using Dijkstra's algorithm.

The second contribution is the integration of this column-generation framework into a branch-and-price algorithm, together with branching strategies on line-frequency variables. Although branching on the original passenger-flow variables may theoretically be required

to obtain fully integer solutions, it was not needed in the computational experiments. Such branching decisions are nevertheless discussed, as their implementation may affect the structure of the pricing subproblems.

The third contribution is the derivation and implementation of valid inequalities tailored to the formulation. These include budget-based inequalities for subsets of lines, strengthened cover inequalities accounting for the frequency bounds at the current node of the search tree, and, in the equal-capacity case, a lower bound on the total frequency required to transport passenger demand. These cuts are complemented by acceleration strategies designed to obtain a first integer solution earlier in the search.

The computational experiments show that the path-based approach avoids the memory bottleneck encountered with the arc-based formulation. On small and relatively easy instances, both approaches rapidly prove optimality. On larger instances, or when the integer-programming heuristic applied at the root node fails to find a feasible solution, the path-based method struggles to obtain optimal integer solutions. Nevertheless, it remains able to compute lower bounds and, for some instances, feasible solutions when the arc-based formulation reaches its memory limits.

The experiments also highlight the remaining limitations of the approach. In particular, degeneracy and highly fractional solutions may lead to large branch-and-bound trees containing nodes with similar lower bounds. Consequently, finding a first feasible integer solution and closing the optimality gap remain difficult on the largest instances.

2.3 Balanced Assignments

Context and Scientific Challenges

The last part of the thesis addresses balanced assignments of periodic tasks. This topic is motivated by cyclic crew rostering, as it seeks assignments relaxing the strong cyclicity requirement. The questions of fairness and periodicity have been studied separately in the literature. This part introduces the notion of balanced assignments, as feasible assignments of periodic tasks achieving long-term exact balancedness among a fixed number of workers, meaning that every worker performs every task with the same asymptotic frequency.

This setting raises two main questions. First, under which conditions does a balanced feasible assignment exist? Second, when such an assignment exists, can it be chosen to be periodic, and what can be said about the length of its period? The latter question is particularly relevant because periodic assignments are desirable in practice, as they admit a finite representation and provide predictable work patterns. The challenge is thus to establish whether balancedness over an infinite horizon can be achieved through a finite repeating pattern.

Scope and Structure

This part consists of three chapters devoted to balanced assignments of periodic tasks.

Chapter 7 introduces the basic version of the problem, in which weekly periodic tasks must be assigned to workers in a non-overlapping manner. It studies the existence of balanced feasible assignments, their periodicity, and some algorithmic counterparts, and links the results to the existence of cyclic rosters.

Chapter 8 extends this setting by requiring the weekly assignments of the tasks to each worker to belong to a given set of admissible schedules. It investigates whether the existence and periodicity results established in Chapter 7 could be extended to this more general framework. The question of periodicity is related to an auxiliary problem on an arc-colored graph, with a pebble placed on each vertex moving according to a common sequence of colors.

Chapter 9 explores further the graph-theoretic framework underlying these results. It connects balanced assignments with balanced color sequences in arc-colored Eulerian digraphs and studies period bounds, particular graph structures, and extensions of the problem.

Contributions

The first contribution of Chapter 7 is a complete characterization of the basic balanced assignment problem. The chapter proves that the following three conditions are equivalent: (i) there exists a feasible assignment in which one worker performs every task at least once; (ii) there exists a balanced feasible assignment; and (iii) there exists a periodic balanced feasible assignment with period q (where q is the number of workers). The proof is constructive and yields a polynomial-time algorithm for deciding existence and computing a periodic balanced assignment.

The second contribution, in Chapter 8, extends the framework to a setting requiring the weekly assignments of the tasks to each worker to belong to a given set of admissible schedules. The problem is related to an auxiliary problem on an arc-colored Eulerian digraph, in which pebbles placed on the vertices move according to a common periodic sequence of colors, visiting each arc in a balanced way. Through this connection, the chapter proves that the existence of a balanced feasible assignment implies the existence of a periodic one whose period is at most $2(q-1)q!$. The chapter further examines whether the resulting periodic assignments can be interpreted as cyclic rosters. Unlike in the basic setting, there is not always a periodic balanced feasible assignment inducing a cyclic roster of any period. Using group-theoretic arguments, the chapter shows that, when q is prime, the existence of a balanced feasible assignment implies the existence of one that also defines a cyclic roster, with period at most $q(q!-1)$.

The third contribution, in Chapter 9, provides complementary perspectives on the framework introduced in Chapter 8.

First, it investigates the particular case where the arc-colored digraph is a Cayley graph, which has additional symmetry properties. In this case, the chapter proves that a periodic balanced sequence of colors exists with period bounded by the number of arcs, leading to improved period bounds for periodic balanced feasible assignments, though ensuring the required Cayley graph structure from a given instance seems difficult in practice.

Second, Chapter 9 establishes a constructive equivalence between the periodic balanced assignment problem and a relaxed problem of finding a periodic balanced sequence of colors. Building on this equivalence, this chapter proposes an integer linear programming formulation to solve the (relaxed) periodic balanced sequence of colors problem with minimum period. This formulation is used to investigate the tightness of the period bounds established in earlier chapters.

Finally, Chapter 9 studies an extension to skill-balanced assignments, where balancedness is defined by a fair distribution of skills, acquired when performing tasks. It establishes connections between this skill-balancedness requirement and the graph framework, and identifies open questions related to the periodicity of such assignments.

Chapter 2

Introduction (français)

***

Cette thèse CIFRE a été réalisée dans le cadre d'un partenariat industriel avec la SNCF (Société nationale des chemins de fer français), et Hexafret, filiale de la SNCF spécialisée dans le transport et la logistique du fret ferroviaire.

Les systèmes de transport reposent sur des infrastructures complexes dans lesquelles des ressources limitées, des contraintes physiques de capacité, une incertitude opérationnelle et de fortes contraintes réglementaires rendent difficile la fiabilité et la compétitivité économique du service. Les systèmes ferroviaires constituent un exemple particulièrement riche de cette complexité. De la conception stratégique à long terme jusqu'à la gestion des opérations quotidiennes soumises à des perturbations en temps réel, les opérateurs de planification ferroviaire doivent prendre des décisions à différentes échelles de temps : quels trains faire circuler, à quelle fréquence et à quels horaires, sur quels itinéraires, avec quel matériel roulant, et avec quel roulement d'agents (c'est-à-dire, quelle équipe d'agents). Ces décisions reposent habituellement sur l'expertise individuelle des planificateurs, mais des outils d'aide à la décision reposant sur des méthodes d'optimisation peuvent rendre la planification des transports plus rapide, plus systématique et plus efficace. La planification ferroviaire et l'optimisation pour les transports publics ont ainsi été largement étudiées en recherche opérationnelle et en science des transports [26, 45, 75].

En pratique, de nombreux processus de planification sont organisés de manière séquentielle. Les lignes, fréquences et horaires sont d'abord conçus ; puis les matériels roulants leur sont affectés ; ensuite les journées de service et les grilles sont construites à partir des décisions précédentes ; et enfin la gestion des perturbations est faite au fur et à mesure de leur apparition. On appelle ici *journée de service* une suite d'activités effectuées par un agent au cours d'une journée de travail, comprenant typiquement une ou plusieurs tâches, des temps d'attente, des pauses, etc. Une grille désigne un planning à plus long terme attribué à un agent, ou à un roulement d'agents. Elle spécifie la succession de journées de service, de remplacements et de repos suivie par les agents tout au long de l'année et doit satisfaire la réglementation du travail et les contraintes opérationnelles. Cette décomposition en étapes de planification séquentielles est naturelle d'un point de vue opérationnel et calculatoire. Cependant, ces décisions sont profondément interdépendantes. Un horaire de train qui paraît attractif peut nécessiter des ressources matérielles ou humaines difficiles à planifier. Une journée de service localement intéressante peut être difficile à insérer dans une grille cyclique. Une grille peu coûteuse en conditions normales de circulation peut devenir fragile lorsque des retards surviennent et se propagent. Ces interactions expliquent pourquoi la planification des transports constitue un

domaine particulièrement exigeant pour l'optimisation mathématique. Un problème récurrent est que l'optimisation séquentielle peut conduire à des décisions globalement sous-optimales lorsque chaque étape de la planification ne prend pas en considération les contraintes et coûts des étapes ultérieures. Cela a été observé pour plusieurs applications à des problèmes de planification, où l'intégration des décisions de planification et de construction des grilles peut significativement améliorer la qualité de la solution finale [13, 36, 37, 59, 63, 86, 98]. Cette thèse s'inscrit dans cet axe de recherche en étudiant des problèmes de planification et d'affectation dans lesquels la faisabilité locale, la structure globale, la robustesse et la tractabilité des expériences numériques doivent être traitées simultanément.

Cette thèse est organisée autour de trois grands thèmes. Le premier thème traite de la planification du personnel pour le fret ferroviaire. Ce problème de planification est généralement décomposé en une étape de construction des journées de service et une étape de construction des grilles cycliques. La construction des journées de service consiste à sélectionner un ensemble de journées de service faisables couvrant tous les trains à conduire. La construction des grilles consiste à assembler ces journées de service en grilles cycliques, chacune associée à un roulement d'agents. De tels problèmes relèvent du champ plus large de la planification du personnel, qui a été largement étudié en recherche opérationnelle [36, 86]. Dans les applications ferroviaires, la présence supplémentaire de contraintes opérationnelles, de réglementations du travail, de contraintes de compatibilité géographique et d'exigences de cyclicité rend le problème particulièrement difficile [48]. Plutôt que de résoudre indépendamment les problèmes de construction des journées de service et de construction des grilles, nous proposons des modèles intégrés qui capturent leurs interactions, dans le but de réduire les sous-optimalités induites par le processus séquentiel. Nous modélisons également la robustesse des grilles aux retards des trains à travers une contrainte stochastique. Les modèles d'optimisation résultants étant trop volumineux pour être résolus par énumération directe, nous nous appuyons sur la génération de colonnes, une technique de décomposition standard pour la programmation en nombres entiers à grande échelle et pour les applications à la planification du personnel [30, 67]. Le deuxième thème de cette thèse concerne la planification des lignes, une étape qui intervient au début dans l'organisation de la planification pour les transports. Le problème de planification des lignes consiste à sélectionner des lignes et leurs fréquences dans un réseau de transport public tout en satisfaisant des contraintes de demande, de capacité et de budget. Dans une planification des lignes orientée voyageurs, l'objectif n'est pas seulement de maîtriser le coût opérationnel, mais aussi de tenir compte du temps de trajet, du temps d'attente et des correspondances des voyageurs. Une manière classique de modéliser les choix des voyageurs repose sur le réseau *change-and-go*, introduit par Anita Schöbel et Susanne Scholl en 2004 [81], qui permet de représenter explicitement l'embarquement, les trajets, les correspondances et le débarquement des passagers [12, 60, 80]. Cependant, les formulations mathématiques existantes de type "arc-based" pour ce problème peuvent devenir trop intensives en mémoire sur des instances de grande taille. Nous explorons donc une formulation de type "path-based", obtenue par une reformulation de Dantzig–Wolfe du modèle "arc-based". Cette approche s'inscrit dans le prolongement de travaux existants sur le problème de planification des lignes. Nous résolvons la formulation obtenue au moyen d'un algorithme de branch-and-price, amélioré par des inégalités valides. Le troisième thème de cette thèse est de nature plus théorique. Il vise à étudier les propriétés combinatoires garantissant l'existence d'affectations équilibrées de tâches périodiques à des agents. Dans ce cadre, des tâches surviennent périodiquement et doivent être affectées à un nombre fixé d'agents. L'objectif est de garantir que chaque agent effectue chaque tâche avec la même fréquence asymptotique. De telles questions sont liées à l'ordonnancement périodique, à la coloration de graphes et à l'équité dans les affectations [14, 57]. De nombreux problèmes de planification des transports s'appuient sur des données périodiques : les demandes voyageurs en planification des lignes est souvent décrite sur une période représentative, les trains en planification des agents sont donnés sur une semaine type et se répètent à l'identique chaque

semaine. Il arrive que la sortie doive elle aussi être périodique, voire cyclique. C'est le cas des grilles cycliques, où la cyclicité est exigée afin de garantir des emplois du temps réguliers et équitables entre les agents. La motivation pour l'introduction des affectations équilibrées part de cette idée : une affectation équilibrée préserve l'équité entre les travailleurs tout en relâchant l'exigence de cyclicité (voire de périodicité).

L'objet de cette introduction est de présenter le contexte opérationnel et mathématique de la thèse, d'expliquer les principaux défis scientifiques, de mettre en avant les contributions, et de décrire l'organisation du manuscrit. Avant de présenter ces trois thèmes en détail, nous introduisons les principaux concepts mathématiques utilisés tout au long de la thèse. L'objectif n'est pas de fournir un état de l'art technique, mais de donner au lecteur le bagage nécessaire pour comprendre pourquoi les problèmes d'affectation, les modèles de graphes et les méthodes de décomposition apparaissent naturellement dans la planification des transports.

1 Quelques concepts mathématiques

1.1 Programmation mathématique

De nombreux problèmes de planification des transports relèvent du vaste domaine de la programmation mathématique et de la recherche opérationnelle. L'idée générale est la suivante : parmi un ensemble de décisions faisables, on cherche une décision qui soit la meilleure au regard d'un critère donné. Dans cette thèse, les décisions peuvent être des journées de service d'agents de conduite, des trajets de voyageurs, des fréquences de lignes, tandis que la faisabilité peut encoder des contraintes variées comme des règles opérationnelles ou de réglementation du travail, la capacité des véhicules, ou des exigences d'équité. Le critère, que l'on cherche à minimiser ou à maximiser, également appelé coût ou objectif, peut représenter des coûts économiques, un temps de trajet, une mesure de robustesse, ou tout autre indicateur de performance.

Nombre des développements modernes de la recherche opérationnelle ont été motivés par des problèmes de transport, de logistique et de planification industrielle. La programmation linéaire s'est imposée comme un outil central au milieu du vingtième siècle, notamment avec la méthode du simplexe introduite par Dantzig en 1947. Dans un programme linéaire, on optimise une fonction objectif linéaire sous des contraintes linéaires d'égalité et d'inégalité. Un programme linéaire s'écrit sous forme générale de la façon suivante :

$$\begin{aligned} \text{trouver un vecteur} \quad & x \in \mathbb{R}^n_{\geqslant 0} \\ \text{minimisant} \quad & \sum_{i=1}^{n} c_i x_i \\ \text{tel que} \quad & \sum_{i=1}^{n} A_{i,j} x_i = b_j \quad \forall j \in [m]\,. \end{aligned}$$

Ce paradigme de modélisation est très utilisé car de nombreux problèmes de transport, de logistique et d'allocation de ressources peuvent s'exprimer sous cette forme, et parce qu'il existe des algorithmes efficaces pour résoudre des instances de grande taille. On sait que la programmation linéaire est résoluble en temps polynomial (bien que ce résultat n'ait été démontré que bien plus tard, en 1979, par Khachiyan [53], au moyen de l'algorithme de l'ellipsoïde, puis par Karmarkar en 1984 [51] avec la méthode des points intérieurs, encore utilisée aujourd'hui en pratique). La dualité est un autre concept fondamental de la programmation linéaire : tout programme linéaire admet un problème lié, appelé le dual, qui fournit des bornes et des certificats d'optimalité, lesquels servent de base à des algorithmes

plus avancés. La programmation linéaire est également utilisée en économie, les variables duales pouvant s'interpréter comme les valeurs marginales des ressources.

Cependant, de nombreuses décisions de planification sont discrètes. Une journée de service est soit sélectionnée, soit non ; une ligne est exploitée à une fréquence entière ou n'est pas exploitée du tout ; etc. De telles décisions discrètes sont naturellement modélisées à l'aide de variables entières. Cela conduit à la programmation linéaire en nombres entiers, où certaines variables doivent prendre des valeurs entières. Un programme linéaire mixte en nombres entiers s'écrit sous forme générale :

$$\begin{aligned} \text{trouver un vecteur} \quad & x \in \mathbb{R}^{n}_{\geqslant 0} \times \mathbb{Z}^{n'}_{\geqslant 0} \\ \text{minimisant} \quad & \sum_{i=1}^{n+n'} c_i x_i \\ \text{tel que} \quad & \sum_{i=1}^{n+n'} A_{i,j} x_i = b_j \quad \forall j \in [m]\,. \end{aligned}$$

La programmation en nombres entiers est bien plus difficile que la programmation linéaire d'un point de vue calculatoire, mais elle couvre un bien plus grand nombre de problèmes d'optimisation. C'est l'un des principaux paradigmes de modélisation utilisés en optimisation des transports [25, 67].

Le problème d'affectation en constitue un exemple élémentaire. Étant donnés un ensemble de tâches et un ensemble de ressources, l'objectif est d'affecter les ressources aux tâches en minimisant le coût. De nombreux problèmes de planification des transports peuvent s'interpréter comme des problèmes d'affectation : des véhicules sont affectés à des trajets, des tâches à des journées de service, des voyageurs à des chemins, etc. Le problème d'affectation basique possède une structure mathématique qui rend sa résolution efficace, ce qui en fait un exemple naturel pour illustrer la puissance de l'optimisation combinatoire. Pourtant, les problèmes de transport inspirés d'applications industrielles sont rarement de simples affectations : on peuvent devoir respecter des contraintes géographiques, des contraintes de compatibilité, des contraintes de capacité ; un agent de conduite (c'est-à-dire un conducteur de train) ne peut être affecté à deux trains circulant en même temps ; le trajet d'un voyageur doit être compatible avec les lignes sélectionnées et leurs fréquences ; dans une affectation équilibrée, un agent doit non seulement suivre un planning faisable chaque semaine, mais surtout un planning qui soit équilibré sur le long terme.

C'est pourquoi les problèmes traités dans cette thèse peuvent être vus comme des problèmes d'affectation enrichis. Ils conservent la structure intuitive consistant à affecter des objets les uns aux autres, mais y ajoutent des contraintes opérationnelles, stochastiques et d'équité. Cet enrichissement est précisément ce qui les rend mathématiquement intéressants et difficiles à résoudre numériquement.

1.2 Graphes et plus courts chemins

Les graphes font aussi partie du vocabulaire commun des problèmes étudiés dans cette thèse. Un graphe est constitué de sommets et d'arcs, et peut représenter un réseau de transport, des déplacements (par exemple un réseau change-and-go), ou des incompatibilités de tâches (par exemple des graphes d'intervalles). Dans le domaine des transports, les graphes apparaissent comme un objet naturel : les sommets peuvent représenter des gares ou des tâches, les arcs des voies ou des compatibilités de tâches. En planification du personnel, un graphe peut représenter les transitions possibles entre tâches. En planification des lignes, un graphe peut représenter les actions disponibles pour les voyageurs : embarquement, trajet, correspondance

et débarquement. Pour les affectations équilibrées, un graphe peut représenter des intervalles de travail et des plannings incompatibles.

L'un des problèmes de graphes les plus connus est le problème du plus court chemin : étant donné un réseau muni de coûts sur les arcs, trouver un chemin de coût total minimal entre deux sommets. Ce problème est central en transport et a été étudié dès les débuts de la théorie algorithmique pour les graphes, avec des algorithmes fondateurs tels que l'algorithme de Dijkstra en 1959 et l'algorithme $A^\star$ en 1968 [46]. Les problèmes de plus court chemin sont numériquement efficaces sous leur forme de base, mais deviennent plus difficiles lorsque des contraintes supplémentaires régissent la faisabilité des chemins. Dans le cadre de cette thèse, pour le problème de planification des lignes, un chemin représente l'itinéraire d'un voyageur.

Cela conduit au problème de plus court chemin sous contraintes de ressources. Dans ce problème, un chemin doit minimiser un coût tout en satisfaisant des contraintes portant sur des ressources telles que le temps, la durée, le nombre de tâches, les pauses, ou la capacité. Les algorithmes résolvant de tels problèmes s'appuient souvent sur les idées clés de l'algorithme $A^\star$. Ce problème de plus court chemin sous contraintes de ressources constitue un ingrédient essentiel de nombreuses applications industrielles, la plupart des contraintes opérationnelles pouvant être modélisées dans ce cadre. Dans le contexte de cette thèse, pour l'application à la planification du personnel, un chemin dans un graphe peut représenter des journées de service consécutives pour un agent de conduite. Les ressources garantissent le respect des contraintes opérationnelles et réglementaires. Les algorithmes de graphes deviennent ainsi un moyen de générer des objets de planification faisables.

Les graphes jouent également un rôle central dans la partie théorique de la thèse. Ils n'y sont alors pas utilisés pour représenter des réseaux ferroviaires physiques, mais des structures d'affectation abstraites. Les graphes orientés et les propriétés de connexité permettent d'obtenir des résultats de périodicité et d'équilibrage des affectations. Cela illustre une caractéristique récurrente de l'optimisation combinatoire : un même objet mathématique, ici un graphe, peut modéliser des réalités très différentes selon l'interprétation donnée à ses sommets et à ses arcs.

1.3 Applications à grande échelle et génération de colonnes

Un obstacle récurrent en optimisation à grande échelle est qu'une formulation en programme linéaire (en nombres entiers) peut comporter un nombre de variables trop important. Par exemple, dans le problème de planification du personnel, on peut définir une variable pour chaque journée de service faisable ou chaque bloc de journées de service consécutives. En planification des lignes, on peut définir une variable pour chaque chemin de voyageur faisable. Bien que chaque variable ait une signification opérationnelle directe, l'ensemble des variables possibles est généralement bien trop grand pour être énuméré explicitement. Des formulations alternatives, comportant un nombre raisonnable de variables, peuvent également être utilisées pour modéliser le même problème ; il est cependant parfois préférable de privilégier une formulation naturelle, où les contraintes de faisabilité sont implicites, au dépens d'un grand nombre de variables. Comme mentionné plus bas pour les reformulations de Dantzig–Wolfe de programmes linéaires en nombres entiers, une telle formulation peut également fournir une relaxation linéaire plus forte, ce qui peut se traduire par des temps de résolution plus rapides.

C'est là qu'intervient la génération de colonnes. Cet algorithme vise à résoudre des programmes linéaires à l'optimalité et sert de brique de base à des méthodes de résolution exacte de programmes linéaires en nombres entiers, telles que le branch-and-price. Plutôt que d'énumérer explicitement toutes les variables du programme linéaire de départ (appelé *problème maître*), on part d'un petit sous-ensemble de variables, puis on résout itérativement un programme linéaire restreint, en recherchant de nouvelles variables susceptibles d'améliorer la solution.

Plus précisément, la génération de colonnes sépare le problème en deux composantes en interaction. Le problème maître restreint détermine une solution optimale en n'utilisant que

les variables (aussi appelées colonnes) générées jusqu'à présent. Le sous-problème de *pricing* recherche de nouvelles colonnes à "coût réduit" négatif, c'est-à-dire des colonnes susceptibles d'améliorer la solution courante. Le coût réduit est calculé à partir des informations duales du problème maître restreint. Bien que cette terminologie puisse sembler technique, l'intuition est la suivante : le problème maître restreint indique quel type d'objet est actuellement intéressant, et le sous-problème de pricing tente de construire un tel objet.

Dans le problème de planification du personnel, le problème maître sélectionne des blocs faisables de journées de service consécutives couvrant tous les trains (éventuellement avec une contrainte stochastique supplémentaire lorsque l'on considère les retards des trains). Le sous-problème de pricing produit alors de nouveaux blocs faisables. Ce sous-problème peut être modélisé comme un problème de plus court chemin avec contraintes de ressources. En planification des lignes, le problème maître sélectionne des chemins de voyageurs et des fréquences de lignes, tandis que le sous-problème de pricing recherche des chemins attractifs dans un réseau, ce qui peut être modélisé comme un problème de plus court chemin.

Face à un programme linéaire (mixte) en nombres entiers, la génération de colonnes peut être combinée à des techniques de branchement afin d'atteindre des solutions entières optimales. Cela donne naissance aux algorithmes de branch-and-price [8]. Les algorithmes de *branch-and-price* s'appuient sur la génération de colonnes et fonctionnent à la manière d'un algorithme de "branch-and-bound". À chaque nœud d'un arbre de recherche, une relaxation linéaire (c'est-à-dire lorsque $x \in \mathbb{Z}^n_{\geqslant 0}$ est relâché en $x \in \mathbb{R}^n_{\geqslant 0}$) est résolue par génération de colonnes, et des décisions de branchement sont utilisées pour finalement imposer l'intégrité des variables. De tels algorithmes sont puissants, mais obtenir de bonnes performances pratiques peut s'avérer difficile. Les décisions de branchement peuvent compromettre ou affaiblir la structure qui rend le sous-problème de pricing rapide à résoudre, par exemple en transformant un simple problème de plus court chemin en un problème sous contraintes de ressources. De plus, le problème maître est souvent fortement dégénéré, ce qui peut entraîner une convergence lente. Concevoir un algorithme de branch-and-price efficace requiert donc davantage que l'application d'un cadre générique : cela nécessite la mise en place des règles de branchement sur mesure, des coupes, et des heuristiques d'accélération.

Les reformulations de Dantzig–Wolfe de programmes linéaires (en nombres entiers) compacts offrent un moyen structuré de construire des formulations comportant un grand nombre de variables, typiquement résolues par des méthodes de génération de colonnes [27]. L'idée sous-jacente à une telle reformulation est qu'un polyèdre convexe peut être défini soit par sa description externe (au moyen de contraintes linéaires), soit par sa description interne (au moyen de points extrêmes et de rayons extrêmes). Cette reformulation s'effectue en identifiant un sous-ensemble de contraintes dans le programme linéaire (en nombres entiers) initial, qui est ensuite, sous des hypothèses raisonnables, remplacé par une combinaison de points extrêmes et de rayons extrêmes d'un polyèdre convexe (défini par les contraintes reformulées dans le cadre continu, ou par leur enveloppe convexe dans le cadre entier). Bien que ces points extrêmes et rayons extrêmes soient difficiles à énumérer explicitement, ils sont souvent décrits par une structure connue, que le sous-problème de pricing exploire pour les générer au moyen d'algorithmes dédiés. De telles reformulations, pour les programmes linéaires en nombres entiers, peuvent produire une relaxation linéaire plus forte que la formulation initiale, en particulier lorsque l'enveloppe convexe des contraintes reformulées est plus serrée que leur relaxation linéaire.

1.4 Optimisation sous incertitude

Les modèles d'optimisation classiques supposent souvent que toutes les données sont connues avec certitude. Dans le cadre de modèles appliqués à une réalité industrielle, cette hypothèse n'est qu'une approximation : les trains peuvent être retardés, les temps de trajet peuvent varier, la demande peut fluctuer. Une solution optimale en situation non perturbée peut se

révéler coûteuse ou infaisable lorsque des perturbations se produisent.

Plusieurs cadres et modèles mathématiques ont été développés pour traiter l'incertitude. La programmation stochastique suppose que certains paramètres sont des variables aléatoires, qui suivent des distributions de probabilité connues, et cherche à déterminer des décisions performantes lorsque les contraintes ou les coûts sont aléatoires [10]. Les modèles stochastiques peuvent incorporer des mesures de risque comme moyen d'évaluer la façon dont une solution fait face à l'incertitude : l'espérance capture le comportement moyen, tandis que d'autres mesures de risque évaluent plutôt les valeurs extrêmes (comme par exemple les mesures de risque utilisant les quantiles). L'optimisation robuste, quant à elle, recherche les meilleures solutions dans le pire des cas, où les paramètres inconnus appartiennent à un ensemble d'incertitude, sans nécessiter de connaissance sur des distributions de probabilité [9]. Ces deux cadres sont largement utilisés lorsque des décisions doivent être prises avant que l'incertitude ne soit pleinement révélée.

En planification ferroviaire, la prise en compte des retards des trains lors de la construction des grilles cycliques est particulièrement importante : un léger retard sur un train peut affecter une correspondance, retardant les trains suivants et rendant la grille infaisable vis-à-vis de la réglementation. Un planning comportant de nombreuses correspondances serrées peut être économiquement attractif en conditions nominales mais fragile en exploitation. Dans cette thèse, nous introduisons une mesure stochastique de fragilité pour le problème de planification du personnel, qui évalue la vulnérabilité des grilles cycliques aux retards. Cette approche s'inscrit dans l'idée plus large selon laquelle la planification devrait anticiper la variabilité opérationnelle avant que les perturbations ne surviennent.

Une abondante littérature aborde différents modèles d'optimisation stochastique et robuste, tels que l'optimisation stochastique et robuste multi-étapes, l'optimisation robuste distributionnelle, l'optimisation robuste dépendante des décisions, etc.

1.5 Optimisation intégrée de processus séquentiels

Dans de nombreuses applications, le processus de planification est décomposé en étapes de décision successives : le résultat d'un problème d'optimisation devient un paramètre ou une contrainte pour le problème suivant. Bien que cette structure séquentielle reflète souvent la manière dont la planification est menée en pratique et permette de réduire l'ensemble des décisions sinon très combinatoire, elle peut conduire à des décisions localement optimales mais globalement sous-optimales. En particulier, la valeur objectif d'une étape intermédiaire n'est souvent pas significative en elle-même ; ce qui compte est la qualité du résultat final une fois toutes les étapes résolues. L'optimisation intégrée vise donc à considérer l'ensemble du processus séquentiel comme un unique problème d'optimisation, dans lequel les décisions amont sont évaluées à travers leurs conséquences sur les problèmes en aval. Dans le problème de planification du personnel, construire des journées de service indépendamment de leur organisation ultérieure en grilles peut conduire à des solutions globalement sous-optimales : des journées de service qui paraissent intéressantes isolément peuvent induire des grilles coûteuses ou fragiles en présence de retards.

Pour illustrer les sous-optimalités inhérentes aux approches séquentielles, considérons un cadre à deux étapes. Une approche séquentielle choisit d'abord

$$x^{\text{seq}} \in \arg\min_{x \in X} f(x)\,,$$

puis, étant donné cette décision de première étape,

$$y^{\text{seq}} \in \arg\min_{y \in Y(x^{\text{seq}})} g(x^{\text{seq}}, y)\,.$$

Le problème intégré consiste au contraire à trouver

$$(x^{\text{int}}, y^{\text{int}}) \in \underset{\substack{x \in X \\ y \in Y(x)}}{\arg\min}\, g(x, y)\,.$$

En général, x^{seq} ne coïncide pas nécessairement avec un x^{int} : une décision qui minimise le coût intermédiaire f peut restreindre l'ensemble faisable de la seconde étape, ou conduire à une mauvaise valeur de l'objectif final g.

Cet effet est encore plus marqué sous incertitude, lorsque les décisions précoces déterminent également la flexibilité du système face aux futurs retards ou perturbations.

1.6 Périodicité dans la planification des transports

Les systèmes de transport sont souvent planifiés sur des périodes représentatives. La demande de voyageurs est décrite sur un horizon temporel répétitif, les horaires sont conçus pour être périodiques, les journées de service se répètent chaque semaine, les grilles sont établies sur une base hebdomadaire et déroulées de manière cyclique pour fournir des plannings annuels aux roulements d'agents. La périodicité est utile car elle rend la planification plus facile à comprendre, à communiquer et à mettre en œuvre. Elle reflète également la nature répétée de nombreuses opérations de transport.

La périodicité soulève cependant des questions mathématiques. Si les données d'entrée sont périodiques, la solution peut-elle elle aussi être choisie périodique ?

Cette question a été abordée dans la littérature pour différents problèmes : le “periodic event scheduling” [14], le problème de routage d'avions [64], etc.

La partie théorique de cette thèse répond à cette question pour un nouveau problème, qui peut être vu comme une relaxation de la cyclicité des grilles. Elle étudie les affectations équilibrées de tâches périodiques et montre que, sous des hypothèses de faisabilité, des solutions équilibrées peuvent être choisies périodiques.

2 Organisation de la thèse

Cette thèse est organisée autour de trois grands thèmes. Les chapitres 3, 4 et 5 sont consacrés à la construction des journées de service et des grilles pour le fret ferroviaire, et proposent des méthodes intégrées déterministes et stochastiques pour le problème de planification du personnel. Le chapitre 6 porte sur un problème de planification des lignes orienté voyageurs et développe une reformulation “path-based”, résolue par branch-and-price. Les chapitres 7, 8 et 9 examinent, d'un point de vue théorique, les affectations équilibrées de tâches périodiques, avec des résultats d'existence, de périodicité et des résultats algorithmiques.

2.1 Planification du personnel pour le fret ferroviaire

Contexte et défis scientifiques

La première partie de la thèse porte sur le problème de planification du personnel pour le fret ferroviaire, motivé par l'application à la construction des emplois du temps des agents de conduite des trains de fret à la SNCF. Le problème consiste à affecter des agents de conduite à des tâches de conduite de trains de fret, tout en satisfaisant des contraintes opérationnelles, des réglementations du travail, des contraintes de compatibilité géographique, et des exigences de cyclicité. En planification du personnel pour le fret ferroviaire, une *journée de service* est une suite d'activités effectuées par un agent de conduite au cours d'une journée de travail. Elle peut comprendre un ou plusieurs trains à conduire, des temps d'attente, des transferts, et des pauses. Une *grille cyclique* est un planning à plus long terme attribué à une

équipe d'agents. Elle spécifie la succession des journées de service et des jours de repos sur une base hebdomadaire, tout en satisfaisant la réglementation du travail et les contraintes opérationnelles. En pratique, le processus de planification est souvent décomposé en deux étapes successives. Premièrement, la construction des journées de service produit un ensemble de journées de service faisables couvrant tous les trains à conduire. Ensuite, la construction des grilles assemble ces journées de service en grilles cycliques, une pour chaque roulement d'agents.

Cette décomposition séquentielle est naturelle d'un point de vue opérationnel et calculatoire, mais elle peut induire des sous-optimalités importantes. Un premier défi consiste à optimiser conjointement les décisions de construction des journées de service et des grilles. Dans l'approche séquentielle standard, les journées de service sont construites avant que leur insertion dans les grilles ne soit évaluée. Des journées de service attractives lorsqu'elles sont considérées isolément peuvent être difficiles à combiner en grilles cycliques, ou peuvent créer des correspondances entre trains fragiles en cas de retard. Inversement, une journée de service légèrement plus coûteuse peut laisser davantage de flexibilité pour la phase de construction des grilles et conduire à une meilleure solution globale. Un second défi provient de l'incertitude. Les trains de fret sont fréquemment sujets à des retards, et ces retards peuvent se propager à travers les successions de trains. Une grille peut donc être faisable et économiquement efficace en conditions nominales, mais fragile en exploitation. Intégrer cet aspect dans le modèle de planification nécessite une mesure stochastique de la fragilité d'une grille, tout en gardant le modèle numériquement viable sur des instances industrielles.

Portée et organisation

Cette partie comprend trois chapitres consacrés à la planification du personnel pour le fret ferroviaire.

Le chapitre 3 introduit le contexte industriel et méthodologique de cette partie. Il décrit le processus de planification du personnel pour le fret ferroviaire, les contraintes opérationnelles rencontrées à la SNCF, ainsi que la décomposition standard entre construction des journées de service et construction des grilles. Il présente également le vocabulaire et des premiers éléments de modélisation, utilisés dans les chapitres suivants, notamment les journées de service, les grilles, les exigences de cyclicité, les contraintes de la réglementation, et le rôle de la génération de colonnes dans les problèmes de planification du personnel à grande échelle.

Le chapitre 4 décrit une méthode de résolution intégrée déterministe du problème de planification du personnel pour le fret ferroviaire. Il propose un modèle d'optimisation dans lequel la construction des journées de service et leur assemblage en grilles cycliques sont traités conjointement. Le modèle repose sur l'introduction des *blocs*, qui sont des objets de planification intermédiaires regroupant des journées de service consécutives et encodant une partie des exigences de compatibilité nécessaires à la construction de grilles cycliques.

Le chapitre 5 étend ce cadre à un contexte stochastique. Il considère l'impact des retards des trains sur les grilles cycliques et introduit une mesure de fragilité des grilles. L'objectif n'est pas seulement de construire des grilles faisables et peu coûteuses, mais aussi de maîtriser leur vulnérabilité à la propagation des retards.

Ces trois chapitres passent progressivement de la description industrielle du problème à l'optimisation intégrée déterministe, puis à l'optimisation intégrée stochastique. Les chapitres 4 et 5 reposent tous deux sur la génération de colonnes pour résoudre les modèles proposés. Comme expliqué dans la section 1.3, cette structure est adaptée aux problèmes de planification du personnel à grande échelle, où le nombre de blocs faisables compatibles avec les grilles est trop important pour être énuméré explicitement.

Contributions

Les contributions de cette partie sont à la fois méthodologiques et calculatoires.

Le chapitre 3 formalise d'abord le problème industriel de planification du personnel pour le fret ferroviaire à la SNCF. Il introduit également une formulation alternative en programme linéaire en nombres entiers pour le problème de construction des grilles à la SNCF, et démontre un lemme préliminaire qui clarifie la structure des grilles cycliques faisables. Cela fournit une base de modélisation rigoureuse pour les approches intégrées développées dans les chapitres suivants.

La première contribution majeure est la conception d'une formulation intégrée déterministe et d'une méthode de résolution pour le problème de planification du personnel à la SNCF. Une telle formulation intégrée constitue un défi car elle combine deux décisions habituellement traitées séparément : la construction des journées de service et leur assemblage en grilles cycliques. L'optimisation conjointe de ces problèmes de construction des journées de service et des grilles conduit à des formulations qui ne sont pas utilisables telles quelles, même sur des instances jouets. Le chapitre 4 introduit une formulation sur les blocs de type "set-covering", dans laquelle les blocs sont des objets de planification intermédiaires regroupant des journées de service tout en encodant les informations de compatibilité nécessaires à la construction de grilles cycliques optimales. Un point clé est que ce modèle de blocs identifie des blocs optimaux, dont l'insertion dans des grilles cycliques préserve l'optimalité. Ainsi, les blocs ne sont pas introduits comme une simplification heuristique, mais comme un outil de reformulation exact. La difficulté est alors reportée sur la génération des blocs. Le nombre de blocs faisables étant extrêmement élevé, ils ne peuvent être énumérés explicitement. Le chapitre 4 développe donc une approche par génération de colonnes dans laquelle les blocs sont générés de manière itérative. Dans ce cadre, le problème maître sélectionne des blocs optimaux couvrant tous les trains. Le sous-problème de pricing associé est un problème de plus court chemin sous contraintes de ressources de grande dimension. Il est particulièrement difficile à résoudre efficacement sur des instances de grande taille, en raison de la haute dimension des ressources et des non-linéarités dans le calcul de la faisabilité et du coût. Des techniques d'accélération algorithmique dédiées sont donc développées pour résoudre efficacement ce sous-problème de pricing dans le cadre de la génération de colonnes.

La seconde contribution majeure, développée au chapitre 5, est l'extension de ce cadre intégré à un contexte stochastique. Une contribution de modélisation est l'introduction d'une mesure de fragilité pour les grilles cycliques. Pour un bloc, la fragilité est définie comme l'espérance d'une fonction mesurant l'impact des heures d'arrivée aléatoires et de la propagation des retards sur ce bloc. La fragilité d'une grille est ensuite obtenue en sommant les fragilités de ses blocs. Cette mesure est intégrée à la formulation de planification du personnel à travers une contrainte stochastique bornant la fragilité totale. L'intégration de cette mesure dans le modèle d'optimisation conduit à un sous-problème de pricing significativement plus difficile, puisque les distributions des retards aléatoires sont stockées dans le problème de plus court chemin sous contraintes de ressources, leur propagation étant effectuée lors de la génération des blocs faisables. Cela suppose que les retards exogènes sont des variables aléatoires indépendantes et que leurs distributions ont un support fini. Le chapitre 5 interroge ensuite cette hypothèse de support fini et montre que le cadre proposé peut néanmoins être utilisé pour obtenir des bornes pour des distributions de retard générales, à l'aide de distributions à support fini approchantes. Sous des hypothèses supplémentaires, il est démontré en outre un résultat d'optimalité pouvant être exploité dans le schéma algorithmique actuel.

Les expériences numériques comparent les approches intégrées proposées à la méthode séquentielle prise comme référence, sur des instances réalistes de fret. Elles montrent que l'intégration peut significativement améliorer les coûts nominaux, et que la formulation stochastique peut substantiellement réduire la fragilité aux retards, tout en améliorant également le coût nominal. Cette partie met ainsi en évidence les deux sources d'amélioration

étudiées dans la thèse : la réduction de la sous-optimalité de la planification séquentielle et l'anticipation de l'incertitude opérationnelle lors de la construction des grilles.

2.2 Planification des lignes orientée voyageurs

Contexte et défis scientifiques

La deuxième partie de la thèse porte sur un problème de planification des lignes orienté voyageurs. Étant donnés un réseau de transport, un ensemble de lignes candidates, et des demandes de voyageurs entre paires origine–destination, ce problème consiste à sélectionner les lignes à exploiter et à déterminer leurs fréquences. La planification des lignes définit ainsi l'offre de transport à partir de laquelle sont ensuite construites les étapes de planification en aval, telles que l'établissement des horaires, la planification des véhicules et la planification du personnel.

Cette partie considère une version du problème orientée voyageurs. L'objectif est de minimiser le temps de trajet total des voyageurs, incluant les temps de déplacement, d'embarquement, de débarquement et de correspondance. Les lignes sélectionnées et leurs fréquences doivent offrir une capacité suffisante pour transporter tous les voyageurs, tout en respectant un budget d'exploitation global et des bornes supérieures sur la fréquence totale exploitée sur chaque arc d'infrastructure.

Les itinéraires des voyageurs sont représentés à l'aide d'un réseau appelé *change-and-go*. Ce réseau auxiliaire décrit explicitement les différentes étapes d'un trajet : entrer dans un véhicule, y voyager, le quitter, et effectuer une correspondance entre lignes. Il permet ainsi de tenir compte des lignes empruntées par les voyageurs et de relier les flux de voyageurs aux services offerts par les lignes et leur fréquence de passage.

Le problème est difficile d'un point de vue algorithmique : les fréquences sont des décisions discrètes, le calcul exact du temps de trajet nécessite l'introduction du réseau change-and-go, bien plus volumineux que le réseau de transport initial, et les décisions de routage des voyageurs doivent être explicites dans la formulation.

Un défi central est la taille de la formulation. Dans le modèle classique "arc-based", la demande de voyageurs est représentée comme un flot multi-commodités dans le réseau change-and-go. Des variables de flot sont introduites pour chaque paire origine–destination et pour un grand nombre d'arcs du réseau. La taille de cette formulation croît donc rapidement avec le nombre de lignes candidates et de commodités de voyageurs. Sur des instances de grande taille, la construction et le stockage explicites du modèle complet peuvent entraîner des dépassements de mémoire avant même que la résolution de l'optimisation ne puisse commencer.

Le principal défi computationnel est donc de concevoir une formulation évitant le goulot d'étranglement mémoire du modèle "arc-based". D'autre part, la formulation alternative doit préserver la nature orientée voyageurs du modèle et fournir des bornes inférieures valides ainsi que des solutions entières.

Portée et organisation

Cette partie ne comporte qu'un seul chapitre, le chapitre 6, consacré à la planification des lignes orientée voyageurs. Le chapitre commence par définir le problème et présente une formulation en programme linéaire mixte en nombres entiers "arc-based", sur un réseau change-and-go, dans laquelle les fréquences de lignes et les flux de voyageurs sont déterminés conjointement.

Pour surmonter les limitations mémoire de cette formulation, le chapitre développe ensuite une formulation "path-based", obtenue par reformulation de Dantzig–Wolfe. Les chemins de voyageurs sont générés dynamiquement en résolvant des problèmes de plus court chemin. Cette

procédure de génération de colonnes est intégrée à un algorithme de branch-price-and-cut, complété par des règles de branchement, des inégalités valides et des stratégies d'accélération.

Enfin, des expériences numériques comparent les approches "arc-based" et "path-based" sur des instances issues de la bibliothèque LinTim.

Contributions

La principale contribution de cette partie est le développement d'un modèle d'optimisation "path-based" pour la planification des lignes orientée voyageurs et sa résolution par un algorithme exact. Son objectif est de surmonter les limitations mémoire de la formulation "arc-based" classique, sans modifier le problème de planification des lignes sous-jacent.

La première contribution est une reformulation de Dantzig–Wolfe des contraintes de flot de voyageurs. Cette reformulation s'appuie sur la structure du polyèdre de flot : les flots de voyageurs faisables peuvent se décomposer en chemins élémentaires et en cycles simples. Les coûts des cycles étant non négatifs dans le problème considéré, il existe toujours une solution optimale ne comportant aucun cycle de ce type. Le routage des voyageurs peut donc être représenté au moyen de chemins élémentaires uniquement.

Cette reformulation ne vise pas à renforcer la relaxation linéaire de la formulation initiale. Son principal avantage est computationnel : elle remplace une formulation compacte volumineuse par une formulation "path-based", adaptée à la génération de colonnes. Les chemins de voyageurs sont générés dynamiquement et les sous-problèmes de pricing correspondants ont une structure de plus court chemin et peuvent être résolus efficacement à l'aide de l'algorithme de Dijkstra.

La seconde contribution est l'intégration de ce cadre de génération de colonnes dans un algorithme de branch-and-price, accompagné de stratégies de branchement sur les variables de fréquence des lignes. Bien que le branchement sur les variables originelles de flot de voyageurs puisse en théorie être nécessaire pour obtenir des solutions entières, cela n'a pas été nécessaire dans les expériences numériques. De telles décisions de branchement sont néanmoins discutées, leur mise en œuvre pouvant affecter la structure des sous-problèmes de génération.

La troisième contribution est la dérivation et la mise en œuvre d'inégalités valides adaptées à la formulation. Celles-ci comprennent des inégalités fondées sur le budget pour des sous-ensembles de lignes, des inégalités de couverture renforcées tenant compte des bornes de fréquence au nœud courant de l'arbre de recherche, et, dans le cas de capacités égales, une borne inférieure sur la fréquence totale nécessaire pour transporter la demande de voyageurs. Ces coupes sont complétées par des stratégies d'accélération conçues pour obtenir plus rapidement une première solution entière.

Les expériences numériques montrent que l'approche "path-based" évite le goulot d'étranglement mémoire rencontré avec la formulation "arc-based". Sur des instances petites et relativement faciles, les deux approches prouvent rapidement l'optimalité. Sur des instances plus grandes, ou lorsque l'heuristique de programmation en nombres entiers appliquée à la racine échoue à trouver une solution faisable, la méthode "path-based" peine à obtenir des solutions entières optimales. Elle demeure néanmoins capable de calculer des bornes inférieures et, pour certaines instances, des solutions faisables lorsque la formulation "arc-based" atteint ses limites mémoire.

Les expériences mettent également en évidence les limites actuelles de l'approche. En particulier, la dégénérescence et des solutions fortement fractionnaires peuvent conduire à un arbre de branchement de grande taille, comportant des nœuds aux bornes inférieures similaires. Par conséquent, trouver une première solution entière faisable et fermer l'écart d'optimalité sont des tâches qui demeurent difficiles sur les plus grandes instances.

2.3 Affectations équilibrées

Contexte et défis scientifiques

La dernière partie de la thèse porte sur les affectations équilibrées de tâches périodiques. Ce thème est motivé par la construction de grilles cycliques, dans la mesure où les affectations équilibrées relâchent la forte exigence de cyclicité tout en préservant certaines propriétés des grilles cycliques. Les questions d'équité et de périodicité sont étudiées séparément dans la littérature. Cette partie introduit la notion d'affectation équilibrée, définie comme une affectation faisable de tâches périodiques atteignant un équilibre exact à long terme entre un nombre fixé d'agents, c'est-à-dire que chaque agent effectue chaque tâche avec la même fréquence asymptotique.

Ce cadre soulève deux questions principales. Premièrement, sous quelles conditions existe-t-il une affectation équilibrée faisable ? Deuxièmement, lorsqu'une telle affectation existe, peut-elle être choisie périodique, et que peut-on dire de sa période ? Cette dernière question est particulièrement importante car les affectations périodiques sont souhaitables en pratique, dans la mesure où elles admettent une représentation finie et fournissent des motifs de travail prévisibles. Le défi consiste donc à établir si l'équilibre sur un horizon infini peut être atteint au moyen d'un motif fini se répétant.

Portée et structure

Cette partie comprend trois chapitres consacrés aux affectations équilibrées de tâches périodiques.

Le chapitre 7 introduit la version de base du problème, dans laquelle des tâches périodiques hebdomadaires doivent être affectées à des agents de façon à ce que les tâches affectées à un agent ne se superposent pas. Ce chapitre étudie l'existence d'affectations faisables équilibrées, leur périodicité, des contreparties algorithmiques, et relie les résultats obtenus à l'existence de grilles cycliques.

Le chapitre 8 étend ce cadre en exigeant que les affectations hebdomadaires des tâches à chaque agent appartiennent à un ensemble donné de plannings admissibles. Ce chapitre cherche à généraliser les résultats d'existence et de périodicité établis au chapitre 7 dans ce cadre plus général. La question de la périodicité est liée à un problème auxiliaire sur un graphe orienté à arcs colorés, dans lequel des jetons placés sur chaque sommet se déplacent selon une séquence commune de couleurs.

Le chapitre 9 explore davantages quelques questions liées au problème de graphe introduit pour les preuves des résultats du chapitre 8. En exploitant le lien entre affectations équilibrées et suites de couleurs équilibrées dans des digraphes eulériens à arcs colorés, il étudie des bornes sur la période, des structures de graphes particulières, ainsi que des extensions du problème.

Contributions

La première contribution du chapitre 7 est une caractérisation complète du problème d'affectation équilibrée dans le cas basique avec pour seule contrainte le non-chevauchement. Le chapitre démontre l'équivalence des trois conditions suivantes : (i) il existe une affectation faisable dans laquelle un agent effectue chaque tâche au moins une fois ; (ii) il existe une affectation faisable équilibrée ; et (iii) il existe une affectation faisable équilibrée périodique de période q. La démonstration est constructive et fournit un algorithme polynomial permettant de décider de l'existence et de calculer une affectation équilibrée périodique.

La seconde contribution, dans le chapitre 8, étend ce cadre en exigeant que les affectations hebdomadaires des tâches à chaque agent appartiennent à un ensemble donné de plannings admissibles. Le problème est relié à un problème auxiliaire sur un digraphe eulérien à arcs

colorés, dans lequel des jetons placés sur les sommets se déplacent selon une séquence périodique commune de couleurs, visitant chaque arc de manière équilibrée. Grâce à cette connexion, le chapitre démontre que l'existence d'une affectation faisable équilibrée implique l'existence d'une affectation périodique dont la période est au plus $2(q-1)q!$. Le chapitre examine ensuite si les affectations périodiques obtenues peuvent être interprétées comme des grilles cycliques. Contrairement au cadre de base, il n'existe pas toujours d'affectation faisable équilibrée périodique induisant une grille cyclique, même en autorisant des grilles cycliques dont la période n'est pas restreinte à une semaine. À l'aide d'arguments issus de la théorie des groupes, le chapitre montre que, lorsque q est premier, l'existence d'une affectation faisable équilibrée implique l'existence d'une affectation qui définit également une grille cyclique, de période au plus $q(q!-1)$.

La troisième contribution, dans le chapitre 9, apporte des discussions complémentaires sur le problème de graphe introduit au chapitre 8.

Premièrement, ce chapitre étudie le cas particulier où le digraphe à arcs colorés est un graphe de Cayley, alors doté de propriétés de symétrie supplémentaires. Dans ce cas, le chapitre démontre l'existence d'une séquence de couleurs équilibrée périodique de période bornée par le nombre d'arcs, conduisant à des bornes de période améliorées pour les affectations faisables équilibrées périodiques, bien que garantir la structure de graphe de Cayley à partir d'une instance donnée semble difficile en pratique.

Deuxièmement, le chapitre 9 établit une équivalence constructive entre le problème d'affectation équilibrée périodique et un problème relâché de recherche d'une séquence de couleurs équilibrée périodique. S'appuyant sur cette équivalence, ce chapitre propose une formulation en programme linéaire en nombres entiers pour résoudre le problème (relâché) de séquence de couleurs équilibrée périodique de période minimale. Cette formulation est utilisée pour discuter la finesse des bornes de période établies dans les chapitres précédents.

Enfin, le chapitre 9 étudie une extension aux affectations équilibrées en compétences, où l'équilibre est défini par une répartition équitable des compétences, acquises lors de l'exécution des tâches. Ce chapitre établit des liens entre cette exigence d'équilibre des compétences et le cadre théorique des graphes, et identifie des questions ouvertes relatives à la périodicité de telles affectations.

PART

Integrating crew scheduling and crew rostering with train delays for rail freight

Introduction

***

3 Planning process in the railway industry

The entire operational planning process in the railway industry has been extensively studied in the operations research literature. This interest is largely driven by the need for cost-efficient resource allocation (including rolling stock, crews, energy, etc.) to accommodate heavy traffic volumes while maintaining a leading position in an increasingly competitive market. With the forthcoming deregulation of the European railway industry, the liberalization of its markets, and the ongoing competition with other modes of transport, achieving economic profitability in rail operations remains a major challenge [3]. Since infrastructure costs are essentially fixed, the primary lever for reducing overall expenses lies in the efficient management and optimization of these resources. Because of the complexity of the overall operational planning process, it is addressed in several planning steps, which differ slightly when dealing with passenger versus freight transportation.

3.1 For rail passenger transportation

In Europe, the operational planning steps for railway passenger transportation are usually as in Figure 2.1. More details can be found in [79]. The first step is the *line planning* step, which consists in determining lines, i.e., sequences of stations, and their frequency in order to meet the passenger demand (Part II addresses this step in more detail). This is followed by the *timetabling* step, where arrival and departure times at each station are determined. Sometimes, a *platforming* step follows, where trains are assigned a platform within each station [23]. The next step is the *vehicle scheduling* step: vehicles are assigned to trips (operations along a given line following a given timetable). Finally, there is the *crew management* step, which is often further divided into a *crew scheduling* step and a *crew rostering* step. Crew scheduling consists in building duties covering all trips, while crew rostering groups these duties into weekly schedules, which are then combined to form rosters, providing the yearly schedules for the crews.

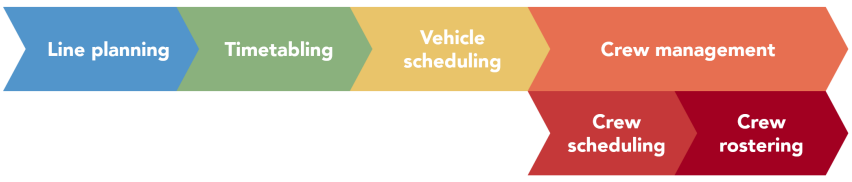


Figure 2.1: Operational planning steps for railway passenger transportation

3.2 For rail freight

The operational planning process for European rail freight transportation differ slightly from the one for passenger services, as summarized in Figure 2.2. The key difference lies in the *train routing* step, which replaces line planning and timetabling. The prevailing paradigm for freight is to select timed network sections to build routes from origin to destination for each wagon instead of relying on fixed lines. This difference primarily stems from the heavy demand placed on the railway network by passenger transportation. Freight and passenger trains share the same infrastructure, but in Europe, passenger trains are typically given priority over freight trains. (This is not the case in North America, where freight trains often take precedence, resulting in fundamentally different planning approaches between the two regions.) As a result, freight trains have only limited timetable options that can be booked with the network operators. The *rolling stock routing* step follows: it assigns vehicles—referred to as *rolling stock* in freight operations—to trips, in a way that is similar to vehicle scheduling for passenger transportation. However, because train composition undergoes numerous modifications in shunting yards, the routes followed by each wagon often differ substantially from the routes followed by the rolling stocks. The final step of the planning process is the crew management problem, as for passenger transportation.



Figure 2.2: Operational planning steps for railway freight transportation

4 Sequential versus integrated optimization

The planning process, as described above, is sequential in nature, since each step is solved using the solution of the previous steps as input. Such a sequential approach to planning in the field of transportation is common because of the practical processes that occur between successive stages of the planning timeline (infrastructure booking, negotiations with trade unions, etc.). It is also justified by the high combinatorial complexity of solving multiple steps simultaneously. However, these steps are highly interdependent, and upstream decisions strongly influence both the feasibility and the cost of the downstream problems. The integration of multiple planning steps in order to reduce the sequentiality gap in the planning process has been the focus of a substantial body of literature [77, 79]. This interest is relatively recent, driven by increased computational power and the improved performance of optimization solvers. The shift from sequential to integrated approaches has also been observed in other application areas [2, 16, 28, 47, 97].

General solution approaches have been developed to tackle multiple optimization problems in an integrated manner. In [79], Schöbel proposes an eigenmodel-based approach that iteratively solves subproblems to address general multi-stage problems. In [77], Schiewe and Schöbel develop a general framework for the integrated formulation of sequential problems and refer to the notion of the *price of sequentiality*, introduced in [75], to quantify how close a sequential solution can get to the optimal solution of the corresponding integrated problem.

In practical applications, a variety of solution methods have been developed to solve integrated problems, including heuristics [2], metaheuristics [47], matheuristics [28, 97], iterative algorithmic schemes [75], branch-and-bound methods [28], and column generation methods [16].

5 Uncertainties in the planning process

The planning process described above is typically carried out under the assumption that all input data—travel times, resource availability, etc.—are perfectly known. In practice, however, railway operations are subject to numerous sources of uncertainty: train delays caused by infrastructure incidents, weather conditions, or traffic congestion; fluctuations in freight demand; and rolling stock or crew unavailability. This is particularly true for freight transportation, since freight trains are given lower priority than passenger trains on the shared infrastructure in Europe, and are therefore more exposed to delays and to their propagation throughout the network. Ignoring these uncertainties at the planning stage can lead to solutions that are optimal in a deterministic sense but fragile in practice, requiring costly last-minute adjustments.

Two main complementary lines of work emerge from the presence of uncertainty in the railway planning process. The first one operates upstream of the realization of the uncertainty, at the planning stage: it consists in building plans (timetables, rolling stock or crew schedules) that anticipate the possibility of disruption, typically by favoring solutions that are inherently less sensitive to perturbations. The second one operates downstream, once the uncertainty has been revealed: it consists in recovering, i.e., locally rerouting or rescheduling resources (trains, rolling stock, crews) to restore a feasible and efficient operation after a disruption has occurred.

For rail freight specifically, the literature has focused almost exclusively on this second line of work. Because freight operations already have limited flexibility at the planning stage—timetable slots being negotiated with infrastructure managers and largely fixed in advance—most of the effort has been devoted to recovery once delays materialize, rather than to building plans that are robust or stochastic by design. This recovery literature typically addresses train routing and rescheduling in the presence of delays or infrastructure disruptions [21].

6 Scientific challenges and contributions

This work addresses the scientific challenge of solving the crew management problem in an integrated manner for a large-scale freight application at SNCF, under both deterministic and stochastic timetable information. The central difficulty addressed in this part is that crew scheduling and crew rostering are strongly interdependent. A duty that appears cost-efficient at the scheduling stage may create infeasibilities or inefficiencies once rosters are built. Conversely, roster-level constraints may require scheduling decisions that are not locally optimal but lead to better global cyclic plans.

The deterministic version of this "integrated" approach, described in Chapter 4, stands out from the literature by its application to rail freight at SNCF, and to large-scale instances. In particular, this framework dispenses with some assumptions made in the literature while taking into consideration specificities of French rail freight.

On top of the inherent suboptimalities stemming from the current two-stage approach used at SNCF, this approach makes it difficult to account for uncertainties because of the paradigm in place, governing the propagation of train delays, and the evaluation of their impact on the rosters. To our knowledge, the integrated resolution of the crew management problem while accounting for train delays has never been addressed in the literature. When timetable information is stochastic, i.e., when trains come with delay distributions, the integrated stochastic approach, described in Chapter 5, builds on the deterministic model by taking these uncertainties into account as an additional stochastic threshold constraint. This stochastic constraint uses a quantitative metric that we introduce in this work. This metric, which we call *fragility*, aims to precisely capture the impact of uncertainty on rosters

in a concrete and meaningful way. The definition and its rationale are given in Section 5.3.2 (assuming the delay distributions are known). The integrated approach we propose is able to handle this stochastic constraint "optimally" when delays are independent random variables with finite support. In practice, we note that delay distributions are often only partially described—for instance, specifying that delays fall within a given interval with some probability. We also discuss (Section 5.6) how rigorous conclusions can still be drawn from our work in the presence of such approximate descriptions.

The deterministic and stochastic integrated approaches then rely on a column generation method that incorporates all working rules. To do so, we overcome the two-fold challenge of solving a resource-constrained shortest path problem that is non-linear and whose dimension is high.

In the deterministic case (with no uncertainties), the integrated approach systematically decreases the number of working days by a significant margin on all of the SNCF instances compared to the current method, as presented in Section 4.6. In the stochastic case (when uncertainties are present), results show that the integrated approach is able to substantially reduce fragility, as well as improve upon the objective value obtained by the current sequential approach as presented in Section 5.7.

7 Outline

In Chapter 3, we state the crew scheduling and crew rostering problems at SNCF, the current resolution methods developed at SNCF for these two problems, and an alternative formulation for the crew rostering problem.

In Chapter 4, we propose a resolution method for the integration of the crew scheduling and crew rostering problems at SNCF in the deterministic case.

In Chapter 5, we propose a resolution method for the integration of the crew scheduling and crew rostering problems at SNCF in the stochastic case. This resolution method is established when delays are independent with finite-support distributions. The finite-support assumption is also challenged in this chapter.

Chapter 3

Crew scheduling and crew rostering at SNCF

***

3.1 Context and contributions . . . 34
3.2 Literature review . . . 34
3.2.1 Crew scheduling problem . . . 34
Resolution methods for the railway industry . . . 35
CSP for rail freight . . . 35
3.2.2 Crew rostering problem . . . 35
Cyclic versus personalized rostering . . . 35
Resolution methods for the railway industry . . . 36
CRP for rail freight . . . 36
3.3 Crew scheduling and crew rostering at SNCF: formulation, specificities and algorithms . . . 36
3.3.1 Crew scheduling at SNCF . . . 36
Problem formulation . . . 36
Problem specificities and algorithm currently used at SNCF . . . 37
3.3.2 Crew rostering at SNCF . . . 38
Problem formulation . . . 38
Problem specificities and algorithm currently used at SNCF . . . 39
Comments and limitations . . . 42
3.4 Alternative model to determine optimal cyclic rosters at SNCF . . . 44
3.5 Numerical experiments . . . 47
3.5.1 Experimental setting and instances . . . 47
3.5.2 Results . . . 47
3.6 Comments . . . 51

3.1 Context and contributions

This chapter introduces the crew scheduling problem and the crew rostering problem at SNCF, together with the methods currently used to solve each of the two problems independently. The crew scheduling problem aims to build duties covering all trips over a defined period (typically a week). The crew rostering aims to build rosters, providing the yearly schedules for the crews, given duties over a defined period. The crew rostering problem is itself solved in multiple stages, the two main stages being: determining "blocks" (representing weekly assignments of tasks) that cover the required trips; and constructing optimal "rosters" from these blocks.

Although the algorithm currently used at SNCF for the crew rostering problem is efficient to solve most instances (see Section 3.5 for the numerical results), it comes with a few limitations, detailed in Section 3.3.2. We propose a methodology that overcomes these drawbacks, via an alternative integer linear program (ILP) with a polynomial number of constraints (Proposition 3.4.1). Moreover, this new ILP is amenable to column generation. As further discussed in Section 3.4, this opens the door to tackling larger instances than those currently solved. By lack of time, we have not explored further this direction. Additionally, as detailed in Section 3.5, the alternative model is able to efficiently solve some instances on which the current resolution method struggles.

We also prove the following: perhaps surprisingly, given any set of blocks covering all trains with a minimal total number of working days, these blocks can be inserted into rosters without any loss of optimality, yielding an optimal solution to the crew rostering problem. This is the second main contribution of this chapter, stated as Lemma 3.4.2. This result further justifies the focus on the first stage—namely, the construction of optimal blocks—in the subsequent chapters. It is also the key argument for the polynomiality of the number of constraints in the ILP of Proposition 3.4.1.

This chapter is not intended to provide technical details, but to introduce the operational vocabulary and the planning problems addressed throughout the subsequent chapters. These planning problems are strongly interdependent, which motivates the integrated models developed in later chapters.

3.2 Literature review

3.2.1 Crew scheduling problem

The crew scheduling problem (CSP) consists of finding a set of duties (sequence of tasks that must satisfy various constraints) covering all trips for a defined period of time, while minimizing the total cost. The trips are assumed to repeat every period (typically a week), and the duties are also built over the same period. The first occurrences of the CSP in the literature appear in the late 60s and early 70s [6, 11]. Since then, this problem has been widely studied, through a number of applications to the healthcare [1], airline [31, 44, 52], bus [82], and railway [23] industries. The CSP has been proven to be NP-hard, even with just maximal working time and elapsed time constraints [66]. While the constraints, costs, and technical vocabulary vary depending on the application, the CSP is typically formulated as a mixed integer linear program, with either a set partitioning formulation [66] or a set covering formulation [82]. In practice, exact formulations quickly become intractable on real-life instances, which has motivated the development of tailored decomposition methods or heuristic approaches to handle large-scale instances [23].

Resolution methods for the railway industry

Regarding the resolution methods, most approaches to the CSP in the railway industry rely on decomposition techniques that separate duty generation from duty selection. While integer programming methods are used on small instances [48], column generation based methods [90] are developed to tackle larger instances where the number of potential duties is large [23, 66]. Heuristics [4] and metaheuristics [54] are also developed for large instances with complex rules or costs. An extensive survey on this topic has been conducted by Heil et al. [48].

CSP for rail freight

While the CSP can be stated identically for passenger transportation and for rail freight, some fundamental differences in the data, the costs and the rules and regulations make it sometimes difficult to transpose the resolution methods of passenger transportation. In particular, freight trains a priori run continuously throughout the day and night (in Europe at least) because of prioritization of passenger traffic [50]. Thus, daily disaggregation techniques sometimes used to accelerate the resolution of the passenger CSP are not applicable for freight.

3.2.2 Crew rostering problem

The crew rostering problem (CRP) consists of finding a set of rosters (a roster being a sequence of working and rest days that must satisfy a variety of constraints) covering all duties for a defined period of time. In practice, the CRP is typically solved sequentially after the CSP, taking the set of generated duties as input and organizing them into rosters. As the duties constructed in the CSP were built over a defined period, they are assumed to be the same every period. The CRP has attracted considerable attention in airline, railway, public transport operations, healthcare, etc. Surveys have been conducted on the CRP regarding to these various applications [19, 36, 55, 86]. A number of variations are considered in the literature under the umbrella of either personalized rostering [96] or cyclic rostering [17]. In the latter case, workers are grouped into teams according to residence and skills, and each worker in a team shares a single roster that is performed in a cyclic manner. These cyclic rosters are operated on a weekly basis (when the duties are defined on a typical week) and provide the yearly schedules for the workers: each worker of a team cycles through the different weeks (or "rows") of the roster. This cyclic variant is often adopted in the railway industry because it inherently promotes fairness and equitable workload distribution among workers of the same team.

Cyclic versus personalized rostering

The railway industry has traditionally favored cyclic rostering, where a single roster is designed for an entire team and repeated cyclically over the planning horizon [17]. This particular case of crew rostering is applicable for data with repeating demand patterns and ensures strict fairness across team members, as all workers follow the same sequence of duties and rest periods.

In contrast, personalized rostering constructs distinct rosters for individuals within a team [96]. This setting for the crew rostering problem is often adopted for call-centers where demand fluctuates without a clear periodic pattern [35]. Such a case of crew rostering also allows staff preferences to be incorporated. In particular, in the airline industry, a bid on individual lines of works is set up. Crews then take bids in order of decreasing seniority [85]. To offer more fairly distributed workloads, equitability in preferential bidding systems, and acyclic crew rostering more generally, are active lines of study in this research area [15, 18].

Resolution methods for the railway industry

Research on the CRP in the railway industry, as with the CSP, is relatively recent compared to its airline counterpart [36]. The (cyclic) CRP comes in the literature with a large variety of constraints and objectives [86]. While some authors focus on the minimization of purely operational costs [34], others argue for the maximization of the roster attractiveness [17]. In general, the number of rows in the final roster is either captured as the operational cost, or used in a constraint, as it corresponds to the number of workers needed to operate the roster. However, and this seems specific to our case study, SNCF takes only the number of working days of the roster into account. In the literature, the CRP is usually modeled with one of the three following formulations: generalized assignment, multi-commodity flow, or set partitioning [17]. Overall, a wide range of resolution methods are used to solve the CRP [86]: integer programming methods are deployed on smaller instances, while column generation, Lagrangian relaxation, constraint programming, metaheuristics, etc. tackle larger real-life instances.

CRP for rail freight

As for the CSP, the CRP can be formulated identically for passenger and for freight transportation. However, the characteristics of rail freight operations impose specific modeling requirements that differ from passenger transportation (freight trains typically operate continuously throughout the day and night). To date, dedicated studies on the CRP specifically for rail freight are limited, and most methodologies rely on adaptations of passenger-oriented approaches, adjusted for the operational characteristics of freight services.

3.3 Crew scheduling and crew rostering at SNCF: formulation, specificities and algorithms

This section introduces the crew scheduling and crew rostering problems as they arise in our real-world application to SNCF rail freight operations. We also focus on the solution methods currently used at SNCF to tackle these two problems.

3.3.1 Crew scheduling at SNCF

Problem formulation

An instance of the crew scheduling problem at SNCF is formed by a collection T of tasks (consisting of driving trains, thus called *trains* from now), where a train t refers to a recurring weekly trip operated from an origin to a destination with a nominal departure time δ_t and a nominal arrival time $\delta_t + \tau_t$, where τ_t is the travel time of t, as illustrated in Figure 3.1. In a typical instance, we are also given a collection I of teams of workers (who are train drivers in this application). Each team is characterized by its residency and its skills. The residency enforces geographic constraints; the ability for a team to drive a train depends on its skills. The goal is to group trains together into "duties," formally defined below, and assign them to teams. These duties will eventually be assigned to drivers, who will be in charge of driving the trains as prescribed in the duty. This will be the focus of the crew rostering problem, detailed in Section 3.3.2. The grouping and assignment process has to comply with working rules (detailed in the paragraph below), which are defined at the level of the duties.

A *duty* is a sequence of trains that has to be performed by a single driver. On top of the trains, three other tasks might appear in a duty: relocations (taxi or walking trips to move from one location to another), "short breaks," and start/end-of-duty. However, given a sequence of trains, departure and arrival locations, and a team's residency, the other tasks

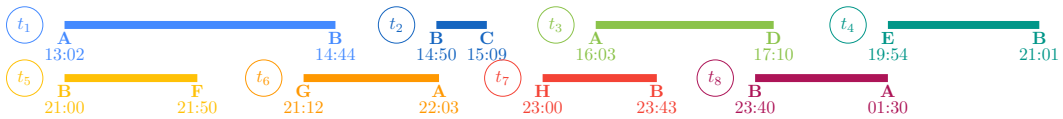


Figure 3.1: An example of trains in an instance, all on the 6th day of the week. The train t_1 departs at 13:02 from location A and arrives at 14:44 at location B.



Figure 3.2: An example of a duty starting at location A and ending at location B, whose train sequence is t_1, t_2, t_3, and which is assigned to a team with residency B: a 5-minute start-of-duty occurs at location A, the train t_1 is driven from A to B, the train t_2 is driven from B to C, a relocation from location C to location I occurs, followed by a short break in location I, a relocation occurs from I to E, the train t_3 is driven from E to B, and finally a 13-minute end-of-duty occurs at location B.

can be determined unambiguously. An example is given in Figure 3.2. A duty has to fit into an 11-hour time window and to comply with working rules, detailed in Section 3.3.1. An *on-call* is a one-day period where a driver is available as a backup.

The crew scheduling problem consists of finding a set of duties of minimal cost, each duty being assigned to a team of drivers, such that each train of the instance is covered by a duty. The cost of a duty is defined as a surrogate cost aiming to capture approximately the later insertion of duties into rosters with the information available at this stage. Details on the definition of the cost are provided in the paragraph below. The crew scheduling problem can be formally written as:

Crew scheduling (CS)
Input. Collection T of trains on a typical week, collection I of teams of drivers.
Output. Covering of trains by duties with minimum surrogate cost, each duty assigned to a team.

This problem is NP-hard because basic versions of the crew scheduling problem can easily be reduced to it, and such basic versions are known to be NP-hard [66].

Problem specificities and algorithm currently used at SNCF

The crew scheduling problem as it arises at SNCF includes technical details, specific to this application. This section discusses such details, as well as the algorithm currently used at SNCF.

In particular, the cost of a duty is defined as a linear combination of multiple factors, aiming to capture the future cost of the insertion of duties within rosters. Using an aggregated cost function is generally a practice to be treated cautiously in Operations Research for several reasons: the difficulty of defining relevant weights for non-comparable terms, the risk of obtaining objective values that lack a concrete interpretation, etc. Moreover, establishing a surrogate cost capturing precisely the insertion of duties within rosters is as difficult as building the rosters themselves.

Working rules applying to the duties are summarized in Table 3.1.

The algorithm currently used at SNCF to address the crew scheduling problem is a heuristic consisting of the following phases:

R1	a task ending at a location must be followed by a task starting at the same location
R2	a team can only drive trains compatible to its set of skills
R3	maximal working range in a daytime duty with a short break = 11h
R4	maximal working range in a daytime duty without a short break = 8h
R5	maximal working range in a nighttime duty = 8h
R6	for a nighttime duty with least 5h of work and 2h of nighttime work, maximal effective work duration = 9h
R7	short breaks cannot happen at nighttime
R8	"meal" short breaks must happen in a dedicated interval, be it at lunch or at dinner
R9	minimal work duration in a duty before and after a short break = 1h30
R10	minimal short break duration = 1h30
R11	given start-of-duty duration (depending on the first task)
R12	given end-of-duty duration (depending on the last task)

Table 3.1: Working rules for duties.

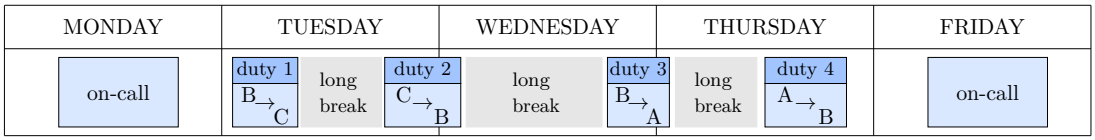


Figure 3.3: An example of a block of five working days.

1. Generation of duties by enumerating paths in a graph with domination-based discarding rules.
2. Integer linear program to select a set of duties of minimum surrogate cost while covering all trains.

3.3.2 Crew rostering at SNCF

Problem formulation

An instance of the crew rostering problem at SNCF is typically formed by a collection of *duties* for each team, each duty being repeated every week. The goal is to group duties together into "rosters," formally defined below, and assign them to teams. The drivers of each team will then be in charge of driving the assigned duties every week. (Each duty is performed by a single driver, and in the final roster of the team, the drivers will cyclically perform this duty over the year.) The grouping and assignment process has to comply with working rules which are defined at the level of "blocks," which we define now, and rosters.

A *block* is a sequence of consecutive duties and on-calls between two rest days, and assigned to a team, where two consecutive duties are separated by a "long break." An example is given in Figure 3.3. It also has to comply with working rules (detailed in the paragraph below). The problem is to cover all trains with blocks, with the objective of minimizing the number of working days. We highlight that, because of the rules R19 to R21, there could be "mandatory" on-calls at the start, the end, or the middle of a block. A day is a *working day* if there is an on-call or a duty overlapping at least partially with this day.

A *roster* is a cyclic sequence of blocks assigned to the same team. An example is given in Figure 3.4. The number of rows of a roster corresponds to the number of weeks it ranges

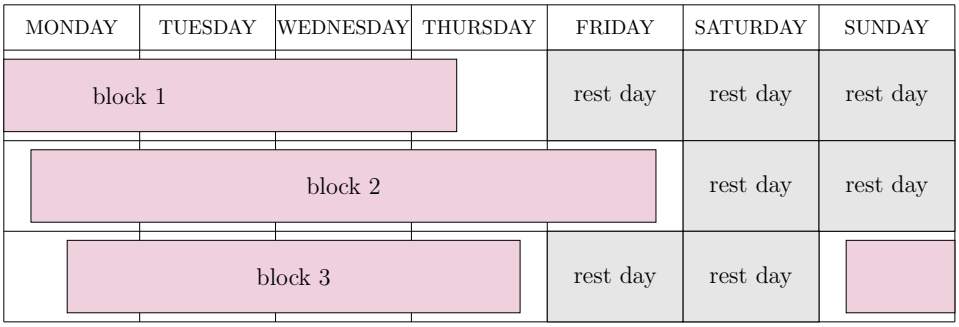


Figure 3.4: An example of a roster for a team of drivers with 14 working days, and 3 rows.

over, or equivalently to the number of workers in the team. It has to comply with working rules (detailed in the paragraph below).

The crew rostering problem consists of finding a set of rosters of minimum number of working days, each roster being assigned to a team of drivers, such that each duty of the instance is covered by a roster. The crew rostering problem can be formally written as:

Crew rostering (CR)
Input. Collection of duties on a typical week, each assigned to a team of drivers.
Output. Covering of duties by rosters with minimum number of working days, each roster assigned to a team.

We emphasize at this point that the number of rows (i.e., the number of workers in each team) is surprisingly to a criterion at SNCF. The approach we develop in Chapters 4 and 5 exploit heavily this property.

Problem specificities and algorithm currently used at SNCF

The crew rostering problem as it arises at SNCF includes technical details, specific to this application. This paragraph discusses such details, as well as the algorithm currently used at SNCF.

There are in fact two types of on-calls at SNCF: the long on-calls and the short on-calls. Given a sequence of duties, the short on-calls are actually the "mandatory" on-calls that can be determined uniquely following the rules R19, R20, and R21 of Table 3.2. Only the short on-calls count in the block cost computation. In other words, when computing the number of working days of a block, we only count the number of days with an overlapping short on-call or duty. On the other hand, the long on-calls are not mandatory per se: from a sequence of duties, any number of long on-calls can be added to a block, while still meeting the rules of Table 3.2. Adding zero, one, or multiple long on-calls to a block defines "variants" of this original block and an example is given in Figure 3.5. Considering such variants of blocks is only useful when building rosters from blocks while taking into account the number of rows as a secondary roster quality criterion. For example, the rules R31 and R32, bounding the number of rest days between two blocks, motivates the introduction of the variants. There can also be a block with no duties and only long on-calls. Such blocks automatically have a number of working days equal to zero.

Working rules applying to the blocks are summarized in Table 3.2. Working rules applying to the rosters are summarized in Table 3.3.

The algorithm currently used at SNCF to address the crew rostering problem consists of the following phases:

1. Generation of blocks without long on-calls by enumerating paths in a graph.

R13	back to team's residency at most every two duties
R14	a duty ending at a location must be followed by a duty starting at the same location
R15	a team can only drive trains compatible to its set of skills
R16	maximal number of days = 6
R17	maximal number of on-calls and duties = 6
R18	maximal number of night duties = 6
R19	a duty starting a block before 6:00 must be preceded by a short on-call
R20	a duty ending a block after 19:00 must be followed by a short on-call
R21	at the team's residency, between two duties with a long break smaller than 38h, there must be a short on-call
R22	minimal long break duration at team's residency = 14h
R23	minimal long break not at team's residency = 8h
R24	a long break shorter than 9h is followed by a long break longer than 15h

Table 3.2: Working rules for blocks.

R25	minimal number of annual rest days = 120
R26	minimal number of annual consecutive rest days = 52
R27	minimal number of annual Sundays that are rest days = 22
R28	minimal number of annual consecutive rest days with a Sunday = 14
R29	minimal number of annual weekends where both days are rest days = 12
R30	maximal number of consecutive worked Sundays = 3
R31	minimal number of rest days between two blocks = 1 if it is a Sunday, 2 otherwise
R32	maximal number of rest days between two blocks = 3
R33	minimal number of rest days after a block with 6 working days = 2
R34	minimal/maximal number of days in each block of long on-calls = 3/5
R35	between two successive blocks with the first block ending with a duty and the second starting with a duty, there must be at least 14h + 24h $\times$ number of full days between the two blocks

Table 3.3: Working rules for rosters.

2. Integer linear program to select a set of blocks of minimum number of working days while covering all trains. This ILP writes

$$\begin{aligned} \text{minimize} \quad & \sum_{i \in I} \sum_{b \in B_i} w_b x_{b,i} & & \\ \text{s.t.} \quad & \sum_{i \in I} \sum_{b \in B_i : t \in b} x_{b,i} \geqslant 1 & \forall t \in T & \qquad \text{(Covering CR)} \\ & x_{b,i} \in \{0,1\} & \forall i \in I, \forall b \in B_i & \end{aligned}$$

   where w_b is the number of working days in the block b, B_i is the set of blocks generated for team i and the variables $x_{b,i}$ take the value 1 if the block $b \in B_i$ is selected for the team $i \in I$. At this stage, the set of blocks in an optimal solution of the ILP covers all trains and minimizes the total number of working days. The remaining two steps aim to build rosters from these blocks or their variants (i.e., blocks augmented with long on-calls) for a better insertion in the rosters.
3. Addition of all possible long on-calls to each block selected by the previous ILP. The "augmented" set of blocks for team i, denoted by $\widetilde{B}_i$, is made of all these variants, along with all possible blocks of long on-calls. An example of all variants of a block is given in Figure 3.5.
4. Integer linear program for each team i to build a set of rosters of minimum number of working days while covering all trains, which is written below. However, some constraints are difficult to express explicitly in a linear form and are therefore enforced dynamically during the resolution process as "no-good cuts" deduced from an incumbent integer solution violating these constraints. Such cuts are detailed after the ILP. To write the ILP, a graph $G = (V, A)$ is built, modeling possible successions of blocks within a roster. This graph has vertex set $\widetilde{B}_i$ and an arc between two blocks whenever they can directly follow one another within the same roster (following rules R31 to R35 of Table 3.3). We denote by day(a) the set of rest days happening between the tail and head of arc a, and by days(b) the number of days (including working days and long on-calls) of block b. The trains assigned to team i are those covered by blocks selected for team i in phase 2. The ILP based method of phase 4 then aims to build a single cycle covering the set $\widetilde{T}_i$ of trains assigned to team i in phase 2.

$$\text{minimize} \sum_{b \in \widetilde{B}_i} w_b x_b \qquad \text{(Rostering)}$$

$$\text{s.t.} \sum_{a \in A : a = (b, b')} y_a = x_b \qquad \forall b \in \widetilde{B}_i \qquad \text{(3.1a)}$$

$$\sum_{a \in A : a = (b', b)} y_a = x_b \qquad \forall b \in \widetilde{B}_i \qquad \text{(3.1b)}$$

$$\sum_{a \in A} |\text{days}(a)| y_a \geqslant \frac{116}{365} \left(\sum_{a \in A} |\text{days}(a)| y_a + \sum_{b \in \widetilde{B}_i} |\text{days}(b)| x_b \right) \qquad \text{(3.1c)}$$

$$\sum_{\substack{a \in A \\ |\text{days}(a)| \geqslant 2}} y_a \geqslant \frac{52}{365} \left(\sum_{a \in A} |\text{days}(a)| y_a + \sum_{b \in \widetilde{B}_i} |\text{days}(b)| x_b \right) \qquad \text{(3.1d)}$$

$$\sum_{\substack{a\in A\\ \text{SUN}\in\text{days}(a)}} y_a \geqslant \frac{22}{365}\left(\sum_{a\in A}|\text{days}(a)|y_a + \sum_{b\in\widetilde{B}_i}|\text{days}(b)|x_b\right) \tag{3.1e}$$

$$\sum_{\substack{a\in A\\ \text{SUN}\in\text{days}(a)\\ |\text{days}(a)|\geqslant 2}} y_a \geqslant \frac{14}{365}\left(\sum_{a\in A}|\text{days}(a)|y_a + \sum_{b\in\widetilde{B}_i}|\text{days}(b)|x_b\right) \tag{3.1f}$$

$$\sum_{\substack{a\in A\\ \{\text{SAT},\text{SUN}\}\subseteq\text{days}(a)}} y_a \geqslant \frac{12}{365}\left(\sum_{a\in A}|\text{days}(a)|y_a + \sum_{b\in\widetilde{B}_i}|\text{days}(b)|x_b\right) \tag{3.1g}$$

$$\sum_{b\in\widetilde{B}_i : t\in b} x_b \geqslant 1 \qquad \forall t\in\widetilde{T}_i \tag{3.1h}$$

$$x_b \in \{0,1\} \qquad \forall b\in\widetilde{B}_i \tag{3.1i}$$

$$y_a \in \{0,1\} \qquad \forall a\in A \tag{3.1j}$$

where the variables y_a take the value 1 if the arc a is selected and the variables x_b take the value 1 if the block b is selected. Constraints (3.1c) to (3.1g) correspond to rules R25 to R29 of Table 3.3. Constraints (3.1h) are the train covering constraints. Constraints (3.1a) and (3.1b) correspond to the cycle defining constraints. However, the constraints ensuring that there is a single cycle and the constraints corresponding to rule R30 are not included in the ILP above as they are difficult to express linearly. As mentioned above, they are included dynamically as no-good cuts from an incumbent integer solution violating these constraints. When an integer solution of (Rostering) is found, we check whether:

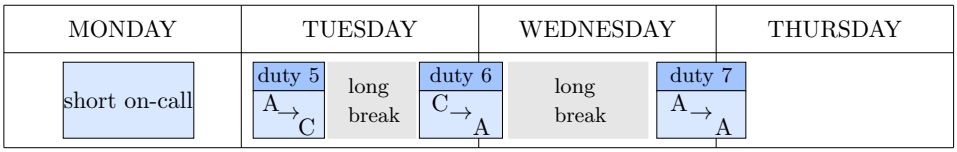
- it decomposes into cycles $C_1 \cup \cdots \cup C_k$ with $k \geqslant 2$, ordered by non-decreasing size. If so, then the cut $\sum_{a\in C_1} y_a \leqslant |C_1| - 1$ is added and the resolution is resumed;
- it decomposes into a single cycle C with 4 or more consecutive worked Sundays. If so, then the cut $\sum_{a\in C} y_a \leqslant |C| - 1$ is added and the resolution is resumed;
- none of the two previous cases are met. If so, then the solution is feasible and optimal for the problem solved by phase 4 (the problem building rosters with minimal number of working days from optimal blocks).

Comments and limitations

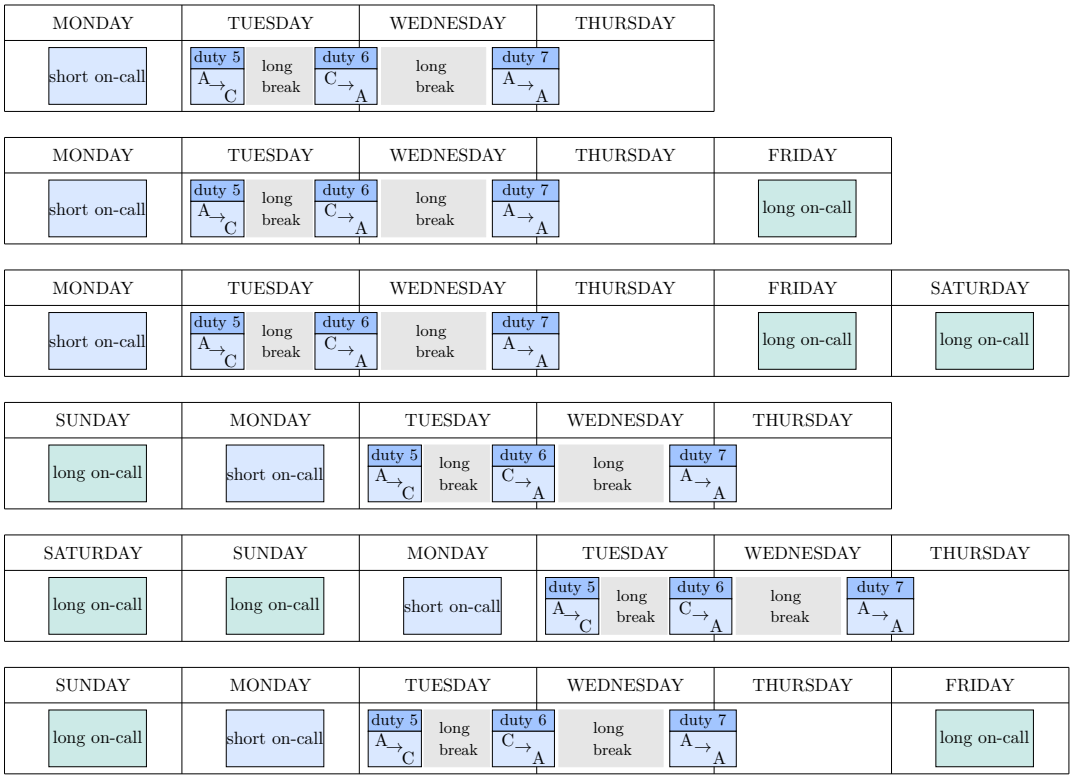
In the following chapters, we will argue that the phases 3 and 4 can be replaced with a straightforward greedy algorithm converting any sequence of blocks to a valid roster. This statement is formalized in the next section as Lemma 3.4.2.

Recall that the number of rows of a roster is the number of weeks it ranges over. Freight planners sometimes prefer to minimize the number of rows (i.e., the number of weeks spanned by the roster) as a primary objective, while ensuring that the total number of working days remains minimal. Such an objective can help planners justify the practicality of the rosters in negotiations with trade unions. In such a setting, the algorithm detailed in the previous paragraph is a valid heuristic, when the ILP (Rostering) is modified as follows:

- the objective function is replaced with $\sum x_b + \sum y_a$, where the first sum ranges over $\{b \in \widetilde{B}_i \colon \text{SUN} \in \text{days}(b)\}$ (i.e., the set of blocks covering a Sunday) and the second sum ranges over $\{a \in A \colon \text{SUN} \in \text{days}(a)\}$ (i.e., the set of arcs with a rest day on a Sunday).
- the following constraint is added $\sum_{b\in\widetilde{B}_i} w_b x_b \leqslant W_i$, where W_i is the number of working days over the blocks of team i selected in the optimal solution of phase 2.



(a) An example of a block b with duties and (mandatory) short on-calls: duty 5 starts before 6:00 and rule R19 applies.



(b) An illustration of all the variants of the block b by addition of all possible long on-calls.

Figure 3.5: An illustration of the augmentation of blocks with long on-calls.

When discussing an alternative ILP in Section 3.4, we will also propose a model suitable for this setting. However, the number of rows is only relevant when the number of working days is kept minimal, as the true financial cost of a roster is its number of working days. Therefore, the discussions on minimizing the number of rows are confined to this chapter: in the subsequent chapters, only the minimization of the number of working days is considered, as it is the sole financially meaningful objective.

Note that, if we do not wish to offer the flexibility to change the objective function to the number of rows, then phase 4 could be simplified as a straightforward greedy heuristic (as the one exhibited in the proof of Lemma 3.4.2).

However, in both settings, the current method solving the crew rostering problem at SNCF shows two main drawbacks:

- the no-good cuts to prevent non-connected cycles and to ensure rule R30 can be time consuming.
- the performances of the algorithm might be highly dependent on the number of blocks $|\widetilde{B}_i|$ as the number of variables and the number of constraints of (Rostering) are both superlinear in $|\widetilde{B}_i|$. This also makes this model poorly suited for a resolution using column generation on the blocks.

3.4 Alternative model to determine optimal cyclic rosters at SNCF

In this section, we propose an alternative model for the problem solved by phase 4 that overcomes the two limitations of the current phase 4 currently used at SNCF; discussed above. First, this alternative model is solved using a standard branch-and-bound algorithm, without having to add no-good cuts throughout the resolution (as the current model does, see Section 3.3.2).

Second, it is amenable for a resolution using column generation (see Appendix A for generalities on column generation) if the number of blocks $\widetilde{B}_i$ for the team i is large. In a similar manner as in the subsequent chapters, this model could be the basis for an integrated resolution of the crew scheduling and crew rostering problems using column generation to tackle the large size of the set of all possible blocks (possibly with long on-calls). This has not been explored further as the focus was made on the minimization of the number of working days, which simplifies the approach towards an integrated resolution (as it is presented in Chapter 4).

Finally, this alternative formulation offers the same flexibility on the objective function as the current model, as it can be used to minimize the number of working days or the number of rows.

Let us denote by:

- $x_{r,i} \in \{0,1\}$ a variable meaning that the row r is used in the roster of team i,
- $y_{r,i} \in \{0,1\}$ a variable meaning that the row r is the last row of the roster of team i.
- $z_{b,r,i} \in \{0,1\}$ a variable meaning that the block b starts in the row r of the roster of team i,

The following proposition ensures that there is an ILP that, from sets of blocks with long on-calls, construct an optimal set of rosters covering all trains. The existence of a (polynomial) upper bound is differed in Lemma 3.4.2.

Proposition 3.4.1. *Let N be an upper bound on the total number of rows. The problem solved by phase 4 of the crew rostering problem can be written as an ILP using the variables $x_{r,i}, y_{r,i}$, and $z_{b,r,i}$ (along with additional auxiliary variables), minimizing the total number of rows across all rosters $\sum_{i\in I}\sum_{r\in[N]} r y_{r,i}$ (or the total number of working days $\sum_{i\in I}\sum_{b\in\widetilde{B}_i}\sum_{r\in[N]} w_b z_{b,r,i}$). Moreover, the number of constraints is polynomial in N.*

Proof. Let us only show that the constraints ensuring that blocks do not overlap can be written linearly with the variables $x_{r,i}, y_{r,i}$, and $z_{b,r,i}$. The other constraints corresponding to the rules of Table 3.3 can be written linearly in a straightforward way (except maybe the one associated with rule R35 which requires a bit more work).

These constraints decompose into three cases, which we call case 1, 2, and 3, as illustrated in Figure 3.6. In case 1, the constraints write:

$$\sum_{\substack{b' \in \widetilde{B}_i \\ d \leqslant d_{b'} \leqslant f}} z_{b',r,i} + 2 \sum_{\substack{b \in \widetilde{B}_i \\ d_b = d, f_b = f}} z_{b,r,i} \leqslant 3 \qquad \forall i \in I, r \in [N], (d,f) \in [7]^2 : d \leqslant f$$

meaning that if there is a block b starting on day d of row r, ending on a day f (with $d \leqslant f$ ensuring that b ends on row r as well), there is at most one block b' starting between the days d and f on row r (b being such a block, there is no other $b' \neq b$). When there is no block between d and f, this means that there are at most three blocks starting between the days d and f on row r, which is always verified. In case 2, the constraints write:

$$\sum_{\substack{b' \in \widetilde{B}_i \\ d'_b \geqslant d}} z_{b',r,i} + 2 \sum_{\substack{b \in \widetilde{B}_i \\ d_b = d, f_b = f}} z_{b,r,i} \leqslant 3 \qquad \forall i \in I, r \in [N], (d,f) \in [7]^2 : d \geqslant f$$

meaning that if there is a block b starting on day d of row r, ending on day f (with $d \geqslant f$ ensuring that b ends on the row following r), there is at most one block b' starting between after the day d on row r (b being such a block, there is no other $b' \neq b$). When there is no block between d and f, this means that there are at most three blocks starting after the day d on row r, which is always verified.

In case 3, the constraints write:

$$\sum_{\substack{b' \in \widetilde{B}_i \\ d_{b'} \leqslant f}} z_{b',r+1,i} + 3 \sum_{\substack{b \in \widetilde{B}_i \\ d_b \geqslant f_b, f_b = f}} z_{b,r,i} \leqslant 3 \qquad \forall i \in I, r \in [1, N-1], f \in [7]$$

meaning that, when row r is not the last row of the roster for team i, if there is a block b starting on row r and ending on day f (with $d_b \geqslant f_b$ ensuring that b ends on the row following r), there is no block b' starting before the day f of row $r+1$. And:

$$\sum_{\substack{b' \in \widetilde{B}_i \\ d_{b'} \leqslant f}} z_{b',1,i} + 3y_{r,i} + 3 \sum_{\substack{b \in \widetilde{B}_i \\ d_b \geqslant f_b, f_b = f}} z_{b,r,i} \leqslant 6 \qquad \forall i \in I, r \in [N], f \in [7]$$

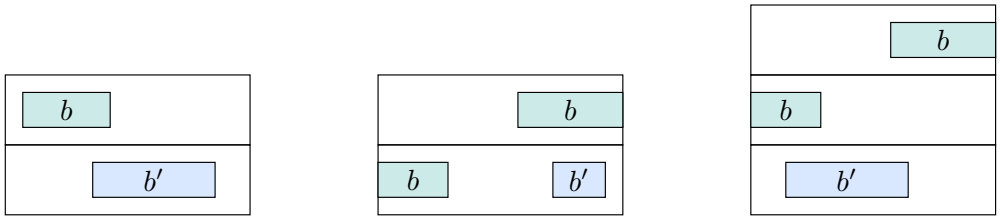
which is when the row r is the last row of the roster for team i.

The total number of constraints is polynomial in $|I|$, N, and $|T|$. □

We end this chapter with the following lemma.

Lemma 3.4.2. *Given blocks $b_1, \ldots, b_k$, there exists a roster covering $b_1, \ldots, b_k$, without adding any working days, and with a number of rows upper bounded by $13k$.*

The consequences of this result are manifold. First, its proof provides a simple greedy heuristic building an optimal solution to the crew rostering problem, given a set of optimal



(a) Case 1: the blocks b and b' cannot start on the same row.

(b) Case 2: the blocks b and b' cannot start on the same row.

(c) Case 3: the blocks b and b' cannot start on following rows.

Figure 3.6: An illustration of the three cases occurring for the constraint ensuring that blocks are not overlapping.

blocks per team (when the number of rows is not taken into account). This means in particular that the current multi-stage approach solving the crew rostering problem at SNCF is actually optimal. It also justifies the focus on the construction of optimal blocks in the subsequent chapters. Finally, it provides an upper bound $N = 13|T|$ on the number of rows. Indeed, the lemma will ever only be applied with blocks $b_1, \dots, b_k$ with each covering at least one new train. Using Proposition 3.4.1 with this upper bound completes the proof of existence of the ILP of Proposition 3.4.1, with a polynomial number of constraints. We however emphasize that this heuristic is purely technical and only aims to show that the focus can be made on finding optimal blocks. In practice, as explained in the last paragraph of Section 3.3.2, this heuristic is replaced by an ILP based resolution method (either the current algorithm for phase 4 or the alternative formulation from Proposition 3.4.1) which constrains the number of working days to the optimal value (found in phase 2) while minimizing the number of rows. Even when using the alternative formulation from Proposition 3.4.1, the bound $13|T|$ is in practice far from being tight. In the numerical results, a tighter bound is computed for the upper bound N via a simple heuristic not detailed in this thesis.

Proof of Lemma 3.4.2. We recall there is a difference between long and short on-calls, detailed in the paragraph on problem specificities at SNCF of Section 3.3.2. This proof consists in constructing a feasible roster with at most $13k$ rows and covering $b_1, \dots, b_k$ without adding any working days (only blocks of long on-calls, which contribute to a number of zero working days). This construction proceeds in two steps. In the first step, we build a "partial" roster with M_1 rows that satisfy rules R30 to R35. We insert blocks of long on-calls between consecutive blocks b_i and b_{i+1} using Figure 3.7 as a lookup table (see its caption for reading instructions). The same procedure is applied after b_k, using b_1 as the next block to close cyclically the partial roster. Since each block b_i spans at most two rows and each transition adds at most two additional rows, the partial roster has at most $M_1 \leqslant 4k$ rows. By inspection of Figure 3.7, the resulting partial roster satisfies rules R30 to R35. After b_k, two cases may appear depending on whether the last inserted long on-calls block overlaps the first row of the partial roster:

- The last long on-calls block overlaps the last and first rows, ending on a Monday, Tuesday or Thursday (these are the only possibilities in Figure 3.7). We call this the "overlap case" and the second step of the roster construction then uses Figure 3.8.
- No block overlaps the last and first rows. We call this the "non-overlap case" and the second step of the roster construction then uses Figure 3.9.

These two figures cover exhaustiverly all configurations produced by the first step.

In the second step, we complete the partial roster by adding $M_2 = 9k$ rows to satisfy additionally the rules R25 to R29. We insert only blocks of long on-calls using as lookup

tables Figure 3.8 in the overlap case and Figure 3.9 in the non-overlap case (see their captions for reading instructions). The full roster satisfies rules R30 to R35 by construction and has at most $M_1 + M_2 \leqslant 13k$ rows. It remains to verify rules R25 to R29.

- For rule R25, the number of rest days in the last $M_2 = 9k$ rows is lower bounded by $27k - 2$. The number of rest days in the first M_1 rows is lower bounded by $4k$ (there is at least 2 rest days before each block and at least one block of on-calls and one other block). Since $31k - 2 \geqslant 13k \cdot 7 \cdot \frac{116}{365}$, the rule is satisfied.
- For rule R26, the number of consecutive rest days in the last M_2 rows is lower bounded by M_2. The number of consecutive rest days in the first M_1 rows is lower bounded by M_1. Since $M_1 + M_2 \geqslant (M_1 + M_2) \cdot 7 \cdot \frac{52}{365}$, the rule is satisfied.
- For rule R27, the number of rest Sundays in the last $M_2 = 9k$ rows is lower bounded by $9k - 1$. The number of rest Sundays in the first M_1 rows is lower bounded by k. Since $10k - 1 \geqslant 13k \cdot 7 \cdot \frac{22}{365}$, the rule is satisfied.
- For rule R28, the number of consecutive rest days with a Sunday in the last $M_2 = 9k$ rows is lower bounded by $9k - 1$. The number of consecutive rest days with a Sunday in the first M_1 rows is lower bounded by k. Since $10k - 1 \geqslant 13k \cdot 7 \cdot \frac{14}{365}$, the rule is satisfied.
- For rule R29, the number of rest weekends in the last $M_2 = 9k$ rows is lower bounded by $9k - 3$. The number of consecutive rest days with a Sunday in the first M_1 rows is lower bounded by 0. Since $9k - 3 \geqslant 13k \cdot 7 \cdot \frac{12}{365}$, the rule is satisfied. □

3.5 Numerical experiments

3.5.1 Experimental setting and instances

The numerical experiments aim to compare the current phase 4 used at SNCF with the alternative model proposed in Section 3.4, when minimizing the number of rows. According to Lemma 3.4.2, we can take the upper bound $N = 13|T|$ for the alternative model. Since the performances of this model should improve with the quality of the bound, we use a simple heuristic providing a tighter upper bound, which value is reported in Table 3.4.

A third resolution method is also experimented. This method is based on an ILP very similar to the one of Section 3.4 (getting rid of the variables $y_{r,i}$), except that it looks for rosters with exactly R rows. By then proceeding with a binary search on R, starting with an upper bound N (which could be determined using the same heuristic as for the alternative model), we can solve an ILP for each value of R and keep the lowest value such that the ILP is feasible.

The instances of the crew rostering problem used here are made from the duties used in concrete rosters for each team of drivers for the planning of year 2024. The data on the composition of the duties is not available from these files. Therefore, the train covering constraint is replaced by a duty covering constraint in all of the ILPs.

The experiments have been conducted on a laptop with an Intel(R) Xeon(R) W-11855M CPU @ 3.20GHz processor. The programming language used is Java 18 and the solver used is CPLEX 22.11.

3.5.2 Results

The results for all instances are summarized in Table 3.4. The caption of the table provides the meaning of the columns.

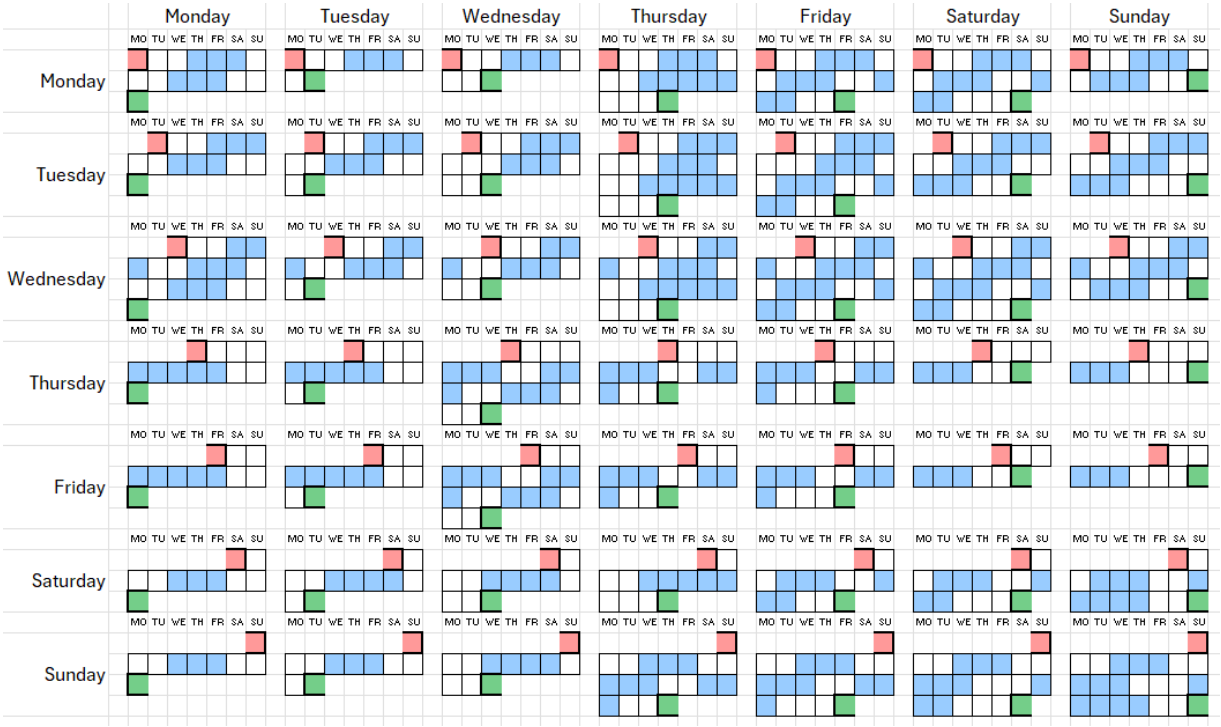


Figure 3.7: The cell in the row corresponding to the ending day of b_i and in the column corresponding to the starting day of b_{i+1} prescribes the block(s) of long on-calls to insert between b_i and $b_i + 1$. In this cell, we indicated in red the last day of block b_i, in blue the days of the block(s) of long on-calls to be added, in green the first day of block b_{i+1}, and in white the rest days between the blocks. By inspection, each cell contains at most three consecutive worked Sundays (rule R30, each cell starts with at most one consecutive worked Sunday and ends with at most two, taking into account the one potentially in the block b_{i+1}), has no single rest day, at least 2 rest days per row, and increases the row count by at most 4 (taking into account the potential additional row stemming from the block b_{i+1}).

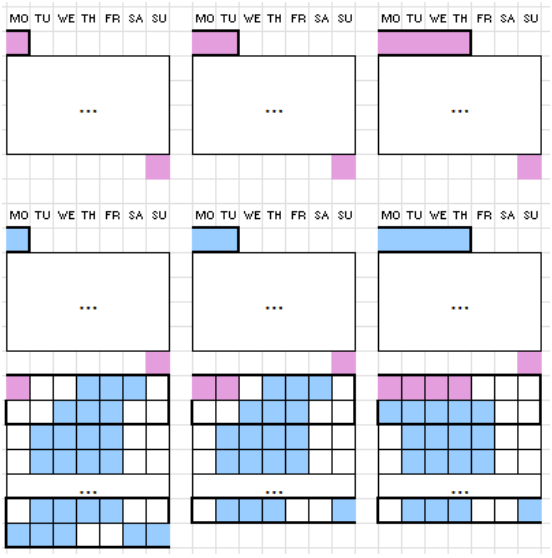


Figure 3.8: The column corresponding to the ending day of the overlapping block in the partial roster prescribes the blocks of long on-calls to append after the partial roster in the overlap case. By inspection of Figure 3.7, the overlapping block can only end on a Monday, Tuesday, or Thursday, hence the three columns of this figure. Each column represents the configuration of the partial roster on top and the final roster after the long on-call blocks insertion on the bottom. In each column, we indicate in purple the days of the overlapping block from its last Sunday to its ending day on the first row, in blue the days of the blocks of long on-calls to be added, and in white the rest days between blocks. The intermediate appended rows (represented by the dots) each contain one block of long on-calls from Tuesday to Friday.

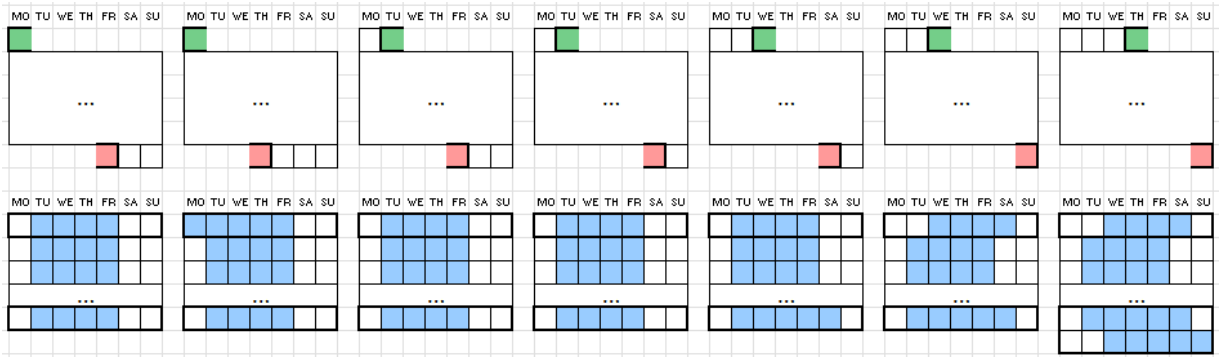


Figure 3.9: The column corresponding to the first day of the first block of the partial roster prescribes the blocks of long on-calls to append after the partial roster, in the non-overlap case. By inspection of Figure 3.7, all non-overlap cases fall into one of the columns of this figure. It is then exhaustive for this case. Each column represents the configuration of the partial roster on top and the final roster after the long on-call blocks insertion on the bottom. In each column, we indicate in green the first day of the first block (which is either b_1 or a block of long on-calls added in the first step between b_k and b_1), in red the last day of the last block, in blue the days of the blocks of long on-calls to be added, and in white the rest days between blocks. The intermediate appended rows (represented by the dots) each contain one block of long on-calls from Tuesday to Friday.

Instance	Number working days	Current algorithm		Alternative ILP			Alternative ILP: binary search				
		Time(s)	Rows	Rows UB	Time(s)	Rows	Rows UB	Rows LB	Time(s)	Iterations	Rows
AF	34	**1.0**	8	17	3.3	8	17	6	7.5	4	8
AMB-2	19	0.7	5	10	0.9	5	10	2	**0.5**	3	5
BGW-2	48	**2.5**	11	15	2.9	11	15	8	8.2	3	11
BGW-3	42	12.8	10	12	**2.5**	10	12	5	4.9	3	10
BYE	18	0.1	4	6	0.3	4	6	3	**0.0**	2	4
CHA	7	0.1	3	5	0.2	3	5	1	**0.0**	2	3
DPY-1	22	**0.2**	6	10	0.5	6	10	3	0.6	3	6
DPY-4	43	56.6	13	14	**2.7**	13	14	6	17.5	4	13
GSY-1	23	0.3	7	9	0.4	7	9	3	**0.2**	3	7
HDE	47	**1.0**	11	14	4.6	11	14	7	14.4	3	11
LAR	60	120.0	(gap=6.3%) 15	16	**5.9**	15	16	8	11.7	4	15
LDV	48	74.3	13	20	5.2	13	20	7	**2.8**	4	13
MAS	27	**0.5**	7	12	2.0	7	12	5	4.2	3	7
PS	17	0.4	5	6	0.5	5	6	2	**0.3**	3	5
PTH	19	**0.5**	5	10	0.8	5	10	2	**0.5**	3	5
RES	15	**0.2**	4	8	0.5	4	8	2	0.3	3	4
SGF	39	12.4	10	17	**2.5**	10	17	6	8.0	4	10
SO	44	3.9	10	15	**1.8**	10	15	7	7.2	3	10
THL-1	39	4.5	10	12	**1.7**	10	12	5	2.4	3	10
THL-2	28	3.0	7	10	**1.1**	7	10	3	4.0	3	7
V	12	**0.1**	4	7	0.3	4	7	2	**0.1**	3	4
VAI	27	**0.2**	6	10	0.6	6	10	4	0.6	3	6
VSG-REF	25	9.8	7	9	**0.6**	7	9	5	1.3	2	7
WPY-2	41	17.2	10	14	**3.3**	10	14	6	8.2	3	10
AMB-1	117	120.0	/	33	120.0	(gap=7.4%) 27	33	18	146.2	4	(gap=7.4%) 27
CLE	57	6.6	14	23	**5.9**	14	23	10	18.0	4	14
MOD	82	**2.3**	19	31	28.5	19	31	13	23.2	4	19
PPN	69	**2.2**	16	21	4.9	16	21	12	20.7	4	16
WPY-3	128	**1.5**	34	39	15.1	34	39	21	76.4	5	34

Table 3.4: Numerical experiments on the freight instances. The column "Row UB" refers to the upper bound on the number of rows computed through a simple heuristic. The column "Row LB" refers to the lower bound on the number of rows computed by taking the maximal number of duties happening on the same day of the week. The reported time takes into account only the last step of the crew rostering algorithm. A time limit of 120 s has been set for the ILP solver (for the binary search method, in particular for the instance AMB-1, this is the time limit for each call to the ILP solver). When only a feasible solution is found under the time limit, the gap is reported. When no feasible solution is found, we leave the corresponding cell empty.

3.6 Comments

The current algorithm, the alternative ILP of Proposition 3.4.1, and the binary search based method have performances that vary depending on the instances. Some instances are solved with comparable times (e.g., AMB-2, BYE, CHA, V, VAI, etc.). Some instances are solved faster with the current algorithm (e.g., MOD, PPN, WPY-3). Some instances are solved faster with the alternative methods compared to the current algorithm (e.g., BGW-3, DPY-4, LAR, SGF, WPY-2). There is even one instance for which the current algorithm is not able to find any feasible solution (possibly due to the inefficient no-good cuts) but the alternative methods are finding feasible solutions, though not optimal. For this instance, further experiments were conducted with an increased time limit of 600 s, with the same results as reported in the table.

While no method clearly outperforms the other two on all instances, the alternative model of Proposition 3.4.1, with the upper bound N provided by a simple heuristic, seems more consistent across instances. Even though we would recommend the alternative model for this reason, the constraints at SNCF change from time to time, and the current model is easier to adapt to such changes, which makes the alternative model less appealing for a potential replacement of the the current model.

Chapter 4

Integrating crew scheduling and crew rostering for rail freight

***

This problem, the methodology, and the numerical results were established in "Integrating crew scheduling and crew rostering for rail freight with train delays," H. Gachet, F. Meunier, and J. Pouzet, *9th International Symposium on Combinatorial Optimization, ISCO 2026* [42].

4.1 Context and contributions . . . 54
4.2 Literature review: integrating crew scheduling and crew rostering . . . 54
4.3 Problem formulation . . . 56
4.4 Model . . . 56
4.4.1 Mathematical program . . . 56
4.5 Solution method . . . 57
4.5.1 Column generation . . . 57
4.5.2 Pricing subproblem as a resource-constrained shortest path problem . . . 58
Resource-constrained shortest path problems: general setting . . . 58
Specialization to the pricing problem . . . 58
Label correcting algorithm . . . 59
4.5.3 Improvements . . . 61
Early feasibility check . . . 61
Warm-starting the label correcting algorithm . . . 61
Set of bounds to discard paths . . . 61
Variable dominance . . . 63
Improving the quality of integer solution . . . 63
4.6 Numerical experiments . . . 64
4.6.1 Experimental setting and instances . . . 64
4.6.2 Results . . . 64
4.6.3 Comments . . . 65

4.1 Context and contributions

This chapter aims to solve in an "integrated" manner the crew scheduling and crew rostering problems at SNCF. We recall that the crew scheduling problem refers to the step in the planning process aiming to build duties from tasks given over a defined time period; the crew rostering problem aims to build (cyclic) rosters from these duties. These two problems are typically solved one after the other, with the output of crew scheduling as the input of crew rostering. We then say that the crew management, consisting of the two problems as a whole, is tackled sequentially, as opposed to an integrated approach addressing the two problems simultaneously.

An important drawback of sequential approaches is that solving the two problems independently generally fails to yield a globally optimal solution. This is a well-identified issue in the operations research community, and this issue has indeed received dedicated attention in the literature [25]. This chapter therefore aims to improve upon the sequential approach currently used at SNCF by handling the two stages simultaneously, so as to reduce suboptimality.

This "integrated" approach, described in Section 5.5, relies on a column generation method that incorporates all working rules. To do so, we overcome the twofold challenge of solving a resource-constrained shortest path problem that is non-linear and whose dimension is high.

This integrated approach consistently and significantly reduces the number of working days across all SNCF instances compared to the current method, as presented in Section 4.6.

4.2 Literature review: integrating crew scheduling and crew rostering

The crew scheduling problem and the crew rostering problem, as they are already complex problems, are often solved sequentially in order to obtain rosters. However, a few articles in the literature propose resolution methods to handle the two problems in an integrated manner. A number of overviews of the literature on the integration of the crew scheduling and rostering problems have been performed [48, 62, 91]. They highlight that only a few articles are applied to the railway industry, in comparison to the airline industry. Among these, various resolution methods are used, sometimes dealing only with special cases (predefined work patterns, constrained number of workers, absence of overnight trains, etc.): heuristics [24, 33], metaheuristics [83], Benders decomposition methods [13, 63], Lagrangian relaxation [98], column generation, and branch-and-price schemes [37, 59, 99].

Among these methods, we now focus on those that are exact, or that at least provide a certified lower bound on the optimal value—a property required to assess solution quality and shared by our own approach. Most existing approaches to the integrated crew management problem tackle the problem under assumptions that do not fully capture cyclic freight operations. This observation motivates the integrated model of this chapter.

Mesquita et al. [63] address the integrated vehicle routing, crew scheduling, and crew rostering problem, where rosters are non-cyclic and respect a predefined cyclic days-off pattern. Their goal is then to simultaneously determine a minimum cost set of vehicle schedules covering all trips, then organized into a minimum cost set of duties, and a minimum cost roster with balanced workload over a planning horizon of 7 weeks. They formulate this problem as an ILP and solve it using Benders decomposition. They have on one side the rostering and on the other an integrated vehicle and crew scheduling (handled via a column generation algorithm with resource-constrained shortest path pricing), which are linked through the Benders subproblem. They implemented branching strategies to obtain integer solutions in a short time. Borndörfer et al. [13] propose a similar approach to the integrated problem to increase drivers satisfaction. They are investigating the construction of non-cyclic rosters

with fixed number of rows with the main objective of maximizing driver satisfaction while taking into consideration a penalty on the deviation from a work range target. The driver satisfaction measure aggregates multiple terms such as: resting weekends, having regular hours, and balancing stress-inducing duties over the weeks. They solve this problem by relying on *duty templates* to generate iteratively coupling Benders cuts, then added in the crew scheduling in a Lagrangian relaxation fashion. Lin and Tsai [59] present a mathematical formulation to integrate the crew scheduling and crew rostering problems for trainmasters in passenger railway transportation, by generating either cyclic or non-cyclic rosters. In both versions, they minimize the number of trainmasters required to perform all mandatory tasks. To solve this problem, they describe a branch-and-price-and-cut algorithm where duties are generated with the pricing subproblem. Zhou et al. [98] formulate the integrated metro crew scheduling and rostering problem as a minimum-cost multi-commodity network flow model, in which each crew member is treated as a commodity flowing through an acyclic directed multi-layer time–space network where each layer represents a duty. A key structural assumption is that each crew member must be assigned exactly one duty or one day-off per day, so the roster is a day-indexed sequence of assignments over the planning horizon. The problem is solved with a Lagrangian relaxation heuristic that decomposes into independent shortest path subproblems for each crew. However, the model considers only one duty type and one days-off pattern — a crew has at least one day-off during the three-day planning horizon — and the number of crews is fixed a priori as the number of commodities in the network. Zhou et al. [99] extend the above work to a more general metro crew management problem that simultaneously considers multiple day-off patterns, duty types, and rostering schemes, while retaining the same day-indexed assignment structure. The problem is solved with a branch-and-price algorithm. However, the day-off patterns remain drawn from a pre-specified discrete set, and the number of roster lines is still fixed in advance. Feng et al. [37] propose an ILP formulation for the integrated crew scheduling and rostering problem applied to urban rail transit, where workers follow a typical work pattern of 3 work days followed by a rest day, and each worker is assigned exactly one duty or one day-off per day. They develop a column generation based algorithm with a resource-constrained shortest path pricing subproblem generating duties.

The problems addressed in the literature above differ from ours in several fundamental ways that we now detail. With the exception of Lin and Tsai [59], who propose a model generating either cyclic or non-cyclic rosters, all existing integrated approaches produce non-cyclic rosters [13, 63] or operate over a short fixed planning horizon without an explicit cyclicity requirement [37, 98, 99]. In our problem, rosters are strictly cyclic: a single sequence of blocks is designed for each team and then repeated in a cyclic manner. Cyclic rostering is operationally attractive in freight rail because it guarantees predictability and regularity, but it introduces annual constraints that couple all rows of the rosters and cannot be handled row by row. A recurring simplification in the literature is to restrict the search to a fixed days-off pattern. Mesquita et al. [63] require rosters to respect a single predefined cyclic days-off pattern given as input. Zhou et al. [98] restrict to a single days-off pattern (at most one rest day within a three-day horizon). Borndörfer et al. [13] fix the rest structure implicitly through pre-defined duty templates. Feng et al. [37] assume a fully prescribed work-rest cycle of 3 consecutive working days followed by a fixed rest day. In our problem, no days-off pattern is given: the placement and number of rest days emerge entirely from the optimization. This flexibility is essential in rail freight, where the irregular daily timetable makes it impossible to commit to a fixed rest rhythm over a full roster. Most existing integrated approaches [13, 37, 59, 63, 98, 99] formulate the rostering problem by requiring each driver to be assigned exactly one duty or one day-off per day. This day-indexed structure inherently restricts work units to a single calendar day. In freight rail, this assumption does not hold: trains have to be operated day and night, thus crews might have multiple duties in a day. For this reason, as to align with the rules and regulations vocabulary, we introduced

blocks—sequences of consecutive duties and on-calls spanning up to six calendar days—as work units, which has no equivalent in any of the existing integrated approaches reviewed above.

In this work, we propose a column generation based approach tackling the integrated problem for the industrial case at SNCF. This approach overcomes the restrictive special cases of the literature, which made the previously developed algorithms difficult to use for our application. Our approach can also be extended to an exact method (see Section A.3 for instance).

4.3 Problem formulation

We refer to the integrated crew scheduling and crew rostering problem as the *crew management problem*. An instance of the crew management problem at SNCF has the same input data as instance of the crew scheduling problem (see Section 3.3.1 for details): it comes with a collection T of trains to be driven (with given origin, destination, departure time, and arrival time), along with a collection I of teams.

The goal is to assign each train to a team, which will be in charge of driving this train every week. (Each train is driven by a single driver, and in the final roster of the team, the drivers will cyclically drive the train over the year.) The assignment process has to comply with working rules, summarized in Tables 3.1, 3.2, and 3.3, defined at the level of duties, blocks and rosters, whose definitions were provided in Sections 3.3.1 and 3.3.2.

As highlighted in Chapter 3, the cost of a roster is its number of working days. Using Lemma 3.4.2, there is an easy greedy algorithm converting any sequence of valid blocks to a valid roster, with the same total number of working days. Thus, rather than searching for a set of optimal rosters, we can equivalently search for a set of optimal blocks; the arrangement of these blocks within rosters can then be handled by the greedy algorithm.

The crew management problem consists of finding a set of blocks that covers every train and minimizes the total number of working days, with each block assigned to a team of drivers. The crew management problem can be formally written as:

Crew management (CM)
Input. Collection T of trains on a typical week, collection I of teams of drivers.
Output. Covering of trains by blocks with minimum number of working days, each block assigned to a team.

4.4 Model

4.4.1 Mathematical program

The crew management problem at SNCF can be modeled as a mixed integer linear program, where w_b is the number of working days in the block b, and where B_i is the set of all blocks that can be performed by team i. This means that the set B_i is defined according to the residency and the skills of i.

$$\begin{aligned} \text{minimize} \quad & \sum_{i \in I} \sum_{b \in B_i} w_b x_{b,i} & & \\ \text{s.t.} \quad & \sum_{i \in I} \sum_{b \in B_i : t \in b} x_{b,i} \geqslant 1 & \forall t \in T & \qquad \text{(Covering CM)} \\ & x_{b,i} \in \{0,1\} & \forall i \in I, \forall b \in B_i \,. & \end{aligned}$$

The variables $x_{b,i}$ take the value 1 if the block $b \in B_i$ is selected for the team $i \in I$.

4.5 Solution method

4.5.1 Column generation

Directly enumerating all feasible blocks is computationally impossible, which motivates a column generation approach.

We propose an algorithm for the resolution of the problem (Covering CM). The algorithm starts by computing an optimal solution of the linear relaxation of (Covering CM) using column generation, where the blocks define the columns. The general algorithm for column generation is given in Algorithm 2, and details are given in Appendix A. Then the restricted version of (Covering CM), where the B_i's are replaced by the subsets B'_i of blocks generated during the column generation, is solved by using an off-the-shelf ILP solver. We emphasize that our approach is not exact and is at this stage a "matheuristic": there might exist blocks in B_i that are required in every optimal solution and that are not in the final B'_i. We are however certain to finish with a feasible solution (when the problem is feasible) since rounding up feasible fractional solutions leads to feasible integer solutions. We note that the overall algorithm is standard; as is common for such approaches, the algorithmic contribution lies in the pricing subproblem, described in Section 4.5.2.

We describe a bit more precisely the column generation used to solve the linear relaxation of (Covering CM). Specializing the setting of Appendix A to (Covering CM), the set of columns is the set of feasible blocks $\bigcup_{i\in I} B_i$. The restricted master problem writes, for subsets $B'_i \subseteq B_i$:

$$\begin{aligned} \text{minimize} \quad & \sum_{i\in I}\sum_{b\in B'_i} w_b x_{b,i} & & \\ \text{s.t.} \quad & \sum_{i\in I}\sum_{b\in B'_i : t\in b} x_{b,i} \geqslant 1 & \forall t \in T & \qquad \text{(RMP(Covering CM))} \\ & x_{b,i} \in [0,1] & \forall i \in I, \forall b \in B'_i\,. & \end{aligned}$$

The first step of the algorithm (line 1) consists here of initializing the B'_i in such a way that all t are covered. This is done with a greedy heuristic, and it is enough to start with a first feasible solution. During the optimization phase, the LP solver is used to identify primal-dual feasible bases (line 11) and the pricing subproblem is solved so as to identify blocks with negative reduced costs (line 12). This is done using Algorithm 1 described in Section A.2.1.

As is common in column generation, the main challenge lies in efficiently solving the pricing subproblems. In our case, there are as many pricing subproblems as teams, written as follows for a team i, where $\lambda_t \geqslant 0$ denotes the dual value associated with the covering constraint for train t:

$$\begin{aligned} \text{minimize } & w_b - \sum_{t\in b} \lambda_t \\ \text{s.t. } & b \in B_i\,. \end{aligned}$$

In other words, for a given team i, we seek to find a block satisfying all rules of Tables 3.1 and 3.2, such that the reduced cost of the associated variable $x_{b,i}$ in (RMP(Covering CM)) is negative.

Each of these subproblems takes the form of a non-linear resource-constrained shortest path (RCSP) problem on an acyclic graph. As explained in the next subsection, we address it with a label-correcting algorithm enhanced with dominance rules and bounding strategies.

4.5.2 Pricing subproblem as a resource-constrained shortest path problem

The pricing problem is the algorithmic core of the method: it generates new feasible blocks with negative reduced cost while enforcing operational constraints.

Resource-constrained shortest path problems: general setting

The RCSP problem, in a quite general setting, can be described as follows; see, e.g., Parmentier [72] for further detail. We are given an acyclic digraph $D = (V, A)$, with two particular vertices o and d, and a partial ordered set $(R, \preccurlyeq)$ of resources, with unique minimal and maximal elements denoted by $\hat{\mathbf{0}}$ and $\hat{\mathbf{1}}$. Each arc a is equipped with a non-decreasing *extension function* $f_a \colon R \to R$. We are also given a non-decreasing *cost function* $c \colon R \to \mathbb{R} \cup \{+\infty\}$. The *resource of a path* P is defined inductively by the relation $r_P \coloneqq f_a(r_{P\setminus\{a\}})$, where a is the last arc of P. The map f_a describes the evolution of the resource of a path when the arc a is added to it. The resource of an empty path reduced to a single vertex is $\hat{\mathbf{0}}$. A path P is *feasible* if $r_P \neq \hat{\mathbf{1}}$. The RCSP problem requires to compute a feasible path P with minimum $c(r_P)$.

Specialization to the pricing problem

The pricing subproblem associated with a team i for the problem (Covering CM) can be modeled as a RCSP problem on an acyclic digraph D. The vertices of the graph are the trains compatible with the team i, along with a source vertex o and a sink vertex d. There is an arc between two vertices when the corresponding trains t and t' can be driven successively within a same block (which can be read off from the arrival time $\delta_t + \tau_t$ of train t, departure time $\delta_{t'}$ of train t', and their locations). In some cases, there are actually several copies of this arc, each copy corresponding to an intermediate location where some long break can be taken. There are moreover arcs from the source o to every train, and arcs from every train to the sink d. One of the working rules is that a block expands over at most six days. This is the reason why, up to duplicating some train, we can assume that D is acyclic.

With this definition, every block $b \in B_i$ encodes a single o-d path, but every o-d path does not necessarily correspond to a valid block. Indeed, as we have mentioned, for a block to be valid, there are numerous working rules to be checked, which take into account elements such as the number of working days in a block, the driving time since the last long break, etc. Moreover, the cost of a path has to be the reduced cost of the corresponding block (up to constant terms). The introduction of carefully chosen poset of resources $(R, \preccurlyeq)$, extension functions f_a, and cost function c, as we explain in more detail now, is so that, conversely, every feasible path P corresponds to a valid block b. With the construction below, we have for an o-d path P and the corresponding block b

$$r_P \neq \hat{\mathbf{1}} \quad \Longleftrightarrow \quad b \text{ is valid}\,,$$

and in the case $r_P \neq \hat{\mathbf{1}}$, we have $c(r_P) = \text{reduced-cost}(x_{b,i})$.

For our problem, we define a resource, i.e., an element of R, as a vector where all components are non-negative real numbers. Each component is associated either with one or multiple working rules, or with an element involved in the computation of the cost function (such as the number of working days, or the dual values). Due to non-linearities, some rules require to keep track of information on the last completed duty, which calls for the introduction of two components for a same rule. The rules R3 to R6 and R9 of Table 3.1 are tracked using multiple components as mentioned above. Each component comes with a partial order, which is actually the usual total order of $\mathbb{R}$ for all components. The partial order equipping R is then just the product order.

For a team i with residency L_i, we illustrate the definition of f_a for the 6th and 7th components of the resource vector, used for the rules R3, R4, and R5 of Table 3.1. The 6th component corresponds to the work range in the current partial duty. The 7th component corresponds to the same as the 6th, except when a has a long break: it then corresponds to the work range in the duty just completed. We have, with $a = (t, t')$:

$$[f_a(r)]_6 = \begin{cases} \Delta^1_{L_i,t'} & \text{if } a \text{ has a long break at } L_i, \\ \Delta^2_{L,t'} & \text{if } a \text{ has a long break at a location } L \neq L_i, \\ [r]_6 + \Delta^3_{t,t'} & \text{otherwise,} \end{cases}$$

and

$$[f_a(r)]_7 = \begin{cases} [r]_6 + \Delta^1_{t,L_i} & \text{if } a \text{ has a long break at } L_i, \\ [r]_6 + \Delta^2_{t,L} & \text{if } a \text{ has a long break at a location } L \neq L_i, \\ [r]_6 + \Delta^3_{t,t'} & \text{otherwise} \end{cases}$$

where $\Delta^1_{t,L_i}, \Delta^1_{L_i,t'}, \Delta^2_{t,L}, \Delta^2_{L,t'}, \Delta^3_{t,t'}$ are buffer times taking into account possible relocations, start-of-duty, and end-of-duty (that depend on the relocations, as mentioned in R11-12 of Table 3.1, but also the type of long break).

Denote by $\lambda_t \geqslant 0$ the dual value associated with the constraint of covering t. The cost function c counts the difference between the number of working days and the partial sum $\sum_t \lambda_t$, where t ranges over the trains met so far. It takes the value $+\infty$ when evaluated at $\hat{\mathbf{1}}$. When the path reaches the vertex d, this accounts exactly for the reduced cost of the variable $x_{b,i}$, i.e., $w_b - \sum_{t \in b} \lambda_t$.

An example of the extension of resources is given in Figure 4.1.

Label correcting algorithm

We solve this RCSP problem with a standard algorithm, the label correcting algorithm. The pseudo-code describing it is given in Algorithm 1, which follows the approach by Parmentier [72]. It implicitly enumerates all possible *o-d* paths. It is implicit since

(i) it discards partial paths that cannot be extended to a path better than the current best path. This relies on bounds β_v attached to the vertices v and that are computed beforehand (as explained, e.g., in the aforementioned paper by Parmentier).

(ii) it maintains only paths that are non-dominated. A path P *dominates* a path Q if $r_P \preccurlyeq r_Q$.

These two ingredients are present on line 11. The bounds β_v belong to the set of resources $(R, \preccurlyeq)$ and are such that $\beta_v \preccurlyeq r_Q$ for every v-d path Q. In particular, $\beta_v = \hat{\mathbf{0}}$ works as a (low-quality) bound. In order to obtain tighter bounds, we use a "backtrack" dynamic programming procedure to compute bounds at every node [70]. This relies on meet computations on resources. In our application, the meet is taken as a componentwise minimum because $\preccurlyeq$ is a product order.

The use of dominance to discard paths (as in (ii) above) is sensitive to the number of resource components and becomes less efficient when it becomes large. We expect thus dominance to be not that useful here (complementary results are provided in Section 4.6). In contrast, efficiency of bounds (as in (i) above) is not directly sensitive to the number of resource components.

The notation $\oplus$ refers to the way of "merging" a resource and a bound [85]. This operator verifies $r_P \oplus r_Q \preccurlyeq r_{P+Q}$ for every vertex v, every o-v path P, and every v-d path Q. For a path P with last vertex v and a bound β_v at vertex v, the resource $r_P \oplus \beta_v$ provides an under-estimation of the lowest resource for an *o-d* path starting with partial path P.

This algorithm can easily be turned into an efficient heuristic for computing feasible paths with negative cost, i.e., blocks with negative reduced cost.

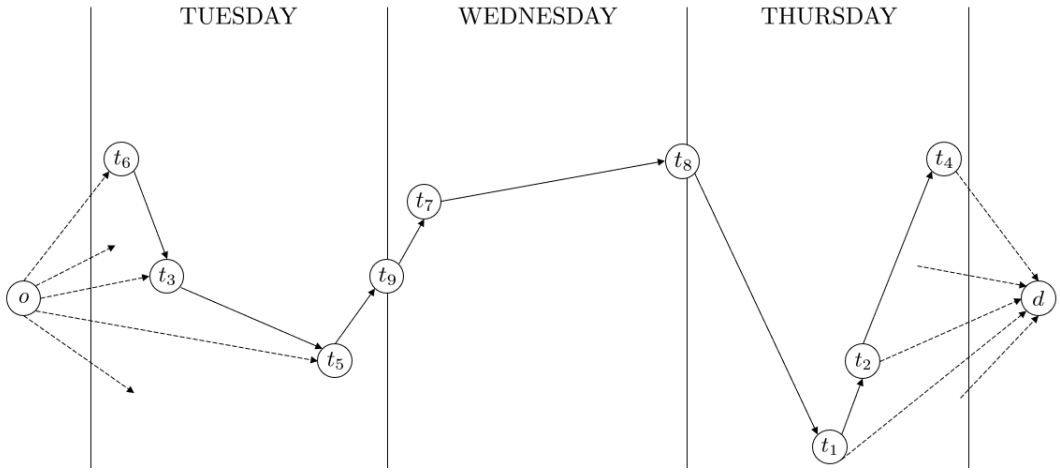


Figure 4.1: Example of a graph D where only the path corresponding to the block of Figure 3.3 is made explicit (the last duty in this block being the one of Figure 3.2). Let $P_1 := o, t_6, t_3, t_5, t_9, t_7, t_8, t_1$ be the path from o to t_1 using the arcs explicited in the graph. The 1st component of r_{P_1} measuring the number of working days in the current partial block is 4. The 6th component of r_{P_1}, measuring the work range in the current partial duty (covering train t_1), is 1h53. The 7th component of r_{P_1}, measuring the work range in the duty just completed (covering train t_8), is 8h13. Let P_2 be the path obtained from P_1 by adding the arc from t_1 to t_2. The 1st component of r_{P_2} is 4. The 6th and 7th components of r_{P_2}, which consider the duty covering trains t_1 and t_2, have the same value equal to 2h12. Let P_3 be the path obtained from P_2 by adding the arc from t_2 to t_4. The 1st component of r_{P_3} is 4. The 6th and 7th components of r_{P_3}, which consider the duty covering trains t_1, t_2, and t_4, have the same value equal to 8h04. Let P_4 be the path obtained from P_3 by adding the arc from t_4 to d. The 1st component of r_{P_4} is 5 (according to rule R20, the last duty must be followed by an on-call, counted when the path is completed). The 6th and 7th components of r_{P_4} have the same value equal to 8h17.

Algorithm 1 Label correcting algorithm for resource-constrained shortest path problem

Require: an acyclic digraph $D = (V, A)$, origin vertex o, destination vertex d, bounds $(\beta_v)_{v \in V}$;

1: `P_best` $\leftarrow$ `undefined`, $r_{\texttt{undefined}} \leftarrow \hat{\mathbf{1}}$, $\mathcal{P} \leftarrow$ {empty path reduced to o}, $D_v \leftarrow$ {empty path reduced to o} for all $v \in V$;
2: **while** $\exists P \in \mathcal{P}$ **do**
3: $\quad \mathcal{P} \leftarrow \mathcal{P} \setminus \{P\}$;
4: $\quad v \leftarrow$ last vertex of P;
5: $\quad$ **if** $v = d$ and $c(r_P) < c(r_{\texttt{P_best}})$ **then**
6: $\quad\quad$ `P_best` $\leftarrow P$;
7: $\quad$ **else**
8: $\quad\quad$ **for all** $a \in \delta^+(v)$ **do**
9: $\quad\quad\quad Q \leftarrow P + a$;
10: $\quad\quad\quad w \leftarrow$ last vertex of Q;
11: $\quad\quad\quad$ **if** $c(r_Q \oplus \beta_w) < c(r_{\texttt{P_best}})$ and Q not dominated by a path in D_w **then**
12: $\quad\quad\quad\quad$ remove from D_w and $\mathcal{P}$ all paths dominated by Q;
13: $\quad\quad\quad\quad$ add Q to D_w and $\mathcal{P}$;
14: **return** `P_best`;

4.5.3 Improvements

This section presents additional algorithmic steps, implemented in the final algorithm, aiming to accelerate the pricing subproblem resolution or generating better variables for the linear relaxation and the ILP (Covering CM).

Early feasibility check

First, we implement an early partial path feasibility check right after line 8 of Algorithm 1. This consists in checking whether some components of $f_a(r_P)$ already violate a rule, without computing the full extension. This is valid whenever such a violation is irreversible, in the sense that no extension can restore feasibility, which is the case for most components in our application.

Such an early check could be a speed-up for two reasons. The most obvious is that it prevents an unnecessary call to the full extension function. The second reason is that infeasibility is not always caught by the bounding strategy: since the bounds $(\beta_v)_v$ underestimate the true resource of any completion, a partial path that is already infeasible with respect to some rules may still satisfy $c(r_{P+a} \oplus \beta_w) < c(r_{P_best})$ and pass the bound-based test of line 11. Its infeasibility is then only detected later, possibly after several further extensions. To avoid wasting the time spent exploring the intermediate infeasible partial paths, the early feasibility check discards such paths as soon as they become infeasible.

Warm-starting the label correcting algorithm

The label correcting Algorithm 1 can be improved by warm-starting the collection of paths to explore $\mathcal{P}$ in line 1. Note that if $\mathcal{P}$ is replaced in this line by a bigger set (made of feasible partial paths) that still contains the empty path reduced to o, the algorithm still returns a shortest path verifying all constraints. The goal is to start the exploration from partial paths known to be feasible in the hope of finding a feasible o-d path faster.

Himmich et al. [49] suggest to use the basic variables of the last restricted master problem for such a warm-starting procedure. They call this the *label storing procedure*. Denote by $\overline{\mathcal{P}}$ the paths associated with basic variables for the last RMP solved. The idea behind this label storing procedure is to replace the initialization of set $\mathcal{P}$ in line 1 of Algorithm 1 by the following:

$$\mathcal{P} \leftarrow \bigcup_{\substack{P \in \overline{\mathcal{P}} \\ Q \text{ subpath of } P \text{ from } o \text{ to } v \neq d}} Q\,.$$

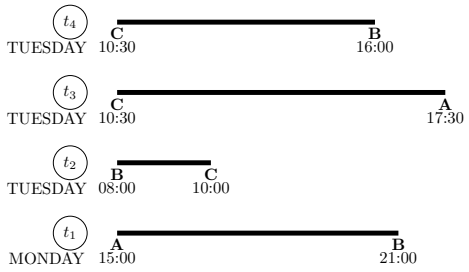
Note that this could be slightly improved by adding to Q any arc $a \in \delta^+(v)$ such that $a \notin P$ (checking feasibility and dominance). Also, the empty path reduced to o still belongs to $\mathcal{P}$. In practice, we bound the number of such paths added for the warm-start.

This procedure helped us gain 3 h2 min for the column generation algorithm on the instance with 1,815 trains. On the instance with 926 trains however, we lost 19 min for the column generation algorithm.

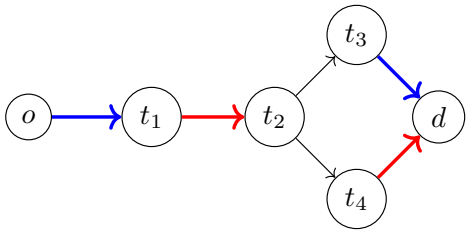
Set of bounds to discard paths

As explained by Parmentier [70], a set of bounds instead of a single bound could be used to discard more efficiently paths in the dominance-based discarding procedure (i). He comes up with two techniques to replace the current lower bound test (line 11) and compute sets of bounds accordingly, namely the *clustered lower bounds test* and the *conditional lower bounds test*. We refer to the original article [70] for more details.

Regarding the clustered lower bounds test, since the graphs considered for the pricing subproblem are typically large, the use of a blown-up larger graph $\mathcal{D}$ for the clustering



(a) An example with an instance with four trains, and a single team with residency A. We assume that no additional relocation tasks are available.



(b) An illustration of the graph D for a team with residency A. The arcs in red are the ones with a long break at a location different from the residency. The arc in blue has a long break at the residency. The arcs in black represent the successions of the trains within a same duty.

Figure 4.2: An illustration of the efficiency of multi-bounds instead of a single bound. Combining rules R4 of Table 3.1 and R13 of Table 3.2, there is no feasible extension of the path $P \coloneqq o, t_1, t_2$. For the single bound computation on vertex t_2, the backtrack dynamic programming procedure computes the meet between resources coming from vertex t_3 and from vertex t_4. This meet results in a single bound β_{t_2} with a work range lower bound of 6 hours and the lower bound on the resource component counting the duties before returning to A equal to 0. This is not enough to say that the path can be extended as $r_P \oplus \beta_{t_2} \neq \hat{\mathbf{1}}$ and infeasibility will only be verified at the next extension step, for the two paths o, t_1, t_2, t_3 and o, t_1, t_2, t_4. We now follow the proposed multi-bound strategy with the number of duties until a return to residency as the criterion. We then have two bounds on vertex t_2: first $\beta^0_{t_2}$ using the meet on the possible path completions with zero duties until a return to A, then $\beta^1_{t_2}$ using the meet on the possible path completions with one duty until a return to A. The first $\beta^0_{t_2}$ then has a work range lower bound of 7 hours 30 minutes and the lower bound on the resource component counting the duties before returning to A equal to 0. The second $\beta^1_{t_2}$ has a work range lower bound of 6 hours and the lower bound on the resource component counting the duties before returning to A equal to 1. None of these bounds allow a feasible completion of the current path as $r_P \oplus \beta^0_{t_2} = r_P \oplus \beta^1_{t_2} = \hat{\mathbf{1}}$.

subroutine on the vertices of the graph D might be too costly. Moreover, as the cost function is non-linear, the use of a morphism for construction of bounds in the context of the conditional lower bounds test is not straightforward.

However, these methods inspired us to establish problem-specific multi-bounds, fast and easy to compute via the same kind of dynamic programming procedure as for the single bound, but more efficient to discard unworthy paths early on in the exploration process. We can keep track of multiple bounds on each vertex using a predefined partitioning criterion (for example the number of duties until a return to residency). For each vertex, the computation of the multi-bound is done by backtracking separately each component, depending on the value of the criterion in the remaining forthcoming path.

Figure 4.2 illustrates the efficiency of such sets of bounds, with the problem-specific sets definition relying on rule R13 of Table 3.2. For the final algorithm, larger sets are defined following the same idea using multiple criteria, established from information on:

- the last day,
- the presence of a short break,
- the presence of a long break.

These criteria were chosen based on numerical experiments.

Variable dominance

The goal of this section is to establish a dominance rule for variables for the master problem. Indeed, even though there is a dominance-based discarding procedure in the label correcting algorithm, this considers the partial order $\preccurlyeq$. Therefore, as an output of the label correcting algorithm when multiple paths with negative reduced cost are generated, there might be two paths where one covers at least the trains covered by the other, while having a smaller or equal cost. This is mainly due to the presence of parallel arcs in the graph. Lemma 4.5.1 states that only one of such paths could be kept for the master problem. The implementation of this dominance rule in practice makes it possible to find integer solutions faster and with better quality in the last step of our algorithm.

Lemma 4.5.1. *Let $i \in I$ and $b, b' \in B_i$ be such that:*

- $w_b \leqslant w_{b'}$,
- $t \in b' \Rightarrow t \in b$.

Then there exists an optimal solution to (Covering CM) *with $x_{b',i} = 0$.*

Proof. Let $x^\star$ be an optimal solution to (Covering CM). Let then $\bar{x}$ be defined as:

$$\bar{x}_{b'',i'} := \begin{cases} \min(1, x^\star_{b,i} + x^\star_{b',i}) & \text{if } b'' = b \text{ and } i' = i\,, \\ 0 & \text{if } b'' = b' \text{ and } i' = i\,, \\ x^\star_{b'',i'} & \text{else}\,. \end{cases}$$

We have that $\bar{x}$ is a feasible solution (since $t \in b' \Rightarrow t \in b$) with value smaller or equal to the one of $x^\star$ (since $w_b \leqslant w_{b'}$). Therefore, $\bar{x}$ is an optimal solution with $\bar{x}_{b',i} = 0$. □

This procedure helped us gain another 3 h 58 min for the column generation algorithm on the instance with 1,815 trains and 35 min on the instance with 926 trains.

Improving the quality of integer solution

As hinted in Section A.3, optimal integer solutions can be obtained via a branch-and-price algorithm. However, the numerous calls to the column generation algorithm makes this approach intractable when tackling large real-life instances.

The alternative for retrieving optimal integer solutions stemming from Lemma A.3.1 requires small adaptations for the label correcting algorithm in order to generate all possible paths with reduced cost smaller or equal to $UB - LB$, where LB is a lower bound and UB an upper bound on the optimal value of the ILP.

This alternative is sometimes adopted when the gap $UB - LB$ is small. However, this shows two main limitations when tackling large instances:

- We cannot rely on the domination-based discarding procedure (ii).
- This procedure might generate a large number of additional variables, making it challenging for the ILP solvers to solve the final ILP.

However, we can rely on this idea to generate a limited number of additional variables in the hope of obtaining good integer solutions of cost lower than the integer solution built from the column generation variables only. To do this, we adapt Algorithm 1 by imposing a stopping condition in line 2 (either a time limit or a maximal number of generated paths), keeping (and returning) all paths with reduced cost under $UB - LB$, dropping the domination-based discarding procedure of line 11, and replacing the comparison to the current best path in line 11 with a comparison to $UB - LB$. Lemma A.3.1 can also be used to discard variables with large reduced cost, which are not useful for the ILP, in order to speed up the ILP solving step (and obtain better integer solutions when a time limit is set).

Overall, multiple iterations of successive variable discarding and generation are performed within a global time limit of 3600 s. After each generation phase, the corresponding ILP is

	Two-stage approach		Integrated approach		
	Time	Objective	Time	LB	UB
116 trains	1 min	50	1 h 8 min	40.3	41
280 trains	3 min	228	1 h 9 min	205.3	207
926 trains	26 min	832	6 h 3 min	711.9	724
1,815 trains	64 min	1633	10 h 2 min	1,395.0	1,477

Table 4.1: Numerical experiments on the four instances. The column "LB" refers to the optimal value of the linear relaxation. The column "UB" refers to the upper bound obtained by solving the integer linear program.

solved and the upper bound UB is updated. When no improvement is made on the value of the upper bound, this iterative procedure is stopped. Each call to the modified label correcting algorithm is set with a time limit of 15 s.

This procedure helped us gain around 0.6% on the best integer solution found for the two largest instances. As a side remark, this observation still holds when the variable dominance and the warm-starting procedures from the previous paragraphs are deactivated.

4.6 Numerical experiments

4.6.1 Experimental setting and instances

The numerical experiments aim to compare the solutions of the two-stage approach currently used at SNCF (explained in Chapter 3) with the integrated approach proposed in Section 5.5. As noted in the introduction, the two-stage approach is not exact by its sequential nature.

The instance with 1,815 trains corresponds to the nation-wide traffic on a typical week for the year 2024. As data acquisition is not straightforward, this is the only nation-wide instance we were able to collect. To put this in contrast with the number of instances of Chapter 3, these instances were made of the duties built by freight experts in 2024, with one instance per team of drivers, whereas here we consider all teams at once for the integrated approach. Moreover, even though there is an automated way to collect the duties built by freight experts in the planned rosters of the past years, there is no software at SNCF that can recover trains from these duties, making data collection a complicated and lengthy task. Finally, even though the instances from Chapter 3 are also taken for the year 2024, we have no guarantee that they were built from the same typical week as the instance with 1,815 trains, so they might not cover the same set of trains. Therefore, the results from Table 3.4 and from Table 4.1 are not intended to be compared.

The other instances are built by isolating parts of the nation-wide instance with 1,815 trains, based on geographical criteria.

The experiments have been conducted on a server with an Intel(R) Xeon(R) CPU E5-2690 v2 @ 3.00GHz processor. The programming language used is Java 18 and the solver used is CPLEX 22.11. No time limit was imposed for the resolutions of the linear relaxation of the restricted master problem and a time limit of 3600 s for the resolutions of the integer linear programs. A time limit of 3600 s was set for the additional variable generation procedure to improve the quality of the integer solution.

4.6.2 Results

The results for the four instances are summarized in Table 4.1. The caption of the table provides the meaning of the columns.

Throughout the column generation algorithm, in every call to the label correcting algorithm, we keep track of the reasons why a partial path is discarded. Recall that there are three

	Number of variables	Number extended paths	Number discarded paths
116 trains	13,500	7,619,648	515,884,281
280 trains	16,821	7,749,733	519,288,691
926 trains	50,635	28,350,131	2,066,420,616
1,815 trains	83,133	50,359,197	4,062,062,782

Table 4.2: Label correcting algorithm statistics

procedures that discard paths:

- the bounding strategy (i),
- the domination strategy (ii),
- the early feasibility check.

Table 4.2 summarizes, for the four instances, the total number of variables generated for the column generation algorithm, the total number of paths explored in the label correcting algorithm (i.e., added to $\mathcal{P}$ in Algorithm 1) and the total number of discarded paths across all calls to the label correcting algorithm. Overall, for all instances, the average percentage of paths discarded with the bounding strategy is around 15%, with the domination strategy it is around 10% and with the early feasibility check it is around 75%. Remark that the number of discarded paths is much larger than the number of extended paths, meaning that the discarding strategies are crucial for the efficiency of the label correcting algorithm.

4.6.3 Comments

The integrated approach provides solutions with a number of working days significantly smaller than the two-stage approach. Since the gain in the number of working days reaches at least 5% for all instances, and this criterion corresponds to large expenses for the freight, these results are very promising for a future use of the integrated approach according to experts at SNCF. The high computation time is actually not prohibitive for the freight application, still according to the experts, since the rosters are designed more than one year in advance.

The bounding and domination strategies are both useful to discard paths early in the exploration process. Along with the early path feasibility check, this makes the graph exploration tractable.

Chapter 5

Crew management with train delays

***

This problem, the methodology, and the numerical results were established in "Integrating crew scheduling and crew rostering for rail freight with train delays," H. Gachet, F. Meunier, and J. Pouzet, *9th International Symposium on Combinatorial Optimization, ISCO 2026* [42].

5.1 Context . . . 68
5.2 Literature review . . . 68
5.3 Problem formulation . . . 69
5.3.1 Setting . . . 69
5.3.2 Modeling delays and "fragility" . . . 69
Modeling delay propagation . . . 69
Modeling "fragility" of rosters . . . 69
5.3.3 Problem formulation . . . 70
5.4 Model . . . 71
5.5 Solution method . . . 71
5.5.1 Column generation . . . 71
Algorithm for the linear relaxation (Covering SCM) . . . 71
From a fractional solution to an integer solution . . . 72
5.5.2 Pricing subproblem with delay distribution as a RCSP problem . . . 72
Poset of resources . . . 72
Extension functions . . . 73
Cost function . . . 73
5.6 Coping with approximated distributions for primary delays . . . 73
5.7 Numerical experiments . . . 76
5.7.1 Experimental setting and instances . . . 76
5.7.2 Results . . . 76
5.7.3 Comments . . . 77
5.8 Final remarks . . . 77

5.8.1 Convergence speed 77
5.8.2 Other risk measures 78
Generalities 78
Conditional value-at-risk 78
5.8.3 Variants of the stochastic crew management problem 79
Minimizing the fragility 79
Delays impacting the number of working days 81

5.1 Context

This chapter aims to tackle the crew management problem under uncertainty. We recall that the crew management problem refers to the planning step aiming to build (cyclic) rosters from tasks given over a defined time period. As highlighted in Chapter 4, a first drawback of the current approach which solves the crew management problem at SNCF sequentially (with first crew scheduling then crew rostering) is that it provides no guarantee of global optimality. Here, the sequential approach makes it moreover difficult for SNCF to account for uncertainties: the outcome of train delays' propagation on a roster, which can result in trains and duties cancellation, can only be measured by considering simultaneously the duties and their organization within the roster. Yet, train delays occur frequently and significantly increase the final cost incurred by the company.

In this chapter, we propose to modify the integrated approach from Chapter 4 by including a stochastic constraint, so as to tackle both suboptimalities induced by sequentiality and train delays.

The stochastic constraint uses a quantitative metric that we introduce in this chapter. This metric, which we call *fragility*, aims to capture exactly the impact of uncertainty on rosters in a concrete and meaningful way. The definition and its rationale are given in Section 5.3.2 (assuming known the distributions of the delays). The integrated approach we propose is able to handle this stochastic constraint when delays are independent random variables with finite support.

Numerical experiments show that this stochastic integrated approach is able to substantially reduce fragility, without deteriorating the objective value obtained by the current sequential approach.

Finally, we note that in practice, delay distributions are often only partially described—for instance, specifying that delays fall within a given interval with some probability. We also discuss (Section 5.6) how rigorous conclusions can still be drawn from our work in the presence of such approximate descriptions.

5.2 Literature review

To our knowledge, articles dealing with crew scheduling with uncertainties only address airline applications [5, 32]. The authors either use a robust or a stochastic setting and model disruptions due to flights (and ground operations) delays in various manners. In the railway industry, the literature rather deals with rescheduling after an uncertainty has happened [56]. In this work, our column generation method includes a stochastic constraint to deal with the integrated problem in the case when there are train delays. We were not able to find papers tackling the integrated crew management problem from a stochastic viewpoint.

5.3 Problem formulation

5.3.1 Setting

We now consider that each train comes with a random variable representing a possible "primary delay," on top of its nominal arrival and departure times. We refer to the integrated crew scheduling and crew rostering problem with train delays as the *stochastic crew management problem*. An instance of the stochastic crew management problem at SNCF is similar to an instance of the crew management problem (see Section 4.3 for details), with an additional "fragility" threshold S in the instance; see below the discussion about primary delays and fragility. To summarize, we are given: a collection T of trains (with given origin, destination, nominal departure and arrival time, and primary delay random variable), a collection I of teams and a threshold S. The goal behind the crew management problem with train delays consists of finding a set of rosters with minimum number of working days, each roster being assigned to a team of drivers, such that each train of the instance is covered by a roster and an additional constraint is verified. This additional constraint consists of keeping the rosters' sensitivity to train delays below the threshold S.

5.3.2 Modeling delays and "fragility"

In this section, we aim to model the propagation and impact of delays on the rosters. The "delay propagation" model described in (5.1) was developed based on discussions with freight experts. In particular, we assume that, in practice, the rules and regulations grant planners some flexibility to cope with unplanned delays: the booked infrastructure and rolling stock can be used slightly outside their planned time windows. This flexibility is nonetheless bounded, so that delays are not allowed to propagate indefinitely along the rosters. We then assume that delay propagation occurs only within duties, and we capture its consequences on the rosters through a fragility score. This score is designed as an operationally interpretable measure for the expected disruption induced by delay propagation. It is intended to capture as closely as possible the actual process followed by freight planners when facing train delays: backup drivers may be called in to operate certain trains or duties removed from the reference rosters, which is, in expectation, what the fragility score measures.

Modeling delay propagation

Exogenous uncertainties might make some trains depart or arrive with a delay, called "primary delay." The primary delay associated with train $t \in T$ is denoted by P_t. When trains are grouped into duties, such primary delays can propagate over the next trains within the duties, causing "secondary delays." More precisely, consider a duty whose train sequence is $t_1, t_2, \ldots, t_k$. The arrival time H_{t_i} of the train t_i is of the form

$$H_{t_i} = \begin{cases} \delta_{t_1} + \tau_{t_1} + P_{t_1} & \text{if } i = 1, \\ \max\big(H_{t_{i-1}} + \Delta_{t_{i-1},t_i}, \delta_{t_i}\big) + \tau_{t_i} + P_{t_i} & \text{if } i \geqslant 2, \end{cases} \tag{5.1}$$

where Δ_{t_{i-1},t_i} is a minimal time to operate consecutively the trains t_{i-1} and t_i taking into account possible relocations and short breaks. Recall from Chapter 3 that δ_t is the nominal departure time of train t and τ_t is its nominal duration.

Modeling "fragility" of rosters

Depending on how the trains are organized into its duties and its blocks, a roster can achieve various degrees of resilience to delays. In particular, when the delays within a duty exceed some thresholds, two things might happen:

(a) the long break between the current duty and the next one in the block does not reach the minimal duration given by the working rules; in this case, the next duty is "disrupted."
(b) the working time in the current duty exceeds a threshold given by the working rules but there are still upcoming trains within the duty that must be driven; in this case, the current duty is "disrupted."

In both cases, the duties or trains must be operated by a backup driver, which leads to an extra and substantial cost. Note that the number of working days of a block is not impacted by the delays. Given a realization h of the arrival times and the composition of a block b, the number $\sigma(h, b)$ of such disruptions is easily computed as follows. Write the duty sequence of b as $d_1, d_2, \ldots, d_q$, and write the train sequence of each duty d_i as $t_{i,1}, t_{i,2}, \ldots, t_{i,k_i}$. The number of disruptions is then given by (with "$h_{t_{0,k_0}} > s_1$" being always false):

$$\sigma(h,b) = \sum_{i=1}^{q} \mathbb{1}\Big(h_{t_{i-1,k_{i-1}}} > s_i \;\text{ or }\; \max_{j=2,\ldots,k_i} \big(h_{t_{i,j-1}} + \Delta_{t_{i,j-1},t_{i,j}}\big) > s'_i\Big), \tag{5.2}$$

where s_i and s'_i have the following meaning:

- s_i is the instant beyond which the arrival of the last train of duty d_{i-1} makes the duty d_i disrupted, as in (a).
- s'_i is the instant beyond which the departure of a train makes the duty d_i disrupted, as in (b). Note that the departure time of train $t_{i,j}$ writes $\max(h_{t_{i,j-1}} + \Delta_{t_{i,j-1},t_{i,j}}, \delta_{t_{i,j}})$ and that $\delta_{t_{i,j}} \leqslant s'_i$ for a (valid) block.

Note that, as expected, $h \mapsto \sigma(h, b)$ is non-decreasing, and that computation of the instants s_i and s'_i is straightforward from the composition of a block.

This makes us introduce the *fragility of a block* b, defined as $\varphi(b) \coloneqq \mathbb{E}\big[\sigma(H, b)\big]$, with $H = (H_t)_{t\in T}$ taking into account both primary and secondary delays. The *fragility of a roster* r is then defined naturally as $\sum_{b\in r} \varphi(b)$. This metric has been validated by the freight experts at SNCF to assess the resilience of a roster r to delays. Note that its meaning is concrete and relevant: it captures the expected number of calls to backup drivers to operate disrupted trains or duties. The ability to cope with train delays in our problem is modeled as a single threshold constraint $\sum_b \varphi(b) \leqslant S$, where b ranges over all blocks of the solution, and S is the threshold value given in the instance.

5.3.3 Problem formulation

As stated in Lemma 3.4.2, there is an easy greedy algorithm converting any sequence of valid blocks to a valid roster, with the same total number of working days. From the discussions of Section 5.3.2, according to the experts at SNCF, the sensitivity of a roster to train delays is measured by the fragility score of a roster, which decomposes over the blocks. The algorithm in the proof of Lemma 3.4.2 then also keeps the total block fragility scores constant. Thus, as for the deterministic crew management problem of Chapter 4, instead of a set of rosters, we can look for a set of blocks. The stochastic crew management problem then consists of finding a set of blocks of minimum number of working days, each block being assigned to a team of drivers, such that each train of the instance is covered by a block and the total fragility scores is below the threshold S. This can be formally written as:

Stochastic crew management (SCM)
Input. Collection T of trains on a typical week, collection I of teams of drivers, threshold S.
Output. Covering of trains by blocks with minimum number of working days, each block assigned to a team, with total fragility below S.

5.4 Model

The stochastic crew management problem at SNCF can be modeled as an integer linear program, where w_b is the number of working days in the block b, and where B_i is the set of all blocks that can be performed by team i.

$$\begin{aligned} \text{minimize} \quad & \sum_{i\in I}\sum_{b\in B_i} w_b x_{b,i} & & \\ \text{s.t.} \quad & \sum_{i\in I}\sum_{b\in B_i:t\in b} x_{b,i} \geqslant 1 & & \forall t \in T \\ & \sum_{i\in I}\sum_{b\in B_i} \varphi(b) x_{b,i} \leqslant S & & \\ & x_{b,i} \in \{0,1\} & & \forall i \in I, \forall b \in B_i\,. \end{aligned} \tag{Covering SCM}$$

The variables $x_{b,i}$ take the value 1 if the block $b \in B_i$ is selected for the team $i \in I$.

Note that this program is exactly the one for the crew management problem (without train delays) (Covering CM), up to adding the fragility constraint.

5.5 Solution method

5.5.1 Column generation

Algorithm for the linear relaxation (Covering SCM)

We propose an algorithm for the resolution of the problem (Covering SCM), which is essentially following the same methodology as in Section 4.5. The algorithm starts by computing an optimal solution of the linear relaxation of (Covering SCM) using column generation, where the blocks define the columns. The general algorithm for column generation is given in Algorithm 2, and details are given in Appendix A. Then the restricted version of (Covering SCM), where the B_i's are replaced by the subsets B'_i of blocks generated during the column generation, is solved by using an off-the-shelf ILP solver. As opposed to the remark made in Section 4.5.1, there might be no integer feasible solution at this point, since the fragility constraint can conflict with the feasibility of the rounding up of a fractional feasible solution. The next section is dedicated to discussing how to retrieve an integer feasible solution after the convergence of the column generation algorithm.

We describe a bit more precisely the column generation used to solve the linear relaxation of the ILP (Covering SCM). As in Section 4.5.1, the set of columns is the set of feasible blocks $\bigcup_{i\in I} B_i$. The restricted master problem writes, for subsets $B'_i \subseteq B_i$:

$$\begin{aligned} \text{minimize} \quad & \sum_{i\in I}\sum_{b\in B'_i} w_b x_{b,i} & & \\ \text{s.t.} \quad & \sum_{i\in I}\sum_{b\in B'_i:t\in b} x_{b,i} \geqslant 1 & & \forall t \in T \\ & \sum_{i\in I}\sum_{b\in B'_i} \varphi(b) x_{b,i} \leqslant S & & \\ & x_{b,i} \in [0,1] & & \forall i \in I, \forall b \in B'_i\,. \end{aligned} \tag{RMP(Covering SCM))}$$

The first step of the algorithm (line 1) consists here of initializing via a greedy heuristic the B'_i in such a way that all t are covered (as in Section 4.5.1). However, because of the fragility constraint, this greedy heuristic does not necessarily provide a first feasible solution. We thus enter the initialization phase of Algorithm 2. During this phase, the LP solver is used to compute (dual) Farkas rays, i.e., rays in the dual of (RMP(Covering SCM)) (line 5) and the

Farkas pricing is solved so as to identify blocks preventing this ray from existing (line 6). This is done using Algorithm 1 described in Section A.2.1. Once a first feasible solution is found, we enter the optimization phase, which follows the same lines as detailed in Section 4.5.1.

Here, as in Section 4.5.1, there are as many pricing subproblems as teams, written as follows for a team i, where $\lambda_t \geqslant 0$ denotes the dual value associated with the covering constraint for train t, and μ denotes the dual value associated with the fragility constraint:

$$\begin{aligned} &\text{minimize } w_b - \sum_{t \in b} \lambda_t + \mu\varphi(b) \\ &\text{s.t. } b \in B_i\,. \end{aligned}$$

In other words, for a given team i, we seek to find a block satisfying all rules of Tables 3.1 and 3.2, such that the reduced cost of the associated variable $x_{b,i}$ in (RMP(Covering SCM)) is negative.

As for the deterministic crew management problem of Chapter 4, each of these subproblems takes the form of a non-linear resource-constrained shortest path (RCSP) problem on an acyclic graph, addressed with the label-correcting algorithm detailed in Section 4.5.2. The pricing subproblem with delay distributions is further explained in Section 5.5.2.

From a fractional solution to an integer solution

As explained in the previous section, we are not guaranteed to obtain a feasible integer solution by solving the ILP (Covering SCM) with a subset of variables corresponding to the last B_i' of the column generation algorithm.

In addition, Lemma A.2.1 only provides a constructive procedure to counter infeasibility in the linear relaxation of (Covering SCM). We however provide below an iterative procedure to overcome this challenge. Nevertheless, this procedure was never used in the numerical experiments, as solving the ILP with the last B_i' always provided a feasible integer solution.

Suppose we have a feasible solution of the linear relaxation of (Covering SCM), obtained via the column generation algorithm. Then, we can slightly modify the label correcting algorithm in order to generate all paths with reduced cost below a given threshold. Using an increasing sequence of cost thresholds, we can iteratively add more variables to the last B_i', with reduced cost below the current threshold, and solve the corresponding ILP with the subset B_i'. This procedure is guaranteed to find an integer solution if one exists, as the whole set of variables B_i is finite. It is however not guaranteed to be efficient.

5.5.2 Pricing subproblem with delay distribution as a RCSP problem

The pricing subproblem associated with a team i for the problem (Covering SCM) can be modeled as a RCSP problem on an acyclic digraph D, built identically as in Section 4.5.2, where every block $b \in B_i$ encodes a single o-d path. For every feasible path P to correspond to a valid block b, we extend the definition of the poset of resources $(R, \preccurlyeq)$, extension functions f_a, and cost function c introduced in Section 4.5.2.

Poset of resources

For our problem, we define a resource, i.e., an element of R, as a vector where all components except the last one are non-negative real numbers and where the last component is a probability distribution. As in Section 4.5.2, each real number component is associated either with one or multiple working rules, or with an element involved in the computation of the cost function (such as the number of working days, the fragility score, etc.). The last component represents the arrival time distribution, taking the primary and secondary delays into account. This is

required to compute the fragility of the blocks. Each real component comes with the usual total order of $\mathbb{R}$. For the last component, the partial order we use is the usual stochastic order, denoted by $\leqslant_{\text{st}}$, and defined as follows. For two real random variables X_A and X_B, we have $X_A \leqslant_{\text{st}} X_B$ if $\mathbb{P}(X_A > x) \leqslant \mathbb{P}(X_B > x)$ for all $x \in \mathbb{R}$. The partial order equipping R is then just the product order.

Extension functions

The extension functions f_a are then extending the resources similarly as in Section 4.5.2 for the resource components common to the deterministic and stochastic versions of the problem. In particular, the example provided for the 6th and 7th components remains valid. The last component requires more attention because we deal here with probability distributions. We explain this in detail now.

Let b be a block whose duty sequence is $d_1, d_2, \ldots, d_q$ and write the train sequence of each duty d_i as $t_{i,1}, t_{i,2}, \ldots, t_{i,k_i}$. As we now see, computing the fragility of a block motivates the assumptions we made on the P_t for our algorithm, namely that they are independent and have finite support. Since they are independent, so are the random variables $H_{t_{i,j}}$ and $H_{t_{i',j'}}$ for $i \neq i'$ and all j, j', and taking the expectation of (5.2), we obtain

$$\varphi(b) = \sum_{i=1}^{q} \mathbb{P}[A_{i-1} > s_i] + \mathbb{P}[B_i > s'_i] - \mathbb{P}[A_{i-1} > s_i] \times \mathbb{P}[B_i > s'_i]$$

where $A_{i-1} = H_{t_{i-1,k_{i-1}}}$ and $B_i = \max_{j=2,\ldots,k_i}(H_{t_{i,j-1}} + \Delta_{t_{i,j-1},t_{i,j}})$.

In order to compute the fragility score, we describe in more detail the computation of the last four components of the extended resource $f_a(r)$, which are used to handle the stochastic constraint. The first of these four components keeps track of $\mathbb{P}[A_{i-1} > s_i]$, the second component keeps track of $\mathbb{P}[B_i > s'_i]$, the third component keeps track of the fragility score of the partial block in construction, and the fourth (i.e., the last component of the resource vectors) keeps track of the distribution of the arrival time of the last train of arc a. For the latter, when the arc a corresponds to a long break between two duties, the last component of $f_a(r)$ is reset to the distribution of $\delta_t + \tau_t + P_t$, where t is the tail of a, following (5.1). Otherwise, the last component of $f_a(r)$ is obtained through a convolution of the last component of r with the primary delay distribution of the tail of a, following (5.1) again. The computation of the convolution is straightforward by the finite-support assumption.

Cost function

Denote by $\lambda_t \geqslant 0$ the dual value associated with the constraint of covering t. The cost function c combines the number of working days, the partial sum $\sum_t \lambda_t$, where t ranges over the trains met so far, and the "partial" fragility. It takes the value $+\infty$ when evaluated at $\hat{\mathbf{1}}$. When the path reaches the vertex d, this accounts exactly for the reduced cost of the variable $x_{b,i}$, i.e., $w_b - \sum_{t \in b} \lambda_t + \mu\varphi(b)$, where $\mu \geqslant 0$ is the dual value associated with the stochastic constraint.

5.6 Coping with approximated distributions for primary delays

SNCF provides only partial information about the true primary delays of the trains, which we denote by $\widetilde{P}_t$ for a train t. It takes the form of a finite sequence of thresholds $0 = \alpha_0 < \alpha_1 < \cdots$ together with the quantities $p_{0,t} := \mathbb{P}(\widetilde{P}_t = 0)$ and $p_{i,t} := \mathbb{P}(\widetilde{P}_t \in (\alpha_{i-1}, \alpha_i])$ for $i = 1, 2, \ldots$. Such a description of the primary delays was used for the numerical results, as shown by

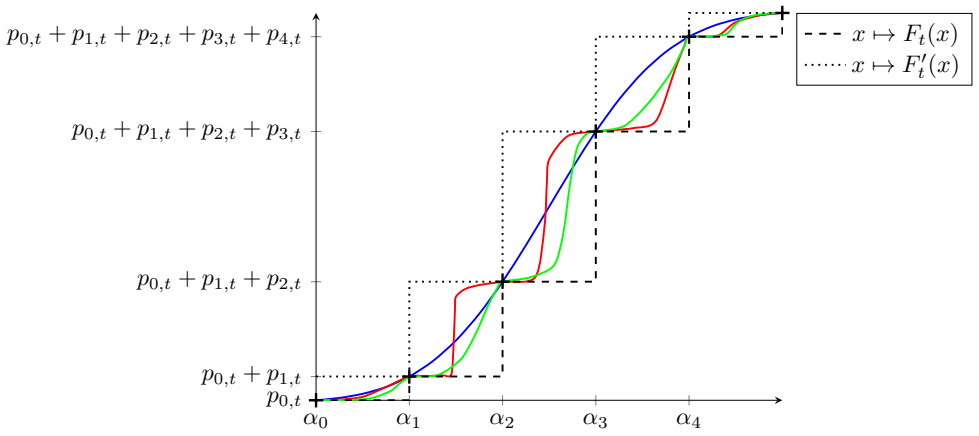


Figure 5.1: An illustration of the cumulative distribution functions (CFD) F_t and F'_t associated with the random variables P_t and P'_t for a train t, given the partial information on the true primary delays. This partial information takes the form of thresholds $0 = \alpha_0 < \alpha_1 < \cdots$ together with quantities $p_{i,t}$ for $i = 0, 2, \ldots$. The CFD of three possible true random variables $\widetilde{P}_t$ are illustrated, each coherent with the partial information, in the sense that $\mathbb{P}(\widetilde{P}_t = 0) = p_{0,t}$ and $\mathbb{P}(\widetilde{P}_t \in (\alpha_{i-1}, \alpha_i]) = p_{i,t}$ for $i = 1, 2, \ldots$. Using the alternative primary delays P'_t provides a lower bound on the optimal value of the problem (with the true delay distributions). If optimal solution under the alternative primary delays P'_t is feasible under the alternative primary delays P_t, then it is an optimal solution of the problem (with the true delay distributions).

Table 5.1. This description of the primary delays does not fit directly with the form of the instances of our problem. Indeed, this means that the precise distribution of the primary delay $\widetilde{P}_t$ of every train t is not known fully, but only a few values taken by the cumulative distribution function (at the points $\alpha_0, \alpha_1, \ldots$) are known. Yet, our framework provides an accurate solution, even with this restriction.

Indeed, we consider instead surrogate primary delays P_t that are independent random variables taking value α_i with probability $p_{i,t}$ for $i = 0, 1, \ldots$. Such variables are of finite support and thus our algorithm applies. Proposition 5.6.1 below ensures that every solution of the problem that is feasible with the surrogate primary delays P_t is feasible for the true primary delays $\widetilde{P}_t$, with the same value of the objective function.

Moreover, taking a "distributionally robust" point of view, the optimal solution obtained for the surrogate primary delays P_t is the best we can hope for: given the thresholds $0 = \alpha_0 < \alpha_1 < \cdots$ and quantities $p_{0,t}, p_{1,t}, \ldots$ for $i = 1, 2, \ldots$, P_t is the (unique) lowest random variable (called join) among the random variables greater than all possible true random variables $\widetilde{P}_t$ such that $p_{0,t} \coloneqq \mathbb{P}(\widetilde{P}_t = 0)$ and $p_{i,t} \coloneqq \mathbb{P}(\widetilde{P}_t \in (\alpha_{i-1}, \alpha_i])$ for $i = 1, 2, \ldots$.

Proposition 5.6.1 shows also the following: using alternative primary delays P'_t defined as independent random variables taking value 0 with probability $p_{0,t} + p_{1,t}$ and value α_i with probability $p_{i+1,t}$ for $i = 1, 2 \ldots$ provides a lower bound on the optimal value of the problem.

Figure 5.1 provides an illustration of the original primary delay $\widetilde{P}_t$ cumulative distribution function for a train t, along with P_t and P'_t. Note that random variables lower than $\widetilde{P}_t$ (for $\leqslant_{\mathrm{st}}$) are above the graph corresponding to $\widetilde{P}_t$ on this figure.

Proposition 5.6.1. *Suppose that for each train t we are given two random variables P^0_t and P^1_t for primary delays, with $P^0_t \leqslant_{\mathrm{st}} P^1_t$ for all t. Then every solution that is feasible under the primary delays $(P^1_t)_t$ remains feasible under $(P^0_t)_t$.*

Proof. Consider a solution that is feasible under $(P_t^1)_t$. Denote by H^0 and H^1 the arrival times of all trains under respectively primary delays $(P_t^0)_t$ and primary delays $(P_t^1)_t$. Since $P_t^0 \leqslant_{\text{st}} P_t^1$ for all t, we have $H_t^0 \leqslant_{\text{st}} H_t^1$ for all t as well, and $\mathbb{E}[\sigma(H^0, b)] \leqslant \mathbb{E}[\sigma(H^1, b)]$. Since the solution is feasible under $(P_t^1)_t$, we have $\sum_b \mathbb{E}[\sigma(H^1, b)] \leqslant S$ where the index b ranges over all blocks of the solution. This, combined with the previous inequality, leads to $\sum_b \mathbb{E}[\sigma(H^0, b)] \leqslant S$ with b still ranging over all blocks of the solution, and the solution is feasible under $(P_t^0)_t$ as well. □

We have been discussing the case when the probability distribution of the true primary delays $\widetilde{P}_t$ is not fully known.

Note that computing the convolutions of continuous distributions can be tricky, even for classical distributions. Our approach requires in particular the distributions to have finite support for this reason: even though classical distributions can be fully described with a finite number of parameters values, it is not necessarily the case with their convolution anymore. Therefore, when $\widetilde{P}_t$ is fully known (but does not meet the finite-support assumption) these discussions still remain relevant.

Proposition 5.6.1 can clearly be used to identify feasible solutions (by taking distributions that are larger than the true ones) and to get lower bounds (by taking distributions that are smaller than the true ones). The—maybe surprising—next theorem shows that we can even go further for such arbitrary distributions: assuming the problem feasible, it is in principle always possible to solve it exactly, provided that the satisfaction of the stochastic constraint with the original primary delays can be checked efficiently on a given solution.

Theorem 5.6.2. *Suppose that the problem is feasible (under the original primary delay distributions $(P_t)_t$). Consider any sequence of alternative primary delays $(P_{t,n}^0)_{t,n}$ with $P_{t,n}^0 \leqslant_{\text{st}} P_t$ for every t and n and $P_{t,n}^0 \xrightarrow{d} P_t$ for every t. Then there exists N such that, for all $n \geqslant N$, the set of feasible solutions under $(P_{t,n}^0)_t$ coincides with the set of feasible solutions under $(P_t)_t$ (and the same holds for the set of optimal solutions).*

Proof. We prove first that, for every (valid) block b, independently of any solution of the problem, we have $\lim_{n\to+\infty} \mathbb{E}_{P_n^0}[\sigma(H_n^0, b)] = \mathbb{E}_P[\sigma(H, b)]$, where H_n^0 corresponds to the random arrival times under the primary delays $P_n^0 = (P_{t,n}^0)_t$ and H corresponds to the random arrival times under the primary delays $P = (P_t)_t$. Since $P_{t,n}^0 \leqslant_{\text{st}} P_t$ holds, we have $\mathbb{E}_{P_n^0}[\sigma(H_n^0, b)] \leqslant \mathbb{E}_P[\sigma(H, b)]$. On the other hand,

$$\begin{aligned}\liminf_{n\to+\infty} \mathbb{P}\big(H^0_{t_{i-1,k_{i-1}},n} > r_i \ \text{ or } \max_{j=2,\dots,k_i} (H^0_{t_{i,j-1},n} + \Delta_{t_{i,j-1},t_{i,j}}) > s_i\big) \\ \geqslant \mathbb{P}\big(H_{t_{i-1,k_{i-1}}} > r_i \ \text{ or } \max_{j=2,\dots,k_i} (H_{t_{i,j-1}} + \Delta_{t_{i,j-1},t_{i,j}}) > s_i\big)\end{aligned}$$

by the special case of the Portmanteau theorem for open sets. The probabilities in the previous expression are nothing else than the expectation of the term in (5.2); hence, we have $\liminf_{n\to+\infty} \mathbb{E}_{P_n^0}[\sigma(H_n^0, b)] \geqslant \mathbb{E}_P[\sigma(H, b)]$. Altogether, we have thus $\lim_{n\to+\infty} \mathbb{E}_{P_n^0}[\sigma(H_n^0, b)] = \mathbb{E}_P[\sigma(H, b)]$.

Using Proposition 5.6.1, every feasible solution under P is feasible under P_n^0. To get the desired conclusion, we prove now that there exists N such that, for all $n \geqslant N$, every feasible solution under P_n^0 is a feasible solution under P. Note that since the objective function does not depend on the random variables, the conclusion about the optimal solutions is then immediate. Suppose thus by contradiction that there is no such N, i.e., there exists an infinite sequence $n_1 < n_2 < \cdots$ such that the following holds for every ℓ: there exists a feasible solution under $P_{n_\ell}^0$ that is not a feasible solution under P. Let x_{n_ℓ} be such a feasible solution under $P_{n_\ell}^0$. Up to extracting a subsequence, we can assume, by finiteness, that every x_{n_ℓ} is the same element x of $\prod_{i\in I}\{0,1\}^{B_i}$. Because we have $\lim_{n\to+\infty} \mathbb{E}_{P_n^0}[\sigma(H_n^0, b)] = \mathbb{E}_P[\sigma(H, b)]$, the feasibility of x_{n_ℓ} under $P_{n_\ell}^0$ shows the feasibility of x under P. This shows that x_{n_ℓ} is a feasible solution under P for every ℓ, which is a contradiction. □

primary delay value	probability
$= 0\,\text{min}$	0.5
$\in (0\,\text{min}, 15\,\text{min}]$	0.2
$\in (15\,\text{min}, 30\,\text{min}]$	0.2
$\in (30\,\text{min}, 60\,\text{min}]$	0.1

Table 5.1: Primary delay distributions used for the numerical experiments (identical for all trains).

We conclude with two complementary remarks. First, under the condition of the theorem, the optimal value of the problem under $(P^0_{t,n})_t$ is the same as the one of the problem under $(P_t)_t$ when $n \geqslant N$. This is just because the objective value does not depend on the primary delay distributions (the sets of feasible solutions coincide by Theorem 5.6.2). Second, these results can be used to obtain an algorithmic counterpart. Proposition 5.6.1 shows that an optimal solution under some delay distributions $(P^0_{t,n})_t$ that satisfies the stochastic constraint with the original distributions $(P_t)_t$ is automatically optimal for $(P_t)_t$ as well. This, together with Theorem 5.6.2, gives a practical algorithm: solve the problem under $(P^0_{t,n})_t$ for increasing values of n, and check whether the resulting optimal solution satisfies the stochastic constraint under the original distributions $(P_t)_t$. By the previous remark, as soon as this check succeeds, the solution obtained is optimal for the original problem, and Theorem 5.6.2 guarantees that this happens after finitely many steps, provided the problem is feasible. This is why the assumption that the stochastic constraint under $(P_t)_t$ can be checked efficiently is key to turning the theorem into a usable exact algorithm.

5.7 Numerical experiments

5.7.1 Experimental setting and instances

We use the same experimental setting as in Section 4.6. The numerical experiments aim to compare the solutions to the stochastic crew management problem obtained with the approach of this chapter and the solutions to the crew management problem obtained with either the two-stage approach currently used at SNCF or the integrated approach of Chapter 4. Note that the fragility score of the crew management problem (without train delays) can be computed a posteriori, independently from the optimization.

After discussions with freight experts, the primary delays are distributed as in Table 5.1 (identical for all trains). Tests have also been run for the largest instance with 1,815 trains with primary delays as in Table 5.2 for the trains departing from or arriving at the shunting yard of Woippy, situated near Metz in France, and primary delays as in Table 5.1 for the other trains. We call "1815 trains bis" the instance defined like this. The fragility threshold for each instance was fixed as 50% of the fragility score of the solution found by the two-stage approach.

The integrated approach is run with a time limit of 24 h for the column generation algorithm, thus providing no lower bound on the value of the problem (Covering SCM) when the convergence of the column generation is not reached.

5.7.2 Results

The results for the instances are summarized in Table 5.3. The caption of the table provides the meaning of the columns.

primary delay value	probability
$= 0\,\text{min}$	0.2
$\in (0\,\text{min}, 15\,\text{min}]$	0.1
$\in (15\,\text{min}, 30\,\text{min}]$	0.2
$\in (30\,\text{min}, 45\,\text{min}]$	0.2
$\in (45\,\text{min}, 60\,\text{min}]$	0.2
$\in (60\,\text{min}, 90\,\text{min}]$	0.1

Table 5.2: Primary delay distributions used for the numerical experiments on the instance "1,815 trains bis": all trains departing from or arriving at the shunting yard at Woippy have primary delay following this distribution, the other trains have primary delay following the distribution of Table 5.1

	Two-stage approach			Integrated approach: deterministic case				Integrated approach: stochastic case			
	Time	Objective	Fragility	Time	LB	UB	Fragility	Threshold S	Time	UB	Fragility
116 trains	1 min	50	0.970	1 h 8 min	40.3	41	2.748	0.485	26 h 11 min	44	0.437
280 trains	3 min	228	2.450	1 h 9 min	205.3	207	6.440	1.225	24 h 19 min	210	1.202
926 trains	26 min	832	8.538	6 h 3 min	711.9	724	28.46	4.269	25 h 44 min	741	4.081
1,815 trains	64 min	1,633	14.900	10 h 2 min	1,395.0	1,477	58.980	7.450	26 h 49 min	1,477	7.432
1,815 trains bis	64 min	1,633	18.410	10 h 2 min	1,395.0	1,477	79.250	9.205	29 h 13 min	1,526	8.767

Table 5.3: Numerical experiments on the four instances. The column "LB" refers to the optimal value of the linear relaxation. The column "UB" refers to the upper bound obtained by solving the integer linear program. The column "Fragility" refers to the total fragility score of the blocks in the solution. For the stochastic integrated approach, the time limit of 24 h is reached for every instance.

5.7.3 Comments

First, we note that the fragility score of the integrated approach in the deterministic case is higher than for the two-stage approach. This could be explained by the fact that the integrated approach tends to create blocks with less idle time, making the primary delays accumulate and the fragility score higher.

In the stochastic case, the column generation for the integrated approach does not converge within the time limit. Still, we are providing a solution better than the two-stage solution both in terms of number of working days and of fragility, with a fragility gain of more than 50%. The solution to the largest instance with 1,815 trains is even as good as the integrated deterministic approach for both criteria. This could be explained by the fact that the time limit of 3600 s on the ILP resolution favors a smaller final gap when the number of variables is smaller (which is the case here, even though the resolution time is larger). It could also be that the variables generated for the stochastic case have better quality for the ILP than the ones generated for the deterministic case.

5.8 Final remarks

5.8.1 Convergence speed

In Theorem 5.6.2, we were not able to provide an absolute upper bound on the value of N, even when carefully choosing the sequence of alternative primary delays. Note that such a bound on N would make it possible to provide an answer even when the problem is unfeasible (under the original primary delays). Denoting by B such a bound for a sequence $(P^0_{t,n})_{t,n}$ verifying the assumptions of Theorem 5.6.2, we can solve the problem under $(P^0_{t,B})_t$. We can then check for this solution if the stochastic constraint with the original delays is

verified. If it is, then we have a feasible (optimal) solution to the desired problem. Otherwise, Theorem 5.6.2 implies that the desired problem is infeasible.

To simplify the following discussion, given the original primary delays $(P_t)_t$ with cumulative distribution functions denoted by $(F_t)_t$, we take the sequence of alternative primary delays $(\widetilde{P}_{t,n})_{t,n}$, with cumulative distribution functions denoted by $(\widetilde{F}_{t,n})_{t,n}$, such that

$$\widetilde{F}_{t,n}(x) = \begin{cases} \widetilde{F}_t(0) & \text{if } x = 0\,, \\ \widetilde{F}_t(1) & \text{if } x > M\,, \\ \widetilde{F}_t(k\frac{M}{n}) & \text{else, where } k = \lceil \frac{nx}{M} \rceil\,, \end{cases}$$

where M is large enough so that for primary delays larger than M, the fragility scores for the $(P_t)_t$ and for $(\widetilde{P}^0_{t,n})_{t,n}$ coincide (typically larger than the maximal duration of a duty).

Note that such alternative primary delays verify $\widetilde{P}_{t,n} \leqslant_{\text{st}} P_t$ for every t and n. Moreover, supposing that the $(F_t)_t$ are continuous functions, even though we do not have $\widetilde{P}_{t,n} \xrightarrow{d} P_t$ for every t, we still have, by the choice of M, that $\lim_{n\to+\infty} \mathbb{E}_{(\widetilde{P}_{t,n})_t}[\sigma(H^0_n, b)] = \mathbb{E}_{(P_t)_t}[\sigma(H, b)]$ (where $\widetilde{H}_n$ corresponds to the random arrival times under the primary delays $(\widetilde{P}_{t,n})_t$ and H corresponds to the random arrival times under the primary delays $(P_t)_t$). The end of the proof of Theorem 5.6.2 thus holds and there exists N such that, for all $n \geqslant N$, the set of feasible solutions under $(\widetilde{P}_{t,n})_t$ coincides with the set of feasible solutions under $(P_t)_t$.

We state the following open question, where the K-Lipschitz assumption on the cumulative distribution functions ensure their continuity and naturally introduces a parameter K which might help with bounding the index N of Theorem 5.6.2 with a constant.

Question 1. Suppose the primary delays $(P_t)_t$ have K-Lipschitz cumulative distribution functions $(F_t)_t$. Is it true that, for all $n \geqslant C(K)$, every feasible solution under $(P_t)_t$ is a feasible solution under $(P^0_{t,n})_t$, where $C(K)$ depends only on K?

5.8.2 Other risk measures

The goal of this section is to discuss other measures of the fragility of a roster. For these discussions, we recall the definition of risk measures.

Generalities

A risk measure ρ is a map taking values in $\mathbb{R} \cup \{+\infty\}$ from a set of random variables. We say that a risk measure ρ is consistent with the usual stochastic order if $X \leqslant_{\text{st}} Y$ implies $\rho(X) \leqslant \rho(Y)$. This means that delays are penalized by the risk measure [71]. As discussed in [71], this is a required property for the resource-constrained shortest path labeling algorithm to output an optimal solution.

Suppose now that the fragility of a roster r can be written in the general form $\sum_{b\in r} \rho(H, b)$ where $\rho(\cdot, b)$ is a risk measure. This generalizes the definition of the fragility of Section 5.3.2 as $\sum_{b\in r} \mathbb{E}[\sigma(H, b)]$, while still ensuring that the fragility of a roster can be decomposed over its blocks, independently of their organization into the roster. Note that until now we considered the measure $\rho(\cdot, b) := \mathbb{E}[\sigma(\cdot, b)]$ where σ is a non-decreasing map. This measure is consistent with the stochastic order, as expected for the label correcting algorithm to be valid. As long as ρ is a monotone risk measure, Proposition 5.6.1 holds for the generalized fragility model.

Conditional value-at-risk

A popular risk measure in stochastic optimization (as well as other domains) is the *conditional value-at-risk* (also called superquantile or expected shortfall), as it generalizes the expectation measure, as well as the worst case scenario. Informally, given a confidence level β, the

conditional value-at-risk measures the average value of the worst $1-\beta$ cases of a random variable. It also presents desirable properties, among which the consistency with the stochastic order.

The Conditional Value-at-Risk of level $\beta \in [0,1)$ is defined as

$$\mathrm{CVaR}_\beta(X) = \mathbb{E}\left[X | X \geqslant \mathrm{VaR}_\beta(X)\right]$$

where $\mathrm{VaR}_\beta(X) = \inf\{x \in \mathbb{R} \colon \mathbb{P}(X \leqslant x) \geqslant \beta\}$ is the Value-at-Risk of level β (also called the quantile of level β).

Beyond its interpretability, a key practical reason for the popularity of the conditional value-at-risk in stochastic optimization is computational. Stochastic optimization problems are typically handled by representing uncertainty through a finite set of scenarios. In this case, the conditional value-at-risk can be linearized [74], and is thus no harder to handle in a scenario-based optimization model than the expectation itself.

In our setting though, the discussion is different. As $\sigma(H,b)$ counts the number of disruptions in the block b under random train arrival times H, it seems natural to want to penalize extreme disruptions. We would then like to replace the expected value with the conditional value-at-risk, and write the fragility of a roster r as $\mathrm{CVaR}_\beta\left(\sum_{b\in r}\sigma(H,b)\right)$. That being said, it is not clear whether this could be written as $\sum_{b\in r}\rho(H,b)$ where $\rho(\cdot,b)$ is a risk measure consistent with the stochastic order.

5.8.3 Variants of the stochastic crew management problem

Minimizing the fragility

As the number of working days is a criterion that corresponds to large expenses for SNCF, freight experts have expressed their wish for solutions to a variant of the stochastic crew management problem, where the threshold constraint is on the number of working days and the objective function is the total expected fragility. This section aims to discuss Proposition 5.6.1 and Theorem 5.6.2 for this variant, as well as some numerical results on the SNCF instances.

This variant can be modeled in the same fashion as the original stochastic crew management problem, with the following ILP.

$$\begin{aligned}
\text{minimize} \quad & \sum_{i\in I}\sum_{b\in B_i} \varphi(b) x_{b,i} & & \\
\text{s.t.} \quad & \sum_{i\in I}\sum_{b\in B_i : t\in b} x_{b,i} \geqslant 1 & & \forall t \in T \\
& \sum_{i\in I}\sum_{b\in B_i} w_b x_{b,i} \leqslant W & & \\
& x_{b,i} \in \{0,1\} & & \forall i \in I, \forall b \in B_i\,.
\end{aligned} \tag{Covering SCM - variant 1}$$

The threshold parameter W on the cost is given in the instance as a replacement for the original fragility threshold S. The variables $x_{b,i}$ take the value 1 if the block $b \in B_i$ is selected for the team $i \in I$.

The discussions of Section 5.6 extend partially to this variant. First, note that the feasibility of a solution does not depend on the primary delays. For this variant, Proposition 5.6.1 is then also true and its proof is straightforward.

The following lemma provides an additional result, which can be used to bound the optimal value of (Covering SCM - variant 1) under partially known delays using finite-support random variables as in Section 5.6.

Lemma 5.8.1. *Suppose that for each train t we are given two random variables P_t^0 and P_t^1 for primary delays, with $P_t^0 \leqslant_{\mathrm{st}} P_t^1$ for all t. Then the optimal value under the primary delays $(P_t^0)_t$ is upper bounded by the optimal value under $(P_t^1)_t$.*

	Two-stage approach			Integrated approach: deterministic case			Integrated approach: stochastic case				
	Time	Nb working days	Fragility	Time	Nb working days	Fragility	Threshold W	Time	Nb working days	LB	UB
116 trains	1 min	50	0.970	1 h 8 min	41	2.748	50	26 min	50	0.0	0.0
280 trains	3 min	228	2.450	1 h 9 min	207	6.440	228	4 min	228	0.0	0.0
926 trains	26 min	832	8.538	6 h 3 min	724	28.46	832	2 h 37 min	832	0.0	0.3
1,815 trains	64 min	1,633	14.900	10 h 2 min	1,477	58.980	1,633	1 h 45 min	1,633	0.0	0.0
1,815 trains bis	64 min	1,633	18.410	10 h 2 min	1,477	79.250	1,633	2 h 24 min	1,633	0.0	0.0

Table 5.4: Numerical experiments on the instances for the variant. The two-stage approach and the integrated approach in the deterministic case aim at minimizing the number of working days (summarized in the columns "Nb working days" which refer to the total number of working days in the solution). For the stochastic integrated approach, the column "LB" refers to the optimal value of the linear relaxation (with the fragility as the objective function), the column "UB" refers to the upper bound obtained by solving the integer linear program.

Proof. Consider an optimal solution under $(P_t^1)_t$. Denote by H^0 and H^1 the arrival times of all trains under respectively primary delays $(P_t^0)_t$ and primary delays $(P_t^1)_t$. Since $P_t^0 \leqslant_{\text{st}} P_t^1$ for all t, we have $H_t^0 \leqslant_{\text{st}} H_t^1$ for all t as well, and $\mathbb{E}[\sigma(H^0, b)] \leqslant \mathbb{E}[\sigma(H^1, b)]$. We then have $\sum_b \mathbb{E}[\sigma(H^0, b)] \leqslant \sum_b \mathbb{E}[\sigma(H^1, b)]$ where the index b ranges over all blocks of the solution. This solution is also feasible under $(P_t^0)_t$, thus the optimal value under $(P_t^0)_t$ is upper bounded by $\sum_b \mathbb{E}[\sigma(H^0, b)]$, which is itself upper bounded by the optimal value under $(P_t^1)_t$. □

We can also recover a convergence result to approximate the optimal value under the original primary delays. Contrary to Theorem 5.6.2, we are however not able to find the optimal value in finitely many steps.

Proposition 5.8.2. *Suppose that the problem is feasible (under the original primary delay distributions $(P_t)_t$). Consider any sequence of alternative primary delays $(P_{t,n}^0)_{t,n}$ with $P_{t,n}^0 \leqslant_{\text{st}} P_t$ for every t and n and $P_{t,n}^0 \xrightarrow{d} P_t$ for every t. Then the optimal value under $(P_{t,n}^0)_t$ converges towards the optimal value under $(P_t)_t$ as n goes to infinity.*

Proof. As in the proof of Theorem 5.6.2, for every block b we have $\lim_{n\to+\infty} \mathbb{E}_{P_n^0}[\sigma(H_n^0, b)] = \mathbb{E}_P[\sigma(H, b)]$, where H_n^0 corresponds to the random arrival times under the primary delays $P_n^0 = (P_{t,n}^0)_t$ and H corresponds to the random arrival times under the primary delays $P = (P_t)_t$. Using Lemma 5.8.1, the optimal value under P_n^0 is upper bounded by the optimal value under P. Up to extracting a subsequence, we can assume, by finiteness, that every optimal solution under P_n^0 is the same element x' of $\prod_{i\in I}\{0,1\}^{B_i}$. Since the set of feasible solutions is the same for the two problems, denoting by $x^\star$ an optimal solution under P, we have:

$$\begin{aligned}\sum_{i\in I}\sum_{b\in B_i} \mathbb{E}_P[\sigma(H,b)]x_{b,i}^\star &\leqslant \sum_{i\in I}\sum_{b\in B_i} \mathbb{E}_P[\sigma(H,b)]x_{b,i}' \\ &= \lim_{n\to+\infty} \sum_{i\in I}\sum_{b\in B_i} \mathbb{E}_{P_n^0}[\sigma(H_n^0,b)]x_{b,i}' \\ &\leqslant \lim_{n\to+\infty} \sum_{i\in I}\sum_{b\in B_i} \mathbb{E}_{P_n^0}[\sigma(H_n^0,b)]x_{b,i}^\star\end{aligned}$$

This shows that the optimal value under P and the limit of the optimal value under P_n^0 are equal which is the desired conclusion. □

Additional numerical experiments have been run for this variant, with the same experimental setting as the one discussed in Section 5.7. The results are summarized in Table 5.4, which details the caption of the columns.

In this variant, in almost all cases, we are able to find solutions with the same number of working days as the current two-stage approach that have no fragility to delays. This

means that in this solution, even the worst case scenario (i.e., when every train realizes its maximum primary delay) does not disrupt any trains nor duties. Indeed, in the solutions, long breaks between duties are long enough to absorb large delays and most duties cover one or two trains (which is not uncommon for freight operations), making delay propagation and accumulation manageable. Moreover, when a duty covers multiple trains, the longest one (in terms of travel time) is typically scheduled last, which limits the risk of that train's departure being pushed beyond the disruption threshold.

Delays impacting the number of working days

Finally, we consider another variant of the stochastic crew management problem in which the train delays impact the feasibility of a solution through the fragility threshold constraint but also the objective function via an additional stochastic term.

This variant could be relevant for SNCF if we were to assume for example that some train or duty disruptions at the end of a block induce additional working days for the assigned driver instead of needing to call for a backup driver.

We discuss the extensions of the results of the previous section for this variant. Rather than the fragility score $\varphi(b)$ of the block b used so far, we now consider more general random scores, which we allow to appear both in the objective function and in the constraints.

Assume then that this variant can be modeled by the following ILP, where $\widetilde{w}_b$ is a parameter, the variables $x_{b,i}$ take the value 1 if the block $b \in B_i$ is selected for the team $i \in I$, and $h \mapsto \sigma^S(h,b)$ and $h \mapsto \sigma^W(h,b)$ are non-decreasing sums of indicator functions over open sets. Formally, this means that $\sigma^S(h,b) = \sum_i \mathbb{1}(h \in \mathcal{O}_i^S)$ and $\sigma^W(h,b) = \sum_i \mathbb{1}(h \in \mathcal{O}_i^W)$ where $\mathcal{O}_i^S$ and $\mathcal{O}_i^S$ are open sets. In particular, note that a union of open intervals is an open set.

$$
\begin{aligned}
\text{minimize} \quad & \sum_{i \in I} \sum_{b \in B_i} (w_b + \mathbb{E}[\sigma^W(H,b)]) x_{b,i} & & \\
\text{s.t.} \quad & \sum_{i \in I} \sum_{b \in B_i : t \in b} x_{b,i} \geqslant 1 & & \forall t \in T \\
& \sum_{i \in I} \sum_{b \in B_i} \mathbb{E}[\sigma^S(H,b)] x_{b,i} \leqslant S & & \\
& x_{b,i} \in \{0,1\} & & \forall i \in I, \forall b \in B_i \,.
\end{aligned}
\tag{Covering SCM - variant 2}
$$

For this variant, Lemma 5.8.1 and Proposition 5.8.2 still hold and are stated as the following corollaries.

Corollary 5.8.3. *Suppose that for each train t we are given two random variables P_t^0 and P_t^1 for primary delays, with $P_t^0 \leqslant_{\text{st}} P_t^1$ for all t. Then the optimal value under the primary delays $(P_t^0)_t$ is upper bounded by the optimal value under $(P_t^1)_t$.*

Proof. Consider an optimal solution under $(P_t^1)_t$. It is feasible under $(P_t^0)_t$ as in the proof of Proposition 5.6.1, with σ^W non-decreasing. Its value is an upper bound on the optimal value under $(P_t^0)_t$ as in the proof of Lemma 5.8.1, with σ^S non-decreasing. □

Corollary 5.8.4. *Suppose that the problem is feasible (under the original primary delay distributions $(P_t)_t$). Consider any sequence of alternative primary delays $(P_{t,n}^0)_{t,n}$ with $P_{t,n}^0 \leqslant_{\text{st}} P_t$ for every t and n and $P_{t,n}^0 \xrightarrow{d} P_t$ for every t. Then the optimal value under $(P_{t,n}^0)_t$ converges towards the optimal value under $(P_t)_t$ as n goes to infinity.*

Proof. Using Corollary 5.8.3, the optimal value under $P_n^0 \coloneqq (P_{t,n}^0)_t$ is upper bounded by the optimal value under $P \coloneqq (P_t)_t$.

As in the proof of Theorem 5.6.2, since σ^S is a non-decreasing indicator function over an open set, there exists N such that, for all $n \geqslant N$, the sets of feasible solutions under P_n^0 and under P coincide. The conclusion then follows the outline of the proof of Proposition 5.8.2, where the inequality remains valid as σ^W takes non-negative values. □

PART

II

Branch-and-price(-and-cut) for line planning with minimal traveling time

Introduction

***

As explained in the introduction of the first part of this work, the *line planning* step, which consists of determining lines, i.e., sequences of stations, and their frequency in order to meet passenger demand is an important step of the planning process for rail passenger transportation. As this step comes first in the operational planning, the decisions taken for line planning greatly influence the solutions found in the forthcoming steps (timetabling, vehicle scheduling, crew scheduling, etc.). Moreover, this problem is relevant for a number of applications as the planning guideline detailed previously is not limited to rail: other public transportation operators follow the same steps, with modifications in the constraints and costs considered at each step. This part aims to discuss two optimization models for the line planning problem, thus turning from Part I's focus on crew-centered planning to passenger-centered network design. Although the operational object changes, the scientific challenges remain similar as we face the same scaling issues in both parts. For the line planning problem, the existing compact formulation reaches a bottleneck on large-scale networks, and column generation based methods can overcome these limitations for a path-based formulation.

Schmidt and Schöbel [78] highlight that the transit line planning step itself can be seen as multiple sequential problems, detailed in Figure 5.2. First comes the *line generation* problem, aiming to construct a set of candidate lines, called the *line pool*. Second comes the *line selection* problem, aiming to select lines to be operated within the line pool, resulting in the *line plan*. Finally comes the *line frequency-setting* problem, aiming to assign frequencies to the selected lines. The transit network design problem integrates line generation and selection at once. The line planning problem integrates line selection and frequency setting. The line planning with full pool (also called the transit network design and frequency setting) integrates line generation, line selection and frequency setting. In this work, we focus on the line planning problem.

Schöbel dedicated an extensive survey to the line planning problem [80]. She points out that this problem is NP-hard even in very basic versions, and highlights two easy special

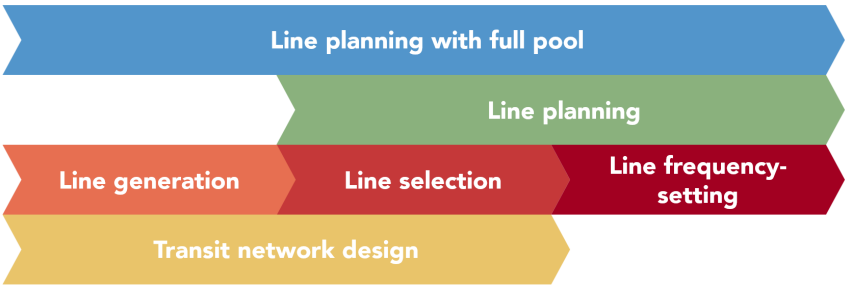


Figure 5.2: Different problems in the line planning step of the operational planning process

cases. She also details a number of models considered for the line planning problem, namely cost-oriented models, passenger-oriented models, game-theoretic models, and location-based models. The models developed in this part fall within the passenger-oriented family, as they aim at minimizing passenger travel time rather than operational cost. A first ILP minimizing total passenger travel time via passengers routing, under a budget constraint, appears in the article by Schöbel and Scholl [81]. As argued in [78], the running frequencies of each line are integer quantities representing the number of vehicles operating the line on a given time period. Other works [12, 38] suggest taking frequencies as (integer) multiples of a given system frequency, or even among a small set of allowed frequencies.

The number of variables of the formulation in [81] becomes rapidly intractable when the size of the network and the number of lines are large. This is a well known observation on arc-based formulations for multi-commodity flow problems.

The remainder of this part consists of a single chapter, which introduces the arc-based formulation for the line planning problem, derives a path-based reformulation via Dantzig–Wolfe reformulation, and solves it using a branch-and-price(-and-cut) algorithm. Numerical experiments on real-world networks conclude the chapter, comparing both formulations and assessing the benefits of the path-based approach.

Chapter 6

Branch-and-price

***

This chapter results from a research stay in the Optimization Research Group at RPTU Kaiserslautern-Landau, under the supervision of Anita Schöbel.

6.1 Problem formulation . . . 87
6.2 Literature review . . . 88
6.2.1 Passenger-oriented line planning . . . 89
6.2.2 Solution methods . . . 89
6.2.3 Different formulations . . . 89
6.3 Model . . . 90
6.3.1 Change-and-go network . . . 90
6.3.2 Arc-based formulation . . . 91
6.3.3 Path-based formulation . . . 92
6.4 Branch-and-price(-and-cut) . . . 94
6.4.1 Column generation . . . 94
6.4.2 Pricing subproblem . . . 94
6.4.3 Branching strategy . . . 95
6.4.4 Valid inequalities . . . 95
6.4.5 Accelerating for the first integer solution . . . 97
6.5 Numerical experiments . . . 98
6.5.1 Experimental setting and instances . . . 98
6.5.2 Results . . . 98
6.5.3 Comments . . . 98

6.1 Problem formulation

An instance of the line planning problem is formed by the following elements, where demands and frequencies are provided on a same planning period:

- A directed graph $G = (V, A)$, called the public transportation network, where each vertex represents a station and each arc represents a route between two stations. Moreover, each arc $a \in A$ comes with a maximal frequency of vehicles allowed to traverse the

arc over the planning period, denoted by $f_a^{\max}$. This plays the role of a capacity on the maximum number of vehicles allowed to travel through arc $a \in A$ on the planning period.

- An integer passenger demand matrix with coefficients $w_{o,d} \in \mathbb{Z}_{\geqslant 0}$ representing, for an entry $(o, d) \in V^2$, the number of passengers wanting to travel from o to d over the planning period. We denote by $\mathcal{D}$ the set of pairs $(o, d) \in V^2$ such that $w_{o,d} > 0$.
- A line pool, i.e., a collection $\mathcal{L}$ of lines. A line is a sequence of vertices, representing the stations serviced by the vehicles operating that line. For a vertex $v \in V$, we denote by $\mathcal{L}(v) \subseteq \mathcal{L}$ the set of lines containing v. We also denote by $A(\ell)$ the arcs serviced by a line $\ell \in \mathcal{L}$. Each line $\ell \in \mathcal{L}$ comes with a cost C_ℓ, representing the financial amount needed to operate one vehicle on this line.
- A vehicle capacity N_ℓ for the vehicles of line $\ell \in \mathcal{L}$.
- A budget B.
- Changing times $\tau^{\text{changing}}(\ell_1, \ell_2, v)$ for two lines ℓ_1 and ℓ_2, and a station $v \in V$, representing the time to change from line ℓ_1 to line ℓ_2 at station $v \in V$.
- Boarding times $\tau^{\text{boarding}}(\ell, v)$ for a line ℓ and a station $v \in V$, representing the time to board line ℓ at station $v \in V$.
- Disembarking times $\tau^{\text{disembarking}}(\ell, v)$ for a line ℓ and a station $v \in V$, representing the time to disembark line ℓ at station $v \in V$.
- Driving times $\tau^{\text{drive}}(\ell, v_1, v_2)$ for a line ℓ and two stations v_1 and v_2 such that $(v_1, v_2) \in A$, representing the time to drive from station v_1 to station v_2 on line ℓ.

The line planning problem consists of finding the lines to operate and their integer frequency, i.e., the number of vehicles to operate on the planning period, minimizing traveling time such that demands are met and minimal and maximal arc frequencies are respected, while complying with the budget. More precisely, meeting demands means that, for every line, the total number of passengers traveling on any of its arcs over the planning period does not exceed the vehicle capacity multiplied by the number of vehicles operated on that line over the same period (i.e., the line frequency). We stress that this constraint is purely aggregated over the planning period: it does not involve any finer temporal structure, such as the actual departure times of individual vehicles. Note that the frequencies are required to be integers because, in real-world applications, frequencies often have to produce a readable timetable.

Line planning with minimal traveling time (LPT)

Input. Public transportation network $G = (V, A)$ with maximal frequencies over the arcs, line pool $\mathcal{L}$, integer demands $w_{o,d}$ for all $(o, d) \in \mathcal{D}$ (number of passengers traveling from o to d in a planning horizon), budget B.

Output. Lines to operate and their frequencies such that: demands are met, the budget is not exceeded, the maximal arc frequencies are respected, and the overall traveling time is minimal.

This problem is NP-hard because a basic version of the line planning problem can easily be reduced to it, and such a basic version has been proven to be NP-hard [20].

6.2 Literature review

The line planning problem asks which lines to operate and at what frequency, typically subject to a budget and capacity constraints, with the objective of minimizing passenger travel time or operating cost.

Early formulations already established the computational difficulty of the problem. Bussieck [20] showed that several variants of line planning are NP-hard.

6.2.1 Passenger-oriented line planning

The literature distinguishes between *cost-oriented* models, which minimize the number of vehicles or operational cost, and *passenger-oriented* models, which minimize travel time or maximize direct travelers [80]. These two objectives often contradict, and various multi-objective formulations have been proposed. This chapter focuses on a passenger-oriented objective within a pool-based setting, where candidate lines are given and the optimization selects a subset along with integer frequencies.

Schöbel and Scholl [81] introduced the *change-and-go network* as a canonical modeling framework for passenger-oriented line planning. This change-and-go network allows great flexibility on the modeling of passenger routing and the associated costs, while encoding passenger trips as flows. In particular, it can capture the stations where passengers may board and alight, the lines along which passengers are driven, the line transfers.

Along with the introduction of this change-and-go network, Schöbel and Scholl [81] propose an ILP formulation for the line planning problem with minimal total traveling time. The present chapter builds directly on this formulation by proposing a path-based counterpart. To our knowledge, such a path-based formulation is new, except for the recent and independent work by van Lieshout and van Rossum [88].

Additionally, papers extended the change-and-go network to model variants of the line planning problem. Lu et al. [60] introduce the line planning problem under crowding, which adds a quadratic penalty crowding term in the users perceived travel time. Goerigk and Schmidt propose a bilevel optimization model with the lower level optimization problem modeling user-optimal routing [43].

6.2.2 Solution methods

Exact methods for line planning have largely relied on integer programming. Arc-based ILP formulations, while compact in the number of constraints, become memory-intensive as instances grow, since the number of arc-flow variables scales with both the change-and-go network size and the number of origin–destination pairs.

Path-based formulations offer an alternative: instead of routing flows arc by arc, each origin-destination pair is served by a set of paths, generating a model with potentially far fewer active variables at any point. The Dantzig–Wolfe reformulation provides an automatic way to derive such formulations by reformulating flow constraints of the arc-based formulation. While such a reformulation does not reach for a better relaxation, as the pricing subproblem has the integrality property [30], its resolution with column generation based approaches aims to overcome the memory limitations of compact arc-based formulations.

6.2.3 Different formulations

A key difficulty of the arc-based formulation on the change-and-go network is that the number of variables and constraints grows rapidly with the number of lines and stops, making exact solution of large instances computationally prohibitive. This scalability limitation is central to the motivation of this chapter's path-based approach, and has recently been examined in depth by van Lieshout and van Rossum [88].

The article by van Lieshout and van Rossum was made available on arXiv during the writing of this chapter. The authors conduct a large-scale comparative study of four models for line planning: from a path-based formulation on the change-and-go network to formulations on smaller networks (like the original public transportation network) where the transfer costs are ignored. They propose solution methods for each model, ranging from integer programming for smaller models to Benders decomposition and branch-and-price.

In particular, they propose a branch-and-price-and-cut resolution for a path-based formulation on the change-and-go network, which appears to be competitive against the other

formulations only on small or easy instances from the Dutch and the Swiss railway networks. Their formulation and resolution approach largely overlaps the work of this chapter, with a difference lying in the valid inequalities and accelerating heuristics. As their work was made public in the writing of this chapter, we kept the discussion below unchanged from before we became aware of their work and refer to their paper for further discussions on the different formulations and their numerical analysis on large networks.

6.3 Model

6.3.1 Change-and-go network

In order to write the line planning model with minimal traveling time as an integer linear program, we use the change-and-go network $\mathcal{G} = (\mathcal{V}, \mathcal{A})$, first introduced by Schöbel and Scholl [81].

They define this larger directed graph as follows. The set of vertices $\mathcal{V}$ is defined as the union of origin nodes $\mathcal{V}_{\text{origin}}$, destination nodes $\mathcal{V}_{\text{destination}}$, and change-and-go nodes $\mathcal{V}_{\text{change\&go}}$, each corresponding to a station and line pair. Formally,

$$\mathcal{V} = \mathcal{V}_{\text{origin}} \cup \mathcal{V}_{\text{destination}} \cup \mathcal{V}_{\text{change\&go}}$$

where

$$\begin{aligned}
\mathcal{V}_{\text{origin}} &:= \big\{(o, \varnothing) \colon (o, d) \in \mathcal{D}\big\}, \\
\mathcal{V}_{\text{destination}} &:= \big\{(\varnothing, d) \colon (o, d) \in \mathcal{D}\big\}, \\
\mathcal{V}_{\text{change\&go}} &:= \big\{(v, \ell) \colon v \in V, \ell \in \mathcal{L}(v)\big\}.
\end{aligned}$$

The set of arcs $\mathcal{A}$ of this change-and-go network is defined as the union of getting in arcs $\mathcal{A}_{\text{in}}$, getting out arcs $\mathcal{A}_{\text{out}}$, driving arcs $\mathcal{A}^{\ell}_{\text{go}}$ for each line ℓ, and line changing arcs $\mathcal{A}_{\text{change}}$. Formally,

$$\mathcal{A} = \mathcal{A}_{\text{in}} \cup \mathcal{A}_{\text{out}} \cup \bigcup_{\ell \in \mathcal{L}} \mathcal{A}^{\ell}_{\text{go}} \cup \mathcal{A}_{\text{change}}$$

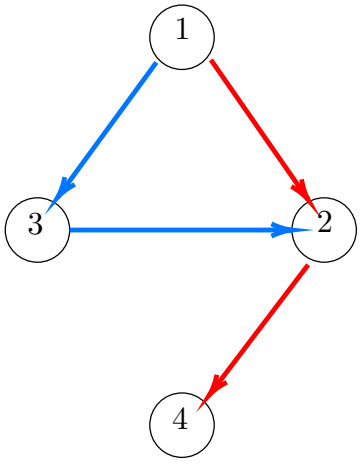
where

$$\begin{aligned}
\mathcal{A}_{\text{in}} &:= \big\{((o, \varnothing), (o, \ell)) \colon (o, d) \in \mathcal{D}, \ell \in \mathcal{L}(o)\big\} \subseteq \mathcal{V}_{\text{origin}} \times \mathcal{V}_{\text{change\&go}}, \\
\mathcal{A}_{\text{out}} &:= \big\{((d, \ell), (\varnothing, d)) \colon (o, d) \in \mathcal{D}, \ell \in \mathcal{L}(d)\big\} \subseteq \mathcal{V}_{\text{change\&go}} \times \mathcal{V}_{\text{destination}}, \\
\mathcal{A}^{\ell}_{\text{go}} &:= \big\{((v_1, \ell), (v_2, \ell)) \colon (v_1, v_2) \in A(\ell)\big\} \subseteq \mathcal{V}_{\text{change\&go}} \times \mathcal{V}_{\text{change\&go}}, \\
\mathcal{A}_{\text{change}} &:= \big\{((v, \ell_1), (v, \ell_2)) \colon v \in V, \ell_1 \neq \ell_2 \in \mathcal{L}(v)\big\} \subseteq \mathcal{V}_{\text{change\&go}} \times \mathcal{V}_{\text{change\&go}}.
\end{aligned}$$

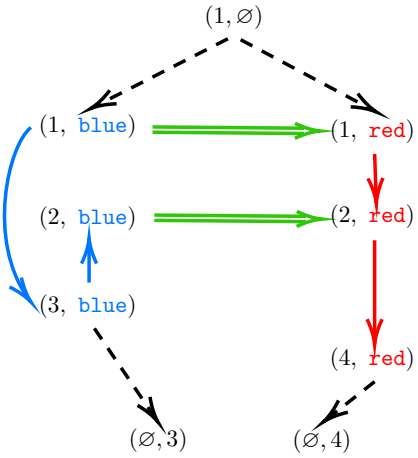
Figure 6.1 provides an example of a small public transportation network with two lines in the line pool, along with the associated change-and-go network.

We define costs on each arc of the change-and-go network, encapsulating the overall time spent by the users from their origin to their destination. Let us then set, for each $a \in \mathcal{A}$, the non-negative cost associated with arc a in the change-and-go network:

$$c_a = \begin{cases}
\tau^{\text{boarding}}(\ell, v) & \text{if } a = ((v, \varnothing), (v, \ell)) \in \mathcal{A}_{\text{in}} \\
\tau^{\text{disembarking}}(\ell, v) & \text{if } a = ((v, \ell), (\varnothing, v)) \in \mathcal{A}_{\text{out}} \\
\tau^{\text{drive}}(\ell, v_1, v_2) & \text{if } a = ((v_1, \ell), (v_2, \ell)) \in \bigcup_{\ell \in \mathcal{L}} \mathcal{A}^{\ell}_{\text{go}} \\
\tau^{\text{changing}}(\ell_1, \ell_2, v) & \text{if } a = ((v, \ell_1), (v, \ell_2)) \in \mathcal{A}_{\text{change}}.
\end{cases}$$



(a) A public transportation network with four stations ($V = \{1, 2, 3, 4\}$), customer traveling from station 1 to station 4 and from station 1 to station 3 ($\mathcal{D} = \{(1, 4), (1, 3)\}$ and with a blue line and a red line ($\mathcal{L} = \{\text{blue}, \text{red}\}$).



(b) The associated change-and-go network. The getting in and getting out arcs are represented in dashed black, the driving arcs in the color of the corresponding line, and the line changing arcs in double green.

Figure 6.1: An illustration of a public transportation network with the associated change-and-go network.

6.3.2 Arc-based formulation

We can write the following integer linear program to solve the line planning problem, introduced by Schöbel and Scholl [81].

$$\text{minimize} \quad \sum_{(o,d)\in\mathcal{D}} \sum_{a\in\mathcal{A}} c_a x^a_{o,d} \tag{LPT}$$

$$\text{s.t.} \quad \sum_{(o,d)\in\mathcal{D}} x^a_{o,d} \leqslant N_\ell f_\ell \qquad \forall \ell \in \mathcal{L}, a \in \mathcal{A}^\ell_{\text{go}} \tag{6.1a}$$

$$\sum_{\ell\in\mathcal{L}} C_\ell f_\ell \leqslant B \tag{6.1b}$$

$$\sum_{\ell\in\mathcal{L}\,:\, a\in A(\ell)} f_\ell \leqslant f^{\max}_a \qquad \forall a \in A \tag{6.1c}$$

$$\sum_{a\in\delta^+((o,\varnothing))} x^a_{o,d} = w_{o,d} \qquad \forall (o,d) \in \mathcal{D} \tag{6.1d}$$

$$\sum_{a\in\delta^-((\varnothing,d))} -x^a_{o,d} = -w_{o,d} \qquad \forall (o,d) \in \mathcal{D} \tag{6.1e}$$

$$\sum_{a\in\delta^+(v)} x^a_{o,d} - \sum_{a\in\delta^-(v)} x^a_{o,d} = 0 \qquad \forall (o,d) \in \mathcal{D}, v \in \mathcal{V}\backslash\{(o,\varnothing),(\varnothing,d)\} \tag{6.1f}$$

$$x^a_{o,d} \in \mathbb{Z}_{\geqslant 0} \qquad \forall (o,d) \in \mathcal{D}, a \in \mathcal{A} \tag{6.1g}$$

$$f_\ell \in \mathbb{Z}_{\geqslant 0} \qquad \forall \ell \in \mathcal{L}\,. \tag{6.1h}$$

The variable $x^a_{o,d}$ is the number of customers traveling from o to d through arc a in the planning period, and the variable f_ℓ is the number of vehicles operated for the line ℓ in the planning period.

Constraints (6.1a) ensure that each line runs at a frequency high enough so that it can convey all passengers using each arc of this line. Constraint (6.1b) ensures that the budget is not exceeded. Constraints (6.1c) are maximal capacity constraints on the arcs of the original public transportation network G, ensuring that the maximal arc frequencies are respected. Constraints (6.1d), (6.1e), and (6.1f) are flow conservation constraints, ensuring that passengers follow a path from their origin to their destination. This also ensures that all passengers can be conveyed in a solution.

6.3.3 Path-based formulation

As suggested in tutorials addressing the multicommodity flow problem, an arc-based formulation might be inefficient in practice when the size of the graph is large. In comparison, a path-based formulation, obtained via a Dantzig–Wolfe reformulation of the flow constraints and solved using column generation, is sometimes preferable [73].

This comment might be even more relevant given that the size of the change-and-go network is very dependent on both the size of the original public transportation network and the number and structure of the lines in the line pool since:

$$|\mathcal{A}| = \sum_{o\in V\,:\,(o,d)\in\mathcal{D}} |\mathcal{L}(o)| + \sum_{d\in V\,:\,(o,d)\in\mathcal{D}} |\mathcal{L}(d)| + \sum_{l\in\mathcal{L}} |A(\ell)| + \sum_{v\in V} 2|\mathcal{L}(v)| - 1\,.$$

Thus, when solving large-size instances, the number of $x^a_{o,d}$ variables makes it difficult for ILP solvers to solve (or even write) the model.

In this section, we propose a path-based formulation for the line planning problem. This path-based formulation is a Dantzig–Wolfe reformulation with respect to the flow constraints (6.1d), (6.1e) and (6.1f). We refer to the book *Branch-and-Price* by Desrosiers, Lübbecke, Desaulniers, and Gauthier [30] for details on Dantzig–Wolfe reformulations for integer linear programs.

More precisely, for $(o,d)\in\mathcal{D}$, denoting by Δ the incidence matrix, and by $\gamma_{o,d}\in\mathbb{Z}^{\mathcal{V}}$ the corresponding right-hand side, we define the integer set:

$$X_{o,d} \coloneqq \{\boldsymbol{x}\in\mathbb{Z}^{\mathcal{A}}_{\geqslant 0}\colon \Delta\boldsymbol{x}=\gamma_{o,d}\} = \mathbb{Z}^{\mathcal{A}}_{\geqslant 0}\cap\{\boldsymbol{x}\in\mathbb{R}^{\mathcal{A}}_{\geqslant 0}\colon \Delta\boldsymbol{x}=\gamma_{o,d}\}\,.$$

The set $\{\boldsymbol{x}\in\mathbb{R}^{\mathcal{A}}_{\geqslant 0}\colon \Delta\boldsymbol{x}=\gamma_{o,d}\}$ is a non-empty (integer) polyhedron. The integrality property comes from the fact that the parameters $\gamma_{o,d}$ are integers and the matrix Δ is totally unimodular.

Denote by $P_{o,d}$ the set of elementary paths from $(o,\varnothing)$ to $(\varnothing,d)$, and by R the set of elementary cycles (independent of the o-d pair). For $p\in P_{o,d}$, let $z^p_{o,d}$ be the vector in $Z^{\mathcal{A}}_{>0}$ obtained by sending $w_{o,d}$ units of flow along p, and for $r\in R$, let z^r be the vector in $Z^{\mathcal{A}}_{>0}$ obtained by sending one unit of flow along r. The extreme points of $\operatorname{conv}(X_{o,d})$ are exactly $\{z^p_{o,d}\}_{p\in P_{o,d}}$, and the normalized extreme rays are exactly $\{z^r\}_{r\in R}$.

We then have, using Theorem 4.1 (p.178) of [30], that:

$$X_{o,d} = \left\{\boldsymbol{x}\in\mathbb{Z}^{\mathcal{A}}_{\geqslant 0}\colon \boldsymbol{x} = \sum_{p\in\mathcal{P}} z^p_{o,d}\lambda^p_{o,d} + \sum_{r\in R} z^r\lambda^r_{o,d}, \sum_{p\in\mathcal{P}}\lambda^p_{o,d}=1, \lambda^p_{o,d}\in\mathbb{R}_{\geqslant 0}, \lambda^r_{o,d}\in\mathbb{R}_{\geqslant 0}\right\}$$

Since the cost $c_r=\sum_{a\in r} c_a$ of every cycle $r\in R$ is non-negative, there exists an optimal solution of (LPT) which decomposes without any extreme rays.

The reformulation then writes as follows, with $c_p=\sum_{a\in p} c_a$ for all $p\in P_{o,d}$.

$$\text{minimize} \sum_{(o,d)\in\mathcal{D}}\sum_{p\in P_{o,d}} c_p w_{o,d}\lambda^p_{o,d} \qquad \text{(LPT_DW)}$$

$$\text{s.t.} \sum_{(o,d)\in\mathcal{D}} \sum_{p\in P_{o,d}\colon a\in p} w_{o,d}\lambda^p_{o,d} \leqslant N_\ell f_\ell \qquad \forall \ell \in \mathcal{L}, a \in \mathcal{A}^\ell_{\text{go}} \qquad (6.2a)$$

$$\sum_{p\in P_{o,d}} \lambda^p_{o,d} = 1 \qquad \forall (o,d) \in \mathcal{D} \qquad (6.2b)$$

$$\sum_{\ell\in\mathcal{L}} C_\ell f_\ell \leqslant B \qquad (6.2c)$$

$$\sum_{\ell\in\mathcal{L}\colon a\in A(\ell)} f_\ell \leqslant f^{\max}_a \qquad \forall a \in A \qquad (6.2d)$$

$$\sum_{p\in P_{o,d}\colon a\in p} w_{o,d}\lambda^p_{o,d} \in \mathbb{Z}_{\geqslant 0} \qquad \forall (o,d) \in \mathcal{D}, a \in \mathcal{A} \qquad (6.2e)$$

$$f_\ell \in \mathbb{Z}_{\geqslant 0} \qquad \forall \ell \in \mathcal{L}\,. \qquad (6.2f)$$

Note that if we were to have an additional minimal arc capacity $f^{\min}_a$, we would have to keep (and generate) the cycles $r \in R$. While an upper bound $f^{\max}_a$ on the arc capacity usually indicates limitations of the underlying infrastructure on the network (e.g., to ensure a regulatory noise level, or prevent road damage [78]), a lower bound $f^{\min}_a$ is sometimes included to take into consideration demand-centered indicators imposed by public institutions (e.g., to ensure service quality levels). With this additional constraint, the reformulation would write as:

$$\text{minimize} \sum_{(o,d)\in\mathcal{D}} \sum_{p\in P_{o,d}} w_{o,d}c_p\lambda^p_{o,d} + \sum_{r\in R} c_r\lambda^r_{o,d} \qquad \text{(LPT_DW_variant)}$$

$$\text{s.t.} \sum_{(o,d)\in\mathcal{D}} \sum_{p\in P_{o,d}\colon a\in p} w_{o,d}\lambda^p_{o,d} + \sum_{r\in R\colon a\in r} \lambda^r_{o,d} \leqslant N_\ell f_\ell \qquad \forall \ell \in \mathcal{L}, a \in \mathcal{A}^\ell_{\text{go}}$$

$$\sum_{p\in P_{o,d}} \lambda^p_{o,d} = 1 \qquad \forall (o,d) \in \mathcal{D}$$

$$\sum_{\ell\in\mathcal{L}} C_\ell f_\ell \leqslant B$$

$$f^{\min}_a \leqslant \sum_{\ell\in\mathcal{L}\colon a\in A(\ell)} f_\ell \leqslant f^{\max}_a \qquad \forall a \in A$$

$$\sum_{p\in P_{o,d}\colon a\in p} w_{o,d}\lambda^p_{o,d} + \sum_{r\in R\colon a\in r} \lambda^r_{o,d} \in \mathbb{Z}_{\geqslant 0} \qquad \forall (o,d) \in \mathcal{D}, a \in \mathcal{A}$$

$$f_\ell \in \mathbb{Z}_{\geqslant 0} \qquad \forall \ell \in \mathcal{L}\,.$$

We highlight that this variant is presented here as a complement but is not used in the rest of this chapter. This variant was motivated by the original arc-based formulation of Schöbel and Scholl [81], as well as the instances used for the numerical experiments.

On the theoretical side, the pricing subproblem of Section 6.4.2 only generates paths $p \in P_{o,d}$ and is not adapted to also generate the cycles $r \in R$ required by (LPT_DW_variant): this would require searching for elementary cycles of negative reduced cost, using a dedicated pricing algorithm.

6.4 Branch-and-price(-and-cut)

The goal of this section is to propose a branch-and-price algorithm with valid inequalities to solve (LPT_DW). The general algorithm for branch-and-price is described in Section A.3.

6.4.1 Column generation

We first describe the general column generation algorithm used at each node of the branch-and-price tree. Note that additional constraints corresponding to the branching decisions are eventually added to the restricted master problem. We describe and write the restricted master problem at the root node, and discuss the modifications to apply for other nodes of the branch-and-price tree.

This algorithm computes an optimal solution of the linear relaxation of (LPT_DW) using column generation, where the paths define the columns. The general algorithm for column generation is given in Algorithm 2, and details are given in Appendix A. The restricted master problem writes, for subsets $P'_{o,d} \subseteq P_{o,d}$:

$$\text{minimize} \sum_{(o,d)\in\mathcal{D}} \sum_{p\in P_{o,d}} w_{o,d} c_p \lambda^p_{o,d} \qquad \text{(RMP(LPT_DW))}$$

$$\text{s.t.} \sum_{(o,d)\in\mathcal{D}} \sum_{p\in P_{o,d}\colon a\in p} w_{o,d}\lambda^p_{o,d} \leqslant N_\ell f_\ell \qquad \forall \ell\in\mathcal{L}, a\in\mathcal{A}^\ell_{\text{go}}$$

$$\sum_{p\in P_{o,d}} \lambda^p_{o,d} = 1 \qquad \forall (o,d)\in\mathcal{D}$$

$$\sum_{\ell\in\mathcal{L}} C_\ell f_\ell \leqslant B$$

$$\sum_{\ell\in\mathcal{L}\colon a\in A(\ell)} f_\ell \leqslant f^{\max}_a \qquad \forall a\in A$$

$$\lambda^p_{o,d} \geqslant 0 \qquad \forall (o,d)\in\mathcal{D}, p\in P_{o,d}$$

$$f_\ell \geqslant 0 \qquad \forall \ell\in\mathcal{L}\,.$$

Finding a first feasible solution is not straightforward, so we enter directly the initialization phase of Algorithm 2. During this phase, the LP solver is used to compute (dual) Farkas rays, i.e., rays in the dual of the linear relaxation of (LPT_DW) (line 5) and the Farkas pricing is solved so as to identify paths preventing this ray from existing (line 6). Then during the optimization phase, the LP solver is used to identify primal-dual feasible bases (line 11) and the pricing subproblem is solved so as to identify paths with negative reduced costs (line 12).

We refer to the following section for both the Farkas pricing and the pricing subproblem resolution.

6.4.2 Pricing subproblem

There are as many pricing subproblems as o-d pairs in $\mathcal{D}$.

The pricing subproblem for a pair $(o,d)\in\mathcal{D}$ writes as follows, where $\mu_{\ell,a}\geqslant 0$ denotes the dual value associated with the vehicle capacity constraint for a line $\ell\in\mathcal{L}$ and an arc $a\in\mathcal{A}^\ell_{\text{go}}$, and $\nu_{o,d}\in\mathbb{R}$ denotes the dual value associated with the convexity constraint:

$$\begin{aligned} \text{minimize} \quad & w_{o,d}\Big(\sum_{a\in p} c_a + \sum_{\ell\in\mathcal{L}} \sum_{a\in\mathcal{A}^\ell_{\text{go}}:\ a\in p} \mu_{\ell,a}\Big) - \nu_{o,d} \\ \text{s.t.} \quad & p \in P_{o,d} \end{aligned}$$

In other words, for a given pair $(o,d)\in\mathcal{D}$, we seek to find a path from $(o,\varnothing)$ to $(\varnothing,d)$ in the change-and-go network such that the reduced cost of the associated variable $\lambda^p_{o,d}$ in (RMP(LPT_DW)) is negative.

We can use Dijkstra's algorithm to compute at once all paths from vertex $(o,\varnothing)$ with minimum cost

$$\sum_{a\in p} c_a + \sum_{\ell\in\mathcal{L}} \sum_{a\in\mathcal{A}^\ell_{\text{go}}:\ a\in p} \mu_{\ell,a}$$

as $c_a \geqslant 0$ and $\mu_{\ell,a}\geqslant 0$.

This also applies for the Farkas pricing with the only difference that $\mu_{\ell,a}\geqslant 0$ and $\nu_{o,d}\in\mathbb{R}$ are then dual rays and the first term in the reduced cost is not accounted for.

6.4.3 Branching strategy

We apply the following branching strategy:

- If there exists a frequency variables with fractional value $\bar{f}_\ell$, then we create three branches. Denote by $[\alpha^{\min},\alpha^{\max}]$ the domain for the variable f_ℓ in the current node, and by $\bar{\alpha}$ the average value $\frac{\alpha^{\min}+\alpha^{\max}}{2}$.
  - If $\lceil\bar{f}_\ell\rceil \leqslant \lfloor\bar{\alpha}\rfloor$, then we define a first branch with the branching decision $f_\ell\leqslant\lfloor\bar{f}_\ell\rfloor$, a second with $\lceil\bar{f}_\ell\rceil\leqslant f_\ell\leqslant\lfloor\bar{\alpha}\rfloor$, and a third with $\lceil\bar{\alpha}\rceil\leqslant f_\ell$.
  - If $\lceil\bar{\alpha}\rceil \leqslant \lfloor\bar{f}_\ell\rfloor$, then we define a first branch with the branching decision $f_\ell\leqslant\lfloor\bar{\alpha}\rfloor$, a second with $\lceil\bar{\alpha}\rceil\leqslant f_\ell\leqslant\lfloor\bar{f}_\ell\rfloor$, and a third with $\lceil\bar{f}_\ell\rceil\leqslant f_\ell$.
  - Otherwise, we define a first branch with the branching decision $f_\ell\leqslant\lfloor\bar{f}_\ell\rfloor-1$, a second with $\lfloor\bar{f}_\ell\rfloor\leqslant f_\ell\leqslant\lceil\bar{f}_\ell\rceil$, and a third with $\lceil\bar{f}_\ell\rceil+1\leqslant f_\ell$.

  We choose the most fractional variable as the candidate for this branching strategy.
- If all frequencies are integer, then we branch on the original $x^a_{o,d}$ variables, i.e., on the sum $\sum_{\substack{p\in P_{o,d}\\ a\in p}} w_{o,d}\lambda^p_{o,d}$. The branching decisions are added in the master problem and additional terms corresponding to these constraints are added in the reduced cost (and the cost for the Farkas pricing). This might induce negative costs in the shortest path problem which invalidates the use of Dijkstra's algorithm. In practice, we never faced a situation where such branching decisions were required in the numerical experiments performed in Section 6.5.

6.4.4 Valid inequalities

We add valid inequalities at the root node and at deeper nodes, in order to speed up the branch-and-price resolution.

On instances where the line costs are close to each other, the inequality of the following lemma is useful to cut fractional solutions. This inequality is added at the root node with different $\mathcal{L}'$: ranking the lines by increasing costs, we take batches of consecutive lines with different sizes and starting points.

Lemma 6.4.1. *For all $\mathcal{L}'\subseteq\mathcal{L}$*

$$\sum_{\ell\in\mathcal{L}'} f_\ell \leqslant \left\lfloor \frac{B}{\min_{\ell\in\mathcal{L}'} C_\ell} \right\rfloor$$

is a valid inequality for the ILP. There exists instances such that this inequality cuts fractional solutions.

Proof. Let $\mathcal{L}' \subseteq \mathcal{L}$. The budget constraint (6.2c) implies:

$$\sum_{\ell \in \mathcal{L}'} f_\ell \leqslant \frac{B}{\min_{\ell \in \mathcal{L}'} C_\ell}\,.$$

Since the frequencies are integers,

$$\sum_{\ell \in \mathcal{L}'} f_\ell \leqslant \left\lfloor \frac{B}{\min_{\ell \in \mathcal{L}'} C_\ell} \right\rfloor$$

is a valid inequality.

Let there be an instance with two vertices o and d such that $w_{o,d} = 1$, with a single line ℓ with cost 15 and capacity $N_\ell = 1$, a budget of 20 and no arc capacities. The fractional solution $f_\ell = 4/3$ is cut by this inequality which writes $f_\ell \leqslant 1$. □

The valid inequalities from the following lemma are searched for and added in all nodes of the branch-and-price tree, using a sorting heuristic to identify potential "covers", i.e. sets $\mathcal{L}' \subseteq \mathcal{L}$ such that $M > 0$.

Lemma 6.4.2. *For every $\ell \in \mathcal{L}$, let $[\alpha_\ell^{\min}, \alpha_\ell^{\max}]$ be the domain of the variable f_ℓ at the current node. Let $\mathcal{L}' \subseteq \mathcal{L}$ be such that:*

$$M \coloneqq \sum_{\ell \in \mathcal{L}'} C_\ell(\alpha_\ell^{\max} - \alpha_\ell^{\min}) - B + \sum_{\ell \in \mathcal{L}} C_\ell \alpha_\ell^{\min} > 0\,.$$

Then, for every $\rho > 0$

$$\sum_{\ell \in \mathcal{L}'} \left\lceil \frac{\min(M, C_\ell)}{\rho} \right\rceil (\alpha_\ell^{\max} - f_\ell) \geqslant \left\lceil \frac{M}{\rho} \right\rceil$$

is a valid inequality for the ILP.

Proof. First, the budget constraint can be rewritten equivalently as follows:

$$\sum_{\ell \in \mathcal{L}} C_\ell f'_\ell \leqslant B'$$

with:

- $f'_\ell \coloneqq f_\ell - \alpha_\ell^{\min}$
- ${\alpha'_\ell}^{\min} \coloneqq 0$
- ${\alpha'_\ell}^{\max} \coloneqq \alpha_\ell^{\max} - \alpha_\ell^{\min}$
- $B' \coloneqq B - \sum_{\ell \in \mathcal{L}} C_\ell \alpha_\ell^{\min}$.

Note in particular that this rewritting verifies $0 = {\alpha'_\ell}^{\min} \leqslant f'_\ell \leqslant {\alpha'_\ell}^{\max}$.

By Atamtürk, Proposition 1, p.34 [7], since $\sum_{\ell \in \mathcal{L}'} C_\ell {\alpha'_\ell}^{\max} - B' = M > 0$, the set $\mathcal{L}'$ is a cover and the following inequality is valid, for every $\rho > 0$,

$$\sum_{\ell \in \mathcal{L}'} \left\lceil \frac{\min(M, C_\ell)}{\rho} \right\rceil ({\alpha'_\ell}^{\max} - f'_\ell) \geqslant \left\lceil \frac{M}{\rho} \right\rceil\,.$$

This rewrites as the inequality of the statement. □

Suppose that for every $(o,d) \in \mathcal{D}$, and every ℓ_1 and ℓ_2 in $\mathcal{L}$, we have $c_{((o,\varnothing),(o,\ell_1))} \leqslant c_{((o,\varnothing),(o,\ell_2))} + c_{((o,\ell_2),(o,\ell_1))}$. This assumption is a mild "triangular-like inequality" on boarding and transfer costs: it holds in particular whenever transfer costs are defined additively from boarding and disembarking times, as is standard in change-and-go networks, and is satisfied by all the instances considered in Section 6.5. Assuming that all lines have the same vehicle capacity N, we also have the following valid inequality.

Lemma 6.4.3. *Assume that the mild triangular-like inequality holds and that all lines have the same vehicle capacity N. For all $o \in V$ such that $(o,\varnothing) \in \mathcal{V}_{origin}$,*

$$\sum_{\ell \in \mathcal{L}(o)} f_\ell \geqslant \left\lceil \sum_{d\colon (o,d)\in\mathcal{D}} \frac{w_{o,d}}{N} \right\rceil$$

is a valid inequality for the ILP.

Proof. Let $o \in V$ be such that $(o,\varnothing) \in \mathcal{V}_{\text{origin}}$. For every $d \in V$ such that $(o,d) \in \mathcal{D}$, we have:

$$\sum_{\ell \in \mathcal{L}(o)} \sum_{\substack{p \in P_{o,d} \\ ((o,\varnothing),(o,\ell)) \in p}} \lambda^p_{o,d} = \sum_{p \in P_{o,d}} \lambda^p_{o,d} = 1\,.$$

Using the costs assumption, we have that for every $\ell \in \mathcal{L}(o)$:

$$\sum_{d\colon (o,d)\in\mathcal{D}} \sum_{\substack{p \in P_{o,d} \\ ((o,\varnothing),(o,\ell)) \in p}} w_{o,d}\lambda^p_{o,d} \leqslant \sum_{d\colon (o,d)\in\mathcal{D}} \sum_{\substack{a \in \mathcal{A}^\ell_{\text{go}} \\ (o,\ell) \text{ tail of } a}} \sum_{\substack{p \in P_{o,d} \\ ((o,\varnothing),(o,\ell)) \in p \\ a \in p}} w_{o,d}\lambda^p_{o,d} \leqslant N f_\ell\,.$$

We thus have, by summing over $\ell \in \mathcal{L}(o)$:

$$\sum_{d\colon (o,d)\in\mathcal{D}} w_{o,d} \leqslant \sum_{\ell \in \mathcal{L}(o)} N f_\ell\,.$$

Since the frequencies are integers,

$$\sum_{\ell \in \mathcal{L}(o)} f_\ell \geqslant \left\lceil \sum_{d\colon (o,d)\in\mathcal{D}} \frac{w_{o,d}}{N} \right\rceil$$

is a valid inequality. □

6.4.5 Accelerating for the first integer solution

While the columns generated at the root node (i.e., at the first column generation resolution) are not enough to obtain an integer feasible solution with the use of an ILP solver, we added frequent calls to an ILP solver after the column generation convergence both at the root node and in deeper nodes of the branch-and-price tree.

We also apply the following "diving" heuristic: we seek to explore deeper nodes by taking multiple branching decisions at once. These decisions are based on the information on the current fractional solutions by checking:

- if there exists a line for which rounding down the frequency does not violate the vehicle capacity constraint in the current fractional solution, then we round its frequency down.
- if there exists a line for which rounding up the frequency does not violate the budget constraint in the current fractional solution (taking into account the cost reduction of rounding down the line frequencies from the first item), then we round its frequency up.

We then explore the deeper node where the frequencies of the lines in the first case are smaller than the rounded down current fractional frequencies and the frequencies of the lines in the second case are larger than the rounded up current fractional frequencies.

Name	$\lvert V\rvert$	$\lvert A\rvert$	$\lvert\mathcal{D}\rvert$	$\lvert\mathcal{L}\rvert$
small	3	3	6	3
toy	8	8	46	8
mandl	15	21	172	21
grid	25	40	567	40
athens	51	52	2,385	59
ring	161	320	25,760	65
bahn-01	250	326	15,424	132

Table 6.1: A summary of the main characteristics of the instances.

6.5 Numerical experiments

6.5.1 Experimental setting and instances

The numerical experiments aim to compare the solutions of the arc-based formulation (LPT) with the path-based formulation (LPT_DW) solved by the branch-and-price(-and-cut) approach.

The experiments have been conducted on a laptop with an AMD Ryzen 7 5700U with Radeon Graphics (1.80 GHz) processor. The programming language used is Python 3.12 and the solver used is Gurobi 12.0.3. For the arc-based formulation, a time limit of 3600 s was set for the resolution of the integer linear program. No time limit was imposed for the resolutions of the linear relaxation of the restricted master problem. A time limit of 60 s was set for the resolutions of the integer linear program in the branch-and-price. An overall time limit of 3600 s has been set for the branch-and-price algorithm.

The instances are taken from LinTim [76], an open source software library for public transportation planning. One of the instances is not accessible in the open source version of LinTim (more precisely: bahn-01). This instance has been constructed using the DeutscheBahn (the main German railway company) network. We note that some of the LinTim instances include minimal arc frequencies $f_a^{\min}$; these are ignored in the experiments of Section 6.5, which are run using formulation (LPT_DW) without minimal frequency constraints. A path-and-cycle-based formulation handling such constraints was introduced in Section 6.3.3, but its algorithmic and numerical treatment is left for future work.

The branch-and-price algorithm from this chapter was developed within LinTim, but is not yet available in its open source version.

6.5.2 Results

The results are summarized in Table 6.2. The caption of the table provides the meaning of the columns.

6.5.3 Comments

These results show that the bottleneck of the arc-based formulation can be overcome by the path-based formulation: on large instance, we are able to obtain at least a lower bound.

Moreover, this method appears to be efficient when the ILP solver call at the root node outputs a feasible solution. Otherwise, we are faced with degeneracy: the nodes in the branch-and-price tree are mostly fractional with the same objective value. To overcome this issue, we should modify the branching strategy to branch on the original flow variables or add cuts involving other variables than the frequency variables. Because of the number of subproblems to solve at each iteration of the column generation algorithm, we made the

	Arc-based: ILP			Path-based: Branch-and-price-and-cut						
				Root node						
	Time (s)	Best integer	Gap (%)	Time (s)	Value	ILP value	Total time (s)	Best integer	Gap (%)	Number nodes
small	0.1	1,048.0	0.0	0.02	1,048.0	1,048.0	0.2	1,048.0	0.0	1
toy	0.1	44,217.0	0.0	0.4	44,217.0	44,217.0	0.4	44,217.0	0.0	1
mandl	3.0	327,430.0	0.0	2.2	326,830.0	/	3,600.0	327,430.0	0.1	865
grid	1.4	76,834.0	0.0	1.8	76,834.0	76,834.0	1.8	76,834.0	0.0	1
athens	103.6	17,569,819.0	0.0	297.8	17,569,819.0	/	3,600.0/	$+\infty$	74	
ring	memory overload			285.5	102,438.8	/	3,600.0	/	$+\infty$	8
bahn-01	memory overload			1,202.4	13,442,535.0	/	3,600.0	/	$+\infty$	4

Table 6.2: Numerical experiments on the LinTim instances. The columns "Best integer" correspond to the best integer solution found under the time limit. The columns "Gap" correspond to the final gap. For the branch-and-price-cut, the column "Time (s)" corresponds to the resolution time at the root node, including column generation and call to the ILP solver as a heuristic for an upper bound. The column "Total time (s)" corresponds to the total resolution time for the branch-and-price-and-cut. The column "Number nodes" corresponds to the total number of nodes explored in the branch-and-price-and-cut before an optimal solution or reaching the time limit.

choice to call for the Dijkstra algorithm once for each origin node. This choice is however often incompatible with the addition of branching constraints and cuts: they often either introduce negative arc costs via the dual values, which invalidates Dijkstra's algorithm, or arc costs depending on the origin-destination pair, which breaks the one-Dijkstra-per-origin speed-up altogether. Addressing these limitations is left for future work.

These results are in line with the conclusions of the recent paper of van Lieshout and van Rossum [88]: their branch-and-price-and-cut approach on the change-and-go network is competitive only on small or easy instances, and is outperformed on larger instances by formulations based on the original network or on intermediate graphs that both approximate travel time. Together with the memory bottleneck of the arc-based formulation and the degeneracy observed here for the path-based formulation, this suggests that the change-and-go network, despite its modeling flexibility, may not be the right level of granularity for large-scale line planning, and that formulations trading off some of this flexibility for a smaller network could be preferable in practice.

PART

III

Balanced assignments of periodic tasks

Introduction

***

The previous parts deal with large-scale planning problems in which tasks must be assigned repeatedly over time under operational constraints. This final part abstracts one of the core structural questions behind such applications: when can periodic tasks be assigned to workers so that the assignment is balanced? And when such an assignment exists, can it always be chosen to be periodic?

Assigning periodic tasks to workers gives rise to problems widely studied in the Operations Research literature. As a first example, the crew management problem from the first part of this work deals with the assignment of train driving tasks, assumed to be repeated every week. Korst et al. [57] consider for instance the problem of assigning periodic operations to processors, while minimizing the total number of such processors. As another example, Orlin [69] studies the problem of minimizing the total number of airplanes while meeting a fixed periodically repeating set of tasks. The question of the periodicity of such assignments is also addressed in the literature; see, e.g., Bortoletto et al. [14] for a "periodic event" scheduling problem and Meunier et al. [64] for the aircraft routing problem.

In this part, we consider tasks that need to be repeated every week at specific times and a number of indistinguishable workers to perform them. We address the problem of assigning each occurrence of these tasks to the workers in a *balanced* way, i.e., so that each worker performs each task with the same frequency over a long period. Two versions of this problem are considered: the basic version, for which the only constraint is that a worker cannot perform two tasks at the same time, and the extended version where there are additional constraints on the feasibility of weekly schedules. As an illustration, the latter version covers the case of a bound on the total number of working hours per week.

Based on the authors' experience, this issue of balancing workloads between workers is present in multiple industries, especially in the transportation sector. For example, at SNCF, the main French railway company, the schedules of the freight train drivers must be of a certain form, called "cyclic rosters" in the literature [17, 96] (see also the first part of this thesis), which automatically ensures such balancedness while helping maintain a similar level of proficiency among the workers.

In Chapter 7, we introduce the basic version of the problem and establish a necessary and sufficient condition of existence of a balanced assignment, which can be checked in polynomial time. We also show that whenever there exists a balanced assignment, there exists one that is periodic, and the period can be made equal to the number of workers.

In Chapter 8, we introduce the extended version of the problem and establish sufficient conditions of existence of a balanced assignment either when no tasks overlap the time $t = 0$ or when every task does. In this setting, we have a result similar to the one in the basic version: namely, whenever there exists a balanced assignment, there exists one that is periodic and we upper bound the period by $2(q-1)q^2$, where q is the number of workers. As a tool to establish these results, we introduce a problem of pebbles moving along arcs on an arc-colored

directed graph and that need to visit each arc with the same frequency in the long term. This latter problem and the results obtained about it might be of independent interest.

Chapters 7 and 8 finish with a discussion on links between the corresponding theoretical results and a potential application to crew rostering.

In Chapter 9, we provide additional results and some open questions.

Chapter 7

Basic version: non-overlapping balanced assignments

***

The problem of this chapter has been introduced in the paper "Balanced assignments of periodic tasks," presented at the conference *ATMOS 2024* [39]. The results presented here come from the updated version [41].

7.1 Introduction . . . 105
7.1.1 Problem formulation and results . . . 105
7.1.2 Positioning with respect to the literature . . . 107
7.2 Main tools for the proofs . . . 109
7.2.1 Building a new feasible assignment from a sequence of feasible assignments 109
7.2.2 Encoding assignments as a directed graph: definition and properties . 110
7.2.3 From a general instance $\mathcal{I}$ to one $\mathcal{I}'$ satisfying $|U(\mathcal{I}')| = q$ . . . 110
7.3 Proof of Theorem 7.1.1 . . . 112
7.4 Proof of Theorem 7.1.2 . . . 117
7.5 Final remarks . . . 119
7.5.1 Discussing possible generalizations of Theorem 7.1.1 . . . 119
7.5.2 Minimizing the number of workers . . . 121
7.5.3 Link to crew rostering . . . 121

7.1 Introduction

7.1.1 Problem formulation and results

Consider a collection of tasks that have to be performed periodically (typically every week). We assume that each task has a given starting time and a given ending time over the period and that its length is at most the period. These tasks have to be performed by a group of indistinguishable workers. Formally, we are given an instance $\mathcal{I}$ with

- a collection of n sub-intervals $[s_i, e_i)$ of the open interval $(-1, 1)$, with $e_i \in (0, 1]$ and $e_i - s_i \leqslant 1$.
- a positive integer q.

Each interval $[s_i, e_i)$ represents a task: the rth occurrence of task i ($r \in \mathbb{Z}_{>0}$) takes place over the time interval $[s_i + r, e_i + r)$. The number q corresponds to the number of workers, whom we identify from now on with the set $[q]$. The interval $(r, r+1]$ will be referred to as *week* r. The tasks that are overlapping the left endpoint of the interval $[0, 1]$ will play an important role in the proofs and, for later purpose, we denote by $U(\mathcal{I})$ the set of such tasks. Formally, $U(\mathcal{I}) = \{i \in [n] \colon s_i \leqslant 0\}$.

Every occurrence of each task has to be assigned to a worker. Such an assignment is *feasible* if it satisfies the *non-overlapping property*: no worker is assigned two occurrences overlapping within $\mathbb{R}_{>0}$. Such an assignment is *balanced* if each task is performed by each worker every q weeks in the long term average. The notion of feasibility of an assignment will be revised in Chapter 8, where the extended version will be defined. Until then, we refer to the basic version of the problem, where feasibility deals only with the non-overlapping property.

In symbols, consider a map $f \colon [n] \times \mathbb{Z}_{>0} \to [q]$, where $f(i, r) = j$ means that the rth occurrence of task i is assigned to worker j. The non-overlapping property writes then:

$$[s_i + r, e_i + r) \cap [s_{i'} + r', e_{i'} + r') \neq \varnothing \quad \Longrightarrow \quad f(i, r) \neq f(i', r') \tag{7.1}$$

for all $i \neq i'$ and all r, r'. (Note that the left-hand side holds only if $|r - r'| \leqslant 1$ and that two occurrences of the same task never overlap.) It is balanced if

$$\lim_{t \to +\infty} \frac{1}{t} \big|\{r \in [t] \colon f(i, r) = j\}\big| = \frac{1}{q} \tag{7.2}$$

for all $i \in [n]$ and all $j \in [q]$. An illustration is given in Figures 7.1 and 7.2, the first figure providing an example with a feasible assignment but no balanced feasible assignment. An assignment f is *periodic* if there exists $h \in \mathbb{Z}_{>0}$ such that $f(i, r) = f(i, r + h)$ for all $i \in [n]$ and $r \in \mathbb{Z}_{>0}$.

We aim to identify conditions under which there exists a balanced feasible assignment and to study the related algorithmic questions.

A few comments are in order. First, note that there exists a feasible assignment if and only if there is no point in $\mathbb{R}$ contained in more than q intervals $[s_i + r, e_i + r)$. This is an elementary result from the coloring of interval graphs. (We actually state it as a proper lemma—Lemma 7.4.2—for sake of completeness.) This means that for the basic version of the problem, feasibility is not the challenge. Second, there are assignments for which the limit in (7.2) is not well-defined. By definition, if the limit is not well-defined, then the assignment is not balanced.

Clearly, a necessary condition for the existence of a balanced feasible assignment is that there is a feasible assignment in which a worker performs each task at least once. Surprisingly, this condition is actually sufficient, as stated by our first theorem.

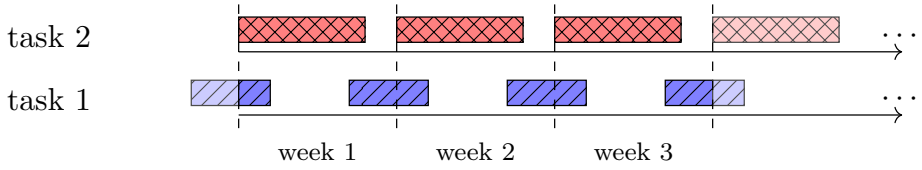
Theorem 7.1.1. *The following statements are equivalent:*

(a) *There exists a feasible assignment with a worker performing each task at least once.*
(b) *There exists a balanced feasible assignment.*
(c) *There exists a periodic balanced feasible assignment with period q.*

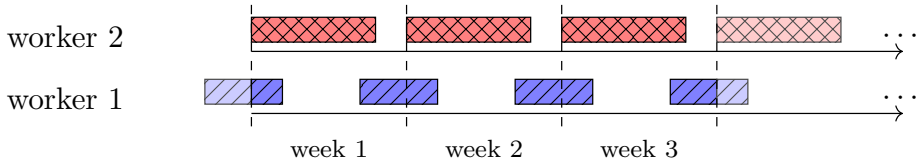
Note that the implications (c) ⇒ (b) and (b) ⇒ (a) (discussed above) are immediate. The proof focuses on showing (a) ⇒ (c).

In the preliminary version [39], the bound on the period of Statement (c) was $q^2 q!$. The current bound was suggested by Rolf Nelson van Lieshout.

Statement (a) of Theorem 7.1.1 is simple enough that it admits an algorithmic counterpart.



(a) The two tasks of $\mathcal{I}$. The first four occurrences of each task are represented. The fourth occurrence of each task is represented in lighter color. Here, $U(\mathcal{I}) = \{1, 2\}$.



(b) A feasible assignment f for $\mathcal{I}$. Assuming that this pattern is repeated along the horizontal axis, each line represents a worker and the rth occurrence of task i is on the line of worker j when $f(i, r) = j$.

Figure 7.1: An example of an instance $\mathcal{I}$ with two tasks ($n = 2$) and two workers ($q = 2$), with a feasible assignment. Up to exchanging workers 1 and 2, this assignment is unique and it is not balanced: without loss of generality, the first occurrence of the hatched blue task is assigned to worker 1, and this determines completely the assignment of all other occurrences of tasks 1 and 2.

Theorem 7.1.2. *Deciding whether there exists a balanced feasible assignment can be done in polynomial time. Moreover, when such an assignment exists, a periodic one with period q can be computed in polynomial time as well.*

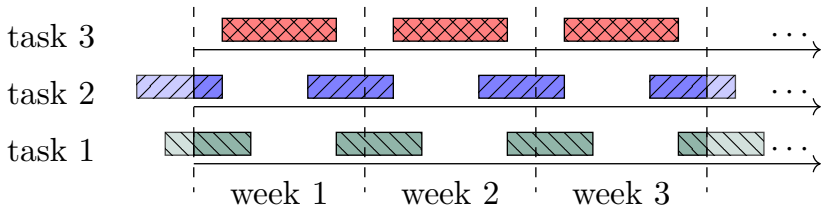
Note that, in this theorem, the computation of the balanced feasible assignment requires an output of size polynomial in n and $\log(q)$ because it is the number of workers that is given in the input, and not the list of them. This means in particular that, when q is large, we are not allowed to output the full first q weeks of the q-periodic balanced feasible assignment. In the proof of this theorem, we will refer to Lemma 7.4.1 to tackle this point.

When there is a point of $[0, 1)$ contained in no interval $[s_i + r, e_i + r)$ with $r \in \{0, 1\}$, then the theorems are trivial: without loss of generality, this point is 0, and any feasible assignment f and any cyclic permutation π of $[q]$ (without fixed points) provide a balanced feasible assignment g, periodic with period q, defined by $g(i, r) \coloneqq (\pi^r \circ f)(i, 1)$ for $i \in [n]$ and $r \in \mathbb{Z}_{>0}$ (where $\pi^r = \underbrace{\pi \circ \cdots \circ \pi}_{r \text{ times}}$).

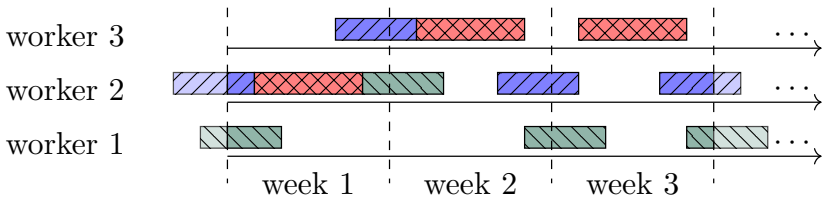
The proof of Theorem 7.1.1 is in Section 7.3 and the proof of Theorem 7.1.2 is in Section 7.4.

7.1.2 Positioning with respect to the literature

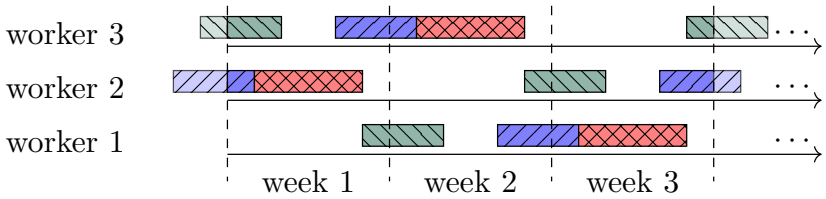
The basic version of this problem has been introduced in the preliminary version of this chapter [39], along with some weaker versions of Theorems 7.1.1 and 7.1.2. To our knowledge, this is the first time this problem has been considered. Although the question of achieving "exact" balanced assignment of tasks in the long term (as we do here) is a new question, the question of balancing workload among workers has been addressed in different settings. For example, de Werra [29] and Meyer [65] introduce the notion of "equitable colorings" as a coloring of either edges or vertices such that the sizes of the color classes differ by at most one element. Such equitable colorings find applications to balance workload: already in his paper, Meyer provides an application to a garbage collection problem. Regarding the paper of de



(a) The three tasks of $\mathcal{I}$. The first three occurrences of each task are represented. The fourth occurrence of tasks 1 and 2 is represented in lighter color. Here, $U(\mathcal{I}) = \{1, 2\}$.



(b) A feasible assignment f for $\mathcal{I}$. Assuming that this pattern is repeated along the horizontal axis, the assignment f is not balanced: worker 3 does not perform task 1 and worker 1 performs 75% of the occurrences of task 1.



(c) A feasible assignment g for $\mathcal{I}$. Assuming that this pattern is repeated along the horizontal axis, the assignment g is feasible and balanced: task 1 is performed equally by workers 1, 2, and 3, and so are tasks 2 and 3. The assignment g is periodic with period $h = 3$.

Figure 7.2: An example of an instance $\mathcal{I}$ with three tasks ($n = 3$) and three workers ($q = 3$), with two feasible assignments.

Werra, an application to a school timetabling problem is provided in [58]. More generally, Martello et al. [61] propose a general technique for addressing optimization problems where the objective is to minimize the "imbalance," measured as the spread of values.

The journal version of the present chapter [41] provides the current version of Theorems 7.1.1 and 7.1.2, and their proofs (along with the extended version of this problem, detailed in Chapter 8). Slightly later, van Lieshout and van Rossum [89] have introduced a similar problem, which they call the *fair periodic assignment problem*, and for which they establish existence and algorithmic results. Some of these results will be discussed further in Section 7.5.2. Corollary 17 of their paper provides an alternative proof of the equivalence (b) ⇔ (c) of our Theorem 7.1.1. They also provide an alternative proof of Theorem 7.1.2. However, their paper and its arXiv version [87] were published after the arXiv version of the present work [40].

To our knowledge, aside from the subsequent article by van Lieshout and van Rossum, the basic version of our problem has never been studied in the literature.

7.2 Main tools for the proofs

This section introduces tools that will be useful for the proofs of the theorems. The constructions and results of this section, except for Lemma 7.2.5, will be revised in Chapter 8 for the extended version of the problem.

7.2.1 Building a new feasible assignment from a sequence of feasible assignments

The goal of this section is to propose a construction of a feasible assignment from a sequence of feasible assignments, for an instance $\mathcal{I}$ such that $|U(\mathcal{I})| = q$. Lemma 7.3.3 (key to prove Theorems 7.1.1 and 7.1.2) uses this construction from a sequence of feasible assignments to build a balanced feasible assignment. The idea behind this construction is to rearrange and concatenate the first weeks of the assignments in the sequence, the rearrangement being uniquely defined through the tasks of $U(\mathcal{I})$. The feasibility is ensured by Lemma 7.2.1.

Let $\mathcal{I}$ be an instance such that $|U(\mathcal{I})| = q$. For every feasible assignment f and every $r \in \mathbb{Z}_{>0}$, we introduce the map $\varphi_{f,r} \colon i \in U(\mathcal{I}) \mapsto f(i,r) \in [q]$. The assumption $|U(\mathcal{I})| = q$ makes this map $\varphi_{f,r}$ a bijection. In the proofs, we will build new feasible assignments from sequences of feasible assignments. Let $f_1, f_2, \dots$ be an infinite sequence of feasible assignments. Define inductively permutations π_r of $[q]$ by the equation $\pi_{r+1} = \pi_r \circ \varphi_{f_r,2} \circ \varphi^{-1}_{f_{r+1},1}$, where π_1 is an arbitrary permutation of $[q]$. This implies in particular

$$(\pi_{r+1} \circ f_{r+1})(i,1) = (\pi_r \circ f_r)(i,2) \quad \forall i \in U(\mathcal{I})\,. \tag{7.3}$$

Lemma 7.2.1. *The map $(i,r) \mapsto (\pi_r \circ f_r)(i,1)$ is a feasible assignment.*

Proof. Let us show that $g \colon (i,r) \mapsto (\pi_r \circ f_r)(i,1)$ satisfies the non-overlapping property by checking the contrapositive of (7.1). Consider $i, i' \in [n]$ with $i \neq i'$ and $r, r' \in \mathbb{Z}_{>0}$. Suppose $g(i,r) = g(i',r')$, i.e., $(\pi_r \circ f_r)(i,1) = (\pi_{r'} \circ f_{r'})(i',1)$. Without loss of generality, suppose that $r \leqslant r'$.

Consider first the case when $r = r'$. Since π_r is a permutation, we have $f_r(i,1) = f_r(i',1)$. Since f_r is feasible, then the contrapositive holds for f_r, namely, $[s_i{+}1, e_i{+}1) \cap [s_{i'}{+}1, e_{i'}{+}1) = \varnothing$, which is equivalent to $[s_i + r, e_i + r) \cap [s_{i'} + r', e_{i'} + r') = \varnothing$, as desired.

Consider now the case when $r{+}1 = r'$. Note first that if $i' \notin U(\mathcal{I})$, then $e_i{+}r < s_{i'}{+}r{+}1$ and so $[s_i{+}r, e_i{+}r) \cap [s_{i'}{+}r', e_{i'}{+}r') = \varnothing$. Suppose now that $i' \in U(\mathcal{I})$. Using the definition of π_{r+1}, we have $(\pi_r \circ f_r)(i,1) = (\pi_r \circ f_r)(i',2)$. Since π_r is a permutation, then $f_r(i,1) = f_r(i',2)$. The contrapositive holds for f_r feasible assignment, so $[s_i + r, e_i + r) \cap [s_{i'} + r', e_{i'} + r') = \varnothing$.

Finally, if $r+2 \leqslant r'$, then $[s_i+r, e_i+r) \cap [s_{i'}+r', e_{i'}+r')$ is necessarily empty. Therefore, g is non-overlapping, and thus feasible. □

7.2.2 Encoding assignments as a directed graph: definition and properties

The goal of this section is to propose a way of encoding the first week of a collection of assignments as a directed graph, for an instance $\mathcal{I}$ such that $|U(\mathcal{I})| = q$. The idea behind this construction is to have the tasks of $U(\mathcal{I})$ as vertices and to encode the first week of each worker as a labeled arc. This graph will be used in order to generate relevant sequences of feasible assignments (as considered in Section 7.2.1) from walks in the graph. We also introduce the notion of *universal* sets of feasible assignments, from which it is easy to decide whether there exists a feasible assignment in which two tasks of $U(\mathcal{I})$ can be performed by a worker on the same week. Different universal sets of feasible assignments will be used in Sections 7.3 and 7.4 (and 8.4 in the next chapter) in order to control key properties of the graph.

Let $\mathcal{I}$ be an instance such that $|U(\mathcal{I})| = q$ and $\mathcal{F}$ be an arbitrary set of feasible assignments for $\mathcal{I}$. We introduce a directed multi-graph $D^{\mathcal{I},\mathcal{F}}$ which will be useful in the rest of this work. Its vertex set is $U(\mathcal{I})$. Its arc set $A^{\mathcal{F}}$ is obtained by introducing q arcs for each $f \in \mathcal{F}$: an arc from $i \in U(\mathcal{I})$ to $i' \in U(\mathcal{I})$ whenever $f(i,1) = f(i',2)$—repetitions are allowed and give rise to parallel arcs—; such an arc is labeled with f.

Note the following properties:

- The q arcs labeled with the same feasible assignment f form a collection of vertex-disjoint directed cycles (possibly loops): each vertex is by construction the head of exactly one arc labeled with f and the tail of exactly one arc labeled with f.
- The number of arcs in $A^{\mathcal{F}}$ is $q|\mathcal{F}|$.
- When $D^{\mathcal{I},\mathcal{F}}$ is weakly connected—i.e., the underlying undirected graph is connected—it is also strongly connected and Eulerian.

We finish by another property of this graph, which requires a proper proof. This property, stated as a lemma, will actually be used within the proofs of Lemmas 7.2.5 and 7.2.6, in the next section. It deals with the notion of "universal" sets of feasible assignments we introduce now. A set $\mathcal{F}$ of feasible assignments is *universal* if the existence of a feasible assignment f such that $f(i,1) = f(i',2)$ for some $i, i' \in U(\mathcal{I})$ implies the existence of a feasible assignment g in $\mathcal{F}$ such that $g(i,1) = g(i',2)$. In other words, given two sets $\mathcal{F}, \mathcal{F}'$ of feasible assignments, with $\mathcal{F}$ being universal, if there is an arc in $A^{\mathcal{F}'}$ from a vertex i to a vertex i', then there is such an arc in $A^{\mathcal{F}}$ as well. Note that universal sets of feasible assignments always exist, and can be built with a straightforward greedy procedure.

Lemma 7.2.2. *Let $\mathcal{F}$ be a set of feasible assignments. Consider the sequence of tasks performed by a worker in some feasible assignment (not necessarily in $\mathcal{F}$). If $\mathcal{F}$ is universal, then the induced sequence in $U(\mathcal{I})$ translates into a walk of $D^{\mathcal{I},\mathcal{F}}$.*

Proof. Let f be a feasible assignment for $\mathcal{I}$ and j be a worker. Suppose that $\mathcal{F}$ is universal. Consider the sequence of tasks performed by j in f and write $i_1, i_2, \ldots$ the induced sequence in $U(\mathcal{I})$. This means that, for every $r \in \mathbb{Z}_{>0}$, we have $f(i_r, r) = j = f(i_{r+1}, r+1)$. Since $\mathcal{F}$ is universal, there exists an arc from i_r to i_{r+1} in $D^{\mathcal{I},\mathcal{F}}$ (the assignment $g_h \colon (i,r) \mapsto f(i, r+h-1)$ is feasible and verifies $g_h(i_h, 1) = g_h(i_{h+1}, 2)$). Therefore, the sequence $i_1, i_2, \ldots$ translates into a walk in $D^{\mathcal{I},\mathcal{F}}$. □

7.2.3 From a general instance $\mathcal{I}$ to one $\mathcal{I}'$ satisfying $|U(\mathcal{I}')| = q$

The goal of this section is to describe a transformation of an instance $\mathcal{I}$ into an instance $\mathcal{I}'$, with the same number of workers q, satisfying $|U(\mathcal{I}')| = q$. This is a desirable property as

already hinted in the previous sections. This transformation, as well as the lemmas established in this section, will be useful for this chapter and the following ones. Lemmas 7.2.3 and 7.2.4 show how desirable properties on the original instance $\mathcal{I}$ are preserved on $\mathcal{I}'$. Lemmas 7.2.5 and 7.2.6 also show how desirable properties of $\mathcal{I}$ translate into properties on the graph built by applying with $\mathcal{I}'$ the construction of Section 7.2.2.

From now on we fix an instance $\mathcal{I}$. Set

$$n' \coloneqq n + q - |U(\mathcal{I})| \quad \text{and} \quad [s_i, e_i) \coloneqq [0, \varepsilon) \text{ for } i \in \{n+1, \dots, n'\},$$

where $\varepsilon > 0$ is chosen small enough, i.e., $\varepsilon \leqslant \min\big(\{s_i : i \in [n] \setminus U(\mathcal{I})\} \cup \{s_i + 1 : i \in U(\mathcal{I})\}\big)$. We can interpret the intervals with $i \in \{n+1, \dots, n'\}$ as new tasks, called *fictitious*. This gives rise to a new instance $\mathcal{I}' = \Big(\big([s_i, e_i)\big)_{i \in [n']}, q\Big)$.

The next two lemmas establish relations between the assignments for $\mathcal{I}$ and for $\mathcal{I}'$ especially regarding their feasibility and balancedness.

Lemma 7.2.3. *If there exists a feasible assignment f for $\mathcal{I}$, then there exists a feasible assignment for $\mathcal{I}'$ whose restriction to the tasks in $\mathcal{I}$ is f. Conversely, if there exists a feasible assignment f' for $\mathcal{I}'$, then its restriction to the tasks in $\mathcal{I}$ is a feasible assignment.*

Proof. Let f be a feasible assignment for $\mathcal{I}$. The assignment f can be extended on the fictitious tasks as follows: for each positive integer r, the assignment f leaves a number of workers idle at time r equal to the number of fictitious tasks; extending f arbitrarily by assigning the rth occurrence of these tasks to these workers leads to a feasible assignment for $\mathcal{I}'$.

Conversely, given a feasible assignment f' for $\mathcal{I}'$, its restriction to the tasks of $\mathcal{I}$ is obviously feasible. □

The next lemma is immediate to prove and adds relevance to the transformation, as it shows that balancedness transfers from $\mathcal{I}'$ to $\mathcal{I}$.

Lemma 7.2.4. *If there exists a balanced feasible assignment f' for $\mathcal{I}'$, then there exists a balanced feasible assignment f for $\mathcal{I}$. Moreover, if f' is periodic, then f can be chosen periodic with the same period as f'.*

Proof. If there exists a balanced feasible assignment f' for $\mathcal{I}'$, then the restriction of this assignment to the tasks of $\mathcal{I}$ is feasible, balanced, and periodic with same period as f' when f' is periodic. □

The next two lemmas provides us with connectivity results for the basic version of the problem. The notion of "universal" set has been introduced in Section 7.2.2.

Lemma 7.2.5. *Let $\mathcal{F}$ be a universal set of feasible assignments for $\mathcal{I}'$. If there exists a feasible assignment for $\mathcal{I}$ with a worker performing each task at least once, then $D^{\mathcal{I}',\mathcal{F}}$ is Eulerian.*

Proof. Suppose there exists a feasible assignment for $\mathcal{I}$ with a worker performing each task at least once. With Lemma 7.2.3, there exists a feasible assignment f' for $\mathcal{I}'$ with a worker $j^\star$ performing each task of $\mathcal{I}$ at least once. We are going to show that $D^{\mathcal{I}',\mathcal{F}}$ is weakly connected. This will be enough with the remark in Section 7.2.2. According to Lemma 7.2.2 applied to f' and worker $j^\star$, there is a walk in $D^{\mathcal{I}',\mathcal{F}}$ visiting every vertex of $U(\mathcal{I})$. The remainder of the proof is devoted to showing that every vertex in $U(\mathcal{I}') \setminus U(\mathcal{I})$ has a neighbor in $U(\mathcal{I})$. Let i' be a task in $U(\mathcal{I}') \setminus U(\mathcal{I})$.

Suppose first that the worker $j^\star$ performs no fictitious task. Let i be a task of $U(\mathcal{I})$ with minimum ending time e_i and r be an integer such that $f'(i, r) = j^\star$. Consider the assignment f'' defined by $f''(\cdot, h) \coloneqq f'(\cdot, r + h)$ for all $h \in \mathbb{Z}_{>0}$. (Note that it is just the assignment

obtained by "forgetting" the first r weeks.) Then swap the first occurrences of i and i' in f'' to obtain yet another assignment. This new assignment is feasible: the task i' overlaps no other task by the choice of ε; the task i overlaps no other task either, otherwise this would prevent $j^\star$ to perform all tasks in f while performing no fictitious task (by the choice of i). This shows the existence of an arc from i' to $U(\mathcal{I})$ in $D^{\mathcal{I}',\mathcal{F}}$, since r was chosen so that $f'(i,r) = j^\star$.

Suppose second that the worker $j^\star$ performs at least one fictitious task i'' and let r be such that $f'(i'',r) = j^\star$. Since swapping the rth occurrences of i' and i'' in f' results in a feasible assignment, there exists an arc from i' to $U(\mathcal{I})$ or the other way around. □

This lemma provides us with a weaker connectivity result, which will be generalized in Chapter 8.

Lemma 7.2.6. *Let $\mathcal{F}$ be a universal set of feasible assignments for $\mathcal{I}'$. If there exists a balanced feasible assignment for $\mathcal{I}$, then $D^{\mathcal{I}',\mathcal{F}}$ is Eulerian.*

Proof. Suppose there exists a balanced feasible assignment for $\mathcal{I}$. In such an assignment, each worker performs each task of $\mathcal{I}$ at least once. With Lemma 7.2.3, there exists a feasible assignment f' for $\mathcal{I}'$ with all workers performing each task of $\mathcal{I}$ at least once. Since the first occurrence of every fictitious task is assigned to a worker who performs every task in $U(\mathcal{I})$ later on in f', every fictitious task is the first vertex of a walk visiting every vertex in $U(\mathcal{I})$ (using Lemma 7.2.2). In case there is no fictitious task, there is a walk visiting every vertex in $U(\mathcal{I})$ as well, just by considering any worker. The graph $D^{\mathcal{I}',\mathcal{F}}$ is thus in any case weakly connected, and we conclude with the remark in Section 7.2.2. □

7.3 Proof of Theorem 7.1.1

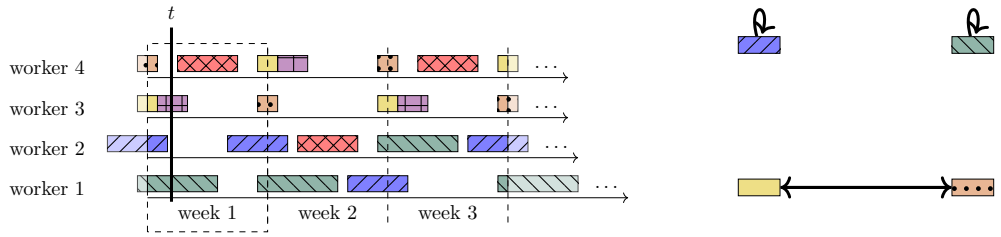
The goal of this section is to prove Theorem 7.1.1. Recall that this theorem applies on an instance $\mathcal{I} = \left(\left([s_i, e_i)\right)_{i\in[n]}, q\right)$ without schedules. The proof relies on the definition of a *merge operation*. Such an operation is performed on a feasible assignment by swapping two workers at a common idle time. To prove Theorem 7.1.1, Lemmas 7.3.1, 7.3.2, and 7.3.3 are established. The first one shows that performing a merge operation decreases the number of connected components in a graph constructed as in Section 7.2.2. The second shows, under some conditions, the existence of an assignment labeling arcs of a Hamiltonian cycle in this graph. The third combines the two results to obtain a periodic balanced feasible assignment of period q. The proof of Theorem 7.1.1 then follows using the results of Section 7.2.

As discussed in Section 7.1.1, we are only left with proving the implication (a) ⇒ (c).

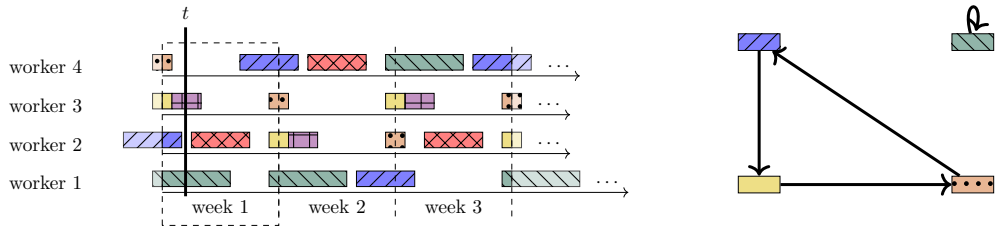
For an instance $\mathcal{I}$, define $\overline{\mathcal{F}_\mathcal{I}}$ as follows. Some feasible assignments induce the same assignment on the first week. For each assignment on the first week, pick a representative among the set of feasible assignments and put it in $\overline{\mathcal{F}_\mathcal{I}}$. Note that $\overline{\mathcal{F}_\mathcal{I}}$ is universal.

Consider an instance $\mathcal{I}$ such that $|U(\mathcal{I})| = q$ and a feasible assignment f. Suppose we are given a "time" $t \in (1,2]$ and two tasks $i_1, i_2 \in U(\mathcal{I})$, belonging to distinct cycles in $D^{\mathcal{I},\overline{\mathcal{F}_\mathcal{I}}}$ labeled with f, such that $f(i_1,1)$ and $f(i_2,1)$ are idle at time t. We define the *merge operation at time t between the tasks i_1 and i_2*, providing a new feasible assignment $\tilde{f}$, by swapping all tasks assigned to the two workers after time t as follows:

$$\tilde{f}(i,r) := \begin{cases} f(i,r) & \text{if } f(i,r) \notin \{f(i_1,1), f(i_2,1)\}, \\ f(i,r) & \text{if } e_i + r \leqslant t, \\ f(i_1,1) & \text{if } e_i + r > t \text{ and } f(i,r) = f(i_2,1), \\ f(i_2,1) & \text{if } e_i + r > t \text{ and } f(i,r) = f(i_1,1). \end{cases}$$



(a) A feasible assignment $f \in \overline{\mathcal{F}}_{\mathcal{I}}$ for an instance $\mathcal{I}$ such that $|U(\mathcal{I})| = q$. The arcs labeled by f in $D^{\mathcal{I},\overline{\mathcal{F}}_{\mathcal{I}}}$ are illustrated on the right. A merge operation can be performed at time t between the two tasks in hatched blue (worker 2) and in dotted orange (worker 4).



(b) The assignment $\tilde{f}$ resulting from the merge operation of Figure 7.3a. The arcs labeled by the representative of $\tilde{f}$ in $D^{\mathcal{I},\overline{\mathcal{F}}_{\mathcal{I}}}$ are illustrated on the right.

Figure 7.3: An illustration of a merge operation on a feasible assignment.

Note that no task i performed by $f(i_1, 1)$ or $f(i_2, 1)$ is such that $t \in [s_i + 1, e_i + 1)$ since both workers are idle at time t. This makes the assignment $\tilde{f}$ feasible.

An illustration of a merge operation is given in Figure 7.3.

Lemma 7.3.1. *Let $\mathcal{I}$ be an instance such that $|U(\mathcal{I})| = q$. Let f be a feasible assignment and $\tilde{f}$ be the feasible assignment resulting from a merge operation on f. The number of cycles in $D^{\mathcal{I},\overline{\mathcal{F}}_{\mathcal{I}}}$ labeled with the representative of $\tilde{f}$ in $\overline{\mathcal{F}}_{\mathcal{I}}$ is smaller by one unit than the number of cycles labeled with the representative of f in $\overline{\mathcal{F}}_{\mathcal{I}}$.*

Proof. Suppose that the merge operation performed to obtain $\tilde{f}$ from f is done at time t between the tasks i_1 and i_2. Consider the arc of tail i_1 (resp. i_2) labeled with the representative of f in $D^{\mathcal{I},\overline{\mathcal{F}}_{\mathcal{I}}}$ and denote by i'_1 (resp. i'_2) the head of this arc. We have $\tilde{f}(i_1, 1) = \tilde{f}(i'_2, 2)$ using the second and third cases of the definition of $\tilde{f}$. Thus the arc leaving i_1 labeled with the representative of $\tilde{f}$ in $\overline{\mathcal{F}}_{\mathcal{I}}$ has head i'_2. Similarly, with the second and fourth cases, we have $\tilde{f}(i_2, 1) = \tilde{f}(i'_1, 2)$. Thus the arc leaving i_2 labeled with the representative of $\tilde{f}$ in $\overline{\mathcal{F}}_{\mathcal{I}}$ has head i'_1. Consider the arcs different from the two going from i_1 to i'_2 and from i_2 to i'_1 and labeled with the representative of $\tilde{f}$ in $\overline{\mathcal{F}}_{\mathcal{I}}$. Each of these arcs is parallel to an arc labeled with f, according to the first case of the definition of $\tilde{f}$. Therefore the number of cycles formed by the arcs labeled with the representative of $\tilde{f}$ in $\overline{\mathcal{F}}_{\mathcal{I}}$ is smaller by one unit than for the representative of f. □

For the next two proofs (and a few other times), the following notation will be useful. For a feasible assignment f and an integer r, define $u_{f,r}(i)$ for $i \in [n]$ as the unique task in $U(\mathcal{I})$ whose rth occurrence is assigned to the same worker as the rth occurrence of i. Formally, $u_{f,r}(i)$ is the unique task in $U(\mathcal{I})$ such that $f(u_{f,r}(i), r) = f(i, r)$.

The following two lemmas will be useful to establish Theorems 7.1.1 and 7.1.2.

Lemma 7.3.2. *Let $\mathcal{I}$ be an instance such that $|U(\mathcal{I})| = q$. Suppose that $D^{\mathcal{I},\overline{\mathcal{F}}_{\mathcal{I}}}$ is Eulerian. If a feasible assignment cannot be subject to a merge operation, then the arcs labeled with its representative in $\overline{\mathcal{F}}_{\mathcal{I}}$ form a Hamiltonian cycle.*

In essence, the proof of this lemma assumes that there are more than two cycles and identifies a common idle time that can be used to perform a merge operation. For some technical reasons, we were however not able to make this scheme as explicit as we wished in the proof.

Proof of Lemma 7.3.2. Take a feasible assignment that cannot be subject to a merge operation and denote by $f^\star$ its representative in $\overline{\mathcal{F}}_{\mathcal{I}}$. For a feasible assignment g, denote by $\rho_g(i)$ the task in $U(\mathcal{I})$ whose first occurrence is assigned to the worker who starts the task i on the first week. Formally,

$$\rho_g(i) = \begin{cases} (\varphi_{g,1}^{-1} \circ \varphi_{g,2})(i) & \text{if } i \in U(\mathcal{I}), \\ u_{g,1}(i) & \text{if } i \notin U(\mathcal{I}). \end{cases}$$

We show now the following property: *For every feasible assignment g and task $i \in [n]$, there is a cycle labeled with $f^\star$ containing the vertices $u_{f^\star,1}(i)$ and $\rho_g(i)$ simultaneously.* Once this property is shown, we finish the proof as follows. Suppose by contradiction that there are at least two cycles labeled with $f^\star$. Choose two such cycles with an arc a of $D^{\mathcal{I},\overline{\mathcal{F}}_{\mathcal{I}}}$ connecting them. Such cycles and arc exist because $D^{\mathcal{I},\overline{\mathcal{F}}_{\mathcal{I}}}$ is Eulerian. Let g be the assignment labeling a, and i' and i be respectively the tail and the head of a. Using the property above, there is a cycle labeled with $f^\star$ containing the vertices $u_{f^\star,1}(i) = i$ and $\rho_g(i) = i'$ simultaneously. This contradicts the fact that i and i' belong to different cycles labeled with $f^\star$.

We will proceed now by induction to prove the property. To do so, for each task $i \in [n]$, we define

$$\tilde{s}_i \coloneqq \begin{cases} s_i + 1 & \text{if } i \in U(\mathcal{I}), \\ s_i & \text{if } i \notin U(\mathcal{I}), \end{cases} \quad \text{and} \quad \tilde{e}_i \coloneqq \begin{cases} e_i + 1 & \text{if } i \in U(\mathcal{I}), \\ e_i & \text{if } i \notin U(\mathcal{I}). \end{cases}$$

We sort the tasks $i_1, \dots, i_n$ by ascending order of $\tilde{s}_i$. Let g be a feasible assignment, and let us show by induction on ℓ that there is a cycle labeled with $f^\star$ containing the vertices $u_{f^\star,1}(i_\ell)$ and $\rho_g(i_\ell)$ simultaneously.

First, let us prove the property for $\ell = 1$. By the definition of $f^\star$, every task $i' \in U(\mathcal{I})$ verifying $[s_{i'}, e_{i'}) \cap [\tilde{s}_{i_1}, \tilde{e}_{i_1}) = \varnothing$ is in the same cycle as $u_{f^\star,1}(i_1)$ (otherwise, a merge operation could be performed right before time $\tilde{s}_{i_1} + 1$ between the tasks $u_{f^\star,1}(i_1)$ and i'). Since g is a feasible assignment, we have $[s_{\rho_g(i_1)}, e_{\rho_g(i_1)}) \cap [\tilde{s}_{i_1}, \tilde{e}_{i_1}) = \varnothing$. Therefore there is a cycle labeled with $f^\star$ containing the vertices $u_{f^\star,1}(i_1)$ and $\rho_g(i_1)$ simultaneously.

Suppose now that the property holds up to $\ell \geqslant 1$. Let us prove the property for $\ell + 1$. Write $C_{\ell+1}$ the cycle labeled with $f^\star$ containing the vertex $u_{f^\star,1}(i_{\ell+1})$. For $i \in U(\mathcal{I}) \setminus C_{\ell+1}$ define $\chi(i)$ as follows. In the assignment $f^\star$, the worker $f^\star(i,1)$ is not idle at time $\tilde{s}_{i_{\ell+1}} + 1$ (otherwise, a merge operation could be performed right before time $\tilde{s}_{i_{\ell+1}} + 1$ between the tasks $u_{f^\star,1}(i_{\ell+1})$ and i). Thus, the worker $f^\star(i,1)$ is busy, performing either:

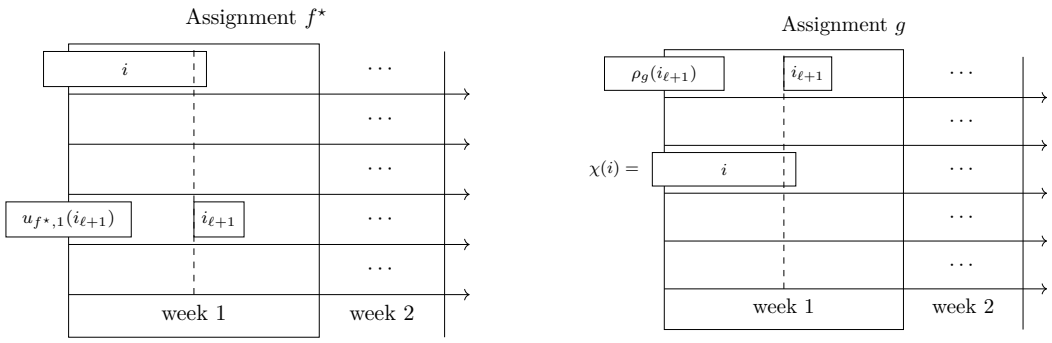
1. the first occurrence of the task $i \in U(\mathcal{I})$.
2. the first occurrence of a task $i_k \notin U(\mathcal{I})$. Note that $k \leqslant \ell$ in that case.
3. the second occurrence of a task $i_k \in U(\mathcal{I})$. Note that $k \leqslant \ell$ in that case.

Set

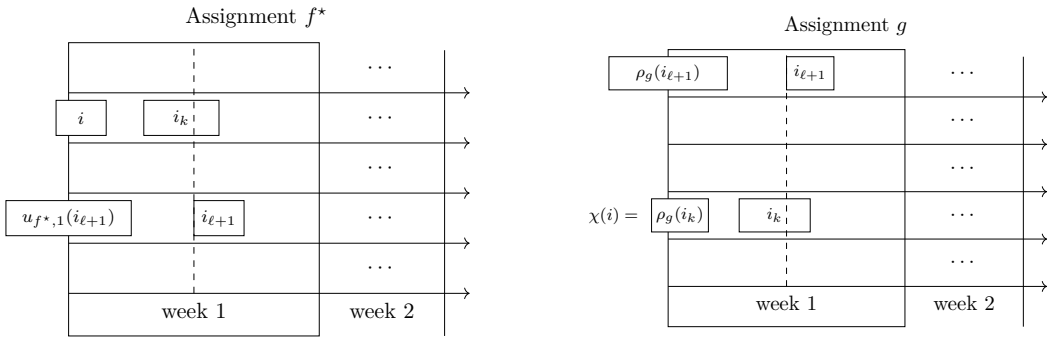
$$\chi(i) \coloneqq \begin{cases} i & \text{in case 1,} \\ \rho_g(i_k) & \text{in cases 2 and 3.} \end{cases}$$

An illustration is given in Figure 7.4. Notice that $g(\chi(i), 1)$ performs a task different from $i_{\ell+1}$ at time $\tilde{s}_{i_{\ell+1}+1}$ in g.

The map χ defines a self-map of $U(\mathcal{I}) \setminus C_{\ell+1}$: in case 1, the task $\chi(i) = i$ does not belong to $C_{\ell+1}$ and in cases 2 and 3, by induction, the task $\chi(i) = \rho_g(i_k)$ belongs to the same cycle



(a) An illustration of the map χ in the case 1: $\chi(i) = i$.



(b) An illustration of the map χ in the cases 2 and 3: $\chi(i) = \rho_g(i_k)$.

Figure 7.4: An illustration of the map χ for the different cases 1, 2, and 3.

as $u_{f^\star,1}(i_k) = i$, which is different from $C_{\ell+1}$ by definition of i. We claim that the map χ is actually injective. Let $i \neq i' \in U(\mathcal{I}) \setminus C_{\ell+1}$. As noted above, with tasks chosen this way, the workers $f^\star(i,1)$ and $f^\star(i',1)$ are busy at time $\tilde{s}_{i_{\ell+1}} + 1$ in the assignment $f^\star$. Let us check that $\chi(i) \neq \chi(i')$.

Suppose first that $\chi(i) = i$ and $\chi(i') = i'$ (case 1 for i and i'). Since $i \neq i'$, we have $\chi(i) \neq \chi(i')$. Suppose now that $\chi(i) = \rho_g(i_k)$ (case 2 for i) and $\chi(i') = i'$ (case 1 for i'). The tasks i_k and i' overlap because $f^\star(i,1)$ and $f^\star(i',1)$ are busy at time $\tilde{s}_{i_{\ell+1}} + 1$ in $f^\star$ with the first occurrences of respectively i_k and i'. Thus $\rho_g(i_k) \neq i'$, i.e., $\chi(i) \neq \chi(i')$. Suppose then that $\chi(i) = \rho_g(i_k)$ (case 2 for i) and $\chi(i') = \rho_g(i_{k'})$ (case 2 for i'). The tasks i_k and $i_{k'}$ overlap because $f^\star(i,1)$ and $f^\star(i',1)$ are busy at time $\tilde{s}_{i_{\ell+1}} + 1$ in $f^\star$ with the first occurrences of respectively i_k and $i_{k'}$. The images of overlapping tasks by ρ_g are distinct, and thus $\chi(i) \neq \chi(i')$. When $\chi(i)$ or $\chi(i')$ is defined according to case 3, the proof of the relation $\chi(i) \neq \chi(i')$ follows along the same lines as for case 2.

The map χ is then bijective, which implies $\{g(\chi(i),1) \colon i \in U(\mathcal{I}) \setminus C_{\ell+1}\} = \{g(i,1) \colon i \in U(\mathcal{I}) \setminus C_{\ell+1}\}$. As noted above, the worker $g(\chi(i),1)$ performs a task different from $i_{\ell+1}$ at time $\tilde{s}_{i_{\ell+1}} + 1$ in the assignment g. Thus, no worker in $\{g(i,1) \colon i \in U(\mathcal{I}) \setminus C_{\ell+1}\}$ performs a task $i_{\ell+1}$ at time $\tilde{s}_{i_{\ell+1}} + 1$ in the assignment g. This implies $\rho_g(i_{\ell+1}) \in C_{\ell+1}$, which concludes the proof of the property. □

Lemma 7.3.3. *Let $\mathcal{I}$ be an instance such that $|U(\mathcal{I})| = q$ and $\mathcal{F}$ be a set of feasible assignments for $\mathcal{I}$. If $D^{\mathcal{I},\mathcal{F}}$ is Eulerian, then there exists a periodic balanced feasible assignment for $\mathcal{I}$ with period q.*

Proof. Suppose that $D^{\mathcal{I},\mathcal{F}}$ is Eulerian. Note that $\mathcal{F}$ is then non-empty (even if $q = 1$) and thus that $\overline{\mathcal{F}}_{\mathcal{I}}$ is non-empty as well. The graph $D^{\mathcal{I},\overline{\mathcal{F}}_{\mathcal{I}}}$ contains a subgraph isomorphic to

$D^{\mathcal{I},\mathcal{F}}$ and they have the same vertex set. The graph $D^{\mathcal{I},\overline{\mathcal{F}}_{\mathcal{I}}}$ is thus weakly connected, and thus Eulerian as noted in Section 7.2.2.

Pick any feasible assignment in $\overline{\mathcal{F}}_{\mathcal{I}}$ and consider the feasible assignment obtained from it by performing merge operations as long as possible. According to Lemma 7.3.1, at most q such operations can be performed, and thus the process finishes. Lemma 7.3.2 implies then that there exists a feasible assignment $g \in \overline{\mathcal{F}}_{\mathcal{I}}$ labeling arcs of a Hamiltonian cycle $i_1, \dots, i_q$ in $D^{\mathcal{I},\overline{\mathcal{F}}_{\mathcal{I}}}$. By definition of $D^{\mathcal{I},\overline{\mathcal{F}}_{\mathcal{I}}}$, we have $g(i_k, 1) = g(i_{k+1}, 2)$ for all $k \in [q]$, which can be rewritten as $\varphi_{g,1}(i_k) = \varphi_{g,2}(i_{k+1})$ (where $i_{k+q} \coloneqq i_k$).

Consider now the permutations π_r as defined in Section 7.2.1 for the infinite sequence $g, g, \dots$ and set $g'(i, r) \coloneqq (\pi_r \circ g)(i, 1)$ for every $i \in [n]$ and every $r \in \mathbb{Z}_{>0}$. According to Lemma 7.2.1, this assignment g' is feasible.

We first show that g' is periodic of period q. We have, for all $k \in [q]$ and $r \in \mathbb{Z}_{>0}$,

$$\pi_r(\varphi_{g,1}(i_k)) = \pi_r(\varphi_{g,2}(i_{k+1})) = (\pi_{r+1} \circ \varphi_{g,1} \circ \varphi_{g,2}^{-1})(\varphi_{g,2}(i_{k+1})) = \pi_{r+1}(\varphi_{g,1}(i_{k+1}))\,.$$

By a direct induction, we get $\pi_r(\varphi_{g,1}(i_k)) = \pi_{r+h}(\varphi_{g,1}(i_{k+h}))$, for all $r, h \in \mathbb{Z}_{>0}$ and $k \in [q]$. Thus,

$$\begin{aligned} g'(i_k, r) &= \pi_r(g(i_k, 1)) = \pi_r(\varphi_{g,1}(i_k)) = \pi_{r+h}(\varphi_{g,1}(i_{k+h})) \\ &= \pi_{r+h}(g(i_{k+h}, 1)) = g'(i_{k+h}, r+h)\ . \end{aligned} \tag{7.4}$$

In particular, we have $g'(i_k, r) = g'(i_{k+q}, r+q) = g'(i_k, r+q)$ for every $k \in [q]$ and $r \in \mathbb{Z}_{>0}$. Therefore, for all $i \in [n]$ and $r \in \mathbb{Z}_{>0}$, we have

$$\begin{aligned} g'(i, r) &= \pi_r(g(i, 1)) = \pi_r(g(u_{g,1}(i), 1)) = g'(u_{g,1}(i), r) = g'(u_{g,1}(i), r+q) \\ &= \pi_{r+q}(g(u_{g,1}(i), 1)) = \pi_{r+q}(g(i, 1)) = g'(i, r+q)\,, \end{aligned}$$

as desired.

We then show that g' is balanced. Consider $i \in [n]$ and $j \in [q]$. Define $i_k \coloneqq u_{g,1}(i)$ and $i_\ell \coloneqq \varphi_{g',1}^{-1}(j)$. We have

$$g'(i, r) = \pi_r(g(i, 1)) = \pi_r(g(u_{g,1}(i), 1)) = g'(u_{g,1}(i), r) = g'(i_k, r) = g'(i_{k-r+1}, 1)\,,$$

where equation (7.4) is used to get the last equality. Since $\varphi_{g',1}$ is bijective, $g'(i, r) = j$ if and only $i_{k-r+1} = i_\ell$, i.e., if and only if $r \equiv k - \ell + 1 \pmod q$. From this, we get

$$\begin{aligned} |\{r \in [t] \colon g'(i, r) = j\}| &= \sum_{r=1}^{t} \mathbb{1}(g'(i, r) = j) = \sum_{\substack{r \in [t] \\ r \equiv k-\ell+1 \pmod q}} \mathbb{1}(g'(i, r) = j) \\ &= |\{r \in [t] \colon r \equiv k - \ell + 1 \pmod q\}|\,, \end{aligned}$$

where $\mathbb{1}(\cdot)$ is the indicator function.

Thus, $\lceil t/q \rceil \geqslant \big|\{r \in [t] \colon g'(i, r) = j\}\big| \geqslant \lfloor t/q \rfloor$ holds. Passing to the limit shows that $\lim_{t\to\infty} \frac{1}{t}\big|\{r \in [t] \colon g'(i, r) = j\}\big|$ exists and is equal to $\frac{1}{q}$. The assignment g' is thus balanced. □

Proof of (a) ⇒ (c) in Theorem 7.1.1. Consider an instance $\mathcal{I}$ (without schedules), and denote by $\mathcal{I}'$ the associated instance defined as in Section 7.2.3. Suppose we are given a feasible assignment for $\mathcal{I}$ with a worker performing each task at least once. According to Lemma 7.2.5, since $\overline{\mathcal{F}}_{\mathcal{I}'}$ is a universal set of feasible assignments as noted at the beginning of this section, the graph $D^{\mathcal{I}',\overline{\mathcal{F}}_{\mathcal{I}'}}$ is Eulerian. Using Lemma 7.3.3, there exists a periodic balanced feasible assignment for $\mathcal{I}'$ with period q. Thus, according to Lemma 7.2.4, there exists a periodic balanced feasible assignment for $\mathcal{I}$ with period q. □

7.4 Proof of Theorem 7.1.2

The goal of this section is to prove Theorem 7.1.2. Recall that this theorem, like Theorem 7.1.1 of which it is the algorithmic counterpart, applies on an instance $\mathcal{I} = \left(\left([s_i, e_i)\right)_{i\in[n]}, q\right)$ without schedules. The proof of this theorem combines the results of Section 7.3 and the following lemmas. Lemma 7.4.1 handles the case when $q \geqslant 2n$. The case when $q < 2n$ is dealt with Lemmas 7.4.2 and 7.4.3. Lemma 7.4.2 shows that a first feasible assignment can be computed in polynomial time. We then proceed as in the proof of Theorem 7.1.1 by performing merge operations and Lemma 7.4.3 shows that this can also be done in polynomial time.

Lemma 7.4.1. *Let $\mathcal{I}$ be an instance such that $q \geqslant 2n$. Then there always exists a periodic balanced feasible assignment with period q, and a closed formula of such an assignment can be computed in polynomial time.*

Proof. For $i \in [n]$ and $r \in \mathbb{Z}_{>0}$, define

$$f(i,r) \coloneqq \begin{cases} r_1 + 2 - 2i & \text{if } 0 < r_1 + 2 - 2i, \\ r_1 + 2 - 2i + q & \text{if } r_1 + 2 - 2i \leqslant 0, \end{cases} \tag{7.5}$$

where r_1 is the remainder of the Euclidean division of r by q. This ensures that $r_1 \leqslant q-1$, and thus that $q-1 \geqslant r_1+2-2i$. So when $0 < r_1+2-2i$, we have $r_1+2-2i \in [q]$. Moreover, we have $r_1 + 2 - 2i \geqslant 2 - 2n \geqslant 2 - q$. So when $r_1 + 2 - 2i \leqslant 0$, we have $r_1 + 2 - 2i + q \in [q]$. Therefore in both cases, $f(i,r) \in [q]$ holds, which makes f an assignment. An illustration is given in Figure 7.5. Let us show that f is balanced, feasible, and periodic with period q.

First, we show that f is feasible. Let $i, i' \in [n]$ and $r, r' \in \mathbb{Z}_{>0}$ be such that $f(i,r) = f(i',r')$. Suppose first $|r - r'| > 1$. Then $[s_i + r, e_i + r) \cap [s_{i'} + r', e_{i'} + r')$ is necessarily empty as noted in Section 7.1. Suppose now that $|r - r'| \leqslant 1$. Without loss of generality, suppose that $r \in \{r', r' + 1\}$. Let r_1 and r'_1 be the remainders of the Euclidean divisions of r and r' by q. Using (7.5), we have then necessarily $r_1 - 2i \equiv r'_1 - 2i' \pmod q$, which can be written $r - 2i \equiv r' - 2i' \pmod q$. We cannot have $r = r' + 1$ because otherwise the equality would rewrite $2i - 1 \equiv 2i' \pmod q$, which would imply $2i - 1 = 2i'$, which is impossible. Thus, $r = r'$ and since $2i, 2i' \in [q]$, we have $i = i'$. Therefore, the assignment f is feasible.

Second, we show that f is periodic with period q. Let $i \in [n]$ and $r \in \mathbb{Z}_{>0}$. Writing $r = r_2 q + r_1$ the Euclidean division of r by q, we have $r + q = (r_2 + 1)q + r_1$ which is the Euclidean division of $r + q$ by q. Therefore $f(i,r) = f(i, r+q)$ which means that f is periodic with period q.

Finally, we show that the assignment f is balanced. Let $i \in [n]$ and $j \in [q]$. We claim that there is a unique $r \in \{q, \dots, 2q-1\}$ such that $f(i,r) = j$. Indeed, we have $r = r_1 + q$ where

$$r_1 = \begin{cases} j + 2i - 2 & \text{if } j + 2i - 2 \in \{0, \dots, q-1\}, \\ j + 2i - 2 - q & \text{if } j + 2i - 2 \in \{q, \dots, 2q-1\}. \end{cases}$$

Since f is periodic with period q, this shows that f is balanced.

Furthermore, since the Euclidean division can be performed in polynomial time, the closed formula (7.5) gives the desired property. □

For the next two lemmas, the first week is understood in a broad manner, including the second occurrences of the tasks of $U(\mathcal{I})$.

Lemma 7.4.2. *Deciding whether there exists a feasible assignment for an instance $\mathcal{I}$ and computing the first week of such an assignment if it exists can be done in polynomial time.*

Proof. Let G be the interval graph built from all the intervals $[s_i, e_i)$ for $i \in [n]$ together with the intervals $[s_i + 1, e_i + 1)$ for $i \in U(\mathcal{I})$. Computing (if it exists) the first week of a feasible

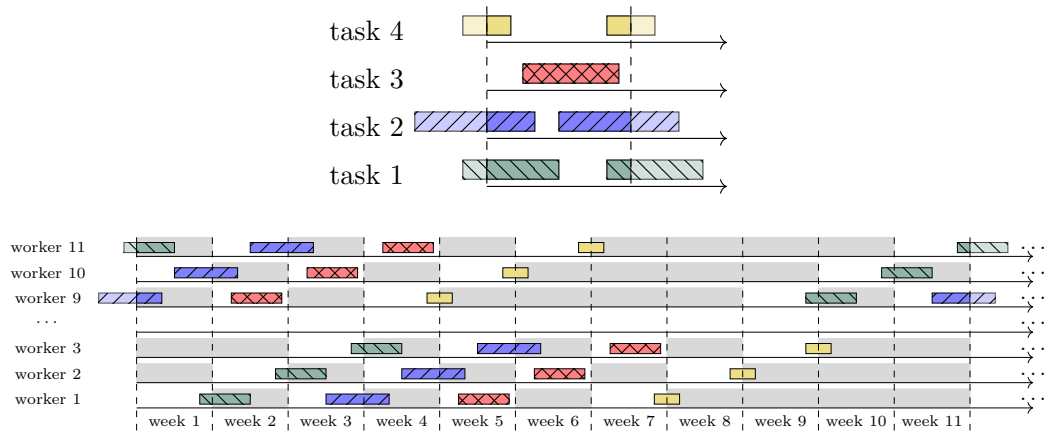


Figure 7.5: An illustration of the assignment given by equation (7.5) in the proof of Lemma 7.4.1. The instance has $n = 4$ tasks (with intervals illustrated above) and $q = 11$ workers. The gray areas are weeks on which no tasks are started.

assignment f is equivalent to computing (if it exists) a proper q-coloring of G, by identifying the colors as the workers. It is well known that this can be done in polynomial time. Note that a proper q-coloring of G induces an assignment on the first week which can be extended into a feasible assignment. □

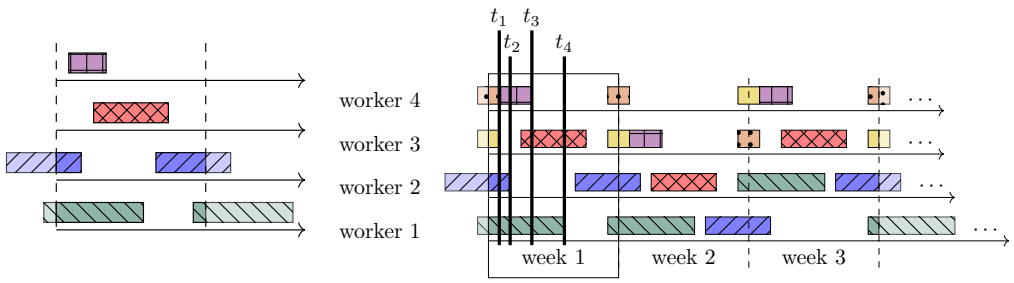
The next lemma discusses the tractability of the merge operation, defined in Section 7.3.

Lemma 7.4.3. *Let $\mathcal{I}$ be an instance such that $|U(\mathcal{I})| = q$ and f be an arbitrary feasible assignment for $\mathcal{I}$. Given a time $t \in (1, 2]$ and two tasks $i_1, i_2 \in U(\mathcal{I})$, deciding whether a merge operation can be performed on f at time t between i_1 and i_2 can be done in polynomial time. Moreover, when such an operation can be done, the restriction on the first week of the assignment resulting from such a merge operation can be computed in polynomial time as well.*

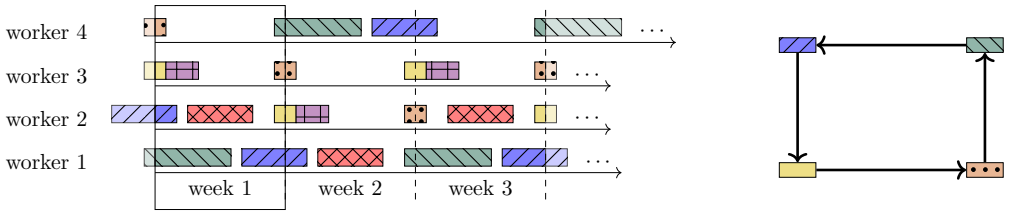
Proof. Computing the cycles of in $D^{\mathcal{I},\overline{\mathcal{F}}_{\mathcal{I}}}$ labeled with $\bar{f}$ the representative of f in $\overline{\mathcal{F}}_{\mathcal{I}}$ can be done in polynomial time in $O(q)$. Indeed, there is an arc from i to i' in $D^{\mathcal{I},\overline{\mathcal{F}}_{\mathcal{I}}}$ labeled with $\bar{f}$ if and only if $f(i, 1) = f(i', 2)$. Checking whether a merge operation can be performed of f at time t between i_1 and i_2 is equivalent to checking if $f(i_1, 1)$ and $f(i_2, 1)$ are both idle at time t and if i_1 and i_2 belong to distinct cycles. Moreover, it is clear from the definition of a merge operation that the first week of the resulting assignment can be computed in polynomial time. □

Proof of Theorem 7.1.2. Let $\mathcal{I} = \left(\left([s_i, e_i)\right)_{i\in[n]}, q\right)$ be an instance (without schedules). The case $q \geqslant 2n$ is dealt with using Lemma 7.4.1, which provides a closed formula for an assignment satisfying the desired properties. We can thus assume for the rest of the proof that $q \leqslant 2n-1$.

Transform $\mathcal{I}$ into $\mathcal{I}' = \left(\left([s_i, e_i)\right)_{i\in[n']}, q\right)$ as described in Section 7.2.3. This can obviously be done polynomial time. Using Lemma 7.4.2, deciding the existence of a feasible assignment for $\mathcal{I}'$, and computing the first week of such an assignment, if it exists, can be done in polynomial time. If no feasible assignment exists for $\mathcal{I}'$, then using Lemma 7.2.3, there is no feasible assignment for $\mathcal{I}$, and we are done. Thus, suppose from now on the existence of a feasible assignment f for $\mathcal{I}'$. Perform merge operations as long as possible starting from f. Notice that these merge operations are only performed at times in $(1, 2]$, hence only the first week of f is needed. An example is given in Figure 7.6. Each merge operation is polynomial according to Lemma 7.4.3 (the time t can in fact be chosen in $\{e_1 + 1, \ldots, e_{n'} + 1\}$) and the



(a) An illustration of an instance $\mathcal{I}$ (left) with four workers and a feasible assignment f for $\mathcal{I}'$ (right). A series of merge operations can be performed: at time t_1 between the plain yellow (worker 3) and dotted orange (worker 4) tasks, then at time t_2 between the hatched blue (worker 2) and plain yellow (worker 4) tasks, and finally at time t_4 between workers 1 and 4.



(b) The assignment $f^\star$ resulting from the series of merge operations described above (at time t_1, t_2, and t_4), starting from f. The assignment $f^\star$ cannot be subject to a merge operation. The arcs labeled by $f^\star$ in $D^{\mathcal{I}', \overline{\mathcal{F}}_{\mathcal{I}'}}$ form a Hamiltonian cycle.

Figure 7.6: An illustration of a series of merge operations on a feasible assignment, until no merge operation can be performed anymore.

number of merge operations is upper bounded by q according to Lemma 7.3.1. Hence, in polynomial time, we can get the first week of an assignment $f^\star$ which cannot be subject to a merge operation. Consider the arcs in $D^{\mathcal{I}', \overline{\mathcal{F}}_{\mathcal{I}'}}$ labeled with the representative of $f^\star$ in $\overline{\mathcal{F}}_{\mathcal{I}'}$. (To get these arcs, only the first week of $f^\star$ is needed.)

Suppose first that they form a Hamiltonian cycle. Following the same lines as in the proof of Lemma 7.3.3, this provides us with a periodic balanced feasible assignment for $\mathcal{I}'$ with period q. An illustration is given in Figure 7.7.

Suppose now that they do not form a Hamiltonian cycle. Using Lemma 7.3.2, the graph $D^{\mathcal{I}', \overline{\mathcal{F}}_{\mathcal{I}'}}$ is not Eulerian. Since $\overline{\mathcal{F}}_{\mathcal{I}'}$ is a universal set of feasible assignments for $\mathcal{I}'$, using Lemma 7.2.6, there is no balanced feasible assignment for $\mathcal{I}$. □

7.5 Final remarks

7.5.1 Discussing possible generalizations of Theorem 7.1.1

In this section, we discuss the possibility of imposing extra constraints on feasible assignments while keeping the results of Theorem 7.1.1, or at least some parts of it. The idea of constraining further feasible assignment will be pursued in the next chapter, with more general constraints than the ones presented in this section. We introduce a new version of the problem as follows. An instance now comes with tasks and a number of workers, and additionally a graph of interdicted transitions. Such a graph has vertex set the set of tasks $[n]$ and arc set encoding interdicted task transitions, with an arc going from vertex i to vertex i' if task i' cannot

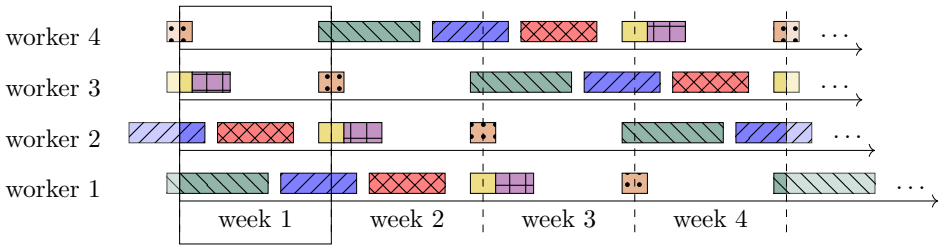
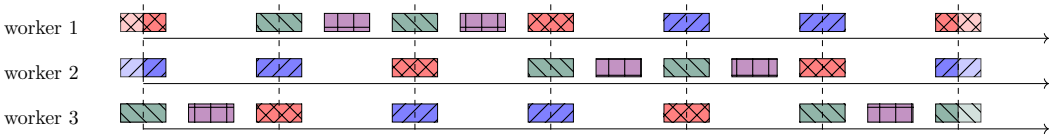


Figure 7.7: The assignment $f^\star$ of Figure 7.6b labels arcs of a Hamiltonian cycle in $D^{\mathcal{I}',\overline{\mathcal{F}}_{\mathcal{I}'}}$. A periodic balanced feasible assignment with period q can be built from the first week of assignment $f^\star$, as illustrated here.



(a) An illustration of an instance $\mathcal{I}$ of the problem with interdicted transitions, with $q = 3 = |U(\mathcal{I})|$. The instance has $n = 4$ tasks (with intervals illustrated on the left) and a graph of interdicted transitions (on the right). For example, task 1 can directly follow task 4 in an assignment, and task 2 can directly follow task 1 in an assignment, but task 2 cannot directly follow task 4 in an assignment.



(b) An illustration of a periodic balanced feasible assignment with period 6 for the instance $\mathcal{I}$.

Figure 7.8: An example of an instance of the problem with interdicted transitions where there exists a balanced feasible assignment but no periodic one with period q (this can be checked by a simple enumeration).

directly follow task i in a feasible assignment. The definition of a feasible assignment is then modified to take into account the interdicted task transitions according to this additional graph. Figure 7.8 provides an example where there exists a balanced feasible assignment but no periodic one with period q for the problem with interdicted transitions. Therefore, Theorem 7.1.1 cannot hold as is for this problem.

In Chapter 8, we introduce the extended version of the problem, with more general constraints. We note here that, when $|U(\mathcal{I})| = q$, the problem with interdicted transitions is a special case of the extended version of Chapter 8. The existence and periodicity results obtained for the extended version (Theorems 8.1.1 and 8.1.2) thus hold for this generalization when $|U(\mathcal{I})| = q$. When $|U(\mathcal{I})| \neq q$, it is not clear whether Theorem 8.1.1 holds for the problem with interdicted transitions. This leads to the following open question:

Question 2. Let $\mathcal{I}$ be an instance of the problem with interdicted transitions such that $|U(\mathcal{I})| \neq q$. Suppose there exists a balanced feasible assignment for $\mathcal{I}$. Is there a periodic balanced feasible assignment for $\mathcal{I}$?

We can also introduce an intermediary problem, which is a special case of the problem with interdicted transitions: each task now comes with an origin and a destination location,

and in a feasible assignment a task i' cannot follow a task i if the destination of i is different from the origin of i'.

As mentioned above, since this problem is a special case of the problem with interdicted transitions, the results of the extended version of Chapter 8 apply when $|U(\mathcal{I})| = q$ (otherwise, the question 2 remains open). Additionally, when $|U(\mathcal{I})| = q$, we can state the following open question:

Question 3. Let $\mathcal{I}$ be an instance of the problem with geographical tasks such that $|U(\mathcal{I})| = q$. If there exists a feasible assignment with a worker performing each task at least once, is there a periodic balanced feasible assignment with period q?

7.5.2 Minimizing the number of workers

A follow-up problem could be finding the minimum number of workers such that there exists a balanced feasible assignment (which is actually equivalent to the existence of a periodic balanced feasible assignment according to Theorem 7.1.1). Within the setting of the basic version, namely without schedules, the problem is polynomial thanks to Theorem 7.1.2 and Lemma 7.4.1. Indeed, we can use Theorem 7.1.2 for each value of q within $[0, 2n]$, the bound $2n$ being given by Lemma 7.4.1.

In a recent work by van Lieshout and van Rossum [87], an upper bound on the number of workers such that there exists a balanced feasible assignment is given. Indeed, they prove that there always exists a *fair periodic assignment* with at most $L + 1$ workers, where L is the maximum number of tasks performed at a time $t \in (0, 1]$ (namely the chromatic number of the interval graph with all tasks overlapping the first week, as in Lemma 7.4.2). It can be checked that every fair periodic assignment is a q-periodic balanced assignment, meaning that the bound $2n$ can be reduced to $L + 1 \leqslant n + 1$, where L can be found in polynomial time.

7.5.3 Link to crew rostering

We define a "cyclic roster on a k-week basis" as an assignment that follows a simple rotating pattern: the assignment for a worker over the first k weeks is picked up by another worker for the next k weeks, and so on, cycling through all q workers before returning to the same worker after kq weeks. Formally, a feasible assignment f is a cyclic roster on a k-week basis if there exists a cyclic permutation π of length q such that, for every $i \in [n]$, every $r \in [k]$, and every $t \in [q]$:

$$f(i, r + kt) = \pi(f(i, r + k(t - 1))) \,.$$

Two remarks are in order. First, it is easy to show that a cyclic roster on a k-week basis is a kq-periodic balanced feasible assignment. Second, one can convince oneself that a cyclic roster on a 1-week basis following this definition corresponds exactly to a cyclic roster for the crew rostering problem of Chapter 3, assuming that the only constraint for the crew rostering problem is that tasks do not overlap in a roster. The number of workers then corresponds to the number of rows of the crew rostering problem.

We can easily show that any q-periodic balanced feasible assignment constructed as in the proof of Theorem 7.1.1 is actually a cyclic roster on a 1-week basis. Remarkably, periodicity and cyclicity, which we deliberately relaxed when introducing balanced assignments, are recovered automatically as soon as a balanced feasible assignment is known to exist. Combining this discussion with the bound on the number of workers of the previous section, this implies that, given periodic tasks, there exists a cyclic roster on a 1-week basis covering these tasks with at most $L + 1$ workers, where L is the maximum number of tasks performed at any time. Moreover, we can find a cyclic roster covering all tasks, and minimizing the number of workers, in polynomial time, as shown by Theorem 7.1.2.

Chapter 8

Extended version: balanced assignments with valid weekly schedules

***

The problem of this chapter has been introduced in the journal article "Balanced assignments of periodic tasks" published in Annals of Operations Research [40]. The results presented here come from the same article.

8.1 Introduction . . . 124
8.1.1 Problem formulation and results . . . 124
8.1.2 Positioning with respect to the literature . . . 126
8.2 Main tools: revisions from the basic versions . . . 126
8.2.1 Building a new feasible assignment from a sequence of feasible assignments 126
8.2.2 Encoding assignments as a directed graph: definition and properties . 127
8.2.3 From a general instance $\mathcal{I}$ to one $\mathcal{I}'$ satisfying $|U(\mathcal{I}')| = q$ . . . 127
8.3 A problem of pebbles on an arc-colored Eulerian directed graph . . . 128
8.4 Proofs of Theorems 8.1.1 and 8.1.2 . . . 130
8.5 Concluding remarks . . . 132
8.5.1 Group action and Cayley graph . . . 132
8.5.2 All feasible assignments are balanced. (Almost.) . . . 133
8.5.3 Open question . . . 134
8.5.4 Link to crew rostering . . . 134

8.1 Introduction

8.1.1 Problem formulation and results

We introduce now an extended version of the previous problem, relying on the notion of "schedules."

An assignment *induces schedules* in the following manner: for each worker and each week r, a schedule is defined by the set of tasks performed over the week r, possibly with a task that has started over week $r-1$ and possibly with a task that will end in week $r+1$. Formally, a *schedule* is a pair (T, W) with $T \subseteq [n]$ and $W = \varnothing$ or $W = \{i\}$ with $i \in U(\mathcal{I})$. The sets T and W can respectively be interpreted as the set of tasks ending in week r and as a potential task starting in week r and ending in week $r+1$. A schedule is *non-overlapping* if for all $i' \in T$ we have

$$\begin{aligned} [s_{i'}, e_{i'}) \cap [s_{i''}, e_{i''}) = \varnothing \text{ for all } i'' \in T \setminus \{i'\} \\ \text{and } [s_{i'}, e_{i'}) \cap [s_i + 1, e_i + 1) = \varnothing \text{ for all } i \in W\,. \end{aligned} \tag{8.1}$$

We extend the previous problem by constraining further the notion of feasible assignment. An instance $\mathcal{I}$ now comes with a set $\mathcal{S}$ of non-overlapping schedules. Formally, an instance $\mathcal{I}$ is a triple of the form $\left(\left([s_i, e_i)\right)_{i\in[n]}, q, \mathcal{S}\right)$ where every schedule in $\mathcal{S}$ is non-overlapping. An assignment is *feasible* if all the schedules it induces belong to $\mathcal{S}$. As an illustration of the relevance of this extended version, it is straightforward to impose constraints such that

- no worker works more than H hours per week.
- no worker performs more than M tasks per week.
- no worker starts two tasks indexed with a prime number in the same week.
- (when $|U(\mathcal{I})| = q$) no worker performs two tasks i and j in a row, where (i, j) is an arc from an interdicting transitions graph (as in the setting of Section 7.5.1).

Note that the basic version is actually the special case of the extended version in which $\mathcal{S}$ is formed by all possible non-overlapping schedules. (The non-overlapping property is the only constraint to take into account in the basic version.)

An implication similar to (b) ⇒ (c) in Theorem 7.1.1 holds for the extended version. In a former version of Theorem 8.1.1 [40], the bound on the period was $q^2q!$. Arthur Léonard suggested that it could be improved (leading to the current $2(q-1)q!$ bound).

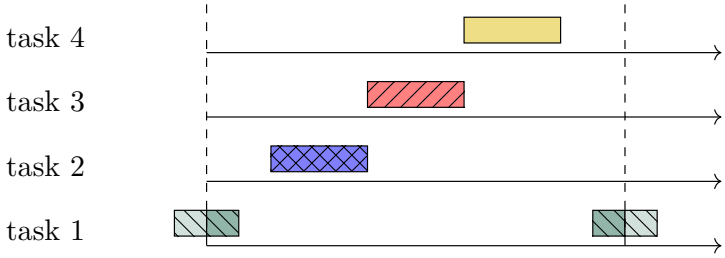
Theorem 8.1.1. *Consider an instance $\mathcal{I}$ with an arbitrary set $\mathcal{S}$ of non-overlapping schedules. If there exists a balanced feasible assignment, then there exists one that is periodic. Moreover, in such a case, the periodic assignment can be chosen so that the period is at most $2(q-1)q!$.*

A consequence of this result is that the existence of a balanced feasible assignment is decidable, i.e., there is an algorithm that decides in finite time whether a balanced feasible assignment exists for any input. It is not clear whether there is a simpler proof of this fact. Note that this was automatically ensured by Theorem 7.1.2 in the basic version.

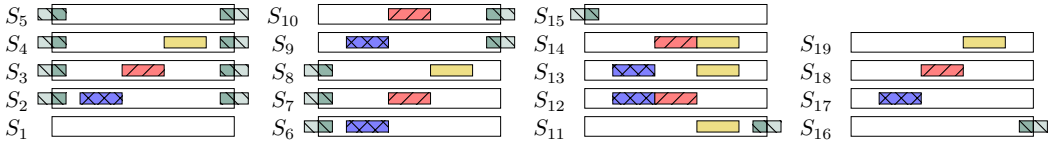
Note that, similarly to for the basic version, when no interval $[s_i + r, e_i + r)$ with $r \in \{0, 1\}$ contains the point 0, Theorem 8.1.1 is trivial for the same reason as for the basic version. In this case, the bound on the period is in fact q.

However, some properties established for the basic version do not hold for the extended version in full generality. First, feasibility is not equivalent anymore to having no point in $\mathbb{R}$ contained in q tasks simultaneously. Second, according to Theorem 7.1.1, in the basic version, as soon as there exists a feasible assignment with a worker performing each task at least once, there exists a balanced feasible assignment. Such a statement does not hold in full generality for the extended version, as illustrated in Figure 8.1.

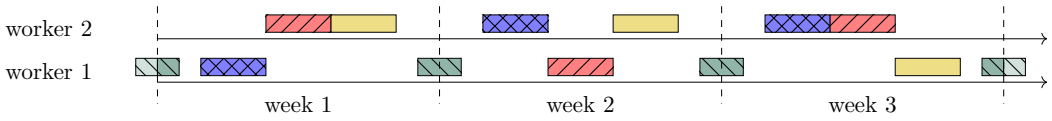
This being said, under an extra condition, we keep in essence the results of Theorem 7.1.1 for the extended version.



(a) The four tasks of $\mathcal{I}$. The first occurrence of tasks 2, 3, and 4 and the first two occurrences of task 1 are represented. Task 1 lasts 12 hours while tasks 2, 3, and 4 last 18 hours.



(b) The set $\mathcal{S} = \{S_k : k \in [19]\}$ gathering all non-overlapping schedules with maximum weekly work duration of 36 hours.



(c) A feasible assignment for $\mathcal{I}$. This assignment is such that worker 1 performs each task at least once; however no balanced feasible assignment exists: an easy case-checking shows that the worker starting with the first occurrence of task 1 performs all the occurrences of this task.

Figure 8.1: An instance $\mathcal{I}$ with four tasks ($n = 4$), a set of non-overlapping schedules $\mathcal{S}$, and two workers ($q = 2$) showing that the implication (a) $\Rightarrow$ (b) given in Theorem 7.1.1 does not hold for the extended version.

Theorem 8.1.2. *Consider an instance $\mathcal{I}$ with an arbitrary set $\mathcal{S}$ of non-overlapping schedules, verifying $|U(\mathcal{I})| = q$, i.e., there are q indices i such that $0 \in [s_i, e_i)$. The following statements are equivalent:*

(a') *There exists a feasible assignment with a worker performing each task at least once.*
(b') *There exists a balanced feasible assignment.*
(c') *There exists a periodic balanced feasible assignment with period bounded by $2(q-1)q!$.*

Note that the implication (b') ⇒ (c') is a direct consequence of Theorem 8.1.1 and the implication (c') ⇒ (a') is straightforward, as for Theorem 7.1.1. The proof focuses on showing (a') ⇒ (b').

Section 8.3 introduces a problem of pebbles moving on an arc-colored Eulerian digraph. The latter problem aims to find a balanced sequence of colors on this graph, i.e., such that each pebble visits each arc with the same asymptotic frequency. This problem and the key result established in Section 8.3 might be of independent interest. The proofs of Theorems 8.1.1 and 8.1.2 are in Section 8.4. They heavily rely on the result of Section 8.3.

8.1.2 Positioning with respect to the literature

Again, as mentioned in Section 7.1.2 of Chapter 7, the question of balancing workload among workers has been addressed in the literature, but the question of achieving "exact" balanced assignment of tasks in the long term (as we do) is never truly addressed. The extended version of this problem has been introduced in the arXiv version of this chapter [40], along with Theorems 8.1.1, and 8.1.2 (with a somewhat weaker bound on the period for the last two theorems) and their proofs. To our knowledge, the extended version of our problem has never been studied in the literature.

8.2 Main tools: revisions from the basic versions

In this section, we essentially follow the same layout as in Section 7.2. In particular, most constructions and lemmas of Section 7.2 are still valid for the extended version.

8.2.1 Building a new feasible assignment from a sequence of feasible assignments

The goal of this section is to recover Lemma 7.2.1 for the extended version. As for the basic version (in Section 7.2.1), Lemma 8.4.1 (key to prove Theorems 8.1.1 and 8.1.2) uses the construction of Section 7.2.1 from a sequence of feasible assignments with the goal of building a balanced feasible assignment.

Let $\mathcal{I}$ be an instance such that $|U(\mathcal{I})| = q$ and $f_1, f_2, \ldots$ be an infinite sequence of feasible assignments. Recall the definition of π_r from Section 7.2.1.

Lemma 8.2.1 (Extended version)**.** *The map $(i, r) \mapsto (\pi_r \circ f_r)(i, 1)$ is a feasible assignment.*

Proof. In the proof of Lemma 7.2.1 in Chapter 7, we built the assignment g and showed that g is non-overlapping. For the extended version, it remains to check that the schedules induced by g belong to the set $\mathcal{S}$ of schedules given in input.

Let (T, W) be a schedule induced by g for the worker j on week r. If $W = \varnothing$, then the pair (T, W) is exactly the schedule induced by f_r for worker $\pi_r^{-1}(j)$ on the first week. Otherwise, let i be the task in $U(\mathcal{I})$ such that $W = \{i\}$. By (7.3), we have $g(i, r+1) = (\pi_r \circ f_r)(i, 2)$. Hence, (T, W) is the schedule induced by f_r for worker $\pi_r^{-1}(j)$ on the first week. Therefore, every schedule induced by g is also induced by some f_r, which means that this schedule belongs to $\mathcal{S}$. □

8.2.2 Encoding assignments as a directed graph: definition and properties

Let $\mathcal{I}$ be an instance for the extended version (with schedules) such that $|U(\mathcal{I})| = q$ and $\mathcal{F}$ be an arbitrary set of feasible assignments for $\mathcal{I}$. The directed multi-graph $D^{\mathcal{I},\mathcal{F}}$ introduced in Section 7.2.2 can be defined similarly for the extended version, where the feasibility of an assignment reads as defined in Section 8.1.1. The notion of universal sets of feasible assignments also applies to the extended version. The properties highlighted in Section 7.2.2 as well as the Lemma 7.2.2 can then be recovered for the extended version. In order to control key properties of the graph, a universal set of feasible assignments will be used in Section 8.4.

Lemma 8.2.2 (Extended version)**.** *Let $\mathcal{F}$ be a set of feasible assignments. Consider the sequence of tasks performed by a worker in some feasible assignment for the extended version (not necessarily in $\mathcal{F}$). If $\mathcal{F}$ is universal, then the induced sequence in $U(\mathcal{I})$ translates into a walk of $D^{\mathcal{I},\mathcal{F}}$.*

Proof. The proof of Lemma 7.2.2 holds as is for the extended version. □

8.2.3 From a general instance $\mathcal{I}$ to one $\mathcal{I}'$ satisfying $|U(\mathcal{I}')| = q$

As in Section 7.2.3, we describe here a transformation of an instance $\mathcal{I}$ (with schedules) into an instance $\mathcal{I}'$, with the same number of workers q, satisfying $|U(\mathcal{I}')| = q$. Lemmas 7.2.3, 7.2.4, and 7.2.6 from Section 7.2.3 can be recovered for the extended version with the following transformation. The transformation and the associated lemmas from this section will be useful for the rest of this chapter.

We fix for now an instance $\mathcal{I} = \Big(\big([s_i, e_i)\big)_{i\in[n]}, q, \mathcal{S}\Big)$. As for the basic version, we introduce a new set of tasks $\big([s_i, e_i)\big)_{i\in[n']}$ made of the tasks of $\mathcal{I}$ along with the additional fictitious tasks. Now for the extended version, we can also get a new set $\mathcal{S}'$ of valid schedules in a natural way by adding in all possible ways these new tasks to the original valid schedules without creating any overlap. This gives rise to a new instance $\mathcal{I}' = \Big(\big([s_i, e_i)\big)_{i\in[n']}, q, \mathcal{S}'\Big)$ for the extended version. (Note that $\mathcal{S}'$ can actually be defined in a formal way: for each pair $(T, W) \in \mathcal{S}$, add to $\mathcal{S}'$ the pair (T', W') with

- $T' = T$ when $T \cap U(\mathcal{I}) \neq \varnothing$.
- $T' = T \cup \{i\}$ for all $i \in \{n+1, \dots, n'\}$, when $T \cap U(\mathcal{I}) = \varnothing$.
- $W' = W$ when $W \neq \varnothing$.
- $W' = \{i\}$ for all $i \in \{n+1, \dots, n'\}$, when $W = \varnothing$.)

An illustration of this transformation is given in Figure 8.2.

The next two lemmas, stated in Section 7.2.3 for the basic version, can be recovered for the extended version.

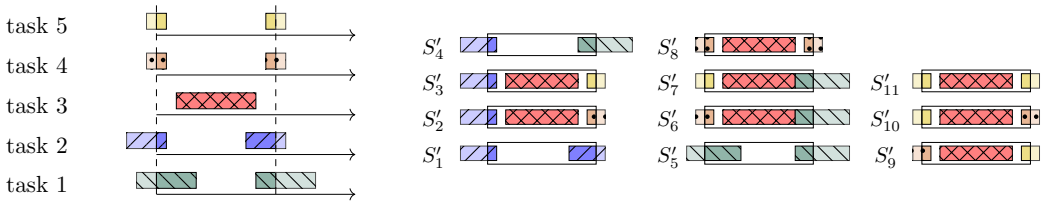
Lemma 8.2.3 (Extended version)**.** *If there exists a feasible assignment f for $\mathcal{I}$, then there exists a feasible assignment for $\mathcal{I}'$ whose restriction to the tasks in $\mathcal{I}$ is f. Conversely, if there exists a feasible assignment f' for $\mathcal{I}'$, then its restriction to the tasks in $\mathcal{I}$ is a feasible assignment.*

Proof. In the proof of Lemma 7.2.3 in Chapter 7, we prove both ways for the basic version. The first part of this result for the extended version comes from the fact that the number ε and the set $\mathcal{S}'$ have been chosen so that this does not create any conflict. Conversely, the second part of this result for the extended version comes from the definition of $\mathcal{S}'$. □

Lemma 8.2.4 (Extended version)**.** *If there exists a balanced feasible assignment f' for $\mathcal{I}'$, then there exists a balanced feasible assignment f for $\mathcal{I}$. Moreover, if f' is periodic, then f can be chosen periodic with the same period as f'.*



(a) An instance $\mathcal{I}$ with three tasks ($n = 3$), a set of schedules $\mathcal{S} = \{S_k : k \in [6]\}$, and four workers ($q = 4$).



(b) The new instance $\mathcal{I}'$ with five tasks ($n' = 5$), a set of schedules $\mathcal{S}' = \{S'_k : k \in [11]\}$, and four workers ($q = 4$).

Figure 8.2: An illustration of the transformation of an instance $\mathcal{I}$ into a new instance $\mathcal{I}'$ verifying $|U(\mathcal{I}')| = q$.

Proof. The proof of Lemma 7.2.4 holds as is for the extended version. □

The next lemma, stated in Section 7.2.3 for the basic version, can be recovered for the extended version.

Lemma 8.2.6 (Extended version)**.** *Let $\mathcal{F}$ be a universal set of feasible assignments for $\mathcal{I}'$. If there exists a balanced feasible assignment for $\mathcal{I}$, then $D^{\mathcal{I}',\mathcal{F}}$ is Eulerian.*

Proof. The proof of Lemma 7.2.6 holds as is for the extended version. □

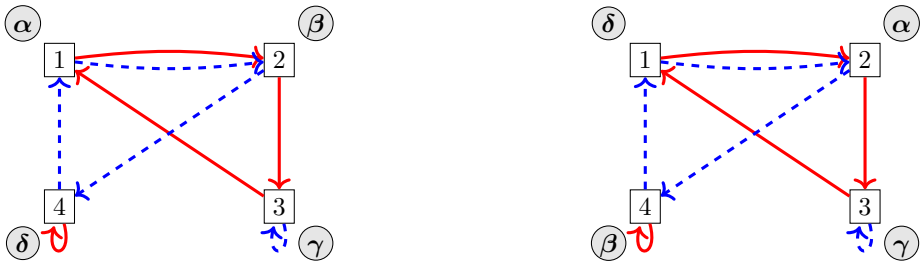
8.3 A problem of pebbles on an arc-colored Eulerian directed graph

The goal of this section is to introduce a problem of pebbles moving on an Eulerian directed graph, which we believe to be interesting on its own. The proofs of Theorems 8.1.1 and 8.1.2 essentially consist in reducing the problem of existence of a balanced feasible assignment to this pebble problem. From now on, this section does not refer anymore to the question of assignments and periodic tasks.

Consider an arc-colored Eulerian directed multigraph $D = (V, A)$ such that each vertex is the head of an arc of every color, and also the tail of an arc of every color. (In other words, each color is a collection of vertex disjoint directed cycles covering the vertex set.) We call such a graph a regular colored Eulerian digraph. Assume we have a pebble on each vertex. We denote by P the set of pebbles, and we have thus $|P| = |V|$.

Now, we explain how a sequence of colors induces a sequence of moves for the pebbles. Given a sequence $c_1, c_2, \ldots$ of colors, each pebble is first moved along the unique arc of color c_1 leaving the vertex on which it is originally placed; then it is moved along the unique arc of color c_2 leaving the vertex it has reached after the first move; and so on. Note that each move sends each pebble to a distinct vertex and so after each move, there is again a pebble on each vertex.

An illustration of such an arc-colored Eulerian directed multigraph and a move of the pebbles is given in Figure 8.3.



(a) Graph D with four vertices $V = \{1, 2, 3, 4\}$ and pebbles $P = \{\boldsymbol{\alpha}, \boldsymbol{\beta}, \boldsymbol{\gamma}, \boldsymbol{\delta}\}$.

(b) Pebbles on the graph D after moving following the color dashed blue.

Figure 8.3: An example of a regular colored Eulerian digraph.

We might ask under which condition there exists an infinite sequence of colors such that the arc visits are "balanced," i.e., each pebble visits each arc with the same frequency. Not only does such a sequence always exist, but it can also be chosen to be periodic.

Proposition 8.3.1. *There always exists a periodic sequence of colors making each pebble visit each arc with the same frequency* $1/|A|$.

The proof shows a bit more: each pebble actually follows a periodic walk on D which has the same period as the sequence of colors, and the latter is upper bounded by $|A|(|V|-1)!$.

The proof of this proposition relies on a larger graph $\widetilde{D} = (\widetilde{V}, \widetilde{A})$ built as follows. The vertex set $\widetilde{V}$ is the set of bijections from P to V. For every color c, define the permutation σ_c of V by setting $\sigma_c(i) = i'$ whenever there is an arc of color c from i to i' in D. The set $\widetilde{A}$ is built as follows: for each bijection $\eta\colon P \to V$ and each color c, introduce an arc $(\eta, \sigma_c \circ \eta)$, and color this arc with color c. The indegree and outdegree of every vertex in $\widetilde{V}$ are equal to the number of colors.

For each pebble j, we introduce a function $p_j\colon \widetilde{A} \to A$. Given an arc $\tilde{a} = (\eta, \eta')$ of $\widetilde{A}$ with color c, we define $p_j(\tilde{a})$ as the arc $(\eta(j), \eta'(j))$ of A with color c.

The graph $\widetilde{D}$ is an encoding of all possible distributions of the pebbles on V and all possible transitions between these distributions. More precisely consider any initial distribution η of the pebbles on V and a sequence of colors $c_1, c_2, \ldots$. The moves induced by the sequence of colors translate into a walk on $\widetilde{D}$. The corresponding sequence of vertices of $\widetilde{D}$ is the sequence of distributions of the pebbles on V induced by the sequence of colors.

Lemma 8.3.2. *Let $j \in P$ and $a \in A$. Denoting by κ the number of connected components of $\widetilde{D}$, we have*

$$|p_j^{-1}(a) \cap A(\widetilde{K})| = \frac{(|V|-1)!}{\kappa}$$

for every connected component $\widetilde{K}$ of $\widetilde{D}$. In particular, the left-hand term is independent of j, a, and $\widetilde{K}$.

Note that weakly and strongly connected components of $\widetilde{D}$ are identical by equality of the in- and outdegrees.

Proof of Lemma 8.3.2. Denote by c the color of a.

We prove first that every connected component $\widetilde{K}$ of $\widetilde{D}$ contains at least one arc from $p_j^{-1}(a)$. Let η be a vertex of such a connected component $\widetilde{K}$. Consider any walk W in D from $\eta(j)$ to the tail of a, and then traversing a. Such a walk exists because D is strongly

connected. With $c_1, c_2, \dots, c$ being the sequence of colors of the arcs traversed by the walk, the sequence

$$\eta, \sigma_{c_1} \circ \eta, \sigma_{c_2} \circ \sigma_{c_1} \circ \eta, \dots, \sigma_c \circ \cdots \circ \sigma_{c_2} \circ \sigma_{c_1} \circ \eta$$

forms a walk in $\widetilde{K}$ starting from η, whose image by p_j is W. Hence, $\widetilde{K}$ contains at least one arc from $p_j^{-1}(a)$.

Second, given two components $\widetilde{K}_1$ and $\widetilde{K}_2$ of $\widetilde{D}$, we build an injective map $\psi\colon A(\widetilde{K}_1) \to \widetilde{A}$ as follows. Pick $\tilde{a}_1 \in p_j^{-1}(a) \cap A(\widetilde{K}_1)$ and $\tilde{a}_2 \in p_j^{-1}(a) \cap A(\widetilde{K}_2)$. According to what we have just proved, these two arcs exist. Write $\tilde{a}_1 = (\eta_1, \sigma_c \circ \eta_1)$ and $\tilde{a}_2 = (\eta_2, \sigma_c \circ \eta_2)$. Then, for an arc $\tilde{a} \in A(\widetilde{K}_1)$ with tail vertex η and color d, set $\psi(\tilde{a})$ as the arc $(\eta \circ \eta_1^{-1} \circ \eta_2, \sigma_d \circ \eta \circ \eta_1^{-1} \circ \eta_2)$ with color d (this arc is unique). Checking that ψ is injective is immediate.

Third, we check that ψ maps elements from $p_j^{-1}(a) \cap A(\widetilde{K}_1)$ to $p_j^{-1}(a) \cap A(\widetilde{K}_2)$. Let $\tilde{a}$ be an arc in $p_j^{-1}(a) \cap A(\widetilde{K}_1)$. It is of the form $(\eta, \sigma_c \circ \eta)$. Its image by ψ is the arc $(\eta \circ \eta_1^{-1} \circ \eta_2, \sigma_c \circ \eta \circ \eta_1^{-1} \circ \eta_2)$ with color c. Denoting i the tail of a, we have $\eta(j) = \eta_1(j) = \eta_2(j) = i$, which implies immediately that $p_j\big(\psi(\tilde{a})\big)$ has the same endpoints as a. Since it has also the same color c, we have $p_j\big(\psi(\tilde{a})\big) = a$.

From the previous two paragraphs, we see that for any two components $\widetilde{K}_1$ and $\widetilde{K}_2$ of $\widetilde{D}$, we have $|p_j^{-1}(a) \cap A(\widetilde{K}_1)| \leqslant |p_j^{-1}(a) \cap A(\widetilde{K}_2)|$. Since the choices of $\widetilde{K}_1$ and $\widetilde{K}_2$ can be arbitrary, we have actually

$$|p_j^{-1}(a) \cap A(\widetilde{K}_1)| = |p_j^{-1}(a) \cap A(\widetilde{K}_2)|\,. \tag{8.2}$$

Finally, an arc $\tilde{a} = (\eta, \eta')$ is mapped to a by p_j precisely when $\tilde{a}$ is colored with color c and $\eta(j) = i$ (where i is the tail of a). The number of bijections η from P to V with $\eta(j) = i$ is $(|V|-1)!$. Hence, $|p_j^{-1}(a)| = (|V|-1)!$. Combining this with equality (8.2), we get the desired conclusion. □

Proof of Proposition 8.3.1. Choose any connected component $\widetilde{K}$ of $\widetilde{D}$. It is Eulerian, since each vertex of $\widetilde{D}$ has equal in- and outdegrees. Consider an arbitrary Eulerian cycle, and denote by $c_1, c_2, \dots$ the sequence of colors of the arcs of this cycle. According to Lemma 8.3.2, every pebble j moved according to this sequence of colors follows a closed walk on D visiting each arc $\frac{(|V|-1)!}{\kappa}$ times. Repeating infinitely many times this sequence of colors provides the desired periodic sequence. □

8.4 Proofs of Theorems 8.1.1 and 8.1.2

The goal of this section is to prove Theorems 8.1.1 and 8.1.2. These two results are direct applications of Lemma 8.4.1. The proof of this lemma relies on the problem of finding a balanced sequence of colors for the pebbles as in Section 8.3.

The following proofs and results are established for the extended version. They are also valid for the basic version, as it is a special case of the extended version, even though Theorems 7.1.1 and 7.1.2 provide us with stronger results.

Lemma 8.4.1. *Let $\mathcal{I}$ be an instance such that $|U(\mathcal{I})| = q$ and $\mathcal{F}$ be an arbitrary set of feasible assignments. If $D^{\mathcal{I},\mathcal{F}}$ is Eulerian, then there exists a periodic balanced feasible assignment for $\mathcal{I}$ with period bounded by $|\mathcal{F}|q!$.*

Proof. Suppose that $D^{\mathcal{I},\mathcal{F}}$ is Eulerian. Place one pebble on each vertex of $D^{\mathcal{I},\mathcal{F}}$. Applying Proposition 8.3.1 on $D^{\mathcal{I},\mathcal{F}}$, with each feasible assignment in $\mathcal{F}$ identified with a color, we get a periodic sequence $f_1, f_2, \dots$ of feasible assignments. Denote by g the resulting feasible assignment given by Lemma 8.2.1 for this sequence, with π_1 being an arbitrary permutation.

Number $j = g(i,1)$ the pebble initially placed on vertex $i \in U(\mathcal{I})$. This makes sure that each pebble gets a distinct number in $[q]$ (by the bijectivity of $\varphi_{g,1}$). We establish now the following claim: *For every $i \in U(\mathcal{I})$ and every $j \in [q]$, pebble j is on vertex i after its rth move if and only if $g(i, r+1) = j$.*

Let us proceed by induction on $r \in \mathbb{Z}_{\geqslant 0}$. This is true for $r = 0$ by the definition of the numbering of the pebbles. Suppose now that the claim if true for some $r \in \mathbb{Z}_{\geqslant 0}$. Consider pebble j and assume it is placed on i after its $r+1$th move. This means that the pebble j was on vertex i' after its rth move then moved along the arc from i' to i labeled with f_{r+1}. Then, using equation (7.3) and the fact that $f_{r+1}(i,2) = f_{r+1}(i',1)$ by definition of $D^{\mathcal{I},\mathcal{F}}$,

$$g(i, r+2) = \pi_{r+2}(f_{r+2}(i,1)) = \pi_{r+1}(f_{r+1}(i,2)) = \pi_{r+1}(f_{r+1}(i',1)) = g(i', r+1) = j\,, \quad (8.3)$$

as desired. Conversely, assume that $g(i, r+2) = j$. Denote by i' the tail of the arc of head i and label f_{r+1}. Then equation (8.3) holds as well, meaning that $g(i', r+1) = j$. By induction, the pebble j was placed on vertex i' after its rth move. It then moves along the arc from i' to i with label f_{r+1}, which concludes the proof of the claim.

We check that g is balanced. According to Proposition 8.3.1, for every $i \in U(\mathcal{I})$, every $j \in [q]$, and every $f \in \mathcal{F}$, we have

$$\lim_{t\to+\infty} \frac{1}{t} \big|\{r \in [t] \colon \text{pebble } j \text{ arrives at } i \text{ along arc labeled with } f \text{ for its } r\text{th move}\}\big| = \frac{1}{|A^{\mathcal{F}}|}\,.$$

With the claim, this equality becomes

$$\lim_{t\to+\infty} \frac{1}{t} \big|\{r \in [t] \colon f_r = f \text{ and } g(i, r+1) = j\}\big| = \frac{1}{|A^{\mathcal{F}}|}\,.$$

This equality is actually also true when $U(\mathcal{I})$ is replaced by the larger set $[n]$. Indeed, we have $g(i, r+1) = g(u_{f_{r+1},1}(i), r+1)$ for every $r \in \mathbb{Z}_{\geqslant 0}$, by definition of g and $u_{f_{r+1},1}(i)$ (the latter being defined in Section 7.3).

Therefore, for every $i \in [n]$ and every $j \in [q]$, we have

$$\begin{aligned}\lim_{t\to+\infty} \frac{1}{t} \big|\{r \in [t] \colon g(i, r+1) = j\}\big| &= \sum_{f\in\mathcal{F}} \lim_{t\to+\infty} \frac{1}{t} \big|\{r \in [t] \colon f_r = f \text{ and } g(i, r+1) = j\}\big| \\ &= \frac{|\mathcal{F}|}{|A^{\mathcal{F}}|} = \frac{1}{q}\,.\end{aligned}$$

This leads to the desired relation $\lim_{t\to+\infty} \frac{1}{t}\big|\{r \in [t] \colon g(i,r) = j\}\big| = 1/q$ since the element $r = 1$ does not count asymptotically.

We check that g is periodic with period upper bounded by $|\mathcal{F}|q!$. According to the comment following Proposition 8.3.1, each pebble follows a periodic walk. Denote by h the period of this walk. Still according to the same comment, we have $h \leqslant |A^{\mathcal{F}}|(q-1)! = |\mathcal{F}|q!$ and that the sequence of the f_r is periodic with period h. With the claim proven above, for each $i \in U(\mathcal{I})$ and $r \in \mathbb{Z}_{>0}$, we thus have $g(i,r) = g(i, r+h)$. This relation is true for all $i \in [n]$ because

$$\begin{aligned} g(i,r) &= \pi_r(f_r(i,1)) = \pi_r(f_r(u_{f_r,1}(i),1)) = g(u_{f_r,1}(i), r) = g(u_{f_r,1}(i), r+h) \\ &= \pi_{r+h}(f_{r+h}(u_{f_r,1}(i),1)) = \pi_{r+h}(f_{r+h}(u_{f_{r+h},1}(i),1)) = \pi_{r+h}(f_{r+h}(i,1)) = g(i, r+h)\,. \end{aligned}$$

Therefore, the assignment g is periodic with period h. □

Proof of Theorem 8.1.1. Denote by $\mathcal{I}'$ the new instance defined from $\mathcal{I}$ as in Section 8.2.3 and take $\mathcal{F}_{\mathcal{I}'}$ a universal set of feasible assignments for $\mathcal{I}'$. Suppose there exists a balanced feasible

assignment for $\mathcal{I}$. According to Lemma 8.2.6, the graph $D^{\mathcal{I}',\mathcal{F}_{\mathcal{I}'}}$ is Eulerian. According to Lemma 8.4.1, there exists a periodic balanced feasible assignment for $\mathcal{I}'$ with period bounded by $|\mathcal{F}_{\mathcal{I}'}|q!$. Therefore there exists such an assignment for $\mathcal{I}$ (by Lemma 8.2.4), and we get the first part of the theorem.

For the second part, note that we can keep the connectivity property of the graph with a smaller set of feasible assignments. To do so, we take $\mathcal{F} \subseteq \mathcal{F}_{\mathcal{I}'}$ such that $D^{\mathcal{I}',\mathcal{F}}$ is Eulerian and $|\mathcal{F}| \leqslant 2(q-1)$. Such a set $\mathcal{F}$ exists because $D^{\mathcal{I}',\mathcal{F}_{\mathcal{I}'}}$ is Eulerian ($D^{\mathcal{I}',\mathcal{F}}$ is the subgraph of $D^{\mathcal{I}',\mathcal{F}_{\mathcal{I}'}}$ keeping only the arcs labeled by assignments in $\mathcal{F}$) and every strongly connected digraph with q vertices has a strongly connected subgraph covering all vertices with at most $2(q-1)$ arcs. We apply Lemmas 8.4.1 and 8.2.4 to conclude like above. □

Proof of Theorem 8.1.2. As discussed in Section 8.1.1, we just have to prove the implication (a') ⇒ (b'). Suppose there exists a feasible assignment for $\mathcal{I}$ with a worker performing each task at least once. Take an arbitrary universal set $\mathcal{F}$ of feasible assignments. According to Lemma 8.2.2, the graph $D^{\mathcal{I},\mathcal{F}}$ is weakly connected, thus Eulerian. According to Lemma 8.4.1, there exists a balanced feasible assignment for $\mathcal{I}$. □

8.5 Concluding remarks

8.5.1 Group action and Cayley graph

In this section, we present a more straightforward and elegant proof of Lemma 8.3.2, proposed by Joakim Tutt, post-doctoral researcher at the University of Geneva. This proof relies on an equivalent reformulation of our arc-colored digraph setting in terms of group action. Lemma 8.3.2 rewrites as Lemma 8.5.1 in this new setting.

For the following discussion, we state the definition of the Cayley graph $\Gamma(G,S)$ of a group G with respect to a generating set S, which is a graph representing group actions. It is defined in its most general setting as follows [94].

Let G be a group and S be a generating set of G. The *Cayley graph* $\Gamma(G,S)$ is an edge-colored directed graph constructed as follows:

- Each element g of G is assigned a vertex: the vertex set of $\Gamma(G,S)$ is identified with G.
- Each element s of S is assigned a color c_s.
- For every $g \in G$ and $s \in S$, there is a directed edge of color c_s from the vertex corresponding to $g \in G$ to the one corresponding to gs.

For a subset S of a group, we denote by $\langle S \rangle$ the subgroup generated by S.

Let $D = (V, A)$ be a regular colored Eulerian digraph and σ_c the permutation associated with color c, as in Section 8.3. Denoting by Σ the set of σ_c, where c ranges over the colors, the vertices of $\Gamma(\langle\Sigma\rangle, \Sigma)$ are the elements of $\langle\Sigma\rangle$ and for every $g \in \langle\Sigma\rangle$ and every $\sigma_c \in \Sigma$, there is an arc from g to $\sigma_c \circ g$ with color c.

For each vertex $i \in V$, we introduce the map $\widetilde{p}_i \colon \Gamma(\langle\Sigma\rangle, \Sigma) \to D$ defined as follows:

- for a vertex g of $\Gamma(\langle\Sigma\rangle, \Sigma)$, set $\widetilde{p}_i(g) \coloneqq g(i) \in V$,
- for an arc $\tilde{a}$ of $\Gamma(\langle\Sigma\rangle, \Sigma)$ going from g to $\sigma_c \circ g$ with color c, set $\widetilde{p}_i(\tilde{a})$ as the arc in A going from $g(i)$ to $(\sigma_c \circ g)(i)$ with color c.

Note that each connected component of the graph $\widetilde{D}$ (constructed in Section 8.3) is isomorphic to $\Gamma(\langle\Sigma\rangle, \Sigma)$. This means in particular that the number of vertices in $\Gamma(\langle\Sigma\rangle, \Sigma)$ is equal to $|\operatorname{Sym}(V)|/\kappa$, where κ is the number of connected components of $\widetilde{D}$. The map $\widetilde{p}_i$ is also closely linked to the map p_j defined in Section 8.3 for a pebble j. Identifying the vertex i with the pebble on vertex i, the map $\widetilde{p}_i$ defined above actually extends the restriction of p_i to the arcs of a connected component of $\widetilde{D}$, identified with $\Gamma(\langle\Sigma\rangle, \Sigma)$. In particular, it is such that $|\widetilde{p}_i^{-1}(a)| = |p_j^{-1}(a) \cap A(\widetilde{K})|$ for all $\widetilde{K}$ connected component of $\widetilde{D}$.

Lemma 8.3.2 can then be reformulated within this new setting as follows. Note that this equality can be rewritten as in Lemma 8.3.2 using the equality $|\langle\Sigma\rangle| = |\operatorname{Sym}(V)|/\kappa = |V|!/\kappa$, where $|\langle\Sigma\rangle|$ is the order of the group $\langle\Sigma\rangle$.

Lemma 8.5.1. *We have $|\widetilde{p}_i^{-1}(a)| = |\langle\Sigma\rangle|/|V|$ for every $i \in V$ and $a \in A$.*

Proof. For every vertex $j \in V$, we have $\widetilde{p}_j^{-1}(j) = \{g \in \langle\Sigma\rangle \colon g(j) = j\}$, which is the stabilizer $\operatorname{Stab}(j)$ of j under the group action of $\langle\Sigma\rangle$. Let $j \in V$ and $\widetilde{g} \in \langle\Sigma\rangle$ such that $\widetilde{g}(i) = j$. We have that $\widetilde{p}_i^{-1}(j) = \{g \in \langle\Sigma\rangle \colon g(i) = j\} = \widetilde{g}(\operatorname{Stab}(i))$. Thus, $\widetilde{p}_i^{-1}(j)$ and $\operatorname{Stab}(i)$ are cosets and $|\widetilde{p}_i^{-1}(j)| = |\operatorname{Stab}(i)|$.

Using the orbit-stabilizer theorem [95], we have $|\operatorname{Stab}(i)| = \frac{|\langle\Sigma\rangle|}{|\operatorname{Orb}(i)|}$ where $\operatorname{Orb}(i) \coloneqq \{j \in V \colon \exists g \in \langle\Sigma\rangle\ j = g(i)\}$ is called the orbit of i. The graph D is strongly connected translates into the group action being transitive, which means that $|\operatorname{Orb}(i)| = |V|$ for all $i \in V$.

Writing j the head of a, j' its tail, and c its color, the set $\widetilde{p}_i^{-1}(a)$ is made of the arcs of $\Gamma(\langle S\rangle, S)$ with color c, going from a $g \in \langle\Sigma\rangle$ such that $g(i) = j$ to $\sigma_c \circ g$. This set is in bijection with the set $\widetilde{p}_i^{-1}(j)$. Therefore $|\widetilde{p}_i^{-1}(a)| = |\widetilde{p}_i^{-1}(j)|$.

We thus have

$$|\widetilde{p}_i^{-1}(a)| = |\widetilde{p}_i^{-1}(j)| = |\operatorname{Stab}(i)| = \frac{|\langle\Sigma\rangle|}{|\operatorname{Orb}(i)|} = \frac{|\langle\Sigma\rangle|}{|V|}\,. \qquad \square$$

Proposition 8.3.1 can then be rewritten using Lemma 8.5.1 to exhibit the period of the sequence of colors.

Corollary 8.5.2. *There always exists a periodic sequence of colors making each pebble follow a closed walk on D visiting each arc with the same frequency $1/|A|$. The period of the sequence of colors is upper bounded by $|A||\langle\Sigma\rangle|/|V|$.*

Proof. As in the proof of Proposition 8.3.1, consider an arbitrary Eulerian cycle of Γ, and denote by $c_1, c_2, \dots$ the sequence of colors of the arcs of this cycle. According to Lemma 8.5.1, every pebble j moved according to this sequence of colors follows a closed walk on D visiting each arc $|\langle\Sigma\rangle|/|V|$ times. Repeating infinitely many times this sequence of colors provides the desired periodic sequence. $\square$

8.5.2 All feasible assignments are balanced. (Almost.)

If we are just investigating the existence of a balanced feasible assignment, and not the periodicity of such an assignment or its computability, we can replace Proposition 8.3.1 by the following lemma in the proof of Theorem 8.1.2. We keep the same setting of an arc-colored Eulerian directed multi-graph $D = (V, A)$ with a distribution of pebbles on its vertices, as in the beginning of Section 8.3.

Lemma 8.5.3. *Consider an infinite sequence of independent random colors drawn uniformly. Then, almost surely, this sequence makes each pebble visit each arc with the same frequency.*

In particular there are infinitely many color sequences making each pebble visit each arc with the same frequency. The proof relies on basic properties of Markov chains. (A standard reference on Markov chains is the book by Norris [68].) The proof does not show how to construct such a sequence of colors. It is not even clear whether the proof could be modified in that regard. So the proof shows that almost all color sequences have the desired property, but does not explain how to construct a single such sequence. Although this might sound surprising, this phenomenon is quite common. *Normal* numbers form an example: (almost) all numbers are normal, but the only known explicit examples are artificially constructed numbers, while it remains unknown whether naturally occurring numbers such as $\pi, e, \sqrt{2}, \ln(2)$ are

normal [92]. The situation of "normal numbers," whose existence status shares some similarity with that of balanced feasible assignments, was pointed out by Jean-François Delmas.

The proof of Proposition 8.3.1 actually provides an alternative proof of the existence of sequences of colors making each pebble visit each arc with the same frequency, with an explicit construction. However, the latter proof does not show that almost all sequences are actually like that. (In counterpart, it shows that such a sequence can be chosen to be periodic.)

Proof of Lemma 8.5.3. Any realization of this random sequence of colors defines a sequence of moves for the pebbles, as described above. Consider an arbitrary pebble. The random sequence of colors translates thus into a random walk of the pebble on the graph D. Denote by X_k the arc along which the pebble performs its kth move. The X_k's form a finite Markov chain. Since the graph is Eulerian, this Markov chain is irreducible, and hence there exists a unique invariant distribution $\boldsymbol{\lambda}$ such that $\boldsymbol{\lambda}^\top = \boldsymbol{\lambda}^\top M$, where M is the transition matrix of the Markov chain.

We claim that $\boldsymbol{\lambda}$ is actually the vector $\frac{1}{|A|}\mathbb{1}$, where $\mathbb{1}$ is the all-one vector. By the uniqueness of the invariant distribution, it is enough to check that $\mathbb{1}$ is a left eigenvector of M with eigenvalue equal to 1. The entry $M_{a,a'}$ of the transition matrix (row a, column a'), which corresponds to the probability of moving along a' just after moving along a, is equal to $1/\ell$ if the head of a is the tail of a', and 0 otherwise. The indegree of each vertex being ℓ, we have

$$\sum_{a \in A} M_{a,a'} = \ell \frac{1}{\ell} = 1\,,$$

and therefore $\mathbb{1}^\top M = \mathbb{1}^\top$.

According to the ergodic theorem, for almost all realizations of the random sequence of colors, the pebble visits any arc a with a frequency equal to the corresponding entry in $\boldsymbol{\lambda}$, which is equal to $\frac{1}{|A|}$ for all arcs since it is a probability distribution proportional to the all-one vector. The previous discussion does not depend on the considered pebble, which implies the desired result: for almost all realizations of the random sequence of colors, every pebble visits every arc with a frequency equal to $\frac{1}{|A|}$. □

8.5.3 Open question

There are open questions remaining for the problem of pebbles introduced in Section 8.3 and also for the extended version of the balanced assignment problem. Given an arc-colored Eulerian digraph $D = (V, A)$ with a pebble on each vertex, we showed that there exists a periodic sequence of colors making each pebble visit each arc with the same frequency (see Corollary 8.5.2). We also achieved an upper bound of $|A|(|V|-1)!/\kappa$ on the period, where κ is the number of connected components of $\widetilde{D}$. We do not know whether this bound is tight in general. The only case where we have been able to prove the tightness of this bound is exemplified in Figure 8.4 (the tightness of the bound is shown using an ILP solver as discussed in Section 9.3).

We can even build an instance of the extended version of the balanced feasible assignment problem for which $q = 4$, the optimal period is $24 \gg 4$, and the example of Figure 8.4 is a valid $D^{\mathcal{I},\mathcal{F}}$ as built in Section 8.2.2 for this instance (details are provided in Section 9.2).

Moreover, for the extended version of the balanced assignment problem, we do not know whether the bound $2(q-1)q!$ given in Theorem 8.1.1 is tight for a single instance.

8.5.4 Link to crew rostering

Contrarily to the discussions of Section 7.5.3, the results obtained in this chapter do not seem to extend to cyclic rosters. In particular, the periodic balanced assignment built in

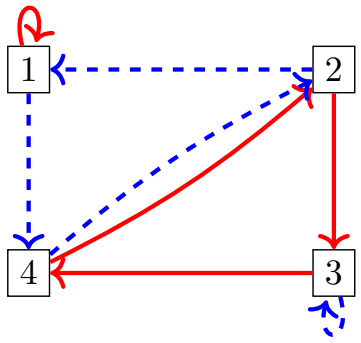


Figure 8.4: An example of a graph with 4 vertices ($q = 4$) and 2 colors such that the minimum period of a periodic sequence of colors making each pebble visit each arc with the same frequency is 24. For this graph, we have $\kappa = 2$.

Theorem 8.1.2 does not necessarily translate into a cyclic roster on a k-week basis for some $k \in \mathbb{Z}_{>0}$.

This actually relates to the subsequent group theory question: given $D = (V, A)$ a regular arc-colored Eulerian digraph with Σ the set of permutations associated with the colors, under what condition is there an element $\omega \in \langle\Sigma\rangle$ such that ω is a cyclic permutation of length $|V|$? In particular, Figure 8.4 provides an example where no such element exists. It can be checked by a simple enumeration of the 12 elements of $\langle\Sigma\rangle$.

The following lemma provides a straightforward condition for such an element to exist, which easily translates to balanced feasible assignments and cyclic rosters as discussed below.

Proposition 8.5.4. *Let $D = (V, A)$ be a regular arc-colored Eulerian digraph with Σ the set of permutations associated with its colors. If $|V|$ is a prime number, then there exists an element $\omega \in \langle\Sigma\rangle$ such that ω is a cyclic permutation of length $|V|$.*

Proof. Using the orbit-stabilizer theorem [95], since the group action of $\langle\Sigma\rangle$ is transitive (as D is Eulerian), we have $|\langle\Sigma\rangle| = |\operatorname{Stab}(i)||V|$ for every $i \in V$. Thus $|V|$ divides the order of $|\langle\Sigma\rangle|$. Using Cauchy theorem [93] with the prime number $|V|$, we thus have the existence of $\omega \in \langle\Sigma\rangle$ of order $|V|$. Decomposing ω into disjoint cycles, each of these cycles have length 1 or $|V|$ as $|V|$ is a prime number. Therefore, ω is a cyclic permutation of length $|V|$. □

Note that, when $|V|$ is not a prime number, the existence of such an element is not guaranteed.

Now let us explicit the relation between cyclic rosters and such a property. Let $\mathcal{I}$ be an instance such that $|U(\mathcal{I})| = q$. Consider a set $\mathcal{F}$ of feasible assignments such that $D^{\mathcal{I},\mathcal{F}}$ is a regular arc-colored Eulerian digraph, and such that there exists a sequence of colors $c_1, \ldots, c_k$ (not necessarily distinct and in no particular order) for which $\omega \coloneqq \sigma_{c_k} \circ \cdots \circ \sigma_{c_1}$ is a cyclic permutation of length q, where σ_c denotes, for every color c of $D^{\mathcal{I},\mathcal{F}}$, the permutation associated with c. Then ω^q (the composition of ω with itself q times) defines a finite sequence of colors of length kq such that each pebble follow a closed walk on D, and within each color, each pebble visits each arc the same number of times. We can show that this sequence of colors defines a cyclic roster on a k-week basis.

Corollary 8.5.5. *Let $\mathcal{I}$ be an instance such that q is a prime number. Suppose there exists a balanced feasible assignment. There exists then a cyclic roster on a k-week basis, with $k \leqslant (q! - 1)$.*

Proof. Denote by $\mathcal{I}'$ the instance defined from $\mathcal{I}$ as in Section 8.2.3. For every color c of $D^{\mathcal{I}',\mathcal{F}_{\mathcal{I}'}}$, denote by σ_c the permutation associated with c and by Σ the set of these permutations.

Lemma 8.2.6 (with $\mathcal{F}_{\mathcal{I}'}$ which is a universal set of feasible assignments) ensures that $D^{\mathcal{I}',\mathcal{F}_{\mathcal{I}'}}$ is a regular arc-colored Eulerian digraph. Since q is prime, Proposition 8.5.4 yields an element $\omega \in \langle\Sigma\rangle$ that is a cyclic permutation of length q. Write $\omega = \sigma_{c_k} \circ \cdots \circ \sigma_{c_1}$ using the generators in Σ, where k is the smallest length of such a sequence $c_1, \ldots, c_k$.

We now show that ω defines a cyclic roster on a k-week basis, namely that it defines a feasible assignment satisfying the equation from Section 7.5.3. Denote by g the feasible assignment for $\mathcal{I}'$ resulting from Lemma 8.2.1 with $\pi_1 = \mathrm{Id}$, applied with the k-periodic sequence $f_{c_1}, \ldots,$ of feasible assignments corresponding to the k-periodic sequence of colors defined from $c_1, \ldots, c_k$. Define $\pi \coloneqq \varphi_{g,1} \circ \omega^{-1} \circ \varphi_{g,1}^{-1}$, which is a cyclic permutation of length q (since $\varphi_{g,1}$ is a bijection).

Let us show by induction on $m \geqslant 1$ that $g(i, m+k) = \pi(g(i,m))$ for all $i \in U(\mathcal{I}')$. For $m = 1$, for every $i \in U(\mathcal{I}')$ we have by construction $g(i, 1+k) = g(\sigma_{c_k}^{-1}(i), k) = \ldots = g(\omega^{-1}(i), 1) = (\varphi_{g,1} \circ \omega^{-1} \circ \varphi_{g,1}^{-1})(g(i,1)) = \pi\big(g(i,1)\big)$. Let $m \geqslant 1$ such that $g(i, m+k) = \pi\big(g(i,m)\big)$ for all $i \in U(\mathcal{I}')$. Let $i \in U(\mathcal{I}')$. Since $f_{c_{m+k}} = f_{c_m}$, we have

$$g(i, m+1+k) = g(\sigma_{c_m}^{-1}(i), m+k) = \pi\big(g(\sigma_{c_m}^{-1}(i)), m\big) = \pi\big(g(i, m+1)\big)$$

which concludes the induction. Thus, for every $i \in U(\mathcal{I}')$, every $r \in [k]$, and every $t \in \mathbb{Z}_{>0}$, we have $g(i, r+kt) = \pi\big(g(i, r+k(t-1))\big)$. This equality extends to all $i \in [n]$ since $g(i, r+kt) = g(u_{f_{c_r},1}(i), r+kt) = \pi\big(g(u_{f_{c_r},1}(i), r+k(t-1))\big) = \pi\big(g(i, r+k(t-1))\big)$ for every $i \in [n]$. Consequently, g is a cyclic roster on a k-week basis for $\mathcal{I}'$, and, the proof of Lemma 8.2.4 can be easily adapted to show that the corresponding restriction is a cyclic roster on a k-week basis for $\mathcal{I}$.

Recall finally that k is the smallest length of a writing of ω using the generators in Σ. In other words, k corresponds to the length of a shortest path from the identity to ω in $\Gamma(\langle\Sigma\rangle, \Sigma)$. The number of vertices in this path is upper bounded by $|\langle\Sigma\rangle|$. We can thus bound k by $|\langle\Sigma\rangle| - 1$, and since $|\langle\Sigma\rangle| \leqslant q!$, we have $k \leqslant q! - 1$ which concludes the proof. □

Chapter 9

Complementary discussions

***

9.1 Special case: Cayley graph . . . 137
9.1.1 A problem of pebbles on a Cayley graph . . . 137
9.1.2 Link to periodic balanced feasible assignments . . . 138
9.2 Equivalence between colors and assignments . . . 139
9.3 An ILP solving the PERIODIC RELAXED BALANCED SEQUENCE OF COLORS problem . . . 142
9.4 Other particular feasible assignments . . . 144
9.4.1 Problem formulation . . . 144
9.4.2 First result . . . 145
9.4.3 A special case . . . 145
9.4.4 Skill-balanced sequence of colors . . . 146
Problem of pebbles . . . 146
From a skill-balanced sequence of colors to a skill-balanced assignment 147

This chapter collects three complementary viewpoints that clarify the scope and limitations of the previous results. It essentially consists of: graph-theoretic interpretations, algorithmic formulations, and open questions.

9.1 Special case: Cayley graph

9.1.1 A problem of pebbles on a Cayley graph

Let us focus on the extended version of the problem. The goal of this section is to show that, under an additional graph assumption, we can obtain a much tighter bound on the period of a periodic balanced sequence of colors. This result is formalized in Proposition 9.1.2.

As reminded in Section 8.5.1, a Cayley graph is defined from a finite group and a generating set. Terence Tao provided on his blog a combinatorial characterization of Cayley graphs.

Lemma 9.1.1 (Tao [84])**.** *A colored directed graph is a Cayley graph if and only if the three following properties hold:*

- *(Regularity) For each color c, every vertex i has a single arc with head i and color c, and a single arc with tail i and color c.*

- *(Connectedness) The graph is connected.*
- *(Homogeneity) For every pair of vertices i, j, there is a unique colored graph isomorphism that maps i to j.*

Note that Cayley graphs are special cases of the graphs as in Section 8.3: the regularity and connectedness properties come from the definition of Section 8.3. However, not all regular colored Eulerian digraphs are Cayley graphs (Figure 8.4 provides an example that is not homogeneous). When D is a Cayley graph, we denote by σ_c the permutation of the vertices associated with color c as in Section 8.3.

The following proposition states that the bound on the period of a balanced sequence of colors can be improved for Cayley graphs. The proof essentially follows the outline of the proof of Corollary 8.5.2.

Proposition 9.1.2. *If D is a Cayley graph, then there exists a periodic balanced sequence of colors with period $|A|$.*

In order to prove Proposition 9.1.2, we establish the following lemma:

Lemma 9.1.3. *Suppose that D is a Cayley graph. Then D is isomorphic to $\Gamma(\langle S\rangle, S)$ where S is the set of σ_c, where c ranges over the colors. In particular the order of $\langle S\rangle$ is $|V|$.*

Proof. Along the proof of 9.1.1, Tao actually proves the following fact: each vertex defines an element in a group G (with an arbitrary vertex as the identity element of G) and each color $c \in C$ defines a generator by looking at the tail of the unique arc going out of the identity vertex with color c. By homogeneity, this is the same as saying that each σ_c (c ranging over the colors) defines a generator. Therefore, the graph D is isomorphic to the Cayley graph $\Gamma(\langle S\rangle, S)$ where S is the set of σ_c, where c ranges over the colors. Therefore, by definition, the number of vertices of $\Gamma(\langle S\rangle, S)$ is $|\langle S\rangle|$, which is then equal to $|V|$. □

Proof of Proposition 9.1.2. Using Lemma 8.5.1 and Lemma 9.1.3 under the homogeneity assumption, we have $|\widetilde{p_i}^{-1}(a)| = 1$ for all $i \in V, a \in A$. The proof of Corollary 8.5.2 can then be revised: considering the colors along a Eulerian cycle of D, every pebble performs a closed walk on D visiting each arc one single time. □

9.1.2 Link to periodic balanced feasible assignments

The discussions of Section 9.1.1 can be extended to periodic balanced assignments in the same manner as in Section 8.4. We state this as Proposition 9.1.4. The proof of this result essentially follows the outline of the proof of Lemma 8.4.1, with a better upper bound on the period when the homogeneity property is assumed. However, this homogeneity assumption seems to be a requirement difficult to ensure from an instance $\mathcal{I}$. This leads to multiple open questions, discussed after stating and proving Proposition 9.1.4.

Proposition 9.1.4. *Let $\mathcal{I}$ be an instance such that $|U(\mathcal{I})| = q$ and $\mathcal{F}$ be an arbitrary set of feasible assignments. If $D^{\mathcal{I},\mathcal{F}}$ is a Cayley graph, then there exists a periodic balanced feasible assignment for $\mathcal{I}$ with period bounded by $|\mathcal{F}|q$.*

Proof. The proof of this result follows the outline of the proof of Lemma 8.4.1, the bound on the period being refined using Proposition 9.1.2. This shows that there exists a periodic balanced sequence of colors with period upper bounded by $|A^{\mathcal{F}}| = |\mathcal{F}|q$. □

In particular, this proposition can be used, along with Proposition 9.2.1, in order to build, from any Cayley graph, an instance such that the periodic balanced feasible assignment with minimal period has a period $|A|$. However, we do not know how to use this result to get better upper bounds more generally.

According to Lemma 9.2.2 of the next section, there exists an instance $\mathcal{I}$ (with $|U(\mathcal{I})| = q$) such that $D^{\mathcal{I},\overline{\mathcal{F}}_{\mathcal{I}}}$ is the same as in Figure 8.4. From here, no set of feasible assignment $\mathcal{F}$ can make the graph $D^{\mathcal{I},\mathcal{F}}$ Eulerian and homogeneous for this instance.

We were not able to identify any nice sufficient condition of existence, checkable directly on the instance, that would allow the use of Proposition 9.1.4 for some set $\mathcal{F}$ of feasible assignments.

9.2 Equivalence between colors and assignments

The goal of this section is to make the link between periodic balanced feasible assignments of minimal period and periodic sequences of colors with minimal period, satisfying the same type of requirement as in Section 8.3, up to a relaxation. We insist that this equivalence is not ensured to be polynomial. The purpose of this section and the following one is not to apply an efficient algorithm to solve the balanced assignment problem in full generality. It is rather to explore the question of the tightness of the bound on balanced assignments from regular colored Eulerian digraphs, which are easy to build. This exploration method could be done via an ILP resolution, as explained in Section 9.3. Let us formally introduce the two problems. Define first the (PBA) problem for the extended version.

Periodic balanced assignment (PBA)
Input. Tasks $([s_i, e_i))_{i\in[n]}$, number q of workers, set $\mathcal{S}$ of non-overlapping schedules.
Output. A periodic balanced feasible assignment with minimal period.

Note that a periodic balanced feasible assignment is a priori an infinite object. Actually, a T-periodic balanced feasible assignment can be "finitely" encoded by taking the first T weeks (over all workers) of the periodic balanced feasible assignment. Therefore, it makes sense to output such an assignment (Theorem 8.1.1 ensures that the period is bounded by $2(q-1)q!$).

Define now the (PRBC) problem, similarly as in Section 8.3 but in a relaxed form: in Section 8.3 we built a periodic sequence of colors making each pebble follow a closed walk on D, each pebble visiting each arc with the same frequency. For the (PRBC) problem, the requirement for the sequence of colors in the output is relaxed (and we now additionally ask for a sequence of minimal length). We emphasize that the assumption that no two colors encode the same permutation is required for the coming results, even though it was not the case in the previous discussions.

Periodic relaxed balanced sequence of colors (PRBC)
Input. A regular colored Eulerian digraph $D = (V, A)$ such as no two colors encode the same permutation.
Output. A finite sequence of colors of minimal length that, in the setting of Section 8.3, makes each pebble follow a closed walk on D, and such that, within each color, each pebble visits each arc the same number of times.

Recall, given an instance $\mathcal{I}$ of the (PBA) problem, the definition of $\overline{\mathcal{F}}_{\mathcal{I}}$ from Section 7.3: some feasible assignments induce the same assignment on the first week. For each assignment on the first week, pick a representative among the set of feasible assignments and put it in $\overline{\mathcal{F}}_{\mathcal{I}}$.

Proposition 9.2.1. *Given an instance $D = (V, A)$ of the* (PRBC) *problem with set of colors C, we can build an instance $\mathcal{I}(D)$ of the* (PBA) *problem with $q = |V|$, $|\overline{\mathcal{F}}_{\mathcal{I}(D)}| = |C|$, and same optimal value in polynomial time.*

Let us prove the following lemma first.

Lemma 9.2.2. *Given an instance of* (PRBC) *with graph D, there is an instance $\mathcal{I}(D)$ of* (PBA) *such that $D^{\mathcal{I}(D),\overline{\mathcal{F}}_{\mathcal{I}(D)}}$ is isomorphic to the graph D. This instance can be built in polynomial time.*

Proof. Let C be the set of colors on $D = (V, A)$. We identify the colors in C with the integers in $[|C|]$ and the vertices in V with the integers in $[|V|]$.

Build the instance $\mathcal{I}(D)$ of (PBA) as follows:

- The number of workers in $\mathcal{I}(D)$ is $|V|$.
- The tasks in $\mathcal{I}(D)$ are the following intervals, with $\varepsilon \coloneqq \frac{1}{2(|C|(|V|-1)+1)}$:
  - $t_0^v \coloneqq [-\varepsilon, \varepsilon)$ for $v \in V$,
  - $t_{c,v} \coloneqq [a_{c,v}, b_{c,v})$, where $a_{c,v} = ((|V|-1)(c-1)+v) \times \varepsilon$ and $b_{c,v} = ((|V|-1)(c-1)+v+1) \times \varepsilon$, for $c \in C$ and $v \in [|V|-1]$,
  - $t'_{c,v} \coloneqq [a'_{c,v}, b'_{c,v})$, where $a'_{c,v} = ((|V|-1)(|C|+c-1)+v) \times \varepsilon$ and $b'_{c,v} = ((|V|-1)(|C|+c-1)+v+1) \times \varepsilon$, for $c \in C$ and $v \in [|V|-1]$.
- The set of schedules in $\mathcal{I}(D)$ is $\mathcal{S} \coloneqq \{(T_{c,v}, W_{c,v}) \colon c \in C, v \in V\}$ where

$$\begin{aligned}
W_{c,v} &\coloneqq \{t_0^{\sigma_c(v)}\},\\
T_{c,1} &\coloneqq \{t_0^1\} \cup \{t_{\ell,v} \colon \ell \neq c, v \in [|V|-1]\} \cup \{t'_{c,1}\},\\
T_{c,v} &\coloneqq \{t_0^v\} \cup \{t_{c,v-1}\} \cup \{t'_{c,v}\} \cup \{t'_{\ell,v-1} \colon \ell \neq c\} \text{ for } 2 \leqslant v \leqslant |V|-1,\\
T_{c,|V|} &\coloneqq \{t_0^{|V|}\} \cup \{t_{c,|V|-1}\} \cup \{t'_{\ell,|V|-1} \colon \ell \neq c\}.
\end{aligned}$$

Note that each schedule in $\mathcal{S}$ is a non-overlapping schedule. This makes the instance $\mathcal{I}(D)$ well-defined and valid. This construction is also polynomial in the size of D.

We finish the proof by showing that $D^{\mathcal{I}(D),\overline{\mathcal{F}}_{\mathcal{I}(D)}}$ is isomorphic to the graph D. Let us define the map γ as follows:

$$\begin{aligned}
\gamma \colon \quad & V \to U(\mathcal{I}(D))\\
& v \mapsto t_0^v
\end{aligned}$$

Let us prove that γ is an isomorphism (preserving the colors). First, γ is a bijection by construction of the tasks of $\mathcal{I}(D)$.

For a color $c \in C$, we denote by g_c the assignment in $\overline{\mathcal{F}}(\mathcal{I}(D))$ such that worker j follows schedule $S_{c,\pi(j)} \coloneqq (T_{c,\pi(j)}, W_{c,\pi(j)})$ on the first week, with π a permutation. Such an assignment exists because the schedules $S_{c,1}, \ldots, S_{c,q}$ are pairwise non-overlapping and cover all task occurrences overlapping the first week.

First, we show that γ is edge-preserving. Let $(u, v) \in A$ with color c. By construction, we have $g_c(t_0^u, 1) = g_c(t_0^{\sigma_c(u)}, 2) = g_c(t_0^v, 2)$. Thus, there is an arc in $D^{\mathcal{I}(D),\overline{\mathcal{F}}_{\mathcal{I}(D)}}$ from $\gamma(u)$ to $\gamma(v)$ labeled by g_c.

For an assignment $g \in \overline{\mathcal{F}}_{\mathcal{I}(D)}$, we claim that there exists a unique color in C, denoted by c_g, such that worker j follows schedule $S_{c_g,\pi(j)}$ on the first week, with π a permutation. Let us show that such a color exists. By contradiction, we assume that there is j and j' with $\pi(j') = \pi(j) + 1$, and $\ell \neq \ell'$ such that the worker j performs $S_{\ell,\pi(j)}$ and the worker j' performs $S_{\ell',\pi(j')}$ on the first week of assignment g. Since $t'_{\ell,\pi(j)} \in T_{\ell,\pi(j)} \cup T_{\ell',\pi(j')}$, there is a contradiction. The color c_g is unique because we assumed D has no colors duplicates.

Let now (t_0^u, t_0^v) be an arc of $D^{\mathcal{I}(D),\overline{\mathcal{F}}_{\mathcal{I}(D)}}$ labeled with g. We have $t_0^u \in T_{c_g,u}$ and $t_0^v \in W_{c_g,u}$ by definition of c_g. Thus, $\sigma_{c_g}(u) = v$ meaning that there is an arc $(u, v) \in A$ with color $c_g \in C$.

Therefore, γ is a graph isomorphism which concludes the proof. □

Proof of Proposition 9.2.1. Let $\mathcal{I}(D)$ be the instance built in polynomial time in the proof of Lemma 9.2.2, in such a way that $D = (V, A)$ is isomorphic to $D^{\mathcal{I}(D),\overline{\mathcal{F}}_{\mathcal{I}(D)}}$ (via the isomorphism

γ). This implies in particular that $q = |V|$ and $|\overline{\mathcal{F}}_{\mathcal{I}(D)}| = |C|$. Let us show that the values are the same.

First, let $c_1, \dots, c_T$ be a finite sequence colors of minimal length satisfying the requirement of (PRBC). Let us prove that there exists a T-periodic balanced feasible assignment for $\mathcal{I}(D)$. This sequence of colors defines a T-periodic balanced feasible assignment as follows. Since D and $D^{\mathcal{I}(D),\overline{\mathcal{F}}_{\mathcal{I}(D)}}$ are isomorphic, this sequence of colors defines a sequence of feasible assignments $g_{c_1}, \dots g_{c_T}$. Denote by g the feasible assignment given by Lemma 8.2.1 for the T-periodic sequence defined from $g_{c_1}, \dots g_{c_T}$, with π_1 being an arbitrary permutation. We show that g is T-periodic and balanced. As in the proof of Lemma 8.4.1, for every $v \in V$ and every $p \in P$, pebble p is on vertex v after its rth move if and only if $g(\gamma(v), r+1) = \gamma(p)$. We check that g is balanced. We have, by definition of the sequence of color, for every $v \in V$, every $p \in P$, and every color $c \in C$:

$$\begin{aligned} N_c &= \big|\{r \in [T] \colon \text{pebble } p \text{ arrives at } v \text{ along arc of color } c \text{ for its } r\text{th move}\}\big| \\ &= \big|\{r \in [T] \colon c_r = c \text{ and } g(\gamma(v), r+1) = \gamma(p)\}\big| \end{aligned}$$

Thus, for every $i \in U(\mathcal{I}(D))$, and every $j \in [q]$, we have that $\big|\{r \in [T] \colon c_r = c \text{ and } g(i, r+1) = j\}\big|$ does not depend on i nor j. We denote by N_c this common value. This holds for every task $i \in [n]$ as $g(i, r+1) = g(u_{g_{c_{r+1}},1}(i), r+1)$. Thus, we have, for every $i \in [n]$, every $j \in [q]$,

$$\lim_{t \to +\infty} \frac{1}{t}\big|\{r \in [t] \colon g(i, r+1) = j\}\big| = \frac{1}{T}\sum_{c \in C}\{r \in [T] \colon c_r = c \text{ and } g(i, r+1) = j\} = \sum_{c \in C}\frac{N_c}{T} = \frac{1}{q}\,.$$

As in the proof of Lemma 8.4.1, g is periodic with period upper bounded by T.

Finally, let us show that g has period exactly T. Suppose by contradiction that it has period $T' < T$. Recall the definition of the schedules of the instance $\mathcal{I}(D)$ from the proof of Lemma 9.2.2. Let us prove that for every color $c \in C$, each worker goes through each schedule $S_{c,1}, \dots, S_{c,|V|}$ the same number of times in the first T' weeks of assignment g. If this holds, then the sequence of colors $c_1, \dots, c_{T'}$ verifies the required property and has period $T' < T$ which is a contradiction.

Let $j \in [q]$ be a worker. The number of times the worker j performs the task t^1_0 in the first T' weeks of g is $|\{r \in [T'] \colon j \text{ performs } S_{c_r,1}\}|$. For a $c \in C$ and a $v \in [|V|-1]$, the number of times the worker j performs the task $t_{c,v}$ in the first T' weeks of g is:

$$\sum_{r=1}^{T'}\sum_{v' \in V}\mathbb{1}(j \text{ performs } S_{c_r,v'} \text{ and } t_{c,v} \in T_{c,v'}) = \sum_{\substack{r=1 \\ c_r \neq c}}^{T'}\mathbb{1}(j \text{ performs } S_{c_r,1}) + \sum_{\substack{r=1 \\ c_r = c}}^{T'}\mathbb{1}(j \text{ performs } S_{c_r,v+1})$$

Since the assignment g is balanced and periodic with period T', both of the tasks t^1_0 and $t_{c,v}$ are performed the same amount of times by the worker j on the first T' weeks of g, for every $c \in C$, $v \in [|V|-1]$. Thus, for every $c \in C$ and $v \in [|V|-1]$,

$$\sum_{\substack{r=1 \\ c_r = c}}^{T'}\mathbb{1}(j \text{ performs } S_{c_r,1}) = \sum_{\substack{r=1 \\ c_r = c}}^{T'}\mathbb{1}(j \text{ performs } S_{c_r,v+1}) \tag{9.1}$$

Furthermore, the number of times a schedule $S_{c,\cdot}$ corresponding to the color c is performed by a worker j_1 is the same as for a worker j_2, thus:

$$\sum_{v \in V}\sum_{\substack{r=1 \\ c_r = c}}^{T'}\mathbb{1}(j_1 \text{ performs } S_{c_r,v}) = \sum_{v \in V}\sum_{\substack{r=1 \\ c_r = c}}^{T'}\mathbb{1}(j_2 \text{ performs } S_{c_r,v}) \tag{9.2}$$

Combining the equalities (9.1) and (9.2), we have that for every color $c \in C$, each worker goes through each schedule $S_{c,1}, \dots, S_{c,|V|}$ the same number of times in the first T' weeks of assignment g, which ends the proof. □

9.3 An ILP solving the periodic relaxed balanced sequence of colors problem

The goal of this section is to propose an ILP model to solve the (PRBC) problem. Using Proposition 9.2.1, this could then be used to build instances of the (PBA) problem associated with a solution of same value as the ILP.

For the following discussions, let $D = (V, A)$ be a regular colored Eulerian digraph. Place a pebble on each vertex of D. In Corollary 8.5.2, we showed that there exists a periodic balanced sequence of colors with period upper bounded by $|A||\langle S\rangle|/|V|$, where S is the set of σ_c, c ranging over the colors. While our method was able to provide us with an upper bound, it could be that the optimal solutions of the (PRBC) problem do not reach this bound, all the more so as (PRBC) relaxes the requirement on the sequence of colors from Section 8.5.1. Denote by P the set of pebbles, by C the set of colors, and by N an upper bound of the optimal number of moves. Note that we can choose $N = |A||\langle S\rangle|/|V|$ using Corollary 8.5.2. For a color $c \in C$, we write A_c the arcs in A with color c. The following ILP model solves the (PRBC) problem. This claim is rigorously stated and proven as Lemma 9.3.1. The meaning of the constraints is delayed to the proof of the lemma. We emphasize beforehand that Constraint 9.7 is generally used as a symmetry-breaking valid inequality in integer linear programs. Here though, the proof of correctness of this ILP relies on this constraint.

$$\text{minimize} \sum_{c\in C}\sum_{k\in[N]} y_{c,k} \qquad \text{(ILP_colors)}$$

$$\text{s.t.} \sum_{k\in[N]} x_{p_1,a,k} = \sum_{k\in[N]} x_{p_2,a,k} \qquad \forall (p_1,p_2)\in P^2, a\in A \qquad (9.3)$$

$$\sum_{k\in[N]} x_{p,a_1,k} = \sum_{k\in[N]} x_{p,a_2,k} \qquad \forall c\in C, p\in P, (a_1,a_2)\in A_c^2 \qquad (9.4)$$

$$x_{p,a,k} \leqslant \sum_{c\in C} y_{c,k+1} \qquad \forall k\in[N-1], p\in P, a\in A, \text{ s.t. head}(a)\neq\text{initial vertex for } p \qquad (9.5)$$

$$x_{p,a,N} = 0 \qquad \forall p\in P, a\in A \text{ s.t. head}(a)\neq\text{initial vertex for } p \qquad (9.6)$$

$$\sum_{c\in C} y_{c,k+1} \leqslant \sum_{c\in C} y_{c,k} \qquad \forall k\in[N-1] \qquad (9.7)$$

$$\sum_{c\in C} y_{c,1} = 1 \qquad (9.8)$$

$$\sum_{c\in C} y_{c,k} \geqslant \sum_{a\in A} x_{p,a,k} \qquad \forall k\in[N], p\in P \qquad (9.9)$$

$$y_{c,1} = x_{p,a,1} \qquad \forall c \in C, p \in P, \quad a = \text{ arc of color } c \text{ s.t. } \text{tail}(a)=\text{initial vertex for } p \tag{9.10}$$

$$y_{c,k+1} + x_{p,a,k} - 1 \leqslant x_{p,a',k+1} \qquad \forall c \in C, k \in [N-1], p \in P, a \in A, \quad a' = \text{ arc of color } c \text{ s.t. } \text{tail}(a')=\text{head}(a) \tag{9.11}$$

$$x_{p,a,k} \in \{0,1\} \qquad \forall p \in P, a \in A, k \in [N] \tag{9.12}$$

$$y_{c,k} \in \{0,1\} \qquad \forall c \in C, k \in [N]\,. \tag{9.13}$$

The variables $y_{c,k}$ take the value 1 if the color c is used for the k-th move and the variables $x_{p,a,k}$ take value 1 if the pebble p goes through the arc a for its k-th move.

Lemma 9.3.1. *The integer program* (ILP_colors) *models correctly the* (PRBC) *problem.*

Proof. First, we take a feasible solution to the (PRBC) problem. We will show that this defines a solution of (ILP_colors) with value at least as good. Let $c_1, \dots c_T$ be a solution of (PRBC). Set $y_{c_k,k} = 1$ for $k \in [T]$, $y_{c,k} = 0$ otherwise. Set $x_{p,a,k} = 1$ when pebble p goes through the arc a for its k-th move, $x_{p,a,k} = 1$ otherwise. Then (x, y) is a solution of (ILP_colors), with objective value T. Indeed:

- Constraint (9.3) means that the pebbles p_1 and p_2 visit the arc a the same number of times.
- Constraint (9.4) means that the pebble p visits the arcs a_1 and a_2 of the same color the same amount of times.
- Constraint (9.5) means that the pebble p ends at its initial vertex, when moves end before N. Constraint (9.6) means the same think when the last move is the Nth one.
- Constraint (9.7) means that if the move k is not used then the next move is not used either.
- Constraint (9.8) means that a single color is chosen for the first move (exactly once since at least one move is performed).
- Constraint (9.9) means that whenever a pebble moves along an arc for its k-th move, at least one color is used.
- Constraint (9.10) means that, on the first move, if the color c is used then all pebbles move according to the color c starting from their initial vertex.
- Constraint (9.11) means that, on the kth move, if the color c is used then all pebbles move according to the color c. More precisely, they take the arc of color c starting from the head of the previous arc. To be more precise, it means that the pebble starts its next move (which is the $k+1$th according to Constraint 9.7) on the vertex it ended after the kth move.
- The objective value is T.

Second, we take a feasible solution of (ILP_colors). We will show that this defines a solution to the (PRBC) problem with value at least as good. Let now (x, y) be a solution of (ILP_colors). Let by T the largest k such that $\sum_{c \in C} y_{c,k} = 1$.

For each k, there is a unique c such that $y_{c,k} = 1$, which we denote by c_k. This is because of constraints (9.8) and (9.7). Note that this also means that the value of the solution (x, y) is T.

It remains to show that the sequence of colors $c_1, \dots, c_T$ is feasible for (PRBC) (it has length at least T by definition).

We start by showing that the pebble p moves along arc a for the k-th move if and only if $x_{p,a,k} = 1$, for every $p \in P$, $a \in A$ and $k \in [N]$. Let us proceed by induction on k. For $k = 1$, Constraint (9.10) ensures that this is true. Indeed, p moves along the arc starting from the initial vertex for p with color c_1 (such that $y_{c_1,1} = 1$). For $k \in [T-1]$: assume the property holds for the k-th move. Suppose that p moves along a' for the $k+1$-th move.

Write a the arc along which p moved for the k-th move. By induction we have $x_{p,a,k} = 1$ and by definition, a' is the arc of color c_{k+1} starting from head(a). Constraint (9.11) then ensures that $x_{p,a',k+1} \geqslant 1$. Constraint (9.9), along with the fact that c_k was uniquely defined, then ensures that $x_{p,a'',k+1} = 0$ for all $a'' \neq a'$. The induction step thus holds.

Moreover, the sequence of colors $c_1, \ldots, c_T$ makes each pebble follow a closed walk on D, directly from Constraints (9.5) and (9.6).

Finally, Constraint (9.3) ensures that two pebbles visit each arc the same number of times and Constraint (9.4) ensures that each pebble visits all arcs of the same color equally often, defining a common number of visits for color c independently of the pebble.

Therefore, the sequence of colors $c_1, \ldots, c_T$ is a solution of the (PRBC) problem. □

We have been able to identify the following family valid inequalities, which helped speed up the ILP resolution in practice.

Lemma 9.3.2. *The following constraints are valid inequalities for* (ILP_colors)*:*

$$\sum_{c \in C} (y_{c,k} - y_{c,k+1}) = 0 \quad \forall k \in [N-1] \colon k \equiv 0 \pmod{|P|} \tag{9.14}$$

Proof. Denoting by N_c the number of times a pebble moves along an arc of color c and by T the value of an optimal solution, we have that $\sum_{c \in C} N_c = T/|P|$. Thus, the length of a solution to the (PRBC) problem is always a multiple of $|P|$. Moreover, we have $\sum_{c \in C} (y_{c,T} - y_{c,T+1}) = 1$ and $\sum_{c \in C} (y_{c,k} - y_{c,k+1}) = 0$ for $k \neq T$. The inequalities are therefore valid. □

Using Lemma 9.3.1, we were able to prove (by solving (ILP_colors) via Gurobi) that a periodic weakly balanced sequence of colors with minimum period for the graph of Figure 8.4 is 24. For this same graph, the upper bound $|A||\langle S \rangle|/|V|$ is also 24. Proposition 9.2.1 shows that an instance of (PBA) can be built so that a periodic balanced assignment with minimum period has period 24 as well.

9.4 Other particular feasible assignments

The goal of this section is to establish additional existence results for feasible assignments beyond balanced ones.

We now assume that there is a set of skills to be trained, and that each task performed by a worker contributes to developing the skills required for that task. We seek feasible assignments where each worker trains each skill at least a given minimum number of times.

As noted in the introduction of this part, cyclic rosters at SNCF are primarily implemented for two reasons: to ensure fairness among workers, and to guarantee a uniform level of proficiency across the workforce.

In this section, we relax the fairness constraint inherent to cyclic rosters and instead focus on ensuring a baseline proficiency level for all workers. This allows for feasible assignments that meet minimum proficiency requirements without necessarily being balanced.

For the following discussions, we refer to the basic version whenever feasibility is defined as in Section 7.1.1 and the extended version whenever feasibility is defined as in Section 8.1.1.

9.4.1 Problem formulation

We denote by K the set of skills. We consider now that each task i comes with skills gains $\alpha^{(i)} \in \mathbb{R}^K_{>0}$, meaning that a worker performing task i gains the corresponding amount $\alpha^{(i)}_k$ of experience for each skill k. We say that a feasible assignment *achieves the frequencies*

$\nu_k \in \mathbb{R}_{\geqslant 0}$ for $k \in K$ when each worker is trained in each skill k with a frequency of at least ν_k. Formally, a feasible assignment f achieves the frequencies ν_k for $k \in K$ if for every $k \in K$ and $j \in [q]$ we have

$$\lim_{t\to+\infty} \frac{1}{t} \sum_{r=1}^{t} \sum_{i=1}^{n} \alpha_k^{(i)} \mathbb{1}(f(i,r) = j) \geqslant \nu_k \,. \tag{9.15}$$

Note that, when summing over $j \in [q]$ in (9.15), the two sides are equal if and only if $\nu_k = \frac{1}{q}\sum_{i=1}^{n} \alpha_k^{(i)}$. When this is the case for every $k \in K$, we say that the assignment f is a *skill-balanced* assignment.

9.4.2 First result

The following lemma provides a first result saying that balanced feasible assignments achieve all achievable frequencies. This lemma also means that skill-balanced assignments achieve all achievable frequencies.

Lemma 9.4.1. *If $\nu_k \leqslant \frac{1}{q}\sum_{i=1}^{n} \alpha_k^{(i)}$ for all $k \in K$, then every balanced feasible assignment achieves the frequencies ν_k for $k \in K$. Otherwise, every feasible assignment fails to satisfy equation (9.15) for some $k \in K$.*

Proof. Let f be a balanced feasible assignment and suppose that $\nu_k \leqslant \frac{1}{q}\sum_{i=1}^{n} \alpha_k^{(i)}$ for all $k \in K$. Using (7.2), f then obviously achieves the frequencies $(\nu_k)_{k\in K}$.

Suppose now that there exists a $k \in K$ such that $\nu_k > \frac{1}{q}\sum_{i=1}^{n} \alpha_k^{(i)}$. Let k be such a skill and f be a feasible assignment. We have:

$$\sum_{j=1}^{q} \lim_{t\to+\infty} \frac{1}{t} \sum_{r=1}^{t} \sum_{i=1}^{n} \alpha_k^{(i)} \mathbb{1}(f(i,r) = j) = \sum_{i=1}^{n} \alpha_k^{(i)} < q\nu_k.$$

Therefore, there exists $j \in [q]$ such that $\lim_{t\to+\infty} \frac{1}{t} \sum_{r=1}^{t} \sum_{i=1}^{n} \alpha_k^{(i)} \mathbb{1}(f(i,r) = j) < \nu_k$, meaning that f does not satisfy equation (9.15) for this k. □

However, even for the basic version, there are examples of ν_k where there exists a feasible assignment achieving frequencies ν_k for $k \in K$ (or even a skill-balanced feasible assignment), but there is no balanced feasible assignment. Figure 7.1 provides such an example, where there is a single skill and $\alpha_1^{(1)} = \alpha_1^{(2)} = 1$ and $\nu_1 = 1$: there is no balanced assignment but the feasible assignment provided is skill-balanced.

We can thus write the following open question:

Question 4. Is it true that, whenever there exists a skill-balanced feasible assignment, there exists a *periodic* one? Is it true that, whenever there exists a feasible assignment achieving frequencies ν_k for $k \in K$, there exists a *periodic* feasible assignment achieving the same frequencies ν_k for all $k \in K$?

9.4.3 A special case

In a special case, we can however recover a converse of Lemma 9.4.1 as a consequence of the results of Chapters 7 and 8.

Suppose that every task is considered to be a distinct skill, namely $K = [n]$ and, for every $i \in [n]$, the skills gains $\alpha^{(i)}$ has all entries equal to zero, except the ith entry, which is equal to one.

In this case, equation (9.15) rewrites as follows, for every $i \in [n]$ and $j \in [q]$:

$$\lim_{t\to+\infty} \frac{1}{t} \sum_{r=1}^{t} \mathbb{1}(f(i,r) = j) \geqslant \nu_i \,. \tag{9.16}$$

Note that, by multiplying each side by a positive real number, we can deal with the case where tasks have different weights similarly as the case $\alpha_i^{(i)} = 1$ presented here (by replacing the frequencies ν_i with $\nu_i/\alpha_i^{(i)}$).

Also, for this special case, skill-balanced feasible assignments are exactly the same as balanced feasible assignments (see (7.2)).

For this special case, we have the following corollary, which follows from Theorems 7.1.1 and 8.1.2. It is not clear whether there is a more straightforward proof of this corollary. This result means in particular, in combination with Lemma 9.4.1, that the existence of a feasible assignment achieving some non-negative frequencies is equivalent to the existence of a balanced feasible assignment. For the basic version, this also means that, along with Theorem 7.1.2, finding a q-periodic feasible assignment achieving some non-negative frequencies is polynomial. This also means that the periodicity result of Theorem 8.1.1 holds for feasible assignment achieving some non-negative frequencies.

Corollary 9.4.2. *In the basic version, if there exists a feasible assignment achieving some non-negative frequencies, then there exists a balanced feasible assignment. In the extended version, if $|U(\mathcal{I})| = q$ and there exists a feasible assignment achieving non-negative frequencies, then there exists a balanced feasible assignment.*

Proof. Let f be a feasible assignment achieving non-negative frequencies. Since $\nu_i > 0$ and $\alpha_i > 0$ for all $i \in [n]$, then there exists a worker performing each task at least once in f. In the basic version, using Theorem 7.1.1, there exists a balanced feasible assignment. In the extended version, assuming that $|U(\mathcal{I})| = q$ and using Theorem 8.1.2, there exists a balanced feasible assignment as well. □

9.4.4 Skill-balanced sequence of colors

The goal of this section is to define the notion of a skill-balanced sequence of colors, in a similar way as in Section 8.3 in order to link this with the existence of (periodic) skill-balanced feasible assignments.

Problem of pebbles

Let $D = (V, A)$ be a regular colored digraph, not necessarily Eulerian. In this section, D comes with additional information encoded on the arcs: each arc a comes with a vector $\alpha^{(a)}$. Place a pebble on each vertex. A sequence of colors induces a sequence of moves for the pebbles exactly as in Section 8.3.

We call a sequence of colors $c_1, c_2, \ldots$ *skill-balanced* if each pebble p follows a walk $a_1(p), a_2(p), \ldots$ on D such that:

$$\lim_{t\to+\infty} \frac{1}{t} \sum_{r=1}^{t} \alpha^{(a_r(p))}$$

does not depend on p. Note that this value, common to all pebbles, is equal to

$$\lim_{t\to+\infty} \frac{1}{t} \sum_{r=1}^{t} \frac{1}{|A_{c_r}|} \sum_{a\in A_{c_r}} \alpha^{(a)}$$

for a sequence of colors $c_1, c_2, \ldots$.

Figure 9.1 exemplifies a regular colored Eulerian digraph such that there exists a skill-balanced sequence of colors but no periodic one. This contrasts with Proposition 8.3.1.

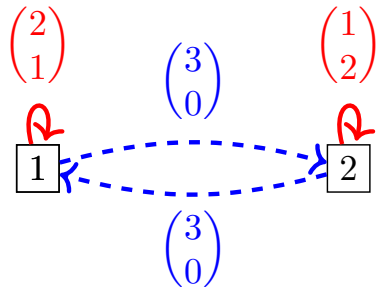


Figure 9.1: An example of a graph with vectors on the arcs such that there exists a skill-balanced sequence of colors (e.g., ●●●

$$= \sum_{r=1}^{t} \sum_{a \in A^{\mathcal{F}}} \sum_{i \in U(\mathcal{I})} \alpha^{(a)} \mathbb{1}\left(\begin{array}{l} a \text{ has head } i \text{ and label } f_r \text{ and pebble} \\ j \text{ is on vertex } a \text{ after its } r\text{th move} \end{array} \right)$$

$$= \sum_{a \in A^{\mathcal{F}}} \alpha^{(a)} \mathbb{1}(\text{pebble } j \text{ moves along arc } a \text{ for its } r\text{th move}) .$$

Since the sequence of colors is skill-balanced, and by definition of the cumulative skills gains $\alpha^{(a)}$, we have, for all $j \in [q]$,

$$\lim_{t \to +\infty} \frac{1}{t} \sum_{r=1}^{t} \sum_{a \in A^{\mathcal{F}}} \alpha^{(a)} \mathbb{1}(\text{pebble } j \text{ moves along arc } a \text{ for its } r\text{th move}) = \lim_{t \to +\infty} \frac{1}{t} \sum_{r=1}^{t} \frac{1}{q} \sum_{a' \in A_{f_r}} \alpha^{(a')}$$

$$= \frac{1}{q} \sum_{i' \in [n]} \alpha^{(i')} .$$

Therefore, for every $j \in [q]$ and t, we have

$$\lim_{t \to +\infty} \frac{1}{t} \sum_{r=1}^{t} \sum_{i=1}^{n} \alpha^{(i)} \mathbb{1}(g(i,r) = j) = \lim_{t \to +\infty} \frac{1}{t} \sum_{r=1}^{t} \sum_{i=1}^{n} \alpha^{(i)} \mathbb{1}(g(i,r+1) = j) = \frac{1}{q} \sum_{i' \in [n]} \alpha^{(i')} ,$$

as desired. □

Even though Figure 9.1 provides an example of a graph such that there exists a skill-balanced sequence of colors but no periodic one, no instance $\mathcal{I}$ and set $\mathcal{F}$ of feasible assignments makes this graph isomorphic to $D^{\mathcal{I},\mathcal{F}}$. It is then still very possible that the first part of Question 4 has a positive answer.

Chapter 10

Conclusion

***

This thesis, carried out as an industrial collaboration with SNCF and Hexafret, has aimed to model optimization problems arising in transportation planning, propose algorithms for large-scale instances and contribute to the field of periodic tasks scheduling. Three guiding threads have structured this work. The first consisted in demonstrating that integrating otherwise sequential decisions yields substantial improvements in the quality and robustness of operational plans, as illustrated by the integrated crew management of freight train drivers at SNCF. The second consisted in proposing a reformulation of the classical arc-based formulation for the passenger-oriented line planning problem, so as to overcome the memory limitations on large networks. The third consisted in establishing the theoretical foundations of fairness and periodicity in the assignment of periodic tasks. For these purposes, we have combined integer programming, column generation, stochastic modeling, branch-and-price-and-cut, and combinatorial optimization.

On real-world instances, the integrated approach significantly reduces costs, while its stochastic variant substantially lowers fragility as well as nominal costs, thereby outperforming the current sequential method with respect to both criteria. This work has been published as:

"Integrating crew scheduling and crew rostering for rail freight with train delays," H. Gachet, F. Meunier, and J. Pouzet, *9th International Symposium on Combinatorial Optimization (ISCO 2026)*.

The second part addressed a passenger-oriented line planning problem, in which one selects lines and their operating integer frequencies so as to minimize total travel time, subject to demand, capacity, budget, and maximum-frequency constraints. We proposed a Dantzig–Wolfe reformulation that replaces arc-by-arc flows with passenger paths. A branch-and-price-and-cut framework reinforced by dedicated branching rules, valid inequalities, and acceleration heuristics was developed for this reformulation, where pricing subproblems are shortest-path problems solved by Dijkstra's algorithm. This reformulation circumvents the memory bottleneck of arc-based formulations on large instances and provides lower bounds—and, in some cases, feasible solutions—where the classical model reaches its limits.

The third part, of a more theoretical nature, investigated balanced assignments of periodic tasks, that is, assignments in which each worker performs each task with the same long-run frequency. We characterized their existence in a basic version of the problem—a balanced assignment exists if and only if there exists a feasible assignment with a worker performing

each task at least once—and exhibited a polynomial time algorithm to verify existence and build a q-periodic balanced feasible assignment (with q being the number of workers). More generally, in the variant involving admissible weekly schedules, we proved that periodicity of a balanced feasible assignment is always achievable, with period bounded by $2(q-1)q!$, and by $q(q!-1)$ when q is a prime number. These results relate to the existence of a balanced sequence of colors orchestrating pebbles' movements on a regular Eulerian arc-colored digraph, which can also be chosen periodic as well, with improved bounds for Cayley graphs. First results for the basic version have been published as

"Balanced Assignments of Periodic Tasks," H. Gachet, F. Meunier, *24th Symposium on Algorithmic Approaches for Transportation Modelling, Optimization, and Systems (ATMOS 2024).*

and results for the extended version were made public in a further arXiv preprint which is in the revision process with a journal:

"Balanced Assignments of Periodic Tasks," H. Gachet, F. Meunier, *Annals of Operations Research, October 2026.*

Beyond these contributions, each of the three parts leaves open natural questions which could be the subject of future research. For the integrated freight crew management problem, the stochastic approach approximates general or continuous delay distributions by finite-support ones, but no absolute bound on the level of refinement required to reach feasibility has yet been established, and remains an open theoretical question. On the methodological side, implementing bidirectional search in the labeling algorithm for the large-scale, nonlinear resource-constrained shortest path pricing subproblem could lead to the convergence of the column generation for the stochastic crew management problem. Alternatively, building on the alternative model of Chapter 9.2, a master problem explicitly constructing cyclic rosters from trains could be imagined, to accommodate richer roster constraints. For line planning, the central difficulty remains the resolution of large-scale instances to optimality: the master problem is highly degenerate, branching on the flow variables is delicate, and there are as many pricing subproblems as origin-destination pairs, which together limit the branch-and-price-and-cut approach; a promising direction is therefore to investigate alternative network representations that overcome these obstacles, as recently explored by van Lieshout and van Rossum [88]. Finally, for balanced assignments of periodic tasks, the period of a periodic balanced assignment is bounded by $2(q-1)q!$ in the extended version (or $q(q!-1)$ when q is prime), yet whether this bound is tight for any instance is unknown, leaving open the more general question of whether periodic balanced assignments have a period bounded by a polynomial in q.

Bibliography

***

[1] Z. A. Abdalkareem, A. Amir, M. A. Al-Betar, P. Ekhan, and A. I. Hammouri. Healthcare scheduling in optimization context: A review. *Health and Technology*, 11(3):445–469, 2021.

[2] N. Absi, C. Archetti, S. Dauzère-Pérès, D. Feillet, and M. G. Speranza. Comparing sequential and integrated approaches for the production routing problem. *European Journal of Operational Research*, 269(2):633–646, 2018.

[3] M. Albers. *Freight Railway Crew Scheduling: Models, Methods, and Applications*. Logos Verlag Berlin GmbH, 2009.

[4] P. Alefragis, P. Sanders, T. Takkula, and D. Wedelin. Parallel Integer Optimization for Crew Scheduling. *Annals of Operations Research*, 99(1):141–166, 2000.

[5] D. Antunes, V. Vaze, and A. P. Antunes. A Robust Pairing Model for Airline Crew Scheduling. *Transportation Science*, 53(6):1751–1771, 2019.

[6] J. P. Arabeyre, J. Fearnley, F. C. Steiger, and W. Teather. The airline crew scheduling problem: A survey. *Transportation Science*, 3(2):140–163, 1969.

[7] A. Atamtürk. Cover and Pack Inequalities for (Mixed) Integer Programming. *Annals of Operations Research*, 139(1):21–38, 2005.

[8] C. Barnhart, E. L. Johnson, G. L. Nemhauser, M. W. P. Savelsbergh, and P. H. Vance. Branch-and-price: Column generation for solving huge integer programs. *Operations Research*, 46(3):316–329, 1998.

[9] A. Ben-Tal, A. Nemirovski, and L. El Ghaoui. *Robust Optimization*. Princeton Series in Applied Mathematics. Princeton University Press, 2009.

[10] J. R. Birge and F. Louveaux. *Introduction to Stochastic Programming*. Springer Series in Operations Research and Financial Engineering. Springer New York, 1997.

[11] J. M. P. Booler. A method for solving crew scheduling problems. *Journal of the Operational Research Society*, 26(1):55–62, 1975.

[12] R. Borndörfer and M. Karbstein. A Direct Connection Approach to Integrated Line Planning and Passenger Routing. In D. Delling and L. Liberti, editors, *12th Workshop on Algorithmic Approaches for Transportation Modelling, Optimization, and Systems*, volume 25 of *Open Access Series in Informatics (OASIcs)*, pages 47–57, Dagstuhl, Germany, 2012. Schloss Dagstuhl – Leibniz-Zentrum für Informatik.

[13] R. Borndörfer, C. Schulz, S. Seidl, and S. Weider. Integration of duty scheduling and rostering to increase driver satisfaction. *Public Transport*, 9(1):177–191, 2017.

[14] E. Bortoletto, R. N. van Lieshout, B. Masing, and N. Lindner. Periodic Event Scheduling with Flexible Infrastructure Assignment. In P. C. Bouman and S. C. Kontogiannis, editors, *24th Symposium on Algorithmic Approaches for Transportation Modelling, Optimization, and Systems (ATMOS 2024)*, volume 123 of *Open Access Series in Informatics (OASIcs)*, pages 4:1–4:18, Dagstuhl, Germany, 2024. Schloss Dagstuhl – Leibniz-Zentrum für Informatik.

[15] K. Boubaker, G. Desaulniers, and I. Elhallaoui. Bidline scheduling with equity by heuristic dynamic constraint aggregation. *Transportation Research Part B: Methodological*, 44(1):50–61, 2010.

[16] M. Bouchard, S. D'Amours, M. Rönnqvist, R. Azouzi, and E. Gunn. Integrated optimization of strategic and tactical planning decisions in forestry. *European Journal of Operational Research*, 259(3):1132–1143, 2017.

[17] T. Breugem, T. Dollevoet, and D. Huisman. Analyzing a Family of Formulations for Cyclic Crew Rostering. In D. Huisman and C. D. Zaroliagis, editors, *20th Symposium on Algorithmic Approaches for Transportation Modelling, Optimization, and Systems (ATMOS 2020)*, volume 85 of *Open Access Series in Informatics (OASIcs)*, pages 7:1–7:16, Dagstuhl, Germany, 2020. Schloss Dagstuhl – Leibniz-Zentrum für Informatik.

[18] T. Breugem, T. Dollevoet, and D. Huisman. Is Equality Always Desirable? Analyzing the Trade-Off Between Fairness and Attractiveness in Crew Rostering. *Management Science*, 68(4):2619–2641, 2022.

[19] E. K. Burke, P. De Causmaecker, G. V. Berghe, and H. Van Landeghem. The State of the Art of Nurse Rostering. *Journal of Scheduling*, 7(6):441–499, 2004.

[20] M. Bussieck. *Optimal Lines in Public Rail Transport*. PhD thesis, Technischen Universität Braunschweig, 1998.

[21] V. Cacchiani, D. Huisman, M. Kidd, L. Kroon, P. Toth, L. Veelenturf, and J. Wagenaar. An overview of recovery models and algorithms for real-time railway rescheduling. *Transportation Research Part B: Methodological*, 63:15–37, 2014.

[22] V. Cacchiani and J.-J. Salazar-González. Optimal solutions to a real-world integrated airline scheduling problem. *Transportation Science*, 51(1):250–268, 2017.

[23] A. Caprara, L. Kroon, M. Monaci, M. Peeters, and P. Toth. *Chapter 3 Passenger Railway Optimization*, volume 14 of *Handbooks in Operations Research and Management Science*. Elsevier, 2007.

[24] A. Caprara, M. Monaci, and P. Toth. A global method for crew planning in railway applications. In S. Voß and J. R. Daduna, editors, *Computer-Aided Scheduling of Public Transport*, pages 17–36. Springer Berlin Heidelberg, 2001.

[25] O. Cats, K. Gkiotsalitis, and A. Schöbel. 50 years of operations research in public transport. *EURO Journal on Transportation and Logistics*, 14:100160, 2025.

[26] A. Ceder. *Public Transit Planning and Operation: Modeling, Practice and Behavior*. CRC Press, 2016.

[27] G. B. Dantzig and P. Wolfe. Decomposition principle for linear programs. *Operations Research*, 8(1):101–111, 1960.

[28] M. Darvish and L. C. Coelho. Sequential versus integrated optimization: Production, location, inventory control, and distribution. *European Journal of Operational Research*, 268(1):203–214, 2018.

[29] D. de Werra. Equitable colorations of graphs. *ESAIM: Mathematical Modelling and Numerical Analysis - Modélisation Mathématique et Analyse Numérique*, 5(R3):3–8, 1971.

[30] J. Desrosiers, M. Lübbecke, G. Desaulniers, and J. B. Gauthier. *Branch-and-Price*. Springer Nature Switzerland, 2026.

[31] M. Deveci and N. C. Demirel. A survey of the literature on airline crew scheduling. *Engineering Applications of Artificial Intelligence*, 74:54–69, 2018.

[32] V. Dück, L. Ionescu, N. Kliewer, and L. Suhl. Increasing stability of crew and aircraft schedules. *Transportation Research Part C: Emerging Technologies*, 20(1):47–61, 2012.

[33] A. Ernst, H. Jiang, M. Krishnamoorthy, H. Nott, and D. Sier. An Integrated Optimization Model for Train Crew Management. *Annals of Operations Research*, 108(1):211–224, 2001.

[34] A. Ernst, H. Jiang, M. Krishnamoorthy, H. Nott, and D. Sier. Rail crew scheduling and rostering optimization algorithms. In S. Voß and J. R. Daduna, editors, *Computer-Aided Scheduling of Public Transport*, pages 53–71. Springer Berlin Heidelberg, 2001.

[35] A. Ernst, H. Jiang, M. Krishnamoorthy, B. Owens, and D. Sier. An Annotated Bibliography of Personnel Scheduling and Rostering. *Annals of Operations Research*, 127(1):21–144, 2004.

[36] A. Ernst, H. Jiang, M. Krishnamoorthy, and D. Sier. Staff scheduling and rostering: A review of applications, methods and models. *European Journal of Operational Research*, 153(1):3–27, 2004.

[37] T. Feng, R. M. Lusby, Y. Zhang, S. Tao, B. Zhang, and Q. Peng. A branch-and-price algorithm for integrating urban rail crew scheduling and rostering problems. *Transportation Research Part B: Methodological*, 183:102941, 2024.

[38] M. Friedrich, M. Hartl, A. Schiewe, and A. Schöbel. System headways in line planning. In *Proceedings of CASPT 2018 (Conference on Advanced Systems in Public Transport and TransitData)*, 2018.

[39] H. Gachet and F. Meunier. Balanced Assignments of Periodic Tasks. In P. C. Bouman and S. C. Kontogiannis, editors, *24th Symposium on Algorithmic Approaches for Transportation Modelling, Optimization, and Systems (ATMOS 2024)*, volume 123 of *Open Access Series in Informatics (OASIcs)*, pages 5:1–5:12, Dagstuhl, Germany, 2024. Schloss Dagstuhl – Leibniz-Zentrum für Informatik.

[40] H. Gachet and F. Meunier. Balanced assignments of periodic tasks, 2025. [arXiv; v2].

[41] H. Gachet and F. Meunier. Balanced assignments of periodic tasks. *Annals of Operations Research*, 2026.

[42] H. Gachet, F. Meunier, and J. Pouzet. Integrating Crew Scheduling and Crew Rostering for Rail Freight with Train Delays. In A. R. Mahjoub, H. Kutucu, S. Küçükyavuz, and V. Kaibel, editors, *Combinatorial Optimization, ISCO 2026*, pages 570–587. Springer Nature Switzerland, 2027.

[43] M. Goerigk and M. Schmidt. Line planning with user-optimal route choice. *European Journal of Operational Research*, 259(2):424–436, 2017.

[44] B. Gopalakrishnan and Ellis. L. Johnson. Airline Crew Scheduling: State-of-the-Art. *Annals of Operations Research*, 140(1):305–337, 2005.

[45] I. Hansen and J. Pachl, editors. *Railway timetabling and operations: Analysis, modelling, optimisation, simulation, performance, evaluation.* Eurail press, 2014.

[46] P. E. Hart, N. J. Nilsson, and B. Raphael. A formal basis for the heuristic determination of minimum cost paths. *IEEE Transactions on Systems Science and Cybernetics*, 4(2):100–107, 1968.

[47] J. He, M. Ge, S. D. Žarković, Z. Li, and P. Hilber. A novel integrated optimization method of micrositing and cable routing for offshore wind farms. *Energy*, 306:132443, 2024.

[48] J. Heil, K. Hoffmann, and U. Buscher. Railway crew scheduling: Models, methods and applications. *European Journal of Operational Research*, 283(2):405–425, 2020.

[49] I. Himmich, I. El Hallaoui, and F. Soumis. A multiphase dynamic programming algorithm for the shortest path problem with resource constraints. *European Journal of Operational Research*, 315(2):470–483, 2024.

[50] S. Jütte, D. Müller, and U. W. Thonemann. Optimizing railway crew schedules with fairness preferences. *Journal of Scheduling*, 20(1):43–55, 2017.

[51] N. Karmarkar. A new polynomial-time algorithm for linear programming. *Combinatorica*, 4(4):373–395, 1984.

[52] A. Kasirzadeh, M. Saddoune, and F. Soumis. Airline crew scheduling: Models, algorithms, and data sets. *EURO Journal on Transportation and Logistics*, 6(2):111–137, 2017.

[53] L. Khachiyan. Polynomial algorithms in linear programming. *USSR Computational Mathematics and Mathematical Physics*, 20(1):53–72, 1980. English version.

[54] E. Khmeleva, A. A. Hopgood, L. Tipi, and M. Shahidan. Fuzzy-Logic Controlled Genetic Algorithm for the Rail-Freight Crew-Scheduling Problem. *KI - Künstliche Intelligenz*, 32(1):61–75, 2018.

[55] N. Kohl and S. E. Karisch. Airline Crew Rostering: Problem Types, Modeling, and Optimization. *Annals of Operations Research*, 127(1):223–257, 2004.

[56] E. König. A review on railway delay management. *Public Transport*, 12(2):335–361, 2020.

[57] J. Korst, E. Aarts, J. K. Lenstra, and J. Wessels. Periodic assignment and graph colouring. *Discrete Applied Mathematics*, 51(3):291–305, 1994.

[58] J. Krarup and D. de Werra. Chromatic optimisation: Limitations, objectives, uses, references. *European Journal of Operational Research*, 11(1):1–19, 1982.

[59] D.-Y. Lin and M.-R. Tsai. Integrated Crew Scheduling and Roster Problem for Trainmasters of Passenger Railway Transportation. *IEEE Access*, 7:27362–27375, 2019.

[60] Y. Lu, R. van Lieshout, L. Martin, and L. Yang. Line planning under crowding: A cut-and-column generation approach, 2025. [arXiv].

[61] S. Martello, W. Pulleyblank, P. Toth, and D. de Werra. Balanced optimization problems. *Operations Research Letters*, 3(5):275–278, 1984.

[62] L. Mertens, L.-A. Wolbeck, D. Rößler, L. Xie, and N. Kliewer. An overview of optimization approaches for scheduling and rostering resources in public transportation, 2023. [arXiv].

[63] M. Mesquita, M. Moz, A. Paias, and M. Pato. A decomposition approach for the integrated vehicle-crew-roster problem with days-off pattern. *European Journal of Operational Research*, 229(2):318–331, 2013.

[64] F. Meunier, A. Parmentier, and N. Tellache. Aircraft routing: periodicity and complexity, 2025. [arXiv].

[65] W. Meyer. Equitable coloring. *The American Mathematical Monthly*, 80(8):920–922, 1973.

[66] A. Mingozzi, M. A. Boschetti, S. Ricciardelli, and L. Bianco. A set partitioning approach to the crew scheduling problem. *Operations Research*, 47(6):873–888, 1999.

[67] G. L. Nemhauser and L. A. Wolsey. *Integer and combinatorial optimization*, volume 18. John Wiley & Sons, Ltd, 1988.

[68] J. R. Norris. *Markov chains*. Number 2 in Cambridge Series in Statistical and Probabilistic Mathematics. Cambridge University Press, 1998.

[69] J. Orlin. Minimizing the number of vehicles to meet a fixed periodic schedule: An application of periodic posets. *Operations Research*, 30:760–776, 1982.

[70] A. Parmentier. Algorithms for non-linear and stochastic resource constrained shortest path. *Mathematical Methods of Operations Research*, 89(2):281–317, 2019.

[71] A. Parmentier and F. Meunier. Stochastic Shortest Paths and Risk Measures, 2014. [arXiv].

[72] A. Parmentier and F. Meunier. Aircraft routing and crew pairing: Updated algorithms at Air France. *Omega*, 93:102073, 2020.

[73] M. Pfetsch. Lecture notes: Multicommodity flows and column generation, 2006.

[74] R. T. Rockafellar and S. Uryasev. Optimization of conditional value-at-risk. *Journal of Risk*, 2(3):21–41, 2000.

[75] P. Schiewe. *Integrated Optimization in Public Transport Planning*, volume 160 of *Springer Optimization and Its Applications*. Springer International Publishing, 2020.

[76] P. Schiewe and A. Schöbel. LinTim, 2007. [Online; accessed 20-July-2026].

[77] P. Schiewe and A. Schöbel. Integrated optimization of sequential processes: General analysis and application to public transport. *EURO Journal on Transportation and Logistics*, 11:100073, 2022.

[78] M. Schmidt and A. Schöbel. Planning and optimizing transit lines, 2024. [arXiv].

[79] A. Schöbel. An eigenmodel for iterative line planning, timetabling and vehicle scheduling in public transportation. *Transportation Research Part C: Emerging Technologies*, 74:348–365, 2017.

[80] A. Schöbel. Line planning in public transportation: Models and methods. *OR Spectrum*, 34(3):491–510, 2012.

[81] A. Schöbel and S. Scholl. Line planning with minimal transfers. Technical report, Georg-August Universität Göttingen, 2004.

[82] B. M. Smith and A. Wren. A bus crew scheduling system using a set covering formulation. *Transportation Research Part A: General*, 22(2):97–108, 1988.

[83] N. Souai and J. Teghem. Genetic algorithm based approach for the integrated airline crew-pairing and rostering problem. *European Journal of Operational Research*, 199(3):674–683, 2009.

[84] T. Tao. Cayley graphs and the geometry of groups, 2010. Website [Online; accessed 24-June-2026].

[85] N. E. Tellache, F. Meunier, and A. Parmentier. Linear lexicographic optimization and preferential bidding system. *Transportation Science*, 58(3):597–613, 2024.

[86] J. Van Den Bergh, J. Beliën, P. De Bruecker, E. Demeulemeester, and L. De Boeck. Personnel scheduling: A literature review. *European Journal of Operational Research*, 226(3):367–385, 2013.

[87] R. van Lieshout and B. van Rossum. The fair periodic assignment problem, 2025. [arXiv; v1].

[88] R. van Lieshout and B. van Rossum. Line planning at scale: Models, methods, and insights, 2026. [arXiv].

[89] R. N. van Lieshout and B. T. C. van Rossum. The Fair Periodic Assignment Problem. In J. Sauer and M. Schmidt, editors, *25th Symposium on Algorithmic Approaches for Transportation Modelling, Optimization, and Systems (ATMOS 2025)*, volume 137 of *Open Access Series in Informatics (OASIcs)*, pages 1:1–1:16, Dagstuhl, Germany, 2025. Schloss Dagstuhl – Leibniz-Zentrum für Informatik.

[90] B. van Rossum. A fast exact pricing algorithm for the railway crew scheduling problem. *Operations Research Letters*, 50(6):707–711, 2022.

[91] Y. Wang, X. He, T. Breugem, and D. Huisman. A decomposition approach to solve the individual railway crew Re-planning problem. *Journal of Rail Transport Planning & Management*, 32:100487, 2024.

[92] Wikipedia. Normal number, 2024. [Online; accessed 1-July-2024].

[93] Wikipedia. Cauchy's theorem (group theory), 2026. [Online; accessed 1-August-2026].

[94] Wikipedia. Cayley graph, 2026. [Online; accessed 15-April-2026].

[95] Wikipedia. Group action - Orbit stabilizer theorem, 2026. [Online; accessed 15-April-2026].

[96] L. Xie and L. Suhl. Cyclic and non-cyclic crew rostering problems in public bus transit. *OR Spectrum*, 37:99–136, 2015.

[97] J. Zhang, A. Zhao, and C. Tang. Integrated optimization of mixed bus fleet replacement and scheduling with emission and budget constraints. *Computers & Industrial Engineering*, 215:111908, 2026.

[98] J. Zhou, X. Xu, J. Long, and J. Ding. Integrated optimization approach to metro crew scheduling and rostering. *Transportation Research Part C: Emerging Technologies*, 123:102975, 2021.

[99] J. Zhou, X. Xu, J. Long, and J. Ding. Metro crew planning with day-off pattern, duty type, and rostering scheme considerations. *Transportation Research Part C: Emerging Technologies*, 143:103832, 2022.

Appendix A

Column generation

***

A.1 Setting

We would like to solve a linear program (called the *master problem* (MP)), with a large set of variables, indexed over the set $\mathcal{I}$. Such a program typically writes under the standard form as:

$$\begin{aligned} \text{minimize} \quad & \sum_{i \in \mathcal{I}} c_i x_i & & \\ \text{s.t.} \quad & \sum_{i \in \mathcal{I}} A_{i,j} x_i = b_j & \forall j \in [m] & \\ & x_i \geqslant 0 & \forall i \in \mathcal{I}\,. & \end{aligned} \tag{MP}$$

where $c_i \in \mathbb{R}$ for all $i \in \mathcal{I}$, $A_{i,j} \in \mathbb{R}$ for all $i \in \mathcal{I}$ and $j \in [m]$, and $b_j \in \mathbb{R}$ for all $j \in [m]$.

A.2 Algorithm

Due to the generally prohibitive size of the set of variables, the master problem is never solved directly as is with a solver. Instead, we iteratively solve the *restricted master problem* (RMP($\mathcal{I}'$)), with non-decreasing subsets of indices $\mathcal{I}' \subseteq \mathcal{I}$:

$$\begin{aligned} \text{minimize} \quad & \sum_{i \in \mathcal{I}'} c_i x_i & & \\ \text{s.t.} \quad & \sum_{i \in \mathcal{I}'} A_{i,j} x_i = b_j & \forall j \in [m] & \\ & x_i \geqslant 0 & \forall i \in \mathcal{I}'\,. & \end{aligned} \tag{RMP($\mathcal{I}'$)}$$

Assume for now that the master problem (MP) is feasible and let $\mathcal{I}' \subseteq \mathcal{I}$ be such that (RMP($\mathcal{I}'$)) is feasible as well.

The column generation algorithm consists then in two phases—the initialization phase (lines 4–9) and the optimization phase (lines 10–15). The initialization phase, also called "Farkas pricing," occurs only if the linear relaxation of (RMP($\mathcal{I}'$)) is infeasible. This is

Algorithm 2 Column generation algorithm for solving (MP)

initialize $\mathcal{I}'$
loop
  solve (RMP($\mathcal{I}'$)); ▷ done with an LP solver;
  if (RMP($\mathcal{I}'$)) is not feasible **then**
    compute a Farkas ray $\boldsymbol{\pi}$ of (RMP($\mathcal{I}'$)); ▷ done with an LP solver;
    if v(FP($\boldsymbol{\pi}$)) < 0 **then** ▷ Farkas pricing;
      add such an index $i \in \mathcal{I}$ to $\mathcal{I}'$;
    else
      return (MP) is infeasible;
  else
    compute the optimal dual values $\boldsymbol{\pi}$ of (RMP($\mathcal{I}'$)); ▷ done with an LP solver;
    if v(SP($\boldsymbol{\pi}$)) < 0 **then** ▷ pricing subproblem;
      add such an index $i \in \mathcal{I}$ to $\mathcal{I}'$;
    else
      return x extended to an optimal solution of (MP);

discussed in more details in Section A.2.2. The optimization phase, also called "pricing subproblem," is discussed in Section A.2.1.

We emphasize that column generation itself is a standard algorithmic scheme; the algorithmic difficulty is mostly shifted to the pricing subproblem, the resolution of which is problem-dependent.

A.2.1 Pricing subproblem

To prove that an optimal solution of (RMP($\mathcal{I}'$)) is optimal for (MP), we must be able to verify if, for any of the indices $i \in \mathcal{I}$, the variable x_i has negative reduced cost $c_i - \sum_{j=1}^{m} \pi_j A_{i,j}$, where π_j is the optimal dual value associated to the jth constraint, obtained from solving (RMP($\mathcal{I}'$)) to optimality. Since the set $\mathcal{I}$ is assumed to be large, it is again prohibitive to perform an exhaustive search on all indices of $\mathcal{I}$ to find one associated to a variable with negative reduced cost. Instead, and since $\mathcal{I}$ is in practice a set defined by some structural constraints (examples are shown in Chapters 4, 5, and 6), we refer to the following surrogate optimization problem, called the "pricing subproblem":

$$\begin{array}{rl} \text{minimize} & c_i - \sum_{j=1}^{m} \pi_j A_{i,j} \\ \text{s.t.} & i \in \mathcal{I}\,. \end{array} \qquad \text{(SP}(\boldsymbol{\pi})\text{)}$$

Note that the pricing subproblem is defined here in a very general setting. As hinted above, this problem can often be written as a linear program or an integer linear program.

Then, denoting by $v((\text{SP}(\boldsymbol{\pi})))$ the optimal value of (SP($\boldsymbol{\pi}$)), we have either:

- $v((\text{SP}(\boldsymbol{\pi}))) < 0$, obtained for $i \in \mathcal{I}$. Then i is added to $\mathcal{I}'$.
- $v((\text{SP}(\boldsymbol{\pi}))) \geqslant 0$. Then the optimal solution of (RMP($\mathcal{I}'$)) is optimal for (MP).

Note that as long as a heuristic provides us with an index $i \in \mathcal{I}$ such that $c_i - \sum_{j=1}^{m} \pi_j A_{i,j} < 0$, this index can be added to $\mathcal{I}'$. Actually, it is usually a good practice in column generation to have a simple heuristic to find such indices fast, combined with an exact pricing subproblem resolution in the case where the heuristic fails to find such an index. It is also often advised to add multiple indices at once in a single iteration.

A.2.2 Farkas pricing

In some applications, a straightforward greedy heuristic can be used to initialize the set $\mathcal{I}'$ so that the first iteration deals with a feasible (RMP($\mathcal{I}'$)) (see Chapter 4). However, in some

cases, such a greedy initialization is not possible (see Chapter 5 and 6). It is in particular the case when column generation is used in a Branch-and-Price resolution (more on this in Chapter 6). In this section, we present a systematic method to initialize the set $\mathcal{I}'$ or to prove the infeasibility of (MP). This method relies on Farkas' lemma:

Lemma A.2.1 (Farkas' lemma)**.** *Let $\boldsymbol{A} \in \mathbb{R}^{m\times n}$ and $\boldsymbol{b} \in \mathbb{R}^m$. Then exactly one of the two following assertions is true:*

1. *There exists $\boldsymbol{x} \in \mathbb{R}^n$ such that $\boldsymbol{A}\boldsymbol{x} = \boldsymbol{b}$ and $\boldsymbol{x} \geqslant \boldsymbol{0}$.*
2. *There exists $\boldsymbol{\pi} \in \mathbb{R}^m$ such that $\boldsymbol{\pi}^\top \boldsymbol{A} \leqslant \boldsymbol{0}$ and $\boldsymbol{\pi} \cdot \boldsymbol{b} > 0$.*

In other words, either the problem (RMP($\mathcal{I}'$)) is feasible (this is condition 1) or there exists a ray in the dual $\boldsymbol{\pi}$ such that $\sum_{j=1}^m \pi_j A_{i,j} \leqslant 0$ for all $i \in \mathcal{I}'$ and $\sum_{j=1}^m \pi_j b_j > 0$. In the latter case, we are either able to find an index $i \in \mathcal{I}$ such that $\sum_{j=1}^m \pi_j A_{i,j} > 0$ (which is then added to the set $\mathcal{I}'$) or, for all $i \in \mathcal{I}$, we have $\sum_{j=1}^m \pi_j A_{i,j} \leqslant 0$. In this second case, since $\boldsymbol{\pi}$ has been chosen to be a dual ray for the current problem (RMP($\mathcal{I}'$)), we also have that $\sum_{j=1}^m \pi_j b_j > 0$. Thus, again using Farkas' lemma, the problem (MP) is infeasible.

Similarly as for the pricing subproblem, given a dual ray $\boldsymbol{\pi}$, we do not actually enumerate the whole set $\mathcal{I}$ to find an index such that $\sum_{j=1}^m \pi_j A_{i,j} > 0$. Instead, we refer to the following optimization problem, called "Farkas pricing":

$$\begin{array}{ll} \text{minimize} & -\sum_{j=1}^m \pi_j A_{i,j} \\ \text{s.t.} & i \in \mathcal{I}\,. \end{array} \qquad \text{(FP}(\boldsymbol{\pi})\text{)}$$

Then, denoting by $v(\text{(FP}(\boldsymbol{\pi})\text{)})$ the optimal value of (FP($\boldsymbol{\pi}$)), we have either:

- $v(\text{(FP}(\boldsymbol{\pi})\text{)}) < 0$, obtained for $i \in \mathcal{I}$. Then i is added to $\mathcal{I}'$.
- $v(\text{(FP}(\boldsymbol{\pi})\text{)}) \geqslant 0$. Then the problem (MP) is infeasible.

Note two things. First, the problem (FP($\boldsymbol{\pi}$)) is very similar to the problem (SP($\boldsymbol{\pi}$)), thus algorithms solving (SP($\boldsymbol{\pi}$)) often handle the resolution of (FP($\boldsymbol{\pi}$)) as well. Second, as for (SP($\boldsymbol{\pi}$)), as long as a heuristic provides us with an index $i \in \mathcal{I}$ such that $\sum_{j=1}^m \pi_j A_{i,j} > 0$, this index can be added to $\mathcal{I}'$.

A.3 Integer linear program

We would now like to solve an integer linear program, with a large set of variables, indexed over the set $\mathcal{I}$. Such a program typically writes under the standard form as:

$$\begin{array}{lll} \text{minimize} & \displaystyle\sum_{i\in\mathcal{I}} c_i x_i & \\ \text{s.t.} & \displaystyle\sum_{i\in\mathcal{I}} A_{i,j} x_i = b_j & \forall j \in [m] \\ & x_i \in \mathbb{Z}_{\geqslant 0} & \forall i \in \mathcal{I} \end{array} \qquad \text{(IMP)}$$

where $c_i \in \mathbb{R}$ for all $i \in \mathcal{I}$, $A_{i,j} \in \mathbb{R}$ for all $i \in \mathcal{I}$ and $j \in [m]$, and $b_j \in \mathbb{R}$ for all $j \in [m]$.

The linear relaxation of this problem can be solved with the column generation algorithm.

A.3.1 Branch-and-price

In order to obtain optimal integer solutions, the column generation algorithm could be embedded into a branch-and-price scheme. This resolution method relies on multiple calls to the column generation algorithm in nodes of a branch-and-bound like tree. An example of branch-and-price algorithm is given in Chapter 6.

Figure A.1 illustrates the principle of branch-and-price for an ILP written as

$$\begin{aligned} \text{minimize} \quad & \sum_{i \in \mathcal{I}} c_i x_i & & \\ \text{s.t.} \quad & \sum_{i \in \mathcal{I}} A_{i,j} x_i = b_j & \forall j \in [m] & \\ & x_i \in \mathbb{Z}_{\geqslant 0} & \forall i \in \mathcal{I} \ . & \end{aligned} \tag{A.1}$$

As in a classical branch-and-bound, we establish a branching strategy, which refers to the choice of the variables to branch on, and the associated decisions to impose in the children nodes. More precisely, we start from a root node where we solve the linear relaxation of the ILP to optimality with the column generation algorithm.

If there exists no feasible solution, then the node is dropped (further decisions will not lead to a feasible solution). If the solution is integer, we update the best integer solution and its value to this solution and drop the node. Otherwise, the solution is fractional, its value is either larger than the one of the best integer solution found so far or not. In the first case, the node is also dropped (further decisions will not lead to an optimal integer solution). In the second case, we build children nodes following the branching strategy. Each of the children nodes carry additional constraints corresponding to the branching decisions. We repeat this process on the children nodes until an optimal integer solution is found. Note that we solve a linear relaxation with additional branching constraints at each node using the column generation algorithm, the branching constraints appearing in the master problem and (/or) in the pricing subproblem.

Additionally, we can add valid inequalities to the master problem, either in the root node or in deeper nodes in order to speed up the branch-and-price algorithm by cutting fractional solutions that would otherwise be explored. When such cuts are added, we often talk about branch-and-price-and-cut. The valid inequalities are added in the master problem and could change the nature of the pricing subproblem.

A.3.2 Enumeration-based gap closing

Alternatively, the pricing subproblem algorithm can generally be adapted in order to generate all possible variables with reduced cost smaller or equal to $UB - LB$, where LB is a lower bound and UB is an upper bound on the optimal value of the ILP (IMP). Then, solving an ILP with the column generation variables, along with these new columns, provides us with an optimal integer solution, as stated by Lemma A.3.1. This lemma can be originally found in a book by Nemhauser and Wolsey [67] (Proposition 2.1, p.389), and is later stated in the following form by Parmentier and Meunier [72] (Lemma 2).

Lemma A.3.1 (Nemhauser and Wolsey [67], Proposition 2.1, p.389)**.** *Consider an ILP with variables $(x_i)_i$ for which the linear relaxation admits a finite optimal value LB. Suppose given an upper bound UB on the optimal value of the ILP. Then for every i such that $\tilde{c}_i > UB - LB$, the variable x_i is equal to 0 in all optimal solutions of the integer program, where $\tilde{c}_i$ denotes the reduced cost of the variable x_i when the linear relaxation has been solved to optimality.*

As highlighted in [22, 72], this alternative approach seems efficient when the gap $UB - LB$ is small. This also requires to have access to an upper bound, which is not always the case (as in Chapter 6).

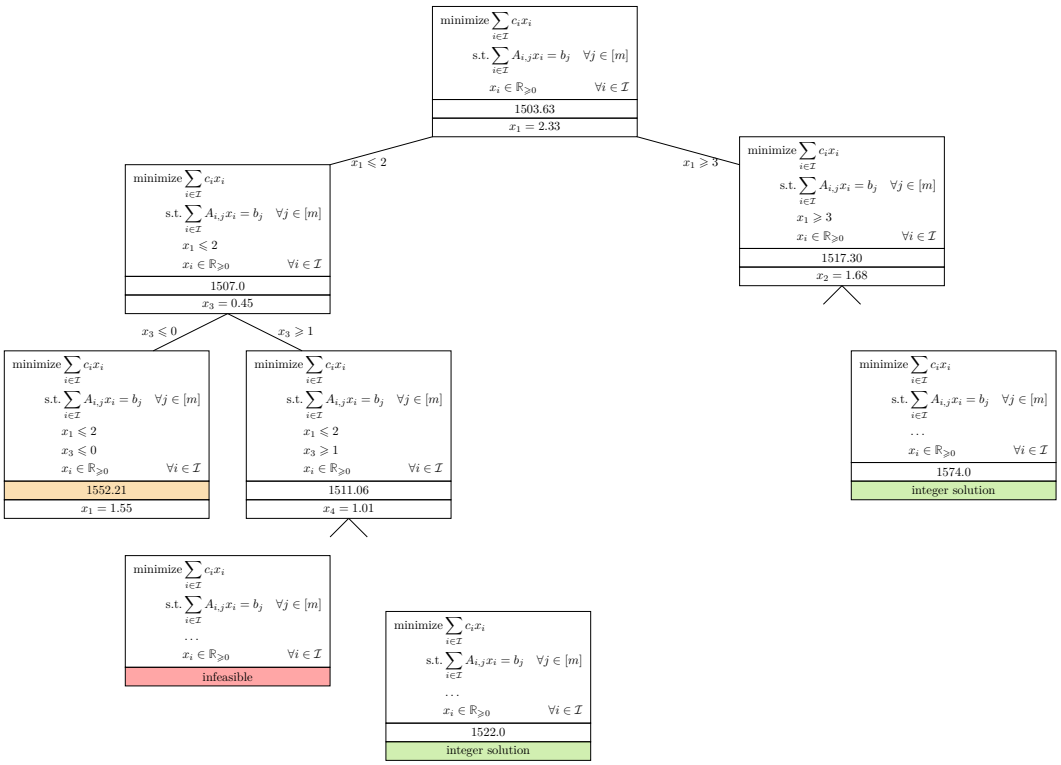


Figure A.1: An illustration of the branch-and-price algorithm: the right side of the tree is explored first and the best integer solution is updated with value 1574.0. Then the left nodes are explored. The node with value 1552.21 does not generate children nodes as its value is worse than the best integer value. Then the infeasible node does not generate any children nodes either. The next best integer solution has value 1522.0. When all nodes have been explored, we have found the optimal integer solution.

ECOLE
DOCTORALE
DE MATHEMATIQUES
HADAMARD

Titre : Planification périodique et optimisation des transports ferroviaires : grilles cycliques pour le fret sous incertitude, planification de lignes et affectations équilibrées

Mots clés : Fret ferroviaire ; Planification périodique ; Génération de colonnes ; Optimisation stochastique ; Planification des lignes ; Affectations équilibrées

Résumé : Cette thèse explore des problèmes d'optimisation en planification des transports et en conception d'emplois du temps périodiques, en combinant programmation en nombres entiers, optimisation combinatoire et modélisation stochastique, avec des applications au fret ferroviaire à la SNCF.

La première contribution traite du problème de gestion des équipes de conducteurs de fret à la SNCF : étant données des tâches périodiques (trains à conduire), l'objectif est de construire des *grilles cycliques* optimales couvrant l'ensemble des tâches, tout en respectant une réglementation riche. La pratique industrielle actuelle décompose ce problème en deux étapes séquentielles — construction des *journées de service* (séquences de tâches journalières affectées à un conducteur) puis leur organisation en grilles cycliques — ce qui introduit des sous-optimalités structurelles et rend difficile la prise en compte des retards. Ce travail propose d'abord un modèle déterministe intégré, formulé comme un programme linéaire en nombres entiers de type "set cover" et résolu par génération de colonnes, avec un sous-problème de pricing modélisé comme un plus court chemin avec contraintes de ressources (RCSP) sur un graphe acyclique. Les expériences numériques sur des instances réelles de la SNCF démontrent des réductions de coûts significatives par rapport à l'approche séquentielle. Ce cadre est ensuite étendu aux retards de trains via un score de *fragilité*, qui mesure le nombre moyen de trains et de journées de service perturbés qui devraient être pris en charge par des conducteurs de remplacement. Le problème stochastique de gestion des équipes minimise alors les coûts sous une contrainte de seuil sur la fragilité moyenne, et est résolu par génération de colonnes avec un RCSP enrichi gérant les distributions de retards. Les résultats confirment que l'approche intégrée stochastique surpasse la méthode séquentielle actuellement en usage à la SNCF, tant en termes de coûts que de robustesse aux perturbations.

La deuxième contribution aborde le problème de *planification de lignes* pour les transports publics. Étant donnés une matrice de demande origine-destination et un ensemble de lignes candidates, l'objectif est de sélectionner des lignes et d'y affecter des fréquences entières afin de minimiser le temps de trajet des voyageurs, sous contraintes de budget et de capacité. Une reformulation de Dantzig–Wolfe d'un modèle basé sur les arcs, sur un graphe appelé "change-and-go network", est résolue par branch-and-price-and-cut, et comparée à la résolution directe du modèle de base dans des expériences numériques.

Enfin, la troisième contribution adopte une perspective plus théorique et étudie les *affectations équilibrées* de tâches périodiques, c'est-à-dire telles que chaque travailleur effectue chaque tâche avec la même fréquence à long terme. Un lien structurel est établi entre l'existence de telles affectations et les propriétés de graphes orientés eulériens arc-colorés. Des théorèmes d'existence et de périodicité, des bornes sur la période minimale et des résultats de complexité sont établis, d'abord pour une version de base, puis pour une version enrichie par des contraintes supplémentaires.

Title : Optimization for Transportation Planning and Periodic Task Scheduling : Rail Freight Crew Management under Train Delays, Line Planning, and Balanced Assignments

Keywords : Rail freight ; Periodic Scheduling ; Column generation ; Stochastic optimization ; Line planning ; Balanced assignments

Abstract : This thesis investigates optimization problems arising in transportation planning and periodic task scheduling, combining integer programming, combinatorial optimization, and stochastic modeling, with a strong application focus on rail freight operations at SNCF.

The first contribution addresses the *crew management* problem for rail freight drivers at SNCF : given periodic tasks (trains to be driven), we aim to build optimal *cyclic rosters* covering all tasks, while complying with numerous rules and regulations. Current industrial practice, motivated by a full body of literature, decomposes this problem into two sequential stages : first, the construction of *duties* (daily task sequences assigned to a single driver), then their organization into cyclic rosters. This sequential approach introduces structural suboptimalities and makes it difficult to account for train delays. This work proposes first a deterministic integrated model formulated as a set-covering integer linear program and solved by column generation. The pricing subproblem is modeled as a Resource-Constrained Shortest Path (RCSP) problem on an acyclic graph, and solved using a label correcting algorithm with additional speeding-up strategies. Numerical experiments on real SNCF instances demonstrate significant cost reductions compared to the sequential pipeline.

Building on this deterministic framework, we then extend the problem to account for train delays. A *fragility* score is introduced to measure the sensitivity of a roster to train delays : as exogenous delays arise and propagate within each duty, this score measures the expected number of disrupted trains and duties that would be operated by backup drivers in practice. This stochastic crew management problem then builds optimal cyclic rosters covering all trains, with an additional threshold constraint on the expected fragility. The corresponding model is solved by column generation with an enriched RCSP pricing subproblem that handles delay distributions. Results confirm that the integrated stochastic approach outperforms the current two-stage sequential approach at SNCF, both in terms of cost and robustness to disruptions.

Shifting to a different application domain, the second contribution of this work tackles the *line planning* problem in public transit networks. Given a passenger origin-destination demand matrix and a pool of candidate lines, the goal is to select lines and assign integer frequencies so as to minimize passengers' travel time, subject to budget and capacity constraints. A Dantzig–Wolfe reformulation of an arc-based model on a "change-and-go network" is solved with branch-and-price-and-cut. Numerical experiments compare the straightforward resolution of the arc-based model with the proposed algorithm.

Finally, the third contribution takes a more theoretical perspective and studies *balanced assignments* of periodic tasks, i.e., such that each worker performs every task with the same frequency over the long term. A structural connection is established between the existence of such assignments and properties of arc-colored Eulerian directed graphs. Existence and periodicity theorems, bounds on the minimal period, and complexity results are established, for a basic version first, then for an extended version introducing additional constraints.

Institut Polytechnique de Paris
91120 Palaiseau, France